

CS100 Lecture 0

Introduction

Contents

- Introduction to programming
- Standardization of C and C++
- Course Information

Introduction to programming

What is programming?

Introduction to programming

Computer programming is the process of writing instructions that get executed by computers.

The instructions, also known as **code**, are written in a **programming language** which the computer can *understand* and use to perform a task or solve a problem.

Machine code

The only language that the computer can directly understand is the **machine code**, which is **binary**!

- For convenience, write binary code in hexadecimal form.

```
gkxx@nodiscard-gkxx ~/tmp> objdump -d a_out
a.out: file format elf64-x86-64

Disassembly of section .init:
0000000000001000 < .init>:
1000:  f3 0f 1e fa          endbr64
1004:  48 83 ec 08          sub    $0x8,%rsp
1008:  48 8b 05 d9 2f 00 00  mov    0x2fd9(%rip),%rax    # 3fe8 <__gmon_start__@Base>
100f:  48 85 c0             test   %rax,%rax
1012:  74 02              je     1016 < .init+0x16>
1014:  ff d0             call   *%rax
1016:  48 83 c4 08          add    $0x8,%rsp
101a:  c3                ret
```

Assembly code

The assembly code is just a more human-readable form of the machine code.

- It can be translated into machine code straightforwardly.

```
gkxx@nodiscard-gkxx ~/tmp> objdump -d a_out
a.out: file format elf64-x86-64

Disassembly of section .init:
0000000000001000 < .init>:
1000:  f3 0f 1e fa          endbr64
1004:  48 83 ec 08          sub    $0x8,%rsp
1008:  48 8b 05 d9 2f 00 00  mov    0x2fd9(%rip),%rax    # 3fe8 <__gmon_start__@Base>
100f:  48 85 c0             test   %rax,%rax
1012:  74 02              je     1016 < .init+0x16>
1014:  ff d0             call   *%rax
1016:  48 83 c4 08          add    $0x8,%rsp
101a:  c3                ret
```

High-level languages

Disadvantages of assembly code:

- Not portable: Different machines (architectures) may have different instruction sets.
- Too difficult to write and understand:
 - Instructions are very primitive and low-level, e.g. load, store, add, jump, ...
 - No high-level structures/abstractions: control-flow statements, classes, ...

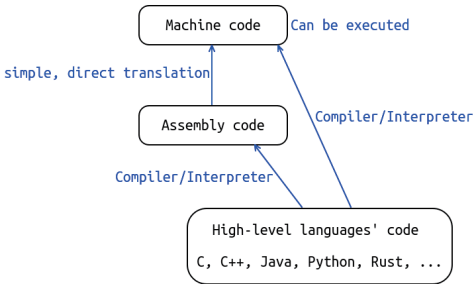
Learn more about them in senior courses (CS110, CS130, etc.)

High-level languages

Examples: C, Java, Python, C++, Rust, JavaScript, ...

- Syntactically rich
- Better abstraction ability
- More structured code
- Easier to code, debug and maintain
- Easier to ensure safety

High-level languages and compilers (including interpreters)

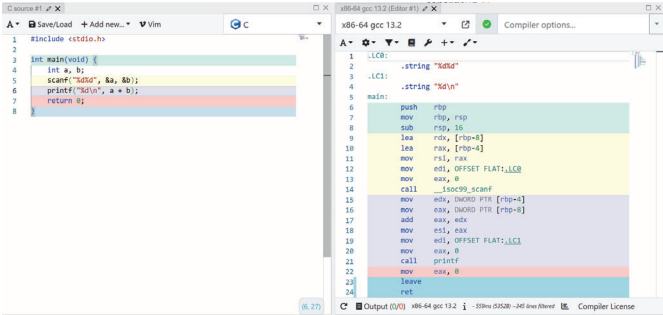


High-level languages and compilers

A **compiler** is a special program that **translates** a programming language's source code into machine code, bytecode or another programming language.

- The high-level code cannot be understood by the computer directly! It needs to be translated by a compiler.
 - The C/C++ compiler will translate the program and generate an executable (**.exe** file).
- Some languages (e.g. Python) use **interpreters**, which may be seen as a special kind of compiler.
 - An interpreter reads and executes the program directly, without earlier compiling it into machine code.
 - When executing a Python program, you don't see a **.exe** file!

High-level languages and compilers



11/33

Standardization of C and C++

Both C and C++ have ISO standards:

- C: ISO/IEC 9899, since 1990
 - C89/90, C99, C11, C17, C23, ...
 - Since C11 (2011), a new standard every 6 years.
- C++: ISO/IEC 14882, since 1998
 - C++98/03, C++11, C++14, C++17, C++20, C++23, ...
 - Since C++11 (2011), a new standard every 3 years.

CS100 is based on C17 and C++17.

16/33

High-level languages and compilers

A compiler is a special program that translates a programming language's source code into machine code, bytecode or another programming language.

```
#include <stdio.h>
int main(void) {
    puts("hello world");
}
```

`gcc hello.c -o hello.exe` ⇒ Generates an **executable** `hello.exe`.

- `gcc` is the C compiler used here.
- `-o xxx` indicates that the name of the generated file is `xxx`.

`.\hello` ⇒ Prints `hello world`.

12/33

What is the value of a standard?

- Offers a unique, standard definition of the language.
- Every compiler should accept the programs that conform to the standard, with some possible compiler extensions.
- If we want our code to be compiled and run with different compilers on different platforms, make it standard-conforming.

17/33

Compilers

Popular C/C++ compilers:

- GCC (GNU Compiler Collection)
- MSVC (Microsoft Visual C compiler)
- Clang
- ICC (Intel C/C++ Compiler)

13/33

How to get the standard?

Possible ways:

- Purchase the standard documentation from ISO (or ANSI) ⇒ Very expensive
- Download the working drafts (free): Some working drafts are really close to the standards.
 - [Useful resources for C](#) and [for C++](#).
- [cppreference](#) ⇐ RECOMMENDED!

18/33

Why C and C++?

- C is a high-level language that is closest to assembly.
- C++ "is as close to C as possible, but no closer".
- C++ is a language that
 - is a better C,
 - supports data abstraction,
 - and supports object oriented programming.
- C and C++ are popular, powerful, general-purpose and efficient, and play an essential role in areas like systems programming, game engines development, high frequency trading, ...

14/33

About [cppreference](#)

The standard documentations are not friendly for learning:

- They aim to provide a **precise definition** of the language, instead of teaching the language.
- The chapters are not organized in a way suitable for learning.
- Changes between different standards are not shown in-place.

Cppreference is a complete online reference for the C and C++ languages and standard libraries.

- A more convenient and friendly version of the standards.

19/33

Standardization of C and C++

Course Information

15/33

20/33

Course objectives

Upon completion of the course, you should be able to:

- Know how to analyze simple problems and design programs.
- Know how to write them in C and C++.
- Know C/C++ syntax.
- Understand some basic ideas of programming.
- Understand sequential and OO programming.

21/33

Course Structure

- 16 weeks teaching
 - C: Week 1 ~ 6
 - C++: Week 7 ~ 16
 - 1 midterm exam, in week 13 ~ 15
- 2 lectures every week
 - Monday 8:15-9:55
 - Wednesday 8:15-9:55
 - Room: Teaching Center 201, 202
- Weekly recitations

22/33

Assessment

Homework 72% + Midterm Exam 22% + Quiz 4% + Recitation attendance checks 2%.

- No final exams
- No projects
- Homework assignments make up a large proportion!

23/33

Recitations

- About 15 recitations
- 2 hours every week
- Include review of important contents, some extensions, homework/quiz/exam solutions, and some coding examples.
- **Attendance mandatory!** (2% in total)
- Videos will be available on Bilibili.

24/33

Homework assignments

- 8 homework assignments:
 - $7\% + 7\% + 7\% + 11\% + 7\% + 11\% + 11\% + 11\% = \mathbf{72\%}$ (may be adjusted)
- **Submit through Online Judge!!**
 - Any other forms of submissions are not accepted, unless with special reasons.
 - For every problem, the score of **the last submission** will be taken as the final score.
- **No plagiarism!!!** The penalty will be heavy.
 - The TAs are really experienced. Don't take any chances.

25/33

*Academic integrity and AI (GPT, Copilot, etc.)

- Anything obtained from AI tools is treated equivalently as the contents from the Internet.
 - If two students copied the same code from the Internet and submitted them $\implies$ **PLAGIARISM!**
 - If two students copied the same answer from AI tools and submitted them $\implies$ **PLAGIARISM!**
- Typical excuses we **don't** accept:
 - "We used the same ChatGPT account, and ChatGPT gave me his code."
 - "Copilot generated the same code for us."

26/33

Quizzes

- 1 or 2 quizzes, 4% in total.
- During regular classes
- Time will not be announced in advance!

27/33

Midterm exam

- 22%
- Will be held in the 13th ~ 15th week
- Covers **everything in regular classes and recitations**, unless otherwise stated.
 - Ranging from the beginning to *operator overloading* (C++).
- Will not be too difficult. Don't worry.

28/33

PIAZZA!!

- All the notifications will be announced on Piazza.
- All the course materials will be available on Piazza.
- Ask and answer questions on Piazza.
- In case you miss any important notifications, we suggest **checking Piazza once every 12 hours**.

29/33

Textbooks and references

Recommended textbooks:

- C++ *Primer* (5th edition) (**NOT C++ *Primer Plus!***) (based on C++11)
- *Effective C++* (based on C++98), *Effective Modern C++* (based on C++14)
- *The C++ Programming Language* (4th edition) (based on C++11), written by **Bjarne Stroustrup**, the inventor of C++

All of the course materials conform to the C17 and C++17 standards, based on cppreference and the standard committee's papers.

30/33

Textbooks and references

Note on textbooks:

- You are not required to read the entire books.
- We will recommend (or require) reading some of the chapters/sections when necessary.
- We will upload the materials to Piazza.

31/33

Brief history of C

3/33

Textbooks and references

Note on cppreference/standard documentations:

- You may find them difficult to read at the beginning. Don't worry!
- We will teach you how to read them (maybe during recitations).
- By the time you have learned some major aspects of the language, you should have less difficulty reading them.

32/33

The UNIX operating system and C

- In 1969, a small group of AT&T Bell Labs led by **Ken Thompson** and **Dennis Ritchie** began to develop UNIX.
- In 1973, UNIX kernel was rewritten in C.
- From 1969 to 1973, Dennis Ritchie developed C in Bell Labs.
- In 1978, Kernighan and Ritchie published the **K&R book**: *The C Programming Language*.

4/33

C/C++ environment setup

- We recommend VSCode + GCC (MinGW on Windows). Follow [this video](#).
- Advanced tools (Visual Studio, CMake, ...) may be needed in further homework assignments.

33/33

Dennis Ritchie and the K&R book



- Dennis M. Ritchie (1941 - 2011)
 - The inventor of C
 - Co-inventor of UNIX
 - ACM Turing Award (1983) with Ken Thompson for UNIX

5/33

CS100 Lecture 1

The First C Program

1/33

Standardization of C

- "K&R C": Informal specification (the K&R book)
- ANSI C: Known as "C89"
 - American National Standards Institute
 - Came out in 1989
- ISO C standard: ISO/IEC 9899
 - International Organization for Standardization
 - First version: "C90" in 1990, the same standard as C89 with only formatting changes
 - C99 (1999), C11 (2011), C17 (2017/2018), C23 (2023)

6/33

Contents

- Brief history of C
- The first C program
 - Functions (basic) and the `main` function
 - `scanf` and `printf`

2/33

The first C program

7/33

Hello World

```
#include <stdio.h>

int main(void) {
    printf("hello world\n");
    return 0;
}
```

- Save the code as `hello.c`.
- `gcc hello.c -o hello.exe` $\Rightarrow$ generates `hello.exe`
- `.\hello.exe` $\Rightarrow$ prints `hello world`, with a newline at the end.

The `main` function

Every C program coded to run in a hosted execution environment contains the definition of a **function** named `main`, which is the designated start of the program.

```
// Other things (functions, structures, ...), if any ...

int main(void) {
    // The program starts here.
    statement_1;
    statement_2;
    // ...
    statement_n;
}
```

* What is a function?

A function in C

A function in mathematics: $f : S \mapsto T$, accepts some **arguments** and **returns** some value.

Example: $f(x) = x^2, x \in \mathbb{Z}$ accepts an integer argument, and **returns** its square.

Write it in C:

```
int f(int x) {
    return x * x;
}
```

A function in C

Example: $f(x, y) = x + y, x, y \in \mathbb{R}$

Write it in C:

```
double f(double x, double y) { // Two arguments
    return x + y;
}
```

`double` : double-precision floating-point number $\Rightarrow$ will be covered in later lectures.

A function in C

Syntax: `ReturnType FunctionName(Parameters) { FunctionBody }`

`FunctionBody` can also contain more complex statements:

```
int max(int a, int b) {
    if (a < b)
        return b;
    else
        return a;
}
```

`if` statement $\Rightarrow$ will be covered in later lectures.

A function in C

Syntax: `ReturnType FunctionName(Parameters) { FunctionBody }`

A function can have no arguments. To define such a function, write `void` in `Parameters` :

```
int always42(void) {
    return 42;
}
```

We will introduce more on **functions** in later lectures.

The `main` function

Every C program coded to run in a hosted execution environment contains the definition of a **function** named `main`, which is the designated start of the program.

According to the standard, the `main` function **must** has one of the following signatures:

1. `int main(void) { ... }`
2. `int main(int argc, char *argv[]) { ... }`
3. `/* another implementation-defined signature */`

For now, we only use the first one: `int main(void) { ... }`.

"signature": consisting of the return-type, parameters, and some other possible information

The `main` function

```
int main(void) {
    printf("hello world\n");
    return 0;
}
```

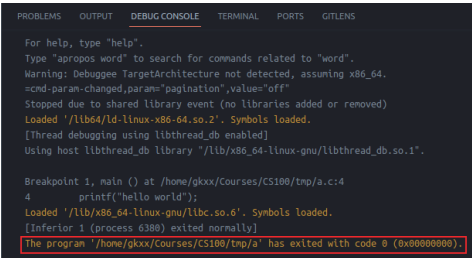
The return value of `main` : Indicates whether the program exits successfully.

A program exits successfully if and only if its `main` function returns `0`.

The `main` function

A program exits successfully if and only if its `main` function returns `0`.

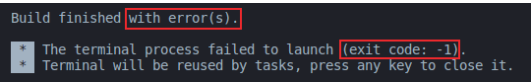
You may also see this somewhere else:



The `main` function

A program exits successfully if and only if its `main` function returns `0`.

You may also see this somewhere else:



The `main` function

It is ok to omit `return 0;` in `main` (but not in other functions):

```
int main(void) {
    printf("hello world\n");
}
```

According to the standard:

If the return type is compatible with `int` and control reaches the terminating `}`, the value returned to the environment is the same as if executing `return 0;`.

18/33

The "A+B" problem

Reads two integers from input (separated by whitespaces), and prints the sum of them.



23/33

`printf`

Declared in the standard library header file `stdio.h`.

- That's why we need `#include <stdio.h>` in the beginning.

Writes something to the **standard output**.

```
printf("hello world\n");
```

- Prints `hello world`, with a newline `\n` at the end.
- Try this out: `printf("hello\nworld\n");`

19/33

`scanf`

Also declared in `stdio.h`.

Reads something from the **standard input**.

Example: Reads two integers from the standard input, separated by whitespaces.

```
scanf("%d%d", &a, &b);
```

- `%d`: Indicates that an integer is expected, and will be stored into an `int` variable.
- `&`: The **address-of** operator $\Rightarrow$ will be covered in later lectures.

For now, just remember to add `&` when passing things to `scanf`.

24/33

Output vs return

```
int main(void) {
    printf("hello world\n");
    return 0;
}
```

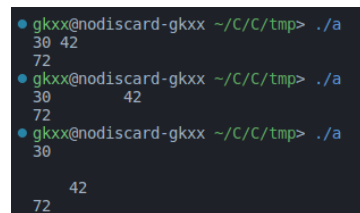
- What is the output of the program?
- What is the return value of `main`?

20/33

`scanf`

```
scanf("%d%d", &a, &b);
```

How should these two integers be separated? Try it out:



25/33

Output vs return

```
int main(void) {
    printf("hello world\n");
    return 0;
}
```

- What is the output of the program? $\Rightarrow$ `hello world` with an ending newline.
- What is the return value of `main`? $\Rightarrow$ `0`.

21/33

`scanf`

```
scanf("%d%d", &a, &b);
```

`%d` will skip any leading whitespaces.

- "whitespace" refers to the character that looks "blank": space `' '`, newline `'\n'`, tab `'\t'`, etc.

More on the rules related to `scanf` will be covered in recitations.

26/33

The "A+B" problem

Reads two integers from input (separated by whitespaces), and prints the sum of them.

```
#include <stdio.h>

int main(void) {
    int a, b; // declares two variables of type "int", named "a" and "b".
    scanf("%d%d", &a, &b);
    printf("%d\n", a + b);
    return 0;
}
```

22/33

`printf` printing an integer

Given `a = 30`, `b = 42` as input:

- `printf("%d\n", a + b);`
 $\Rightarrow$ prints `72`, with a newline in the end.
- `printf("%d + %d equals %d\n", a, b, a + b);`

What is the output?

27/33

printf printing an integer

Given `a = 30`, `b = 42` as input:

- `printf("%d\n", a + b);`
⇒ prints `72`, with a newline in the end.
- `printf("%d + %d equals %d\n", a, b, a + b);`
⇒ prints `30 + 42 equals 72`, with a newline in the end.

28/33

Summary

Understand the following two programs:

```
#include <stdio.h>

int main(void) {
    printf("hello world\n");
    return 0;
}
```

- Why is `#include <stdio.h>` needed?
- What does `int main(void)` mean?
- What is the meaning of `return 0;`? Can it be omitted?
- How do we represent a newline?
- What does `%d` mean?
- How are whitespaces handled when reading integers with `scanf`?

```
#include <stdio.h>

int main(void) {
    int a, b;
    scanf("%d", &a, &b);
    printf("%d + %d equals %d\n",
        a, b, a + b);
    return 0;
}
```

29/33

Summary

- Why is `#include <stdio.h>` needed?
 - `scanf` and `printf` are declared in the standard library header file `stdio.h`.
- What does `int main(void)` mean?
 - The `main` function is where the program starts. `int` is the return type and `void` indicates that this function accepts no arguments.
- What is the meaning of `return 0;`? Can it be omitted?
 - A program returns `0` if it exits successfully. The `main` function of C will execute `return 0` automatically at the end if we don't write it explicitly.

30/33

Summary

- How do we represent a newline?
 - `'\n'`.
- What does `%d` mean?
 - Indicates that the type of the data being read or printed is `int`.
- How are whitespaces handled when reading integers with `scanf`?
 - When reading `int` with `%d` in `scanf`, leading whitespaces are ignored.

31/33

Exercises

1. Run the following code. Try to understand what it means.

```
#include <stdio.h>

int max(int a, int b) {
    if (a < b)
        return b;
    else
        return a;
}

int main(void) {
    int a, b;
    scanf("%d%d", &a, &b);
    printf("%d\n", max(a, b));
    return 0;
}
```

32/33

Exercises

2. Try to modify the program above. Add some prompts for input and output. For example:

```
gkxx@nodiscard-gkxx ~/C/C/tmp> ./a
Input two integers: 30 42
The larger one is 42.
```

33/33

CS100 Lecture 2

Variables I and Arithmetic Types

1/52

Contents

- Variable declaration
- Arithmetic types
 - Bits and bytes
 - Integer types
 - Real floating types
 - Character types
 - Boolean type

2/52

Variable declaration

Type of a variable

Every variable in C has a type.

- The type is **fully deterministic** and cannot be changed.
- The type is **known even when the program is not run**.
 - ⇔ The type is known at **compile-time**.
 - ⇔ C is **statically-typed**¹. ⇔ C has a **static type system**.
 - In contrast, Python is **dynamically-typed**.

4/52

Statically-typed vs dynamically-typed

Python: dynamically typed

```
a = 42          # Type of a is int.
a = "hello"     # Type of a becomes str.
```

The type of a variable

- can be changed, and
- is not necessarily known until we run the program.

C: statically-typed

```
int a = 42; // Type of a is int.
a = "hello"; // Error! Types mismatch!
```

The type of a variable

- is explicitly written on declaration, and
- is known at compile-time, and
- cannot be changed.

A type-related error in C is (usually) a compile error:

- It stops the compiler. The executable will not be generated.

Readability matters

[Best practice] Declare the variable when you first use it!

- If the declaration and use of the variable are too separated, it will become much more difficult to figure out what they are used for as the program goes longer.

[Best practice] Use meaningful names!

- The program would be a mess if polluted with names like `a`, `b`, `c`, `d`, `x`, `y`, `cnt`, `cnt_2`, `flag1`, `flag2`, `flag3` everywhere.
- Use meaningful names: `sumOfScore`, `student_cnt`, `open_success`, ...

Readability is very important. Many students debug day and night simply because their programs are not human-readable.

Declare a variable

To declare a variable, we need to specify its **type** and **name**.

```
Type name;
```

Example:

```
int x; // Declares a variable named 'x', whose type is 'int'.
double y; // Declares a variable named 'y', whose type is 'double'.
```

We may declare multiple variables of a same type in one declaration statement, separated by `,`:

```
int x, y; // Declares two variables 'x' and 'y', both having type 'int'.
```

Use of global variables

One reason for using global variables is to have them shared between functions:

```
int input;
void work(void) {
    printf("%d\n", input);
}
int main(void) {
    scanf("%d", &input);
    work();
}

void work(void) {
    // Error: 'input' was not decared
    // in this scope.
    printf("%d\n", input);
}
int main(void) {
    int input;
    scanf("%d", &input);
    work();
}
```

⇒ More about scopes and name lookup in later lectures / recitations.

Declare a variable

A variable declaration can be placed

- inside a function, which declares a **local variable**, or
- outside of any functions, which declares a **global variable**.

```
#include <stdio.h>

int x, y; // global variables

int main(void) {
    scanf("%d%d", &x, &y);
    printf("%d\n", x + y);
}

#include <stdio.h>

int main(void) {
    // local variables in 'main'
    int x, y;
    scanf("%d%d", &x, &y);
    printf("%d\n", x + y);
}
```

Initialize a variable

A variable can be **initialized** on declaration.

```
int x = 42; // Declares the variable 'x' of type 'int',
            // and initializes its value to 42.
int a = 0, b, c = 42; // Declares three 'int' variables, with 'a' initialized
                    // to 0, 'c' initialized to 42, and 'b' uninitialized.
```

This is syntactically **different** (though seems equivalent) to

```
int x; // Declares 'x', uninitialized.
x = 42; // Assigns 42 to 'x'.
```

[Best practice] Initialize the variable if possible. Prefer initialization to later assignment.

⇒ More on initialization in later lectures.

Local variables vs global variables

Which one do you prefer?

```
#include <stdio.h>

int x, y; // global variables

int main(void) {
    scanf("%d%d", &x, &y);
    printf("%d\n", x + y);
}

#include <stdio.h>

int main(void) {
    // local variables in 'main'
    int x, y;
    scanf("%d%d", &x, &y);
    printf("%d\n", x + y);
}
```

Arithmetic types

Refer to [this page](#) for a complete, detailed and standard documentation.

Integer types

Is `int` equivalent to $\mathbb{Z}$?

- Is there a limitation on the numbers that `int` can represent?

What are these variables used for?

```
#include <stdio.h>
// Other #includes

int x, y; // What are these two variables used for?

int moveSpaceShuttle(SpaceShuttle *shuttle, Coord to, Vehicle *by) {
    // 109 lines
}
int makePreparations(Environment *env, Task tasks[], Time time) {
    // 73 lines
}
LaunchResult launchSpaceShuttle(SpaceShuttle *shuttle, Task tasks[]) {
    // 35 lines
}
// Other 136 functions, 3325 lines in total
int main(void) {
    // 120 lines
}
```


Integer types

Is `int` equivalent to $\mathbb{Z}$?

- Is there a limitation on the numbers that `int` can represent?

Experiment:

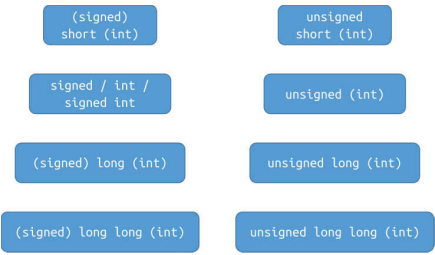
```
#include <stdio.h>

int main(void) {
    int x = 1;
    while (1) {
        printf("%d\n", x);
        x *= 2; // x = x * 2
        getchar();
    }
}
```

- On 64-bit Ubuntu 22.04 and compiled with GCC 13, after printing 1073741824 (2^{30}), the output becomes negative, and then 0.

```
1073741824
-2147483648
0
0
```

Integer types



Bits and bytes

Information is stored in computers in binary.

- $42_{\text{ten}} = 101010_{\text{two}}$.

A bit is either 0 or 1.

- The binary representation of 42 consists of 6 bits.

A byte is 8 bits² grouped together like 10001001.

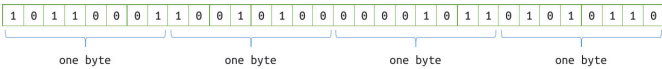
- At least 1 byte is needed to store 42.
- At least 3 bytes are needed to store $142857_{\text{ten}} = 100010111000001001_{\text{two}}$

Integer types

- The keyword `int` is optional in types other than `int`:
 - e.g. `short int` and `short` name the same type.
 - e.g. `unsigned int` and `unsigned` name the same type.
- "Unsigned-ness" needs to be written explicitly: `unsigned int`, `unsigned long`, ...
- Types without the keyword `unsigned` are signed by default:
 - e.g. `signed int` and `int` name the same type.
 - e.g. `signed long int`, `signed long`, `long int` and `long` name the same type.

Bits and bytes

A 32-bit number: $2979269462_{\text{ten}} = 10110001100101000000101101010110_{\text{two}}$.



Suppose now we have n bits.

- How many different values can be represented?
- What is the largest integer that can be represented?
- How do we represent negative numbers? Non-integer values? ...

Width of integer types

type	width (at least)	width (usually)
<code>short</code>	16 bits	16 bits
<code>int</code>	16 bits	32 bits
<code>long</code>	32 bits	32 or 64 bits
<code>long long</code>	64 bits	64 bits

- A signed type has the same width as its `unsigned` counterpart.
- It is also guaranteed that $\text{sizeof}(\text{short}) \leq \text{sizeof}(\text{int}) \leq \text{sizeof}(\text{long}) \leq \text{sizeof}(\text{long long})$.
 - $\text{sizeof}(T)$ is the number of bytes that `T` holds.

Bits and bytes

Suppose now we have n bits.

- How many different values can be represented?
 - 2^n .
- What is the largest integer that can be represented?
 - $2^n - 1 = \underbrace{111 \dots 1}_n_{\text{two}}$.
- How do we represent negative numbers? Non-integer values? ...
 - There are several different signed number representations, among which two's complement is widely used.
 - About floating-point numbers: IEEE754
 - Details are not covered in CS100.

Implementation-defined behaviors

The standard states that the exact width of the integer types is implementation-defined.

- Implementation:** The compiler and the standard library.
- An implementation-defined behavior depends on the compiler and the standard library, and is often also related to the hosted environment (e.g. the operating system).

Integer types

An integer type in C is either **signed** or **unsigned**, and has a **width** denoting the number of bits that can be used to represent values.

Suppose we have an integer type of n bits in width.

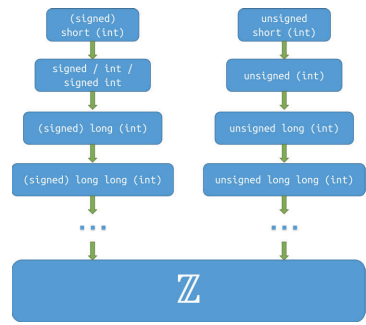
- If the type is **signed**³, the range of values that can be represented is $[-2^{n-1}, 2^{n-1} - 1]$.
- If the type is **unsigned**, the range of values that can be represented is $[0, 2^n - 1]$.

Which one should I use?

`int` is the most optimal integer type for the platform.

- Use `int` for integer arithmetic by default.
- Use `long long` if the range of `int` is not large enough.
- Use smaller types (`short`, or even `unsigned char`) for memory-saving or other special purposes.
- Use `unsigned` types for special purposes. We will see some in later lectures.

Which one is the real world, the integer types or $\mathbb{Z}$?



Exercise

Write the "A+B" program for real numbers. Which type do you decide to use? How do you read and print the values?

Exercise

Write the "A+B" program for real numbers. Which type do you decide to use? How do you read and print the values?

```
#include <stdio.h>

int main(void) {
    double a, b;
    scanf("%lf%lf", &a, &b);
    printf("%lf\n", a + b);
    return 0;
}
```

Real floating types

"Floating-point": The number's radix point can "float" anywhere to the left, right, or between the significant digits of the number.

Real floating-point types can be used to represent *some* real values.

- Real floating-point types $\neq \mathbb{R}$.

Real floating types

C has three types for representing real floating-point values:

- `float`: single precision. Matches IEEE754 binary32 format if supported.
- `double`: double precision. Matches IEEE754 binary64 format if supported.
- `long double`: extended precision. A floating-point type whose precision and range are at least as good as those of `double`.

Details of IEEE754 formats are not required in CS100.

Range of values can be found in [this table](#).

Character types

The C standard provides three different character types: `signed char`, `unsigned char` and `char`.

Let $T \in \{ \text{signed char}, \text{unsigned char}, \text{char} \}$. It is guaranteed that

$1 == \text{sizeof}(T) \leq \text{sizeof}(\text{short}) \leq \text{sizeof}(\text{int}) \leq \text{sizeof}(\text{long}) \leq \text{sizeof}(\text{long long})$.

- `T` takes exactly 1 byte.

Question: What is the valid range of `signed char`? `unsigned char`?

Character types

Question: What is the valid range of `signed char`? `unsigned char`?

- `signed char`: $[-128, 127]$.
- `unsigned char`: $[0, 255]$.

What? A character is an integer?

Which one should I use?

Use `double` for real floating-point arithmetic by default.

- In some cases the precision of `float` is not enough.
- Don't worry about efficiency! `double` arithmetic is not necessarily slower than `float`.

Do not use floating-point types for integer arithmetic!

scanf / printf

Refer to the table in [this page](#).

type	format specifier
short	%hd
int	%d
long	%ld
long long	%lld

- %f for float, %lf for double, and %Lf for long double.

type	format specifier
unsigned short	%hu
unsigned	%u
unsigned long	%lu
unsigned long long	%llu

ASCII (American Standard Code for Information Interchange)

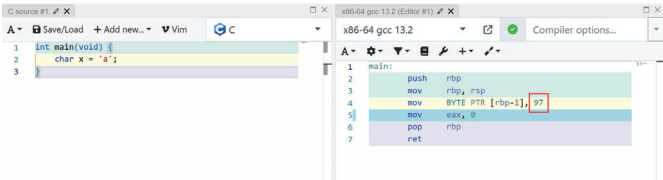
A character is represented in computers as its ASCII code, which is a small integer.

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	(NULL)	32	20	(SPACE)	64	40	@	96	60	>
1	1	(START OF HEADING)	33	21	!	65	41	A	97	61	a
2	2	(START OF TEXT)	34	22	"	66	42	B	98	62	b
3	3	(END OF TEXT)	35	23	#	67	43	C	99	63	c
4	4	(END OF TRANSMISSION)	36	24	\$	68	44	D	100	64	d
5	5	(ENQUIRY)	37	25	%	69	45	E	101	65	e
6	6	(ACKNOWLEDGE)	38	26	&	70	46	F	102	66	f
7	7	(BELL)	39	27	'	71	47	G	103	67	g
8	8	(BACKSPACE)	40	28	(	72	48	H	104	68	h
9	9	(HORIZONTAL TAB)	41	29	)	73	49	I	105	69	i
10	A	(LINE FEED)	42	2A	*	74	4A	J	106	6A	j
11	B	(VERTICAL TAB)	43	2B	+	75	4B	K	107	6B	k
12	C	(FORM FEED)	44	2C	,	76	4C	L	108	6C	l
13	D	(CARRIAGE RETURN)	45	2D	-	77	4D	M	109	6D	m
14	E	(SHIFT OUT)	46	2E	.	78	4E	N	110	6E	n
15	F	(SHIFT IN)	47	2F	/	79	4F	O	111	6F	o
16	10	(DATA LINK ESCAPE)	48	30	0	80	50	P	112	70	p
17	11	(DEVICE CONTROL 1)	49	31	1	81	51	Q	113	71	q
18	12	(DEVICE CONTROL 2)	50	32	2	82	52	R	114	72	r
19	13	(DEVICE CONTROL 3)	51	33	3	83	53	S	115	73	s
20	14	(DEVICE CONTROL 4)	52	34	4	84	54	T	116	74	t
21	15	(NEGATIVE ACKNOWLEDGE)	53	35	5	85	55	U	117	75	u
22	16	(SYNCHRONOUS IDLE)	54	36	6	86	56	V	118	76	v
23	17	(END OF TRANS. BLOCK)	55	37	7	87	57	W	119	77	w
24	18	(CANCEL)	56	38	8	88	58	X	120	78	x
25	19	(END OF MEDIUM)	57	39	9	89	59	Y	121	79	y
26	1A	(SUBSTITUTE)	58	3A	:	90	5A	Z	122	7A	z
27	1B	(ESCAPE)	59	3B	;	91	5B	[	123	7B	{
28	1C	(FILE SEPARATOR)	60	3C	<	92	5C	\	124	7C	
29	1D	(GROUP SEPARATOR)	61	3D	=	93	5D	]	125	7D	}
30	1E	(RECORD SEPARATOR)	62	3E	>	94	5E	^	126	7E	~
31	1F	(UNIT SEPARATOR)	63	3F	?	95	5F	_	127	7F	(DEL)

ASCII (American Standard Code for Information Interchange)

A character is represented in computers as its **ASCII code**, which is a small integer.

- We only consider the so-called *ASCII characters* here.



A character is **nothing but** an integer! In C, there is no "conversion" between characters and ASCII code!

Character types

For almost all cases, use `char` (or, sometimes `int`) to represent characters.

`signed char` and `unsigned char` are used for other purposes.

To read/print a `char` using `scanf` / `printf`, use `%c`.

ASCII (American Standard Code for Information Interchange)

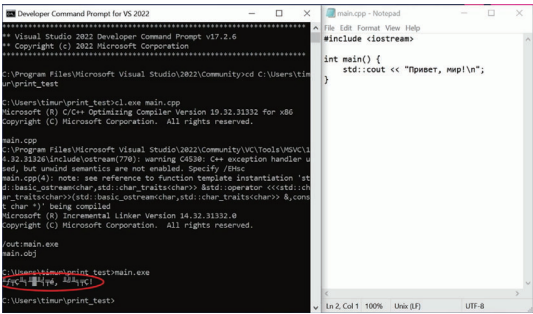
Important things to remember:

- `'0'`, `'9'` = [48, 57].
- `'A'`, `'Z'` = [65, 90].
- `'a'`, `'z'` = [97, 122].

Example: Given a lowercase letter, return its uppercase form.

```
char to_uppercase(char x) {
    return x - 32;
}
```

Sad story: Handling non-ASCII characters? ...



[Best practice] Avoid magic numbers

What is the meaning of `32` here? $\Rightarrow$ a magic number.

```
char to_uppercase(char x) {
    return x - 32;
}
```

Write it in a more human-readable way:

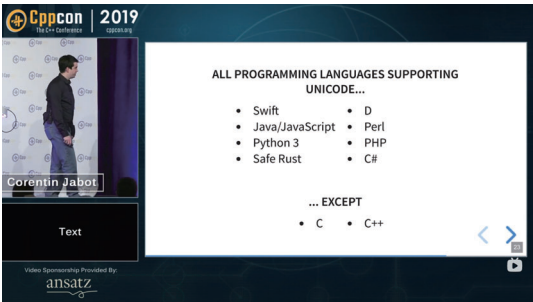
```
char to_uppercase(char x) {
    return x - ('a' - 'A');
}
```

Sad story: Handling non-ASCII characters? ...

Even though the standard provides `wchar_t`, `char8_t` (since C23), `char16_t` and `char32_t` to handle wide/Unicode characters, there are still a lot of problems.

C++23 has some improvement.

That's why Python people laugh at us ...



Escape sequence

Some special characters are not directly representable: newline, tab, quote, ...

We use **escape sequences**, e.g.

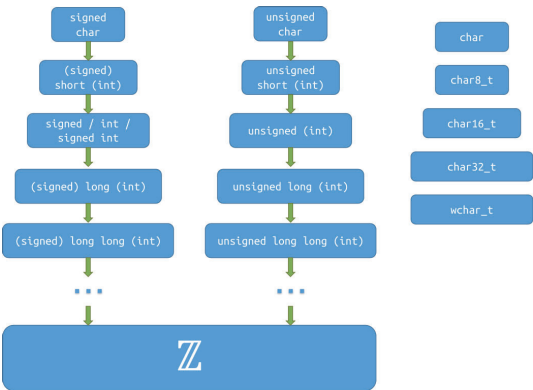
escape sequence	description
<code>\'</code>	single quote
<code>\"</code>	double quote
<code>\\</code>	backslash

escape sequence	description
<code>\n</code>	newline
<code>\r</code>	carriage return
<code>\t</code>	horizontal tab

Character types

`char`, `signed char` and `unsigned char` are three different types.

- Whether `char` is signed or unsigned is **implementation-defined**.
- If `char` is signed (unsigned), it represents the same set of values as the type `signed char` (`unsigned char`), but **they are not the same type**.
 - In contrast, `T` and `signed T` are the same type for `T` $\in$ { `short`, `int`, `long`, `long long` }.



Boolean type: `bool` (since C99)

A type that represents true/false, 0/1, yes/no, ...

To access the name `bool`, `true` and `false`, `<stdbool.h>` is needed. (until C23)

Example: Define a function that accepts a character and returns whether that character is a lowercase letter.

Before C99, using `int`, `0` and `1`:

```
int is_lowercase(char c) {
    if (c >= 'a' && c <= 'z')
        return 1;
    else
        return 0;
}
```

Since C99, using `bool`, `false` and `true`:

```
bool is_lowercase(char c) {
    if (c >= 'a' && c <= 'z')
        return true;
    else
        return false;
}
```

Summary

- Arithmetic types
 - Width, signed-ness, valid range
 - Which type to choose
 - Characters: ASCII code, escape sequence
- Boolean

Boolean type: `bool` (since C99)

Before C99, using `int`, `0` and `1`:

```
int is_lowercase(char c) {
    if (c >= 'a' && c <= 'z')
        return 1;
    else
        return 0;
}
```

Since C99, using `bool`, `false` and `true`:

```
bool is_lowercase(char c) {
    if (c >= 'a' && c <= 'z')
        return true;
    else
        return false;
}
```

Both return values can be used as follows:

```
char c; scanf("%c", &c);
if (is_lowercase(c)) {
    // do something when c is lowercase ...
}
```

Exercise

Write a simple calculator that handles input of the form `x op y`, where `x` and `y` are floating-point numbers and `op` $\in$ { `'+'`, `'-'`, `'*'`, `'/'` }. You may use a group of `if - else` statements like this:

```
if (op == '+') {
    // ...
} else if (op == '-') {
    // ...
} else if (op == '*') {
    // ...
} else if (op == '/') {
    // ...
} else {
    // report an error
}
```

[Best practice] Simplify your code

Just return the result of the condition expression.

```
int is_lowercase(char c) {
    return c >= 'a' && c <= 'z';
}

bool is_lowercase(char c) {
    return c >= 'a' && c <= 'z';
}
```

We will introduce the operators (`&&`, `<=`, `>=`) involved here in later lectures.

Notes

- ¹ The type of every expression in C is determined at compile-time except for *variable-length arrays* (since C99).
- ² A byte is 8 bits on most platforms, but we do have exceptions: [36-bit computing](#).
- ³ There are several different signed number representations, but all popular machines and almost all compilers use **two's complement**. Before C23 and C++20, the C/C++ standards allow for all possible representations, so the minimal valid range for a n -bit integer is $[-2^{n-1} + 1, 2^{n-1} - 1]$, which is the range for *one's complement* and *sign-and-magnitude*. Since C23 and C++20, the only representation allowed is two's complement, so the valid range is guaranteed to be $[-2^{n-1}, 2^{n-1} - 1]$. In C5100 we still assume that two's complement is used, even though we are based on C17/C++17.

Summary

- Variable declaration
 - Type + name
 - Multiple variables in one declaration statement
 - Global vs local
 - Initialization

CS100 Lecture 3

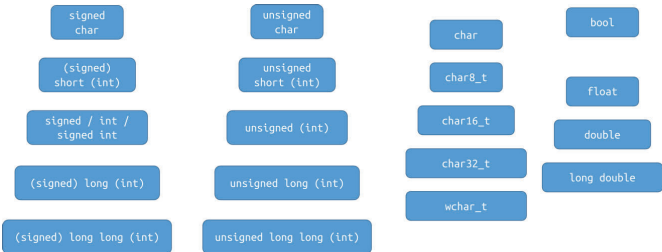
Operators and Control Flow I

Contents

- Operators
 - `+`, `-`, `*`, `/`, `%`
 - Compound assignment operators
 - Signed integer overflow
 - `++` and `--`
- Control flow
 - `if - else`
 - `while`
 - `for`

Summary

- Arithmetic types



Operators

The calculator

Accept input of the form `x op y`, where `x` and `y` are floating-point numbers and `op` $\in \{ '+', '-', '*', '/' \}$. Print the result.

```
#include <stdio.h>

int main(void) {
    double x, y;
    char op;
    scanf("%lf %c %lf", &x, &op, &y);
    if (op == '+')
        printf("%lf\n", x + y);
    else if (op == '-')
        printf("%lf\n", x - y);
    else if (op == '*')
        printf("%lf\n", x * y);
    else if (op == '/')
        printf("%lf\n", x / y);
    else
        printf("Invalid operator.\n");
    return 0;
}
```

3/55

Binary +, -, and *, /

`a + b`, `a - b`, `a * b`, `a / b`

If any one operand is of floating-point type and the other is an integer, the integer will be implicitly converted to that floating-point type.

Example:

```
double pi = 3.14;
int diameter = 20;
WhatType c = pi * diameter; // What is the type of this result?
```

8/55

Binary +, -, and *, /

`a + b`, `a - b`, `a * b`, `a / b`

If any one operand is of floating-point type and the other is an integer, the integer will be implicitly converted to that floating-point type.

Example:

```
double pi = 3.14;
int diameter = 20;
double c = pi * diameter; // 62.8
```

The value of `diameter` is implicitly converted to a value of type `double`. Then, a floating-point multiplication is performed, yielding a result of type `double`.

* Does this rule make sense?

9/55

Overview of arithmetic operators

Operator	Operator name	Example	Result
+	unary plus	+a	the value of a after promotions
-	unary minus	-a	the negative of a
+	addition	a + b	the addition of a and b
-	subtraction	a - b	the subtraction of b from a
*	product	a * b	the product of a and b
/	division	a / b	the division of a by b
%	remainder	a % b	the remainder of a divided by b
~	bitwise NOT	~a	the bitwise NOT of a
&	bitwise AND	a & b	the bitwise AND of a and b
	bitwise OR	a b	the bitwise OR of a and b
^	bitwise XOR	a ^ b	the bitwise XOR of a and b
<<	bitwise left shift	a << b	a left shifted by b
>>	bitwise right shift	a >> b	a right shifted by b

5/55

Binary +, -, and *, /

`a + b`, `a - b`, `a * b`, `a / b`

If any one operand is of floating-point type and the other is an integer, the integer will be implicitly converted to that floating-point type.

Example:

```
double pi = 3.14;
int diameter = 20;
double c = pi * diameter; // 62.8
```

The value of `diameter` is implicitly converted to a value of type `double`. Then, a floating-point multiplication is performed, yielding a result of type `double`.

* Does this rule make sense? - Yes, because $\mathbb{Z} \subseteq \mathbb{R}$.

10/55

+ , - , * , / , %

- + and - have two versions: unary (+a, -a) and binary (a+b, a-b).
 - The unary +/− and binary +/− are **different operators**, although they use the same notation.
- Operator precedence:
 - { unary +, unary - } > { *, /, % } > { binary +, binary - }
 - e.g. `a + b * c` is interpreted as `a + (b * c)`, instead of `(a + b) * c`.
 - ⇒ We will talk more about operator precedence later.

6/55

Binary +, -, and *, /

`a + b`, `a - b`, `a * b`, `a / b`

If any one operand is of floating-point type and the other is an integer, the integer will be implicitly converted to that floating-point type, and the result type is that floating-point type.

Similarly, if the operands are of types `int` and `long long`, the `int` value will be implicitly converted to `long long`, and the result type is `long long`.¹

11/55

Binary +, -, and *, /

`a + b`, `a - b`, `a * b`, `a / b`

Before the evaluation of such an expression, the operands (`a`, `b`) undergo a sequence of **type conversions**.

- The detailed rules of the conversions are very complex,
 - including *promotions*, conversions between `signed` and `unsigned` types, conversions between integers and floating-point types, etc.
 - We only need to remember some common ones.
- In the end, the operands will be converted to a **same type**, denoted `T`. The result type is also `T`.

7/55

Division: a / b

Assume `a` and `b` are of the same type `T` (after conversions as mentioned above).

- Then, the result type is also `T`.

Two cases:

- If `T` is a floating-point type, this is a floating-point division.
- If `T` is an integer type, this is an integer division.

12/55

Division: `a / b`

Two cases:

- If `T` is a floating-point type, this is a floating-point division.
 - The result is no surprising.
- If `T` is an integer type, this is an integer division.
 - The result is **truncated towards zero** (since C99 and C++11) ².
 - What is the result of `3 / -2`?

Let `a` and `b` be two integers.

- What is the difference between `a / 2` and `a / 2.0`?
- What does `(a + 0.0) / b` mean? What about `1.0 * a / b`?

Signed integer overflow

If a **signed integer type** holds a value that is not in the valid range, **overflow** is caused.

Suppose `int` is 32-bit and `long long` is 64-bit.

Do the following computations cause overflow?

```
int ival = 100000; long long llval = ival;
int result1 = ival * ival;
long long result2 = ival * ival;
long long result3 = llval * ival;
long long result4 = llval * ival * ival;
```

Division: `a / b`

If `T` is an integer type, this is an integer division.

- The result is **truncated towards zero** (since C99 and C++11) ².
- What is the result of `3 / -2`?
 - `-1.5` truncated towards zero, which is `-1`.

What is the difference between `a / 2` and `a / 2.0`?

- `a / 2` yields an integer, while `a / 2.0` yields a `double`.

What does `(a + 0.0) / b` mean? What about `1.0 * a / b`?

- Both use floating-point division to compute $\frac{a}{b}$. The floating-point numbers `0.0` and `1.0` here cause the conversion of the other operands.

Signed integer overflow

Suppose `int` is 32-bit and `long long` is 64-bit.

Do the following computations cause overflow?

```
int ival = 100000; long long llval = ival;
int result1 = ival * ival; // (1) overflow
long long result2 = ival * ival; // (2) overflow
long long result3 = llval * ival; // (3) not overflow
long long result4 = llval * ival * ival; // (4) not overflow
```

- (1) $(10^5)^2 = 10^{10} > 2^{31} - 1$.
 - (2) The result type of the multiplication `ival * ival` is `int`, which causes overflow.
- This is not affected by the type of `result2`.

Remainder: `a % b`

Example: `15 % 4 == 3`.

`a` and `b` must have integer types.

If `a` is negative, is the result negative? What if `b` is negative? What if both are negative?

Signed integer overflow

Suppose `int` is 32-bit and `long long` is 64-bit.

Do the following computations cause overflow?

```
int ival = 100000; long long llval = ival;
int result1 = ival * ival; // (1) overflow
long long result2 = ival * ival; // (2) overflow
long long result3 = llval * ival; // (3) not overflow
long long result4 = llval * ival * ival; // (4) not overflow
```

- (3) Since `llval` is of type `long long`, the value of `ival` will be implicitly converted to `long long`, and then the multiplication yields a `long long` value.
 - (4) `*` is **left-associative**, so the expression `a * b * c` is interpreted as `(a * b) * c`.
- ⇒ We will talk about associativity in later lectures.

Remainder: `a % b`

Example: `15 % 4 == 3`.

`a` and `b` must have integer types.

If `a` is negative, is the result negative? What if `b` is negative? What if both are negative?

For any integers `a` and `b`, the following always holds:

```
(a / b) * b + (a % b) == a
```

Undefined behavior

Signed integer overflow is **undefined behavior**: There are no restrictions on the **behavior of the program**. Compilers are not required to diagnose undefined behavior (although many simple situations are diagnosed), and the compiled program is not required to do anything meaningful.

- It may yield some garbage values, or zero, or anything else;
- or, this statement may be removed if the compiler is clever enough;
- or, the program may crash;
- or, any other results beyond imagination.

⇒ More on undefined behaviors in recitations.

Compound assignment operators

`+=`, `-=`, `*=`, `/=`, `%=`

- `a op= b` is equivalent to `a = a op b`.
- e.g. `x *= 2` is equivalent to `x = x * 2`.
- [Best practice] Learn to use these operators, to make your code clear and simple.

Unsigned integers never overflow

Unsigned integer arithmetic is always performed *modulo* 2^n , where n is the number of bits in that integer type.

For example, for `unsigned int` (assuming it is 32-bit)

- adding one to $2^{32} - 1$ gives 0 because $2^{32} \equiv 0 \pmod{2^{32}}$, and
- subtracting one from 0 gives $2^{32} - 1$ because $-1 \equiv 2^{32} - 1 \pmod{2^{32}}$.

* "wrap-around"

if - else

Conditionally executes some code.

Syntax:

- (1) `if (condition) statementT`
- (2) `if (condition) statementT else statementF`

where `condition` is an expression of arithmetic type or pointer type.

⇒ We will talk about pointers in later lectures.

If `condition` compares not equal to the integer zero, `statementT` is executed.

In (2), if `condition` compares equal to the integer zero, `statementF` is executed.

23/55

28/55

if - else

- (1) `if (condition) statementT`
- (2) `if (condition) statementT else statementF`

`statementT`, `statementF`: Either a **statement** or a **block** (any number of statements surrounded by `{ }`).

- If more than one statements are to be executed, they must be surrounded with a pair of braces `{ }`.

```
int i = 42;
if (i == 42)
    printf("hello");
    printf("world\n"); // This is NOT a part of statement!
                      // This code is incorrectly indented.
```

24/55

29/55

if - else

- (1) `if (condition) statementT`
- (2) `if (condition) statementT else statementF`

`statementT`, `statementF`: Either a **statement** or a **block** (any number of statements surrounded by `{ }`).

- If more than one statements are to be executed, they must be surrounded with a pair of braces `{ }`.

```
int i = 42;
if (i == 42) {
    printf("hello");
    printf("world\n");
}
```

25/55

30/55

The calculator example

```
double x, y;
char op;
scanf("%lf %c %lf", &x, &op, &y);
if (op == '+') {
    printf("%lf + %lf == %lf\n", x, y, x + y);
} else if (op == '-') {
    printf("%lf - %lf == %lf\n", x, y, x - y);
} else if (op == '*') {
    printf("%lf * %lf == %lf\n", x, y, x * y);
} else if (op == '/') {
    printf("%lf / %lf == %lf\n", x, y, x / y);
} else {
    printf("Unknown operator!\n");
}
```

26/55

31/55

The calculator example

If there is only one statement, the braces can be omitted.

```
double x, y;
char op;
scanf("%lf %c %lf", &x, &op, &y);
if (op == '+')
    printf("%lf + %lf == %lf\n", x, y, x + y);
else if (op == '-')
    printf("%lf - %lf == %lf\n", x, y, x - y);
else if (op == '*')
    printf("%lf * %lf == %lf\n", x, y, x * y);
else if (op == '/')
    printf("%lf / %lf == %lf\n", x, y, x / y);
else
    printf("Unknown operator!\n");
```

27/55

32/55

Increment/decrement operators

Unary operators that increment/decrement the value of a variable by `1`.

Postfix form: `a++`, `a--`

Prefix form: `++a`, `--a`

- `a++` and `++a` increment the value of `a` by `1`.
- `a--` and `--a` decrement the value of `a` by `1`.

Increment/decrement operators

Unary operators that increment/decrement the value of a variable by `1`.

Postfix form: `a++`, `a--`

The result of the **postfix** increment/decrement operators is the value of `a` **before** incrementation/decrementation.

* What does "result" mean?

Increment/decrement operators

Unary operators that increment/decrement the value of a variable by `1`.

Postfix form: `a++`, `a--`

The result of the **postfix** increment/decrement operators is the value of `a` **before** incrementation/decrementation.

```
int x = 42;
printf("%d\n", x++); // x becomes 43, but 42 is printed.
int y = x++; // y is initialized with 43. x becomes 44.
```

25/55

Increment/decrement operators

Unary operators that increment/decrement the value of a variable by `1`.

Prefix form: `++a`, `--a`

The result of the **prefix** increment/decrement operators is the value of `a` **after** incrementation/decrementation.

```
int x = 42;
printf("%d\n", ++x); // x becomes 43, and 43 is printed.
int y = ++x; // y is initialized with 44. x becomes 44.
```

26/55

Control flow

The calculator example

Nested `if` statements are possible:

```
double x, y;
char op;
scanf("%lf %c %lf", &x, &op, &y);
if (op == '+')
    printf("%lf + %lf == %lf\n", x, y, x + y);
else if (op == '-')
    printf("%lf - %lf == %lf\n", x, y, x - y);
else if (op == '*')
    printf("%lf * %lf == %lf\n", x, y, x * y);
else if (op == '/') {
    if (y == 0)
        printf("Error: division by zero.\n");
    else
        printf("%lf / %lf == %lf\n", x, y, x / y);
} else
    printf("Unknown operator!\n");
```

33/55

while

```
while (n > 0) {
    // loop body
    --n;
}
```

How many times is this loop body executed?

After the loop, what is the value of `n` ?

Dangling else

The `else` is always associated with the closest preceding `if`.

```
if (i == 1)
    if (j == 2)
        printf("aaa\n");
else // Incorrectly indented! This `else` is associated with `if (j == 2)`.
    printf("bbb\n");
```

If you want this `else` to be associated with `if (i == 1)`, add a pair of braces.

```
if (i == 1) {
    if (j == 2)
        printf("aaa\n");
} else
    printf("bbb\n");
```

34/55

while

```
while (n > 0) {
    // loop body
    --n;
}
```

How many times is this loop body executed?

- `n`.

After the loop, what is the value of `n` ?

- `0`, which makes `n > 0` not satisfied.

The formatter tells you the truth.

Unlike Python, indentations are not part of the C syntax. The following code is completely legal, but with extremely low readability.

```
int a,b;scanf("%d%d",&a,&b);int s=a+b;printf("%d\n",s);if(a==1)if(b==2)printf("aaa\n");else printf("b\n");
```

[Best practice] Use a formatter! If you use VSCode, press Shift+Alt+F (Windows), ⌘⇧F (MacOS) or Ctrl+Shift+I (Linux) to format the code.

- Other editors may also have this functionality.

The formats are customizable. Explore it on your own if you are interested.

while

A simpler and common way to write this loop:

```
while (n--) {
    // loop body
}
```

How many times is this loop body executed?

After the loop, what is the value of `n` ?

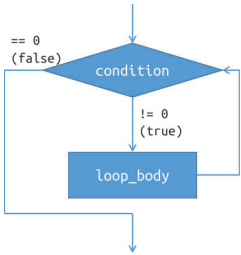
while

Executes some code repeatedly under certain condition.

Syntax:

```
while (condition) loop_body
```

where `condition` is an expression of arithmetic type or pointer type, and `loop_body` is a statement or a block.



while

A simpler and common way to write this loop:

```
while (n--) {
    // loop body
}
```

How many times is this loop body executed?

- `n`.

After the loop, what is the value of `n` ?

- `-1`. When `n == 0`, the result of `n--` is `0`, which not only makes the loop terminated but also decrements `n` by `1`.

while

Read an integer `n`, followed by `n` integers. Print the sum of the `n` integers.

```
int n;
scanf("%d", &n);
int sum = 0;
while (n > 0) {
    int x;
    scanf("%d", &x);
    sum += x;
    --n;
}
printf("%d\n", sum);
```

break

`break;` causes the enclosing loop to terminate.

```
while (n--) {
    int x;
    scanf("%d", &x);
    // If x == 42, the loop is terminated and control goes to (*).
    if (x == 42)
        break;
    sum += x;
}
// (*)
printf("%d\n", sum);
```


continue

`continue`; causes the remaining portion of the enclosing loop body to be skipped (in the current iteration).

```
while (n--) {
    int x;
    scanf("%d", &x);
    // If x == 42, the rest of the loop body is skipped and control goes to (*).
    if (x == 42)
        continue;
    sum += x;
    // (*)
}
printf("%d\n", sum);
```

43/55

break vs continue

`break`: terminates the loop.

`continue`: skips the rest of the loop body.

```
while (n--) {
    int x;
    scanf("%d", &x);
    if (x == 42)
        break; // goto (*)
    sum += x;
    // (*)
}
printf("%d\n", sum);
```

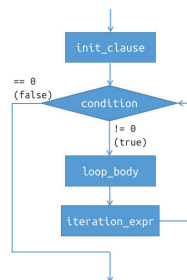
```
while (n--) {
    int x;
    scanf("%d", &x);
    if (x == 42)
        continue; // goto (*)
    sum += x;
    // (*)
}
printf("%d\n", sum);
```

44/55

for

Syntax: `for (init_clause; condition; iteration_expr) loop_body`

- `init_clause` may be an expression or a declaration (since C99).
- `condition` is an expression of arithmetic type or pointer type. It is evaluated before the loop body.
- `iteration_expr` is an expression evaluated after the loop body.
- `loop_body` is either a statement or a block.



45/55

for

Typical usage: execute the body for `n` times.

The following are equivalent.

```
while (n--) {
    int x;
    scanf("%d", &x);
    sum += x;
}
```

```
for (int i = 0; i < n; ++i) {
    int x;
    scanf("%d", &x);
    sum += x;
}
```

46/55

for

Syntax: `for (init_clause; condition; iteration_expr) loop_body`

It is equivalent to

```
{
    init_clause;
    while (condition) {
        loop_body
        iteration_expr;
    }
}
```

47/55

for

A loop with iteration variable `i` incremented by `2` every time:

```
for (int i = 0; i < n; i += 2)
    // ...
```

Loop until $\lfloor \sqrt{n} \rfloor$:

```
for (int i = 0; i * i <= n; ++i) // We don't need sqrt.
    // ...
```

A loop with iteration variable `i` going downward from `n` to `0`:

```
for (int i = n; i >= 0; --i)
    // ...
```

* Can we use `unsigned` here?

48/55

for

What will happen?

```
for (unsigned i = n; i >= 0; --i)
    // ...
```

49/55

for

What will happen?

```
for (unsigned i = n; i >= 0; --i)
    // ...
```

`i` is an `unsigned` integer, so `i >= 0` holds forever.

- When `i == 0`, `--i` makes the value of `i` become `4294967295` (if `unsigned` is 32-bit).
- An unsigned integer never overflows. Unsigned integer arithmetic is performed modulo 2^n .

50/55

for

[Best practice] Always declare and initialize the iteration variable inside the `for` statement, if possible.

Do not write code of the last century:

Write it in a modern, standard way:

```
main() {
    int i, j;
    /* ... */
    for (i = 0; i < n; ++i)
        for (j = 0; j < m; ++j)
            /* ... */
}
```

```
int main(void) {
    // ...
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < m; ++j)
            // ...
}
```

51/55

Summary

Operators

- `a + b`, `a - b`, `a * b`, `a / b`: `a` and `b` are converted to a common type `T` according to some rules, and the result type is also `T`.
- Integer division: The result is truncated towards zero.
- Remainder: `(a / b) * b + (a % b) == a` always holds.
- `+=`, `-=`, `*=`, `/=`, `%=`: `a op= b` $\Leftrightarrow$ `a = a op b`
- Signed integer overflow: **undefined behavior**.
- Unsigned integer arithmetic: modulo 2^n , never overflows.
- `++a` and `--a` return the original value. `++a` and `--a` return the new value.

52/55

Summary

Control flow

- `if - else`
- `while`, `break`, `continue`
- `for`

53/55

Exercises

1. Write a program that reads an **unknown number** of integers from input until `0` is entered. Print the sum of these integers.
2. Make the calculator a loop that runs repeatedly, instead of handling only one input.

54/55

Notes

¹ Similar conversions also happen for operands with types `int + long`, `long + long`, `long`, `unsigned + unsigned long long`, etc. However, if any operand is of type with *rank* less than *rank* of `int`, **integer promotion** is first applied. For example, if `a` is of type `signed char` and `b` is of type `short`, both of them are converted to `int` first and the result type is `int`, not `short`.

Integer promotion is also applied to the operand of unary `+` and `-`. For example, `+x` is of type `int` if `x` is a `short` variable.

² Before C99 and C++11, the result of integer division is rounded in an implementation-defined direction.

55/55

CS100 Lecture 4

Operators and Control Flow II, Functions

Contents

- Operators
 - Operator precedence, associativity and evaluation order
 - Comparison operators `<`, `<=`, `>`, `>=`, `==`, `!=`
 - Logical operators `&&`, `||`, `!`
 - Conditional operator `?:`
 - Assignment operator `=`
- Control Flow
 - `do - while`
 - `switch - case`
- Functions

2/49

Operators

Operator precedence

Operator precedence defines the order in which operators are bound to their arguments.

Example: `*` and `/` have higher precedence than `+` and `-`, so `a + b * c` is interpreted as `a + (b * c)` instead of `(a + b) * c`.

Operator precedence does not determine evaluation order.

- `f() + g() * h()` is interpreted as `f() + (g() * h())`, but the order in which `f`, `g` and `h` are called is **unspecified**.

3/49

4/49

Associativity

Each operator is either **left-associative** or **right-associative**.

Operators with the same precedence have the same associativity.

Example: `+` and `-` are **left-associative**, so `a - b + c` is interpreted as `(a - b) + c`, instead of `a - (b + c)`.

Associativity does not determine evaluation order.

- `f() - g() + h()` is interpreted as `(f() - g()) + h()`, but the order in which `f`, `g` and `h` are called is **unspecified**.

5/49

Evaluation order

Unless otherwise stated, the order in which the operands are evaluated is **unspecified**.

- We will see that `&&`, `||` and `?:` (and also `,` in recitations) have specified evaluation order of their operands.

Examples: In the following expressions, it is **unspecified** whether `f` is called before `g`.

- `f() + g()`
- `f() == g()`
- `some_function(f(), g())` (Note that the `,` here is not the **comma operator**.)
- ...

6/49

Evaluation order and undefined behavior

Let `A` and `B` be two expressions. The behavior is **undefined** if

- the order in which `A` and `B` are evaluated is unspecified ¹, and
- both `A` and `B` modify an object, or one modifies an object and the other uses its value ².

Examples:

```
i = ++i + i++; // undefined behavior
i = i++ + 1;   // undefined behavior
printf("%d, %d\n", i, i++); // undefined behavior
```

Recall that **undefined behavior** means "everything is possible". We cannot make any assumptions about the behavior of the program.

7/49

Terminology: Return type/value of an operator

When it comes to "the return type/value of an operator", we are actually viewing the operator as a function:

```
int operator_plus(int a, int b) {
    return a + b;
}
int operator_postfix_inc(int &x) { // We must use a C++ notation here.
    int old = x;
    x += 1;
    return old;
}
```

The "return value" of an operator is the value of the expression it forms.
The "return type" of an operator is the type of its return value.

Comparison operators

Comparison operators are binary operators that test a condition and return `1` if that condition is logically `true` and `0` if it is logically `false`.

Operator	Operator name	Operator	Operator name
<code>a == b</code>	equal to	<code>a > b</code>	greater than
<code>a != b</code>	not equal to	<code>a <= b</code>	less than or equal to
<code>a < b</code>	less than	<code>a >= b</code>	greater than or equal to

For most cases, the operands `a` and `b` are also converted to a same type, just as what happens for `a + b`, `a - b`, ...

Comparison operators

Note: Comparison operators in C cannot be chained.
Example: `a < b < c` is interpreted as `(a < b) < c` (due to left-associativity), which means to

- compare `(a < b)` first, whose result is either `0` or `1`, and then
- compare `0 < c` or `1 < c`.

To test `a < b < c`, use `a < b && b < c`.

Logical operators

Logical operators apply standard boolean algebra operations to their operands.

Operator	Operator name	Example
<code>!</code>	logical NOT	<code>!a</code>
<code>&&</code>	logical AND	<code>a && b</code>
<code> </code>	logical OR	<code>a b</code>

Logical operators

`!a`, `a && b`, `a || b`

Recall the boolean algebra:

<i>A</i>	<i>B</i>	$\neg A$	$A \wedge B$	$A \vee B$
True	True	False	True	True
True	False	False	False	True
False	True	True	False	True
False	False	True	False	False

For C logical operators:

<i>a</i>	<i>b</i>	<code>!a</code>	<code>a && b</code>	<code>a b</code>
<code>!= 0</code>	<code>!= 0</code>	<code>0</code>	<code>1</code>	<code>1</code>
<code>!= 0</code>	<code>== 0</code>	<code>0</code>	<code>0</code>	<code>1</code>
<code>== 0</code>	<code>!= 0</code>	<code>1</code>	<code>0</code>	<code>1</code>
<code>== 0</code>	<code>== 0</code>	<code>1</code>	<code>0</code>	<code>0</code>

Logical operators

Precedence: `!` > comparison operators > `&&` > `||`.

Typical example: lexicographical comparison of two pairs (a_1, b_1) and (a_2, b_2)

```
int less(int a1, int b1, int a2, int b2) {
    return a1 < a2 || (a1 == a2 && b1 < b2);
}
```

The parentheses are optional here, but it improves readability.

Avoid abuse of parentheses

Too many parentheses reduce readability:

```
int less(int a1, int b1, int a2, int b2) {
    return (((a1) < (a2)) || (((a1) == (a2)) && ((b1) < (b2))));
    // Is this a1 < b1 || (a1 == b1 && a2 < b2)
    // or (a1 < b1 || a1 == b1) && a2 < b2 ?
}
```

[Best practice] Use one pair of parentheses when two binary logical operators meet.

Short-circuit evaluation

`a && b` and `a || b` perform short-circuit evaluation:

- For `a && b`, `a` is evaluated first. If `a` compares equal to zero (is logically `false`), `b` is not evaluated.
 - `False ∧ p ≡ False`
- For `a || b`, `a` is evaluated first. If `a` compares not equal to zero (is logically `true`), `b` is not evaluated.
 - `True ∨ p ≡ True`

The evaluation order is specified!

Conditional operator `?:`

Syntax: `condition ? expressionT : expressionF`,

where `condition` is an expression of scalar type.

The evaluation order is specified!

- First, `condition` is evaluated.
- If `condition` compares not equal to zero (is logically `true`), `expressionT` is evaluated, and the result is the value of `expressionT`.
- Otherwise (if `condition` compares equal to zero, which is logically `false`), `expressionF` is evaluated, and the result is the value of `expressionF`.

Conditional operator `?:`

Syntax: `condition ? expressionT : expressionF`,

Example: `to_uppercase(c)` returns the uppercase form of `c` if `c` is a lowercase letter, or `c` itself if it is not.

```
char to_uppercase(char c) {
    if (c >= 'a' && c <= 'z')
        return c - ('a' - 'A');
    else
        return c;
}
```

Use `?:` to rewrite it:

```
char to_uppercase(char c) {
    return c >= 'a' && c <= 'z' ? c - ('a' - 'A') : c;
}
```

Conditional operator ?:

Syntax: `condition ? expressionT : expressionF`

Use it to replace some simple and short `if - else` statement.

Avoid abusing it! Nested conditional operators reduces readability significantly.

```
int result = a < b ? (a < c ? a : c) : (b < c ? b : c); // Um ...
```

[Best practice] Avoid more than two levels of nested conditional operators.

18/49

Assignment operator =

`lhs = rhs`

The assignment operator **returns the value of lhs** after assignment.

Moreover, the assignment operator is **right-associative**, making it possible to write "chained" assignments:

```
int a = 0, b = 1, c = 2;
a = b = c; // interpreted as a = (b = c)
           // Both a and b are assigned with 2.
```

19/49

Control Flow

do - while

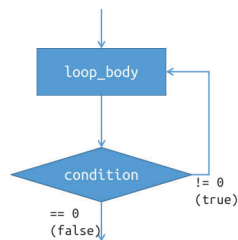
Syntax: `do loop_body while (condition);`

Executes `loop_body` repeatedly until the value of `condition` compares equal to zero (is logically `false`).

Example:

```
int i = 0;
do {
    printf("%d", i++);
} while (i < 5);
```

Output: `01234`



21/49

do - while

Note that in each iteration, the condition is tested **after** the body is executed.

```
int i = 0;
do {
    printf("%d", i++);
} while (i < n);
```

Even if `n == 0`, `0` is printed. The loop body is always executed at least once.

22/49

do - while

Exercise: Rewrite a `do - while` loop using a `while` loop.

```
do {
    // loop_body
} while (condition);
```

23/49

do - while

Exercise: Rewrite a `do - while` loop using a `while` loop.

```
do {
    // loop_body
} while (condition);
```

Use `while (1)` and `break` :

```
while (1) {
    // loop_body
    if (!condition)
        break;
}
```

24/49

switch - case

The calculator example:

```
int main(void) {
    double a, b;
    char op;
    scanf("%lf %c %lf", &a, &op, &b);
    if (op == '+')
        printf("%lf\n", a + b);
    else if (op == '-')
        printf("%lf\n", a - b);
    else if (op == '*')
        printf("%lf\n", a * b);
    else if (op == '/')
        printf("%lf\n", a / b);
    else
        printf("Invalid operator!\n");
    return 0;
}
```

25/49

switch - case

Rewrite it using `switch - case` :

```
if (op == '+')
    printf("%lf\n", a + b);
else if (op == '-')
    printf("%lf\n", a - b);
else if (op == '*')
    printf("%lf\n", a * b);
else if (op == '/')
    printf("%lf\n", a / b);
else
    printf("Invalid operator: %c\n", op);
```

```
switch (op) {
    case '+':
        printf("%lf\n", a + b); break;
    case '-':
        printf("%lf\n", a - b); break;
    case '*':
        printf("%lf\n", a * b); break;
    case '/':
        printf("%lf\n", a / b); break;
    default:
        printf("Invalid operator!\n");
        break;
}
```

26/49

switch - case

`switch (expression) { ... }`

```
switch (op) {
    case '+':
        printf("%lf\n", a + b); break;
    case '-':
        printf("%lf\n", a - b); break;
    case '*':
        printf("%lf\n", a * b); break;
    case '/':
        printf("%lf\n", a / b); break;
    default:
        printf("Invalid operator!\n");
        break;
}
```

- First, `expression` is evaluated.
- Control finds the `case` label to which `expression` compares equal, and then goes to that label.
- Starting from the selected label, **all subsequent statements are executed until a `break`; or the end of the `switch` statement is reached.**
- Note that `break`; here has a special meaning.

27/49

switch - case

```
switch (expression) { ... }
```

```
switch (op) {
case '+':
    printf("%lf\n", a + b); break;
case '-':
    printf("%lf\n", a - b); break;
case '*':
    printf("%lf\n", a * b); break;
case '/':
    printf("%lf\n", a / b); break;
default:
    printf("Invalid operator!\n");
    break;
}
```

- If no `case` label is selected and `default:` is present, the control goes to the `default:` label.
- `default:` is optional, and often appears in the end, though not necessarily.
- `break;` is often needed. Modern compilers often warn against a missing `break;`.

28/49

switch - case

The expression in a `case` label must be an integer *constant expression*, whose value is known at compile-time, such as `42`, `'a'`, `true`, ...

```
int n; scanf("%d", &n);
int x = 42;
switch (value) {
case 3.14: // Error: It must have an integer type.
    printf("It is pi.\n");
case n: // Error: It must be a constant expression (known at compile-time)
    printf("It is equal to n.\n");
case 42: // OK.
    printf("It is equal to 42.\n");
case x: // Error: 'x' is a variable, not treated as "constant expression".
    printf("It is equal to x.\n");
}
```

29/49

switch - case

Another example: Determine whether a letter is vowel or consonant.

```
switch (letter) {
case 'a':
case 'e':
case 'i':
case 'o':
case 'u':
    printf("%c is vowel.\n", letter);
    break;
default:
    printf("%c is consonant.\n", letter);
}
```

30/49

Functions

Call and return

```
#include <stdlib.h>
#include <stdio.h>

double divide(int a, int b) {
    if (b == 0) {
        fprintf(stderr, "Division by zero!\n");
        exit(EXIT_FAILURE);
    }
    return 1.0 * a / b;
}

int main(void) {
    int x, y; scanf("%d%d", &x, &y);
    double result = divide(x, y);
    printf("%lf\n", result);
    // ...
}
```

- For the *call expression* `divide(x, y)`: First the arguments `x` and `y` are passed into `divide` as if the parameters are initialized³ as follows:

```
int a = x;
int b = y;
```

Then control is transferred into the function `divide`, starting from the first statement.

32/49

Call and return

```
#include <stdlib.h>
#include <stdio.h>

double divide(int a, int b) {
    if (b == 0) {
        fprintf(stderr, "Division by zero!\n");
        exit(EXIT_FAILURE);
    }
    return 1.0 * a / b;
}

int main(void) {
    int x, y; scanf("%d%d", &x, &y);
    double result = divide(x, y);
    printf("%lf\n", result);
    // ...
}
```

`return` does two things:

- passes a value out to the *call site*.
 - This value is the result of the **call expression**.
 - Such a value does not exist if the function return type is `void`.
- transfers the control to the *call site*.

33/49

Call and return

```
#include <stdlib.h>
#include <stdio.h>

double divide(int a, int b) {
    if (b == 0) {
        fprintf(stderr, "Division by zero!\n");
        exit(EXIT_FAILURE);
    }
    return 1.0 * a / b;
}

int main(void) {
    int x, y; scanf("%d%d", &x, &y);
    double result = divide(x, y);
    printf("%lf\n", result);
    // ...
}
```

- The parentheses `()` in the expression `divide(x, y)` is the **function-call operator**.
- Even if the function accepts no parameters, the function-call operator should not be omitted.
- A statement like `f;` without the function-call operator is **valid**.
 - It is a statement that has no effect, just like `5;`, `2+3;`, `;` or `{}`.

34/49

Call and return

If a function has return type `void`, the function does not pass a value to the call site.

For a non-`void` function:

- A `return something;` statement must be executed to return something.
- If control reaches the end of the function without a `return` statement, the return value is undefined. **The behavior is undefined** if such value is used.

```
int do_something(int i) {
    printf("%d\n", i);
}
```

```
a.c: In function 'do_something':
a.c:5:1: warning: control reaches end of non-void function [-Wreturn-type]
  5 | }
```

35/49

[Best practice] Avoid unnecessary `if`

```
int abs_int(int x) {
    if (x < 0)
        return -x;
    else if (x == 0)
        return 0;
    else if (x > 0)
        return x;
}
```

```
a.c: In function abs_int:
a.c:8:1: warning: control reaches end of non-void function [-Wreturn-type]
  8 | }
```

36/49

[Best practice] Avoid unnecessary `if`

The last `if (x > 0)` is not needed:

```
int abs_int(int x) {
    if (x < 0)
        return -x;
    else if (x == 0)
        return 0;
    else // x > 0 must hold. No need to test it
        return x;
}
```

It can be simplified as

```
int abs_int(int x) {
    return x < 0 ? -x : x;
}
```

37/49

Function declaration and definition

A **definition** of a function contains the function body.

```
int sum(int a, int b) {
    return a + b;
}
```

A **declaration** of a function contains only its return type, its name and the types of its parameters, ended with `;`.

The following statements declare **the same function**:

```
int sum(int, int);
int sum(int x, int y);
int sum(int a, int);
```

Scopes and name lookup

[Best practice] Declare a variable right before the use of it. Declare it in a scope as small as possible.

[Best practice] Don't worry about the same names in different scopes.

```
// The three `i`'s are local to the three loops. They will not collide.
for (int i = 0; i < n; ++i)
    do_something(i);
for (int i = 0; i < n; ++i)
    do_another_thing(i);
if (condition()) {
    for (int i = 0; i < n; ++i)
        do_something_else(i);
}
```

Function declaration and definition

- A function should have only one definition, but can be declared many times.
- A definition is also a declaration, since it contains all the information that a declaration has.
- When a function is called, its declaration must be present.

```
int sum(int, int);    // declares the function
int main(void) {
    int x = sum(2, 3); // ok
}
int sum(int x, int y) { // gives its definition afterwards
    return x + y;
}
```

Summary

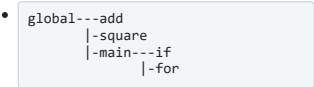
Operators

- Precedence, associativity
- Precedence and associativity do not determine the evaluation order of operands.
 - Things like `i = i++` is undefined behavior.
- Comparison operators
 - `a < b < c` is not `a < b && b < c`. Use `a < b && b < c` instead.
- Logical operators
 - `&&` and `||` are short-circuited.
- Conditional operator
- Assignment operator
 - Chained assignment is allowed: `a = b = c` is interpreted as `a = (b = c)`.

Scopes

```
int add(int x, int y) {
    return x + y;
}
int square(int x) {
    return x * x;
}
int main(void) {
    int x; scanf("%d", &x);
    printf("%d\n", square(x));
    if (x == 42) {
        int x = 35;
        printf("%d\n", square(square(x)));
    }
    for (int x = 1; x <= 10; ++x)
        printf("%d\n", square(x + 1));
    return 0;
}
```

- The scopes form a tree structure:



Summary

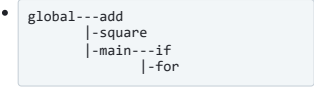
Control Flow

- `do - while` tests the condition after the body is executed.
- `switch - case`
 - Don't forget to write `break;`.

Name lookup

```
int add(int x, int y) {
    return x + y;
}
int square(int x) {
    return x * x;
}
int main(void) {
    int x; scanf("%d", &x);
    printf("%d\n", square(x));
    if (x == 42) {
        int x = 35;
        printf("%d\n", square(square(x)));
    }
    for (int x = 1; x <= 10; ++x)
        printf("%d\n", square(x + 1));
    return 0;
}
```

- The scopes form a tree structure:



- When a name `x` is referenced, the **name lookup** for `x` is performed:
 - Only the declarations before the current position can be seen.
 - Lookup is performed from the innermost scope to the outer scopes, until a declaration is found.

Summary

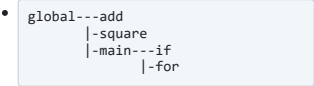
Functions

- call and return
- `()` is the function-call operator.
- Non- `void` function must return something.
- declaration vs definition
- scope and name lookup
 - Only the declarations before the current position can be seen.

Name lookup

```
int add(int x, int y) {
    return x + y;
}
int square(int x) {
    return x * x;
}
int main(void) {
    int x; scanf("%d", &x);
    printf("%d\n", square(x));
    if (x == 42) {
        int x = 35;
        printf("%d\n", square(square(x)));
    }
    for (int x = 1; x <= 10; ++x)
        printf("%d\n", square(x + 1));
    return 0;
}
```

- The scopes form a tree structure:



- A declaration in an inner scope may hide a declaration in an outer scope that introduces the same name.

Exercise

Write a function `yes_or_no` that asks if the user agrees (`Y / y`, `N / n`). Repeat until the input is valid. Returns `1` if the user agrees, `0` otherwise.

```
int main(void) {
    int response = yes_or_no();
    if (response)
        printf("Your response is yes.\n");
    else
        printf("Your response is no.\n");
    return 0;
}
```

Notes

¹ If the evaluation order of `A` and `B` is *unspecified*, there are two possibilities:

1. evaluations of `A` and `B` are **unsequenced**: they may be performed in any order and **may overlap**.
2. evaluations of `A` and `B` are **indeterminably-sequenced**: they may be performed in any order but **may not overlap**: either `A` will be complete before `B`, or `B` will be complete before `A`.

To constitute undefined behavior, the evaluations of `A` and `B` should be **unsequenced**. An example is `f() + g()` where both `f` and `g` modify a global variable: The C standard says that **two functions calls are indeterminably sequenced**, so `f() + g()` is not undefined behavior.

² Here "an object" is a scalar object. For example, it is okay if two unsequenced operations modify two different members of a `struct` object.

48/49

An "only-once" function

Suppose we want to design a function that can only be called once.

- When it is called for the second time, report an error.

Notes

³ The standard says that

Assignment is performed to copy the value of each argument to the corresponding function parameter, ignoring any type qualifiers on the parameter type and its possibly recursive elements of members, if any.

In short, argument passing is an **assignment** with type qualifiers on the parameter type ignored, instead of **initialization**. A post-C11 defect report [DR427](#) proposed change of such semantics from assignment to initialization, but was closed as not-a-defect.

In C++, this is initialization undoubtedly.

49/49

An "only-once" function

Suppose we want to design a function that should be called only once.

- When it is called for the second time, report an error.

```
bool start_game_called = false;

void start_game(Player *p1, Player *p2, int difficulty, GameWorld *world) {
    if (start_game_called)
        report_an_error("You cannot start the game twice!");
    start_game_called = true;
    // ...
}
```

An "only-once" function

What if there are many such functions?

```
bool start_game_called = false;
bool end_game_called = false;
bool magic_power_used = false;
bool special_skill_used = false;
// ...
```

The global name space is **polluted** with these variables.

- Is there a way to declare these variables inside the corresponding functions?

CS100 Lecture 5

Variables II, Pointers and Arrays I

Contents

- Variables
 - Local `static` variables
 - Initialization
 - `const` type qualifier
- Pointers
- Arrays

Variables

An "only-once" function

This does not work. Why?

```
void start_game(Player *p1, Player *p2, int difficulty, GameWorld *world) {
    bool called = false; // Note that we can use a shorter name,
                        // because it is local to this function.
    if (called)
        report_an_error("You cannot start the game twice!");
    called = true;
    // ...
}
```

An "only-once" function

This does not work. Why?

```
void start_game(Player *p1, Player *p2, int difficulty, GameWorld *world) {
    bool called = false; // Note that we can use a shorter name,
                        // because it is local to this function.
    if (called)
        report_an_error("You cannot start the game twice!");
    called = true;
    // ...
}
```

- Every time `start_game` is called, `called` is created and initialized to zero.
- Every time `start_game` returns, `called` is destroyed.
- We need a "local" variable whose lifetime is longer!

Local `static` variables

```
void start_game(Player *p1, Player *p2, int difficulty, GameWorld *world) {
    static bool called = false;
    if (called)
        report_an_error("You cannot start the game twice!");
    called = true;
    // ...
}
```

The lifetime of a local `static` variable is as long as that of a global variable. (They both have *static storage duration*.)

- A local `static` variable is initialized during program startup, and is destroyed on program termination.

It behaves just like a global variable, but its name is inside a function, which does not pollute the global name space.

9/51

Initialization

If we declare a variable without explicit initialization, what is the value of it?

Experiment:

```
#include <stdio.h>

int global;

int main(void) {
    int local;
    static int local_static;
    printf("%d, %d, %d\n", global, local,
        local_static);
    return 0;
}
```

- Compiled without `-O2` (a kind of optimization):

0, 22031, 0

- Compiled with `-O2`:

0, 0, 0

10/51

Implicit initialization [Very important]

If a variable is declared without explicit initialization:

- For local non-`static` variables, they are initialized to **indeterminate values**. In other words, they are **uninitialized**.
- For global or local `static` variables, they are **empty-initialized**¹:
 - Pointers are initialized to *null pointer values* of their types. (later in this lecture)
 - Objects of integral types are initialized to `0`.
 - Objects of floating types are initialized to positive zero (`0.0`).
 - Other cases will be discussed in later lectures.

* Intuitively, such variables are initialized to some kind of "zero"². This is called **zero-initialization** in C++.

11/51

Uninitialized garbage can be deadly!

The **behavior is undefined** if the value of an object is used while it is indeterminate.

Example: Schrödinger's `bool`

```
int main(void) {
    bool p;
    if (p)
        printf("p is true.\n");
    if (!p)
        printf("p is false.\n");
}
```

- Possible output (observed with an older version of gcc):

p is true.
p is false.

- `p` is in a "superposition state" of true and false?

12/51

Uninitialized garbage can be deadly!

Example: For a local variable declared with `int n;` (uninitialized), `n` may not compare equal to itself and it may appear to change its value on subsequent reads.

```
// in some function
int n; // uninitialized
// n == n is not necessarily true and involves undefined behavior.
int m = n;
// m == n is not necessarily true and involves undefined behavior.
```

Uninitialized variables may also have *trap representations*, which on some architectures can cause serious problems.

13/51

Uninitialized garbage can be deadly!

[Best practice] Always initialize the variable.

Except in certain cases, e.g.

```
// in some function
int n; // uninitialized
scanf("%d", &n); // A value is assigned to `n` immediately. This is OK.
// Now the value of `n` is not indeterminate. It can be used normally.
```

14/51

`const` type qualifier

Each type `T` (not `const`-qualified) has a `const`-qualified version of that type, written as `T const` or `const T`.

Any direct modification of variables with `const`-qualified types is not allowed:

```
const int n = 100; // Type of `n` is `const int`.
++n; // Error.
```

(Any indirect modification of `const` variables is undefined behavior; see in later lectures.)

15/51

`const` type qualifier

A `const` variable cannot be modified after initialization.

Therefore, an uninitialized `const` local non-`static` variable is almost a non-stop ticket to undefined behavior.

```
// in some function
const int n; // `n` has indeterminate values
n = 42; // Error: cannot modify a const variable.
scanf("%d", &n); // Error: cannot modify a const variable.
```

In C++, `const` variables of built-in types must be initialized.

16/51

Pointers

Pointers

A pointer *points to* a variable. The **value** of a pointer is the address of the variable that it points to.

```
int i = 42;
int* pi = &i;
printf("%d\n", *pi);
```



- `int* pi;` declares a pointer named `pi`.
 - The type of `pi` is `int*`.
 - The type of the variable that `pi` points to ("pointee") is `int`.
- `&` is the **address-of operator**, used for taking the address of a variable.
- `*` in the expression `*pi` is the **indirection (dereference) operator**, used for obtaining the variable that a pointer points to.

18/51

Pointers

A pointer *points to* a variable.

We can access and modify a variable through its address (or a pointer pointing to it).

```
int num = 3;
int* ptr = &num;
printf("%d\n", *ptr); // 3
*ptr = 10;
printf("%d\n", num); // 10
++num;
printf("%d\n", *ptr); // 11
```

19/51

Declare a pointer

To declare a pointer: `PointeeType* ptr;`

- The type of `ptr` is `PointeeType*`.
 - Pointer types with different pointee types are **different types**: `int*` and `double*` are different.
- The asterisk `*` can be placed near either `PointeeType` or `ptr`:
 - `PointeeType* ptr;` and `PointeeType *ptr;` are the same declaration.
 - `PointeeType * ptr;`, `PointeeType * ptr;` and `PointeeType*ptr;` are also correct.

20/51

Declare a pointer

The asterisk `*` can be placed near either `PointeeType` or `ptr`:

- `PointeeType* ptr;` may be more intuitive?

However, when declaring more than one pointers in one declaration statement, an asterisk is needed for **every identifier**:

```
int* p1, p2, p3; // 'p1' is of type 'int*', but 'p2' and 'p3' are ints.
int *q1, *q2, *q3; // 'q1', 'q2' and 'q3' all have the type 'int*'.
int* r1, r2, *r3; // 'r1' and 'r3' are of the type 'int*',
// while 'r2' is an int.
```

[Best practice] Either `PointeeType *ptr` or `PointeeType* ptr` is ok. Choose one style and stick to it. But if you choose the second one, never declare more than one pointers in one declaration statement.

21/51

& and *

`&var` returns the address of the variable `var`.

- The result type is `Type *`, where `Type` is the type of `var`.
- `var` must be an object that has an identity (an *lvalue*)³: `&42` or `&(a + b)` are not allowed.

`*expr` returns **the variable** whose address is the value of `expr`.

- `expr` must have a pointer type `PointeeType *`. The result type is `PointeeType`.
- The **variable** is returned, not only its value. This means that we can modify the returned variable: `++*ptr` is allowed.

22/51

*

In a **declaration** `PointeeType *ptr`, `*` is a part of the pointer type `PointeeType *`.

In an **expression** like `*ptr`, `*` is the **indirection (dereference) operator** used to obtain the variable whose address is the value of `ptr`.

Do not mix them up!

23/51

The null pointer

The **null pointer value** is the "zero" value for pointer types⁴.

- It can be obtained from the macro `NULL`, which is available from many standard library header files (e.g. `<stddef.h>`, `<stdio.h>`, `<stdlib.h>`):

```
int *ptr = NULL; // 'ptr' is a null pointer.
```

- It can also be obtained from the integer literal `0`.

```
double *ptr = 0; // same as 'double *ptr = NULL;'
```

- Conversion from a null pointer to an integer type results in `0`.

Note: Better null pointer values (`nullptr`) are available in [C23](#) and in [C++11](#).

24/51

The null pointer

The **null pointer value** is the "zero" value for pointer types⁴.

A null pointer compares unequal to any pointer pointing to an object.

It is used for representing a pointer that "points nowhere".

Dereferencing a null pointer is undefined behavior, and often causes severe runtime errors!

- Because it is not pointing to an object.

```
int *ptr = NULL;
printf("%d\n", *ptr); // undefined behavior
*ptr = 42; // undefined behavior
```

25/51

Implicit initialization of pointers

If a pointer is not explicitly initialized:

- Global or local `static`: Initialized to the null pointer value.
- Local non-`static`: Initialized to indeterminate values, or in other words, **uninitialized**.
 - Uninitialized pointers are often called **wild pointers**.

A wild pointer do not point to a specific object, and is not a null pointer either.

Dereferencing a wild pointer is undefined behavior, and often causes severe runtime errors.

[Best practice] Avoid wild pointers.

26/51

Pointers that are not dereferenceable

A pointer `ptr` is dereferenceable. $\Leftrightarrow$ `*ptr` has no undefined behavior. $\Leftrightarrow$ `ptr` points to an existing object.

A pointer that does not point to an existing object may be

- uninitialized (wild), or
- a null pointer, or
- dangling (We will discuss this in later lectures), or
- holding some other meaningless address: `int *p = 123;`

27/51

Pointers that are not dereferenceable

Dereferencing such a pointer is undefined behavior, and usually causes severe runtime errors.

- Recall the "short-circuit" evaluation of binary logical operators:

```
if (ptr != NULL && *ptr == 42) { /* ... */ }
```

When `ptr` is a null pointer, the right-hand side operand `*ptr == 42` won't be evaluated, so `ptr` is not dereferenced.

28/51

Argument passing

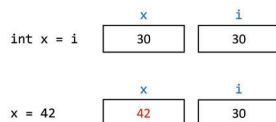
What is the output? Is the value of `i` changed to `42`?

```
void fun(int x) {
    x = 42;
}
int main(void) {
    int i = 30;
    fun(i);
    printf("%d\n", i);
}
```

29/51

Argument passing

```
void fun(int x) {
    x = 42;
}
int main(void) {
    int i = 30;
    fun(i);
    printf("%d\n", i); // 30
}
```



The output is still `30`. `i` is not changed.

- The parameter `x` is initialized as if `int x = i`; , thus obtaining the value of `i`.
 - `x` and `i` are two independent variables.
- Modification on `x` does not influence `i`.

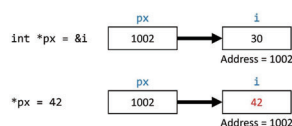
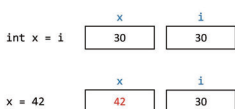
30/51

Argument passing

To make the modification happen on a variable outside, pass its address into the function.

```
void fun(int x) {
    x = 42;
}
int main(void) {
    int i = 30;
    fun(i); // int x = i;
    printf("%d\n", i); // 30
}
```

```
void fun(int *px) {
    *px = 42;
}
int main(void) {
    int i = 30;
    fun(&i); // int *px = &i
    printf("%d\n", i); // 42
}
```



31/51

Exercise: swap

Write a `swap` function to exchange the values of two integers. How do you design the parameters and the return type?

This does not work:

```
void swap(int a, int b) {
    int tmp = a;
    a = b;
    b = tmp;
}
```

32/51

Exercise: swap

Write a `swap` function to exchange the values of two integers. How do you design the parameters and the return type?

```
void swap(int *pa, int *pb) {
    int tmp = *pa;
    *pa = *pb;
    *pb = tmp;
}
int main(void) {
    int i = 42, j = 35;
    swap(&i, &j);
    // Now i = 35, j = 42.
}
```

* Why do we need to write `&` when calling `scanf`, but not in `printf`?

33/51

Exercise: swap

Write a `swap` function to exchange the values of two integers. How do you design the parameters and the return type?

```
void swap(int *pa, int *pb) {
    int tmp = *pa;
    *pa = *pb;
    *pb = tmp;
}
int main(void) {
    int i = 42, j = 35;
    swap(&i, &j);
    // Now i = 35, j = 42.
}
```

* Why do we need to write `&` when calling `scanf`, but not in `printf`?

- `scanf` needs to modify the variable, but `printf` doesn't.

34/51

Arrays

Arrays

An array is a sequence of `N` objects of an *element type* `ElemType` stored **contiguously** in memory, where $N \in \mathbb{Z}_+$ is the *length* of it.

```
ElemType arr[N];
```

`N` must be a **constant expression** whose value is known at compile-time.

```
int a1[10]; // OK. A literal is a constant expression.
#define MAXN 10
int a2[MAXN]; // OK. `MAXN` is replaced with `10` by the preprocessor.
int n; scanf("%d", &n);
int a[n]; // A C99 VLA (Variable-Length Array), whose length is
// determined at runtime.
```

For now, we do not recommend the use of VLAs. We will talk more about it in recitations.

36/51

Array type

An array is a sequence of `N` objects of an *element type* `ElemType` stored **contiguously** in memory, where $N \in \mathbb{Z}_+$ is the *length* of it.

```
ElemType arr[N]; // The type of `arr` is `ElemType [N]`.
```

The type of an array consists of two parts:

- the element type `ElemType`, and
- the length of the array `[N]`.

37/51

Array subscript

```
ElemType arr[N];
```

Use `arr[i]` to obtain the `i`-th element of `arr`, where $i \in [0, N)$.

```
int a[10];
bool find(int value) {
    for (int i = 0; i < 10; ++i)
        if (a[i] == value)
            return true;
    return false;
}
int main(void) {
    int n; scanf("%d", &n);
    for (int i = 0; i < n; ++i)
        scanf("%d", &a[i]);
    for (int i = 0; i < n; ++i)
        a[i] *= 2;
    // ...
}
```

The subscript `i` is an integer within the range $[0, N)$. **Array subscript out of range is undefined behavior, and usually causes severe runtime errors.**

38/51

Array subscript

The subscript `i` is an integer within the range $[0, N)$. **Array subscript out of range is undefined behavior, and usually causes severe runtime errors.**

The compiler may assume that the program is free of undefined behaviors:

```
int a[4];

bool find(int value) {
    for (int i = 0; i <= 4; ++i)
        if (a[i] == value)
            return true;
    return false;
}
```

- This code may be compiled as if it is

```
bool find(int value) {
    return true;
}
```

demo

39/51

Initialization of nested arrays

Things have been stated quite clearly [on cppreference](#).

```
int a[4][3] = { // array of 4 arrays of 3 ints each (4x3 matrix)
    { 1 },      // row 0 initialized to {1, 0, 0}
    { 0, 1 },   // row 1 initialized to {0, 1, 0}
    { [2]=1 },  // row 2 initialized to {0, 0, 1}
    // row 3 initialized to {0, 0, 0}
};
int b[4][3] = { // array of 4 arrays of 3 ints each (4x3 matrix)
    1, 3, 5, 2, 4, 6, 3, 5, 7 // row 0 initialized to {1, 3, 5}
};
// row 1 initialized to {2, 4, 6}
// row 2 initialized to {3, 5, 7}
// row 3 initialized to {0, 0, 0}
int y[4][3] = {[0][0]=1, [1][1]=1, [2][0]=1}; // row 0 initialized to {1, 0, 0}
// row 1 initialized to {0, 1, 0}
// row 2 initialized to {1, 0, 0}
// row 3 initialized to {0, 0, 0}
```

44/51

Array initialization

If an array is declared without explicit initialization:

- Global or local `static`: Empty-initialization $\Rightarrow$ Every element is empty-initialized.
- Local non-`static`: Every element is initialized to indeterminate values (uninitialized).

40/51

Summary

Variables

- Local `static` variables: A variable whose name is local to a function but whose lifetime is as long as that of a global variable.
- Implicit initialization:
 - Global or local `static`: empty-initialization (some kind of "zero" value)
 - Local non-`static`: initialized to indeterminate values (uninitialized)
 - Use of indeterminate values is undefined behavior.
- `const`: not modifiable.

45/51

Array initialization

Arrays can be initialized from [brace-enclosed lists](#):

- Initialize the beginning few elements:

```
int a[10] = {2, 3, 5, 7}; // Correct: Initializes a[0], a[1], a[2], a[3]
int b[2] = {2, 3, 5};    // Error: Too many initializers
int c[] = {2, 3, 5};     // Correct: 'c' has type int[3].
int d[100] = {};         // Correct in C++ and since C23.
```

- Initialize designated elements (since C99):

```
int e[10] = {[0] = 2, 3, 5, [7] = 7, 11, [4] = 13};
```

41/51

Summary

Pointers

- Type of a pointer: `PointeeType *`
- `&var` returns the address of `var`.
- `*expr` returns the variable whose address is the value of `expr`.
- Null pointer value: The "zero" value for pointer types.
 - Obtained from `NULL` or the integer literal `0`.
 - Representing a pointer that "points nowhere".

46/51

Array initialization

If an array is explicitly initialized, all the elements that are not explicitly initialized are **empty-initialized**.

```
int main(void) {
    int a[10] = {1, 2, 3}; // a[3], a[4], ... are all initialized to zero.
    int b[100] = {0};      // All elements of b are initialized to zero.
    int c[100] = {1};      // c[0] is initialized to 1,
                          // and the rest are initialized to zero.
}
```

`= {x}` is not initializing all elements to `x` !

42/51

Summary

Pointers

- Implicit initialization:
 - Global or local `static`: null pointer
 - Local non-`static`: "wild pointer"
 - Dereferencing such pointers is undefined behavior.
- Used for argument passing
 - To modify a variable outside the function, pass its address into the function.

47/51

Nested arrays

The C answer to "multidimensional arrays" is **nested arrays**, which is in fact **arrays of arrays**:

Type `a[4][3]`

a[0] a[1] a[2] a[3]

```
int a[10][20];

bool find(int value) {
    for (int i = 0; i < 10; ++i)
        for (int j = 0; j < 20; ++j)
            if (a[i][j] == value)
                return true;
    return false;
}
```

43/51

Summary

Arrays

- Type of an array: `ElemType [N]`
- `a[i]` returns the `i`-th element in the array `a`.
 - $i \in [0, N)$. Out-of-range access is undefined behavior.
- Implicit initialization: performed recursively on each element.
- Initialization from brace-enclosed lists: `int a[10] = {2, 3, 5};`
 - The rest elements are initialized to zero.
- Nested arrays: Arrays of arrays

48/51

Exercises

1. The `swap` function on page 33 swaps the values of two integers. What if we want to swap the values of two pointers?

```
void swap(/* ??? */, /* ??? */) {
    // ???
}
int main(void) {
    int i = 42, j = 50;
    int *p1 = &i, *p2 = &j;
    swap(/* ??? */);
    // Now 'p1' should point to 'j', and 'p2' should point to 'i'.
}
```

Pointers and Arrays

Exercises

2. Write a program that reads a matrix $M \in \mathbb{Z}^{n \times m}$ and prints its transpose M^T . You may assume $n, m \leq 100$ and the entries are representable by `int`.

Sample input:

```
3 5
1 2 3 4 5
6 7 8 9 10
11 12 13 14 15
```

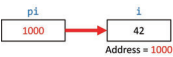
Sample output:

```
1 6 11
2 7 12
3 8 13
4 9 14
5 10 15
```

Recap: Pointers

A pointer *points to* an object. The value of a pointer is the address of the object that it points to.

```
int i = 42;
int *pi = &i;
```



To declare a pointer: `PointeeType *ptr;`

- What is the type of `ptr`?

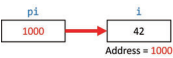
Notes

- ¹ The [c++reference page for empty-initialization](#) does not mark the term "empty-initialization" as C23, but in fact it does not appear in any standards prior to C23. Looking through the edit history of this page, we can see that this term was not introduced until the C23 things were added. For convenience, we use this term to represent this kind of initialization.
- ² The "zero" value is not necessarily all-bits zero. For example, the floating-point positive zero `+0.0` may not be all-bits zero in [a special representation used by the CCS C compiler](#). The null pointer value is not all-bits zero on some platforms either, details of which are available on [comp.lang.c FAQ](#).
- ³ We will not talk about the concepts related to value categories until we meet them in C++, although [C also has value categories](#).
- ⁴ See [2](#).

Recap: Pointers

A pointer *points to* an object. The value of a pointer is the address of the object that it points to.

```
int i = 42;
int *pi = &i;
```



To declare a pointer: `PointeeType *ptr;`

- What is the type of `ptr`? `PointeeType *`

How can we make `ptr` point to `var`?

CS100 Lecture 6

Pointers and Arrays II

Contents

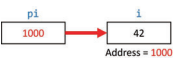
Pointers and Arrays

- Pointer arithmetic
- Array-to-pointer conversion
- Pass an array to a function
- Pass a nested array to a function
- Do we need an array?

Recap: Pointers

A pointer *points to* an object. The value of a pointer is the address of the object that it points to.

```
int i = 42;
int *pi = &i;
```



To declare a pointer: `PointeeType *ptr;`

- What is the type of `ptr`? `PointeeType *`

How can we make `ptr` point to `var`?

- First, `var` should be of type `PointeeType`.
- `PointeeType *ptr = &var;` (initialize)
- `ptr = &var;` (assign)

Recap: Arrays

An array is a sequence of `N` objects of type `ElemType` stored contiguously.

To declare an array: `ElemType arr[N];`

- What is the type of `arr`?

Recap: Arrays

An array is a sequence of N objects of type `ElemType` stored contiguously.

To declare an array: `ElemType arr[N];`

- What is the type of `arr`? `ElemType [N]`

To access the i -th element: `arr[i]`

- What is the valid range of i ?

8/44

Recap: Arrays

An array is a sequence of N objects of type `ElemType` stored contiguously.

To declare an array: `ElemType arr[N];`

- What is the type of `arr`? `ElemType [N]`

To access the i -th element: `arr[i]`

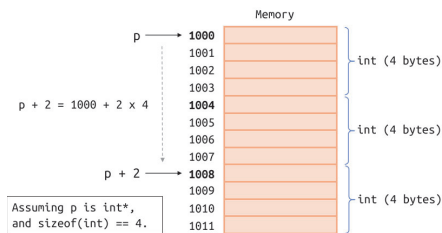
- What is the valid range of i ? $[0, N)$

9/44

Pointer arithmetic

Let p be a pointer of type `T*` and let i be an integer.

- $p + i$ returns the address equal to the value of $(\text{char} *)p + i * \text{sizeof}(T)$. In other words, pointer arithmetic uses the unit of the pointed-to type.

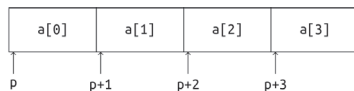


10/44

Pointer arithmetic

Let p be a pointer of type `T*` and let i be an integer.

- $p + i$ returns the address equal to the value of $(\text{char} *)p + i * \text{sizeof}(T)$. In other words, pointer arithmetic uses the unit of the pointed-to type.
- If we let $p = \&a[0]$ (where a is an array of type `T[N]`), then
 - $p + i$ is equivalent to $\&a[i]$, and
 - $*(p + i)$ is equivalent to $a[i]$.



- Arithmetic operations $i + p$, $p + i$, $p - i$, $p - i$, $++p$, $p++$, $--p$, $p--$ are defined in the same way.

11/44

Array-to-pointer conversion

If we let $p = \&a[0]$ (where a is an array of type `T[N]`), then

- $p + i$ is equivalent to $\&a[i]$, and
- $*(p + i)$ is equivalent to $a[i]$.

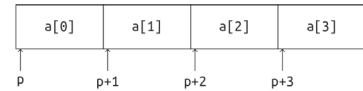


Considering the close relationship between arrays and pointers, an array can be **implicitly converted** to a pointer to the first element: $a \rightarrow \&a[0]$, $T[N] \rightarrow T*$.

- $p = \&a[0]$ can be written as $p = a$ directly.
- $*a$ is equivalent to $a[0]$.

12/44

Array-to-pointer conversion



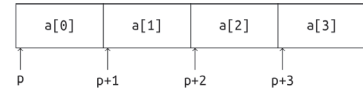
We can use pointers to traverse an array:

```
int a[10];

bool find(int value) {
    for (int *p = a; p < a + 10; ++p)
        if (*p == value)
            return true;
    return false;
}
```

13/44

Subtraction of pointers



Let a be an array of length N . If $p1 = a + i$ and $p2 = a + j$ (where i and j are nonnegative integers), the expression $p1 - p2$

- has the value equal to $i - j$, and
- has the type `ptrdiff_t`, which is a **signed** integer type declared in `<stddef.h>`.
 - The size of `ptrdiff_t` is implementation-defined. For example, it might be 64-bit on a 64-bit machine, and 32-bit on a 32-bit machine.
- Here $i, j \in [0, N]$ (closed interval), i.e. $p1$ or $p2$ may point to the "past-the-end" position of a .

14/44

Pointer arithmetic

Pointer arithmetic can only happen within the range of an array and its "past-the-end" position (indexed $[0, N]$). For other cases, the **behavior is undefined**.

Examples of undefined behaviors:

- $p1 - p2$, where $p1$ and $p2$ point to the positions of two different arrays.
- $p + 2 * N$, where p points to some element in an array of length N .
- $p - 1$, where p points to the first element $a[0]$ of some array a .

Note that the evaluation of the innocent-looking expression $p - 1$, without dereferencing it, is still undefined behavior and may fail on some platforms.

15/44

Pass an array to a function

The only way¹ of passing an array to a function is to **pass the address of its first element**.

The following declarations are equivalent:

```
void fun(int *a);
void fun(int a[]);
void fun(int a[10]);
void fun(int a[2]);
```

In all these declarations, the type of the parameter a is `int *`.

- How do you verify that?

16/44

Pass an array to a function

```
void fun(int a[100]);
```

The type of the parameter a is `int *`. How do you verify that?

```
void fun(int a[100]) {
    printf("%d\n", (int)sizeof(a));
}
```

Output: (On 64-bit Ubuntu 22.04, GCC 13)

```
8
```

- If the type of a is `int[100]` as declared, the output should be `400` (assuming `int` is 32-bit).

17/44

Pass an array to a function

Even if you declare the parameter as an array (either `T a[N]` or `T a[]`), its type is still a pointer `T*`: You are allowed to pass anything of type `T*` to it.

- Array of element type `T` with any length is allowed to be passed to it.

```
void print(int a[10]) {
    for (int i = 0; i < 10; ++i)
        printf("%d\n", *(a + i));
}
int main(void) {
    int x[20] = {0}, y[10] = {0}, z[5] = {0}, w = 42;
    print(x); // OK
    print(y); // OK
    print(z); // Allowed by the compiler, but undefined behavior!
    print(&w); // Still allowed by the compiler, also undefined behavior!
}
```

18/44

Pass an array to a function

Even if you declare the parameter as an array (either `T a[N]` or `T a[]`), its type is still a pointer `T*`: You are allowed to pass anything of type `T*` to it.

- Array of element type `T` with any length is allowed to be passed to it.

The length `n` of the array is often passed explicitly as another argument, so that the function can know how long the array is.

```
void print(int *a, int n) {
    for (int i = 0; i < n; ++i)
        printf("%d\n", *(a + i));
}
```

19/44

Subscript on pointers

```
void print(int *a, int n) {
    for (int i = 0; i < n; ++i)
        printf("%d\n", a[i]); // Look at this!
}
```

Subscript on pointers is also allowed! `a[i]` is equivalent to `*(a + i)`.²

20/44

Return an array?

There is no way of returning an array from the function.

Returning the address of its first element is ok, but be careful:

This is OK:

```
int a[10];

int *foo(void) {
    return a;
}
```

This returns an invalid address! (Why?)

```
int *foo(void) {
    int a[10] = {0};
    return &x;
}
```

21/44

Return an array?

These two functions have made the same mistake: returning the address of a local variable.

```
int *foo(void) {
    int a[10] = {0};
    return a;
}
int main(void) {
    int *a = foo();
    a[0] = 42; // undefined behavior
}

int *fun(void) {
    int x = 42;
    return &x;
}
int main(void) {
    // undefined behavior
    printf("%d\n", *fun());
}
```

- When the function returns, all the parameters and local objects are destroyed.
 - `a` and `x` no longer exist.
- The objects on the returned addresses are "dead" when the function returns!

22/44

Exercise

Write a function that accepts an array of `int`s, and copies the odd numbers to another array in reverse order.

```
int main(void) {
    int a[5] = {1, 2, 3, 5, 6}, b[5];
    copy_odd_reversed(a, b); // your function
    // Now `a` is unchanged, and the values in `b` are {5, 3, 1}.
}
```

Design the usage of your function (parameters and return values) on your own.

23/44

Exercise

Write a function that accepts an array of `int`s, and copies the odd numbers to another array in reverse order.

```
int copy_odd_reversed(int *from, int n, int *to) {
    int cnt = 0;
    for (int i = n - 1; i >= 0; --i)
        if (from[i] % 2 == 1)
            to[cnt++] = from[i]; // What does this mean?
    return cnt;
}
int main(void) {
    int a[5] = {1, 2, 3, 5, 6}, b[5];
    int b_length = copy_odd_reversed(a, 5, b); // b_length == 3.
    // Now `a` is unchanged, and the values in `b` are {5, 3, 1}.
}
```

Since arrays cannot be returned, we often create the result array on our own, and pass it to the function.

24/44

Pointer type (revisited)

The type of a pointer is `PointeeType *`.

For two different types `T1` and `T2`, the pointer types `T1 *` and `T2 *` are different types, although they may point to the same location.

```
int i = 42;
float *fp = &i;
++*fp; // Undefined behavior. It is not ++i.
```

In C, pointers of different types can be implicitly converted to each other (with possibly a warning). This is **extremely unsafe** and an error in C++.

Dereferencing a pointer of type `T1 *` when it is actually pointing to a `T2` is *almost* always undefined behavior.

- We will see one exception in the next lecture.³

25/44

Pass a nested array to a function

When passing an array to a function, we make use of the **array-to-pointer conversion**:

- `Type [N]` will be implicitly converted to `Type *`.

What about nested arrays?

26/44

Pass a nested array to a function

When passing an array to a function, we make use of the **array-to-pointer conversion**:

- `Type [N]` will be implicitly converted to `Type *`.

A "2d-array" is an "array of array":

- `Type [N][M]` is an array of `N` elements, where each element is of type `Type [M]`.
- What is the conversion result of `Type [N][M]`?

27/44

Pass a nested array to a function

When passing an array to a function, we make use of the **array-to-pointer conversion**:

- `Type [N]` will be implicitly converted to `Type *`.

A "2d-array" is an "array of array":

- `Type [N][M]` is an array of `N` elements, where each element is of type `Type [M]`.
- `Type [N][M]` should be implicitly converted to a "pointer to `Type[M]`".

What is a "pointer to `Type[M]`"?

28/44

Pass a nested array to a function

In each of the following declarations, what is the type of `a`? Does it accept an argument of type `int[N][M]`?

1. `void fun(int a[N][M])`
2. `void fun(int (*a)[M])`
3. `void fun(int (*a)[N])`
4. `void fun(int **a)`
5. `void fun(int *a[])`
6. `void fun(int *a[N])`
7. `void fun(int a[100][M])`
8. `void fun(int a[N][100])`

33/44

Pointer to array

A pointer to an array of `N` `int`'s:

An array of `N` pointers (pointing to `int`):

```
int (*parr)[N];
```

```
int *arrp[N];
```

Too confusing! How can I remember them?

29/44

Pass a nested array to a function

In each of the following declarations, what is the type of `a`? Does it accept an argument of type `int[N][M]`?

1. `void fun(int a[N][M])`: A pointer to `int[M]`. Yes.
2. `void fun(int (*a)[M])`: Same as 1.
3. `void fun(int (*a)[N])`: A pointer to `int[N]`. Yes iff `N == M`.
4. `void fun(int **a)`: A pointer to `int *`. No.
5. `void fun(int *a[])`: Same as 4.
6. `void fun(int *a[N])`: Same as 4.
7. `void fun(int a[100][M])`: Same as 1.
8. `void fun(int a[N][100])`: A pointer to `int[100]`. Yes iff `M == 100`.

34/44

Pointer to array

A pointer to an array of `N` `int`'s:

An array of `N` pointers (pointing to `int`):

```
int (*parr)[N];
```

```
int *arrp[N];
```

Too confusing! How can I remember them?

- `int (*parr)[N]` has a pair of parentheses around `*` and `parr`, so
 - `parr` is a pointer (`*`), and
 - points to something of type `int[N]`.
- Then the other one is different:
 - `arrp` is an array, and
 - stores `N` pointers, with pointee type `int`.

30/44

Exercise

We wrote a "transpose" program in last lecture's exercise, which accepts a matrix from input and prints its transpose.

Rewrite this functionality as a function. Suppose the size of the given matrix is 3×4 .

```
int main(void) {
    int a[3][4];
    for (int i = 0; i < 3; ++i)
        for (int j = 0; j < 4; ++j)
            scanf("%d", &a[i][j]);
    int b[4][3];
    transpose(a, b); // Your function
    for (int i = 0; i < 4; ++i) {
        for (int j = 0; j < 3; ++j)
            printf("%d ", b[i][j]);
        printf("\n");
    }
}
```

35/44

Pass a nested array to a function

The following declarations are equivalent: The parameter is of type `int (*)[N]`, which is a pointer to `int[N]`.

```
void fun(int (*a)[N]);
void fun(int a[][N]);
void fun(int a[2][N]);
void fun(int a[10][N]);
```

We can pass an array of type `int[K][N]` to `fun`, where `K` is arbitrary.

- The size for the second dimension must be `N`.
 - `T[10]` and `T[20]` are different types, so the pointer types `T(*)[10]` and `T(*)[20]` are not compatible.

31/44

Do we need an array?

Write a program that reads an integer n and prints the n -th Fibonacci number. Assume that the numbers are representable by `int`.

$$F_n = \begin{cases} 0, & n = 0, \\ 1, & n = 1, \\ F_{n-1} + F_{n-2}, & n > 1. \end{cases}$$

36/44

Pass a nested array to a function

```
void print(int (*a)[5], int n) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < 5; ++j)
            printf("%d ", a[i][j]);
        printf("\n");
    }
}
int main(void) {
    int a[2][5] = {{1, 2, 3, 4, 5}, {6, 7, 8, 9, 10}};
    int b[3][5] = {0};
    print(a, 2); // OK
    print(b, 3); // OK
}
```

32/44

Do we need an array?

Write a program that reads an integer n and prints the n -th Fibonacci number. Assume that the numbers are representable by `int`.

$$F_n = \begin{cases} 0, & n = 0, \\ 1, & n = 1, \\ F_{n-1} + F_{n-2}, & n > 1. \end{cases}$$

```
int calc_fib(int n) {
    int fib[100] = {0, 1};
    for (int i = 2; i <= n; ++i)
        fib[i] = fib[i - 1] + fib[i - 2];
    return fib[n];
}
```

Is this array necessary?

37/44

Do we need an array?

Is this array necessary?

```
int calc_fib(int n) {
    int fib[100] = {0, 1};
    for (int i = 2; i <= n; ++i)
        fib[i] = fib[i - 1] + fib[i - 2];
    return fib[n];
}
```

We only need two (three) variables!

```
int calc_fib(int n) {
    int a = 0, b = 1;
    for (int i = 2; i <= n; ++i) {
        int new = a + b;
        a = b;
        b = new;
    }
    return b;
}
```

38/44

Do we need an array?

In the following cases, do we need an array?

- Read n integers, then print the sum of them.
- Read n integers, then print them in reverse order.
- Read n integers, then print the maximum of them.
- Read n integers, then print the second maximum of them.

39/44

Do we need an array?

In the following cases, do we need an array?

- Read n integers, then print the sum of them. **No**
- Read n integers, then print them in reverse order. **Yes**
- Read n integers, then print the maximum of them. **No**
- Read n integers, then print the second maximum of them. **No**

40/44

Summary

Pointer arithmetic

- can only happen within the range of an array and its "past-the-end" position. Other cases are undefined behaviors.
- $p + i$ returns the address $(\text{char} *)p + i * \text{sizeof}(T)$, i.e. $i * \text{sizeof}(T)$ bytes away from p .
- $p1 - p2$ is equal to $i - j$.

Array-to-pointer conversion

- An array a can be implicitly converted to $\&a[0]$.
- $T[N] \rightarrow T^*$.

41/44

Summary

Pass an array to a function

- You cannot declare an array parameter. It is always a pointer.
- We often use another parameter to indicate the length of the array.
- Instead of returning an array, we create the result array and pass it to the function.
- $T[N][M] \rightarrow T^*[M]$

Avoid unnecessary arrays.

42/44

Notes

¹ In fact, you can pass the address of an array:

```
void print_array_10(int (*pa)[10]) {
    for (int i = 0; i < 10; ++i)
        printf("%d\n", (*pa)[i]);
}

int main(void) {
    int a[10], b[100], c = 42;
    print_array_10(&a); // OK
    print_array_10(&b); // Error
    print_array_10(&c); // Error
}
```

In the function `print_array_10` above, the parameter type is `int (*)[10]`, a pointer to an array of 10 `int`s. The pointee type must be `int[10]`. Passing the address of anything else to it would not work.

43/44

Notes

² In fact, [the subscript operator](#) is defined by the standard for pointers. In expressions like `a[i]` where `a` is an array, `a` undergoes the implicit array-to-pointer conversion. [Such implicit conversion is so common](#) that some people even treat arrays and pointers as the same thing, which is a common misunderstanding.

³ See [strict aliasing](#) for detailed rules.

44/44

CS100 Lecture 7

Pointers and Arrays III, Dynamic memory, Strings

1/51

Contents

- Pointers and Arrays
 - Pointers and `const`
 - The `void *` type
- Dynamic memory
- Strings

2/51

Pointers and Arrays

3/51

Recap: `const` variables

A `const` variable cannot be modified after initialization.

- It has type `const T` (or equivalently `T const`), where `T` is its original (unqualified) type.
- Any direct modification to a `const` variable is not allowed.

The type of the expression `&x`, where `x` is of type `const T`, is `const T *`.

4/51

Pointer to `const`

A pointer to `const` is a pointer whose pointee type is `const`-qualified:

```
const int x = 42;
int *pi = &x; // Dangerous: It discards the const qualifier.
const int *cpi = &x; // Better.
```

The type of `cpi` is `const int *` (or equivalently, `int const *`), which is a pointer to `const int`.

5/51

`const` can be helpful

It tells the compiler "this variable should not be modified!".

```
int count(const int *a, int n, int value) {
    int cnt = 0;
    for (int i = 0; i < n; ++i)
        if (a[i] == value) // Error: cannot modify a[i]
            ++cnt;
    return cnt;
}
```

[Best practice] Use `const` whenever possible.

We will see more usage of `const` in C++.

9/51

Top-level `const` ness

A pointer itself can also be `const`. The type of such pointer is `PointeeType *const`.

- Such `const` ness is often called "top-level `const` ness".

```
int x = 42;
int *const pc = &x;
++*pc; // OK.
int y = 30;
pc = &y; // Error.
```

A `const` pointer cannot switch to point to other variables after initialization.

A pointer can have both low-level and top-level `const` ness:

```
const int *const cipc = &x;
```

10/51

`void *`

A special pointer type:

- Any pointer can be implicitly converted to that type.
- A pointer of type `void *` can be implicitly converted to any pointer type.
 - This must happen explicitly in C++.
- Use `printf("%p", ptr);` to print the value of a pointer `ptr` of type `void *`.
 - If `ptr` is a pointer of some other type, a conversion is needed:

```
printf("%p", (void *)ptr);
```

The C type system is weak. Many kinds of implicit conversions are very dangerous, although allowed by C.

11/51

`void *`

C does not have a static type system as powerful as C++'s. `void *` is often used to represent "pointer to anything", "location of some memory", or even "any object".

- Typically, the memory allocation function `malloc` (see below) returns `void *`, the address of the block of memory allocated.
 - Memory does not have types. - We say "a disk of 1TB" instead of "a disk that can hold 2^{38} `int`'s".

12/51

Dynamic memory

`const` is a "lock"

`const` is like a lock, guarding against modifications to the variable.

It is very dangerous to let a pointer to non-`const` point to a `const` variable: It is an attempt to remove the lock!

- Warning in C, error in C++.

```
const int x = 42;
int *pi = &x; // Dangerous: It discards the const qualifier.
const int *cpi = &x; // Better.
++*pi; // No error is generated by the compiler, but actually undefined behavior.
```

Any indirect modification to a `const` variable is **undefined behavior**.

6/51

`const` is a "lock"

`const` is like a lock, guarding against modifications to the variable.

A pointer to `const` can point to a non-`const` variable: This is adding a lock.

```
int x = 42;
int *pi = &x;
const int *cpi = &x; // OK.
++*pi; // Correct, same as ++x.
++*cpi; // Error!
```

7/51

`const` is a "lock"

A pointer to `const` can also point to a non-`const` variable: This is adding a lock.

- A pointer to `const` *thinks* that it is pointing to a `const` variable. Therefore, it does not allow you to modify the variable through it.

```
int x = 42;
int *pi = &x;
const int *cpi = &x; // OK.
++*pi; // Correct, same as ++x.
++*cpi; // Error!
```

Such `const` ness on the **pointee type** is often called "low-level `const` ness".

8/51

13/51

A "dynamic array"

Create an "array" whose size is determined at runtime?

14/51

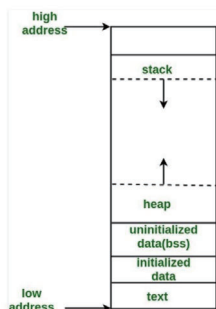
A "dynamic array"

Create an "array" whose size is determined at runtime?

- We need a block of memory, the size of which can be determined at runtime.
- If we run out of memory, we need to know.
- We may require a pretty large chunk of memory.

15/51

Stack memory vs heap (dynamic) memory



- Stack memory is generally smaller than heap memory.
- Stack memory is often used for storing local and temporary objects.
- Heap memory is often used for storing large objects, and objects with long lifetime.
- Operations on stack memory is faster than on heap memory.
- Stack memory is allocated and deallocated automatically, while heap memory needs manual management.

16/51

Use malloc and free

Declared in `<stdlib.h>`.

```
void *malloc(size_t size);
```

Allocates `size` bytes of uninitialized storage on heap.

If allocation succeeds, returns the starting address of the allocated memory block.

If allocation fails, a null pointer is returned.

- `size_t`: A type that can hold the size (number of bytes) of any object. It is
 - declared in `<stddef.h>`, and
 - is an **unsigned** integer type,
 - whose size is implementation-defined. For example, it may be 64-bit on a 64-bit machine, and 32-bit on a 32-bit machine.

17/51

Use malloc and free

Declared in `<stdlib.h>`.

```
void *malloc(size_t size);
```

```
T *ptr = malloc(sizeof(T) * n); // sizeof(T) * n bytes
for (int i = 0; i != n; ++i)
    ptr[i] = /* ... */
// Now you can use 'ptr' as if it points to an array of 'n' objects of type 'T'
// ...
free(ptr);
```

To avoid **memory leaks**, the starting address of that block memory must be passed to `free` when the memory is not used anymore.

18/51

Use malloc and free

Declared in `<stdlib.h>`.

```
void free(void *ptr);
```

Deallocates the space previously allocated by an allocation function (such as `malloc`).

If `ptr` is a null pointer, this function does nothing.

- There is no need to do a null check before calling `free`!

The **behavior is undefined** if `ptr` is not equal to an address previously returned by an allocation function.

19/51

Use malloc and free

Declared in `<stdlib.h>`.

```
void free(void *ptr);
```

Deallocates the space previously allocated by an allocation function (such as `malloc`).

The **behavior is undefined** if the memory area referred to by `ptr` has already been deallocated.

- In other words, "double `free`" is undefined behavior (and often causes severe runtime errors).

After `free(ptr)`, `ptr` no longer points to an existing object, so it is no longer dereferenceable.

- Often called a "dangling pointer".

20/51

Use malloc and free

We can also create one single object dynamically (on heap):

```
int *ptr = malloc(sizeof(int));
*ptr = 42;
printf("%d\n", *ptr);
// ...
free(ptr);
```

But why? Why not just create one normal variable like `int ival = 42; ?`

21/51

Use malloc and free

Benefit: The lifetime of a dynamically allocated object goes beyond a local scope.

It is not destroyed until we `free` it.

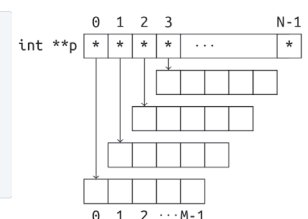
```
int *create_array(void) {
    int a[N];
    return a; // Returns the address of the local object 'a'.
              // When the function returns, 'a' will be destroyed, so that
              // the returned address becomes invalid.
              // Dereferencing the returned address is undefined behavior.
}
int *create_dynamic_array(int n) {
    return malloc(sizeof(int) * n); // OK. The allocated memory is valid until
                                   // we free it.
}
```

22/51

Use malloc and free

Create a "2-d array" on heap?

```
int **p = malloc(sizeof(int *) * n);
for (int i = 0; i < n; ++i)
    p[i] = malloc(sizeof(int) * m);
for (int i = 0; i < n; ++i)
    for (int j = 0; j < m; ++j)
        p[i][j] = /* ... */
// ...
for (int i = 0; i < n; ++i)
    free(p[i]);
free(p);
```



23/51

Use malloc and free

Create a "2-d array" on heap? - Another way: Create a 1-d array of length `n * m`.

```
int *p = malloc(sizeof(int) * n * m);
for (int i = 0; i < n; ++i)
    for (int j = 0; j < m; ++j)
        p[i * m + j] = /* ... */ // This is the (i, j)-th entry.
// ...
free(p);
```

24/51

Use calloc

Declared in `<stdlib.h>`

```
void *calloc(size_t num, size_t each_size);
```

Allocates memory for an array of `num` objects (each of size `each_size`), and initializes all bytes in the allocated storage to zero¹.

Similar as `malloc(num * each_size)`². Returns a null pointer on failure.

25/51

malloc, calloc and free

The behaviors of `malloc(0)`, `calloc(0, N)` and `calloc(N, 0)` are **implementation-defined**:

- They may or may not allocate memory.
- If no memory is allocated, a null pointer is returned.
- They may allocate *some* memory, for some reasons. In that case, the address of the allocated memory is returned.
 - You cannot dereference the returned pointer.
 - It still constitutes **memory leak** if such memory is not `free` d.

26/51

Arrays vs malloc

- An array has limited lifetime (unless it is global or `static`). It is destroyed when control reaches the end of its scope.
- Objects allocated by `malloc` are not destroyed until their address is passed to `free`.
- The program crashes if the size of an array is too large (running out of stack memory). There is no way of recovery.
- Attempt to `malloc` a block of memory that is too large results in a null pointer. We can know if there is no enough heap memory by doing a null check.

```
int *ptr = malloc(1ull << 60); // unrealistic size
if (!ptr)
    report_an_error("Out of memory.");
```

27/51

Summary

Pointer to `const`

- A pointer to `const` **thinks** that it is pointing to a `const` variable (though it may not), so it prevents you from modifying the pointed-to variable through it.
- Use `const` whenever possible.

`void *`

- A pointer type that can contain anything.
- Often used for representing "any pointer", "any object", or memory address.

28/51

Summay

Dynamic memory

```
void *malloc(size_t size);
void *calloc(size_t num, size_t each_size);
void free(void *ptr);
```

`malloc`: Allocates `size` bytes of **uninitialized** memory and returns its starting address.

`calloc`: Allocates `num * each_size` bytes of memory², each byte initialized to zero, and returns its starting address.

Both `malloc` and `calloc` return a null pointer on failure.

`free`: Deallocates the memory block starting at `ptr`.

29/51

Strings

C-style strings

C does not have a special construct for "string".

A string is a sequence of characters stored contiguously. We often use an array or a pointer to the first character to represent a string.

- It can be stored in an array, or in dynamically allocated memory.
- It must be **null-terminated**: There should be a null character `'\0'` at the end.

```
char s[10] = "abcde"; // s = {'a', 'b', 'c', 'd', 'e', '\0'}
printf("%s\n", s); // prints abcde
printf("%s\n", s + 1); // prints bcde
s[2] = ';'; // s = "ab;de"
printf("%s\n", s); // prints ab;de
s[2] = '\0';
printf("%s\n", s); // prints ab
```

The position of the first `'\0'` is the end of the string. Anything after that is discarded.

31/51

The null character `'\0'`

`'\0'` is the "null character" whose **ASCII** value is 0.

It is the **only** way to mark the end of a C-style string.

Every standard library function that handles strings will search for `'\0'` in that string.

- If there is no `'\0'`, they will search nonstop, and eventually go out of range (undefined behavior).

```
char s[5] = "abcde"; // OK, but no place for '\0'.
printf("%s\n", s); // undefined behavior (missing '\0')
```

Remember to allocate one more byte storage for `'\0'` !

32/51

Empty string

An empty string contains no characters before the null character.

```
char empty[] = ""; // `empty` is of type char[1], which contains only '\0'.
printf("%s\n", empty); // Prints only a newline.
printf(""); // Nothing is printed
```

33/51

String I/O

`scanf / printf: "%s"`

- `%s` in `scanf` matches a sequence of **non-whitespace** characters.
 - Leading whitespaces are discarded.
 - Reading starts from the first non-whitespace character, and stops right before the next whitespace character.
 - `'\0'` will be placed at the end.

Suppose the input is `123 456`:

```
char str[100] = "abcdef";
scanf("%s", str); // Reads "123". `str` becomes {'1', '2', '3', '\0', 'e', 'f'}
printf("%s\n", str); // Prints "123".
// 'e' and 'f' are not considered as part of the string.
```

34/51

String I/O

`scanf` is not memory safe:

```
char str[10];
scanf("%s", str);
```

- `str` is decayed (implicitly converted) to `char *` when passed as an argument.
- `scanf` receives only a pointer `char *`. It has **no idea how big the array is**.
- If the input content has more than 9 characters, it causes disaster!

That's why it is banned by MSVC. An alternative is to use `scanf_s`, but not necessarily supported by every compiler.

35/51

String I/O

`gets` reads a string without bounds checking. It has been removed since C11.

- An alternative for `gets` that does bounds checking is `gets_s`, but not supported by every compiler.

The best alternative: `fgets`. It is more portable, more generic, and safer (with bounds checking).

```
char str[100];
fgets(str, 100, stdin);
```

`puts(str)`: Prints the string `str`, followed by a newline.

36/51

String I/O

Homework Read the [cppreference documentation for fgets](#). Answer the following questions:

- How many characters does it read at most?
- When does it stop?

37/51

String manipulation / examination

Some common standard library functions: declared in `<string.h>`.

- `strlen(str)`: Returns the length of the string `str`.
- `strcpy(dest, src)`: Copies the string `src` to `dest`.
- `strcat(dest, src)`: Appends a copy of `src` to the end of `dest`.
- `strcmp(s1, s2)`: Compares two strings in lexicographical order.
- `strchr(str, ch)`: Finds the first occurrence of `ch` in `str`.

This page is only a brief introduction which cannot be relied on. The detailed documentations can be found [here](#).

38/51

String manipulation / examination

Read the documentation of a function before using it.

- Is `'\0'` counted in `strlen`?
- Does `strcpy` put a null character at the end? What about `strncpy`?
- For `strcpy(dest, src)`, what will happen if `dest` and `src` refer to the same memory address? What if they overlap? What about `strcat`?
- What is the result of `strcmp`? Is it $\in \{-1, 0, 1\}$? Is it `true` / `false`?

If you use the function without making these clear, you are heading for late-night debugging sessions!

39/51

Exercise: Implement strlen

Implement your own `strlen`, which accepts a pointer to the first character of a string and returns its length.

The length of a string does not count the null character `'\0'` in.

- `strlen("hello")` is 5, not 6.

40/51

Exercise: Implement strlen

Implement your own `strlen`, which accepts a pointer to the first character of a string and returns its length.

```
size_t my_strlen(const char *str) {
    size_t ans = 0;
    while (*str != '\0') {
        ++ans;
        ++str;
    }
    return ans;
}
```

```
size_t my_strlen(const char *str) {
    const char *end = str;
    while (*end != '\0')
        ++end;
    return end - str;
}
```

41/51

Exercise: Implement strlen

Implement your own `strlen`, which accepts a pointer to the first character of a string and returns its length.

```
size_t my_strlen(const char *str) {
    size_t ans = 0;
    while (*str++ != '\0') // Understand and learn to use this.
        ++ans;
    return ans;
}
```

42/51

strlen

There is no magic in `strlen`. It traverses the entire string and counts the characters, which is very slow.

Do not call `strlen` repeatedly!

```
for (size_t i = 0; i < strlen(s); ++i) // very slow
    // ...
```

- At the end of each iteration, the condition `i < strlen(s)` is evaluated, which causes the entire string to be traversed.
- The time complexity of this loop is $O(n^2)$.

43/51

strlen

There is no magic in `strlen`. It traverses the entire string and counts the characters, which is very slow.

Do not call `strlen` repeatedly!

```
for (size_t i = 0; i < strlen(s); ++i) // very slow
// ...
```

Change it to:

```
int n = strlen(s);
for (int i = 0; i < n; ++i)
// ...
```

44/51

Conversions to and from numeric formats

Full list can be found [here](#).

`strtol`, `strtoll`, `strtoul`, `strtoull`, `strtof`, `strtod`, `strtold`: Extracts the numeric value from a string.

- These are better alternatives to `atoi`, `atol`, ...: The base can be customized, and they have better error-reporting mechanisms.

45/51

String literals

A string literal is something like `"abcde"`, surrounded by double quotes `"`.

- The type of a string literal is `char [N+1]`, where `N` is the length of the string.
 - `+1` is for the terminating null character.
- But a string literal will be placed in read-only memory!!
 - In C++, its type is `const char [N+1]`, which is more reasonable.

When initializing a pointer with a string literal,

```
char *p = "abcde";
```

we are actually letting `p` point to the address of the string literal.

46/51

String literals

Using a pointer to non-`const` to point to a string literal is **allowed in C** (not allowed in C++), but **very dangerous**:

```
char *p = "abcde"; // OK
p[3] = 'a'; // No compile-error, but undefined behavior,
// and possibly severe runtime-error.
```

Correct ways:

Use low-level `const` ness to protect it:

```
const char *str = "abcde";
str[3] = 'a'; // compile-error
```

Copy the contents into an array:

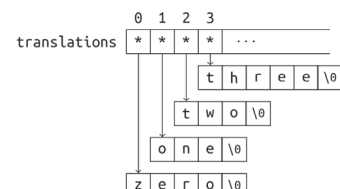
```
char arr[] = "abcde";
arr[3] = 'a'; // OK.
// `arr` contains a copy of "abcde".
```

47/51

Array of strings

```
const char *translations[] = {
    "zero", "one", "two", "three", "four",
    "five", "six", "seven", "eight", "nine"
};
```

- `translations` is an array of pointers, where each pointer points to a string literal.
- `translations` is not a 2-d array!



48/51

Summary

A C-style string is a sequence of characters stored contiguously, with `'\0'` at the end.

- Never forget the null character `'\0'`.
- String literals are not modifiable, even though their types are not `const`.
 - It's better to use a pointer to `const` to point to a string literal.
- I/O: `scanf` / `printf`, `fgets`, `puts`
- String manipulation / examination functions like `strlen`, `strcmp`, `strcpy`, ...
- Conversions to and from numeric formats: `strtol`, `strtoll`, ...

49/51

Exercises

Implement your own `strlen`, `strchr`, `strcmp`, `strcpy` and `strcat`. This will be in homework.

50/51

Notes

¹ "All bytes zero" does not necessarily mean `0.0` for floating point or null pointer value for pointers, although it does for most modern computers and compilers.

² Due to the alignment requirements, the number of allocated bytes is not necessarily equal to `num * each_size`.

51/51

CS100 Lecture 8

Dynamic Memory and Strings Revisited

1/33

Contents

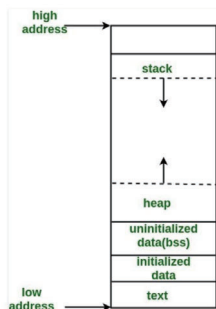
- Recap
- Command line arguments
- Example: Read a string of unknown length

2/33

Recap

3/33

Stack memory vs heap (dynamic) memory



- Stack memory is generally smaller than heap memory.
- Stack memory is often used for storing local and temporary objects.
- Heap memory is often used for storing large objects, and objects with long lifetime.
- Operations on stack memory is faster than on heap memory.
- Stack memory is allocated and deallocated automatically, while heap memory needs manual management.

4/33

Use malloc

- Allocate memory for a "2-d" array with n rows and m columns?

Another way: Allocate a "1-d" array of nm elements:

```
int *p = malloc(sizeof(int) * n * m);
for (int i = 0; i < n; ++i)
    for (int j = 0; j < m; ++j)
        scanf("%d", &a[i * m + j]); // a[i * m + j] is the (i, j)-th entry
```

8/33

Use free

- What pointer should be passed to `free`?
- What does `free(NULL)` do?
- What if we forget to `free`?
- After a call to `free(ptr)`, what is the value of `ptr`?
- What will happen if we `free` an address twice?

9/33

Use malloc

- Allocate memory for an `int`?
- Allocate memory for 100 `int`s?
- Allocate memory for a "2-d" array with n rows and m columns?
- Test allocation failure?

5/33

Use free

- What pointer should be passed to `free`?
 - The pointer must be either `null` or equal to a value returned earlier by an allocation function (one of `malloc`, `calloc`, `aligned_alloc` and `realloc`).
- What does `free(NULL)` do?
 - Nothing.
- What if we forget to `free`?
 - Memory leak.

10/33

Use malloc

- Allocate memory for an `int`?

```
int *p = malloc(sizeof(int));
*p = 42;
printf("%d\n", *p);
```

- Allocate memory for n `int`s?

```
int *p = malloc(sizeof(int) * n);
for (int i = 0; i < n; ++i)
    scanf("%d", p + i); // What does 'p + i' mean?
```

6/33

Use free

- After a call to `free(ptr)`, what is the value of `ptr`?
 - `ptr` becomes a **dangling pointer**, which cannot be dereferenced.
- What will happen if we `free` an address twice?
 - Undefined behavior (and is often severe runtime error).

11/33

Use malloc

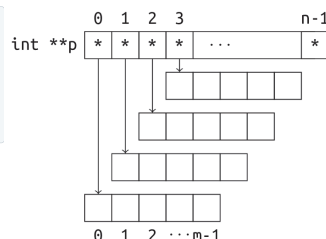
- Allocate memory for a "2-d" array with n rows and m columns?

```
int **p = malloc(sizeof(int *) * n);
for (int i = 0; i < n; ++i)
    p[i] = malloc(sizeof(int) * m);

for (int i = 0; i < n; ++i)
    for (int j = 0; j < m; ++j)
        scanf("%d", &p[i][j]);
```

`p` is a pointer to pointer to `int`,

- pointing to a sequence of pointers,
- each pointing to a sequence of `int`s.



7/33

Use malloc and free

Which of the following pieces of code deallocate(s) the memory correctly?

```
int *p = malloc(sizeof(int) * 100);
```

- `free(p);`
- `for (int i = 0; i < 100; ++i) free(p + i);`
- `free(p + 50); free(p);`
- `for (int i = 0; i < 10; ++i) free(p + i * 10);`

12/33

Use malloc and free

Which of the following pieces of code deallocate(s) the memory correctly?

- ```
int *p = malloc(sizeof(int) * 100);
```
- `free(p);` Yes
  - `for (int i = 0; i < 100; ++i) free(p + i);` No
  - `free(p + 50); free(p);` No
  - `for (int i = 0; i < 10; ++i) free(p + i * 10);` No

You cannot deallocate only a part of the memory block!

Command line arguments

Command line arguments

The following command executes `gcc.exe`, and tells it the file to be compiled and the name of the output:

```
gcc hello.c -o hello
```

How are the arguments `hello.c`, `-o` and `hello` passed to `gcc.exe`?

- It is definitely different from "input".

Strings

- What is a string in C?
- How can we obtain the length of a string?
- How do we read / write a string?
- How does a function accept and handle a string?

A new signature of main

```
int main(int argc, char **argv) { /* body */ }
```

Run this program with some arguments: `.\program one two three`

```
int main(int argc, char **argv) {
 for (int i = 0; i < argc; ++i)
 puts(argv[i]);
}
```

Output:

```
.\program
one
two
three
```

Strings

- What is a string in C?
  - A sequence of characters stored contiguously, with `'\0'` at the end.
- How can we obtain the length of a string?
  - `strlen(s)`
- How do we read / write a string?
  - `scanf` / `printf` with `"%s"`
  - `fgets`, `puts`

A new signature of main

```
int main(int argc, char **argv) { /* body */ }
```

where

- `argc` is a non-negative value representing the number of arguments passed to the program from the environment in which the program is run.
- `argv` is a pointer to the first element of an array of `argc + 1` pointers, of which
  - the last one is null, and
  - the previous ones (if any) point to strings that represent the arguments.

If `argv[0]` is not null (or equivalently, if `argc > 0`), it points to a string representing the program name.

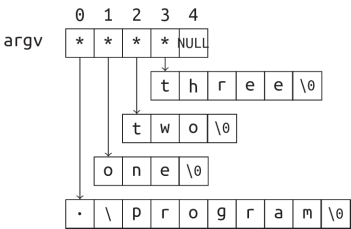
Strings

- How does a function accept and handle a string?
  - The function accepts a `char *`, indicating the start of the string.
  - The end of the string is found by searching for the first appearance of `'\0'`.
  - What is the result of `printf(NULL)`?

Command line arguments

```
int main(int argc, char **argv) { /* body */ }
```

`argv` is an array of pointers that point to the strings representing the arguments:



Strings

- How does a function accept and handle a string?
  - The function accepts a `char *`, indicating the start of the string.
  - The end of the string is found by searching for the first appearance of `'\0'`.
  - What is the result of `printf(NULL)`?
- Undefined behavior! `printf` expects a string for the first argument, which should contain at least a character `'\0'`.

\* Differentiate between the null character `'\0'`, the empty string `""` and the null pointer `NULL`.

## Example: Read a string of unknown length

### Read a string

`fgets(str, count, stdin)` reads a string, but at most `count - 1` characters.

`scanf("%s", str)` reads a string, but not caring about whether the input content is too long to fit into the memory that `str` points to.

For example, the following code is likely to crash if the input is `responsibility`:

```
char word[6];
scanf("%s", word);
```

`scanf` does nothing to prevent the disaster.

- It does not even know how long the array `word` is!

### Read a string of unknown length

Suppose we want to read a sequence of non-whitespace characters, the length of which is unknown.

- Use `malloc` / `free` to allocate and deallocate memory dynamically.
- When the current buffer is not large enough, we allocate a larger one and copies the stored elements to it!

### Ignore leading whitespaces:

```
char *read_string(void) {
 char c = getchar();
 while (isspace(c))
 c = getchar();
}
```

### Set a buffer with initial capacity.

```
char *read_string(void) {
 char c = getchar();
 while (isspace(c))
 c = getchar();

 char *buffer = malloc(INITIAL_SIZE);
 int capacity = INITIAL_SIZE;
 int cur_pos = 0; // The index at which we store the input character.

}
```

### Write a loop to read and store characters.

```
char *read_string(void) {
 // ignore leading whitespaces

 char *buffer = malloc(INITIAL_SIZE);
 int capacity = INITIAL_SIZE;
 int cur_pos = 0; // The index at which we store the input character.

 while (!isspace(c)) {
 if (cur_pos == capacity - 1) { // '-' is for '\0'.

 }
 buffer[cur_pos++] = c;
 c = getchar();
 }
}
```

### When the buffer is full, allocate a new one twice as large.

```
char *read_string(void) {
 // ...
 while (!isspace(c)) {
 if (cur_pos == capacity - 1) { // '-' is for '\0'.
 char *new_buffer = malloc(capacity * 2);
 memcpy(new_buffer, buffer, cur_pos); // copy everything we have stored
 // to the new buffer
 capacity *= 2;
 buffer = new_buffer;
 }
 buffer[cur_pos++] = c;
 c = getchar();
 }
}
```

\* Are we done?

### Do not forget to free!

```
char *read_string(void) {
 // ...
 while (!isspace(c)) {
 if (cur_pos == capacity - 1) { // '-' is for '\0'.
 char *new_buffer = malloc(capacity * 2);
 memcpy(new_buffer, buffer, cur_pos); // copy everything we have stored
 // to the new buffer
 free(buffer); // !!!!!!!!!!!!!
 capacity *= 2;
 buffer = new_buffer;
 }
 buffer[cur_pos++] = c;
 c = getchar();
 }
}
```

### Don't consume more than what we need from the input.

```
char *read_string(void) {
 // ...
 while (!isspace(c)) {
 if (cur_pos == capacity - 1) { // '-' is for '\0'.
 // ...
 }
 buffer[cur_pos++] = c;
 c = getchar();
 }

 // Now, 'c' is a whitespace. This is not part of the contents we need.
 ungetc(c, stdin); // Put that whitespace back to the input.

 return buffer;
}
```

\* Are we done?

### Don't forget the null character!

```
char *read_string(void) {
 // ...
 while (!isspace(c)) {
 if (cur_pos == capacity - 1) { // '-' is for '\0'.
 // ...
 }
 buffer[cur_pos++] = c;
 c = getchar();
 }

 // Now, 'c' is a whitespace. This is not part of the contents we need.
 ungetc(c, stdin); // Put that whitespace back to the input.

 buffer[cur_pos] = '\0'; // Remember this!!!

 return buffer;
}
```



## struct type

The name of the type defined by a `struct` is `struct Name`.

- Unlike C++, the keyword `struct` here is necessary.

```
struct Student stu; // `stu` is an object of type `struct Student`
struct Point3d polygon[1000]; // `polygon` is an array of 1000 objects,
 // each being of type `struct Point3d`.
struct TreeNode *pNode; // `pNode` is a pointer to `struct TreeNode`.
```

\* The term "*object*" is used interchangeably with "*variable*".

- *Objects* often refer to variables of `struct` (or `class` in C++) types.
- But in fact, there's nothing wrong to say "an `int` object".

33/33

5/39

## Members of a struct

Use `obj.mem`, the member-access operator `.` to access a member.

```
struct Student stu;
stu.name = "Alice";
stu.id = "2024533000";
stu.entrance_year = 2024;
stu.dorm = 8;
printf("%d\n", student.dorm);
++student.entrance_year;
puts(student.name);
```

1/39

6/39

## Dynamic allocation

Create an object of `struct` type dynamically: Just allocate `sizeof(struct Student)` bytes of memory.

```
struct Student *pStu = malloc(sizeof(struct Student));
```

Member access through a pointer: `ptr->mem`, or `(*ptr).mem` (not `*ptr.mem`!).

```
pStu->name = "Alice";
pStu->id = "2024533000";
(*pStu).entrance_year = 2024; // equivalent to pStu->entrance_year = 2024;
printf("%d\n", pStu->entrance_year);
puts(pStu->name);
```

As usual, don't forget to `free` after use.

```
free(pStu);
```

2/39

7/39

## Size of a struct

```
struct Student {
 const char *name;
 const char *id;
 int entrance_year;
 int dorm;
};
```

```
struct Student *pStu = malloc(sizeof(struct Student));
```

What is the value of `sizeof(struct Student)` ?

3/39

8/39

## Size of a struct

Try these:

```
struct A {
 int x;
 char y;
 double z;
};
```

```
struct B {
 char x;
 double y;
 int z;
};
```

```
printf("%zu\n", sizeof(struct A));
```

```
printf("%zu\n", sizeof(struct B));
```

Possible result: `sizeof(struct A)` is 16, but `sizeof(struct B)` is 24 (on Ubuntu 22.04, GCC 13).

4/39

9/39

## Use

```
int main(void) {
 char *content = read_string();
 puts(content);
 free(content);
}
```

Remember to `free` it after use!

## CS100 Lecture 9

`struct`, Recursion

## Contents

- `struct`
- Recursion
  - Factorial
  - Print a non-negative integer
  - Selection-sort

## struct

## Define a struct

A `struct` is a **type** consisting of a sequence of **members** whose storage is allocated in an ordered sequence.

Simply put, place several things together to form a new type.

```
struct Student {
 const char *name;
 const char *id;
 int entrance_year;
 int dorm;
};
```

```
struct Point3d {
 double x, y, z;
};
struct Line3d {
 // P(t) = p0 + tv
 struct Point3d p0, v;
};
```

## Size of `struct`

```
struct A {
 int x; // 4 bytes
 char y; // 1 byte
 // 3 bytes padding
 double z; // 8 bytes
};
```

- `sizeof(struct A) == 16`

```
struct B {
 char x; // 1 byte
 // 7 bytes padding
 double y; // 8 bytes
 int z; // 4 bytes
 // 4 bytes padding
};
```

- `sizeof(struct B) == 24`

It is guaranteed that

$$\text{sizeof}(\text{struct } X) \geq \sum_{\text{member} \in X} \text{sizeof}(\text{member}).$$

The inequality is due to **memory alignment requirements**, which is beyond the scope of CS100.

10/39

## Implicit initialization

What happens if an object of `struct` type is not explicitly initialized?

```
struct Student gStu;

int main(void) {
 struct Student stu;
}
```

11/39

## Implicit initialization

What happens if an object of `struct` type is not explicitly initialized?

```
struct Student gStu;

int main(void) {
 struct Student stu;
}
```

- Global or local `static`: "empty-initialization", which performs **member-wise** empty-initialization.
- Local non-`static`: every member is initialized to indeterminate values (in other words, uninitialized).

12/39

## Explicit initialization

Use an initializer list:

```
struct Student stu = {"Alice", "2024533000", 2024, 8};
```

Use C99 designators: (highly recommended)

```
struct Student stu = {.name = "Alice", .id = "2024533000",
 .entrance_year = 2024, .dorm = 8};
```

The designators greatly improve the readability.

**[Best practice]** Use designators, especially for `struct` types with lots of members.

13/39

## Compound literals

```
struct Student *student_list = malloc(sizeof(struct Student) * n);
for (int i = 0; i != n; ++i) {
 student_list[i].name = A(i); // A, B, C and D are some functions
 student_list[i].id = B(i);
 student_list[i].entrance_year = C(i);
 student_list[i].dorm = D(i);
}
```

Use a **compound literal** to make it clear and simple:

```
struct Student *student_list = malloc(sizeof(struct Student) * n);
for (int i = 0; i != n; ++i) {
 student_list[i] = (struct Student){.name = A(i), .id = B(i),
 .entrance_year = C(i), .dorm = D(i)};
}
```

14/39

## `struct`-typed parameters

The semantic of argument passing is **copy**:

```
void print_student(struct Student s) {
 printf("Name: %s, ID: %s, dorm: %d\n", s.name, s.id, s.dorm);
}

print_student(student_list[i]);
```

In a call `print_student(student_list[i])`, the parameter `s` of `print_student` is initialized as follows:

```
struct Student s = student_list[i];
```

The copy of a `struct`-typed object: **Member-wise copy**.

15/39

## `struct`-typed parameters

In a call `print_student(student_list[i])`, the parameter `s` of `print_student` is initialized as follows:

```
struct Student s = student_list[i];
```

The copy of a `struct`-typed object: **Member-wise copy**. It is performed as if

```
s.name = student_list[i].name;
s.id = student_list[i].id;
s.entrance_year = student_list[i].entrance_year;
s.dorm = student_list[i].dorm;
```

16/39

## Return a `struct`-typed object

Strictly speaking, returning is also a **copy**:

```
struct Student fun(void) {
 struct Student s = something();
 some_operations(s);
 return s;
}

student_list[i] = fun();
```

The object `s` is returned as if

```
student_list[i] = s;
```

**But in fact, the compiler is more than willing to optimize this process.** We will talk more about this in C++.

17/39

## Array member

```
struct A {
 int array[10];
 // ...
};
```

Although an array cannot be copied, an **array member can be copied**.

The copy of an array is **element-wise copy**.

```
int a[10];
int b[10] = a; // Error!

struct A a;
struct A b = a; // OK
```

18/39

## Summary

A `struct` is a type consisting of a sequence of members.

- Member access: `obj.mem`, `ptr->mem` (equivalent to `(*ptr).mem`, but better)
- `sizeof(struct A)`, no less than the sum of size of every member.
  - But not necessarily equal, due to memory alignment requirements.
- Implicit initialization: recursively performed on every member.
- Initializer-lists, designators, compound literals.
- Copy of a `struct`: member-wise copy.
- Argument passing and returning: copy.

19/39

Exercise

Consider a 3-d coordinate point  $(x, y, z)$  and a line  $\mathbf{P}(t) = \mathbf{P}_0 + t\mathbf{v}$ . Define some `struct` `s` to represent these concepts.

Write some functions to calculate the distance between points and lines, to calculate the point  $\mathbf{P}(t_0)$  for a given  $t_0$ , and to print some information.

Learn and try to use `initializer-lists`, `designators` and `compound literals`.

```
double dist(struct Point3d p, struct Line3d line);
struct Point3d line_at(struct Line3d line, double t);
```

Recursion

Problem 1. Calculate  $n!$

A piece of cake!

```
int factorial(int n) {
 int result = 1;
 for (int i = 1; i <= n; ++i)
 result *= i;
 return result;
}
```

You should be able to write this in 30 seconds.

Calculate  $n!$

Consider the recurrence relation:

$$n! = \begin{cases} 1, & n = 0, \\ n \cdot (n-1)!, & n > 0. \end{cases}$$

Translate it directly into C:

```
int factorial(int n) {
 return n == 0 ? 1 : n * factorial(n - 1);
}
```

This is perfectly valid and reasonable C code!

- The function `factorial` recursively calls itself.

Calculate  $n!$

```
int fac(int n) {
 if (n == 0)
 return 1;
 else
 return n * fac(n - 1);
}
```

Problem 2. Print a non-negative integer

If we only have `getchar`, how can we read an integer?

- We have solved this in recitations.

If we only have `putchar`, how can we print an integer?

- Declared in `<stdio.h>`.
- `putchar(c)` prints a character `c`. That's it.

For convenience, suppose the integer is non-negative (unsigned).

Print a non-negative integer

To print  $x$ :

- If  $x < 10$ , just print the digit and we are done.
- Otherwise ( $x \geq 10$ ), we first print  $\lfloor \frac{x}{10} \rfloor$ , and then print the digit on the last place.

```
void print(unsigned x) {
 if (x < 10)
 putchar(x + '0'); // Remember ASCII?
 else {
 print(x / 10);
 putchar(x % 10 + '0');
 }
}
```

Simplify the code

To print  $x$ :

- If  $x \geq 10$ , we first print  $\lfloor \frac{x}{10} \rfloor$ . Otherwise, do nothing.
- Print  $x \bmod 10$ .

```
void print(unsigned x) {
 if (x >= 10)
 print(x / 10);
 putchar(x % 10 + '0');
}
```

Print a non-negative integer

To print  $x$ :

- If  $x \geq 10$ , we first print  $\lfloor \frac{x}{10} \rfloor$ . Otherwise, do nothing.
- Print  $x \bmod 10$ .

```
void print(unsigned x) {
 if (x >= 10)
 print(x / 10);
 putchar(x % 10 + '0');
}
```

Exercise: Allow for negative numbers.

Design a recursive algorithm

Suppose we are given a problem of scale  $n$ .

- Divide the problem into one or more **subproblems**, which are of smaller scales.
- Solve the subproblems **recursively** by calling the function itself.
- Generate the answer to the big problem from the answers to the subproblems.

\* Feels like mathematical induction?

## Exercise: Quick power

Calculate  $x^n$  following the recurrence relation:

$$x^n = \begin{cases} 1, & n = 0, \\ x \cdot x^{n-1}, & n \text{ is odd,} \\ (x^{n/2})^2, & n \text{ is even.} \end{cases}$$

30/39

## Exercise: Quick power

Calculate  $x^n$  following the recurrence relation:

$$x^n = \begin{cases} 1, & n = 0, \\ x \cdot x^{n-1}, & n \text{ is odd,} \\ (x^{n/2})^2, & n \text{ is even.} \end{cases}$$

```
unsigned long long quick_power(unsigned long long x, int n) {
 if (n == 0)
 return 1;
 if (n % 2 == 1)
 return x * quick_power(x, n - 1);
 else {
 unsigned long long t = quick_power(x, n / 2);
 return t * t;
 }
}
```

31/39

## Problem 3. Selection-sort

How do you sort a sequence of  $n$  numbers? (In ascending order)

Do it **recursively**.

32/39

## Problem 3. Selection-sort

How do you sort a sequence of  $n$  numbers  $\langle a_0, \dots, a_{n-1} \rangle$ ? (In ascending order)

Do it **recursively**: Suppose we are going to sort  $\langle a_k, a_{k+1}, \dots, a_{n-1} \rangle$ , for some  $k$ .

- If  $k = n - 1$ , we are done.
- Otherwise ( $k < n - 1$ ):
  - i. Find the minimal number  $a_m = \min \{a_k, a_{k+1}, \dots, a_{n-1}\}$ .
  - ii. Put  $a_m$  at the first place by swapping it with  $a_k$ .
  - iii. Now  $a_k$  is the smallest number in  $\langle a_k, \dots, a_{n-1} \rangle$ . All we have to do is to sort the rest part  $\langle a_{k+1}, \dots, a_{n-1} \rangle$  **recursively**.

33/39

## Selection-sort

Do it **recursively**: Suppose we are going to sort  $\langle a_k, a_{k+1}, \dots, a_{n-1} \rangle$ , for some  $k$ .

```
void sort_impl(int *a, int k, int n) { // "impl" stands for "implementation"
}
void sort(int *a, int n) {
 sort_impl(a, 0, n); // Why do we set k = 0 here?
}
```

34/39

## Selection-sort

Do it **recursively**: Suppose we are going to sort  $\langle a_k, a_{k+1}, \dots, a_{n-1} \rangle$ , for some  $k$ .

- If  $k = n - 1$ , we are done.

```
void sort_impl(int *a, int k, int n) {
 if (k == n - 1)
 return;
}
```

35/39

## Selection-sort

Do it **recursively**: Suppose we are going to sort  $\langle a_k, a_{k+1}, \dots, a_{n-1} \rangle$ , for some  $k$ .

- If  $k < n - 1$ :
  - i. Find the minimal number  $a_m = \min \{a_k, a_{k+1}, \dots, a_{n-1}\}$ .

```
void sort_impl(int *a, int k, int n) {
 if (k == n - 1)
 return;

 int m = k;
 for (int i = k + 1; i < n; ++i)
 if (a[i] < a[m])
 m = i;
}
```

36/39

## Selection-sort

Do it **recursively**: Suppose we are going to sort  $\langle a_k, a_{k+1}, \dots, a_{n-1} \rangle$ , for some  $k$ .

- If  $k < n - 1$ :
  - i. Find the minimal number  $a_m = \min \{a_k, a_{k+1}, \dots, a_{n-1}\}$ .
  - ii. Put  $a_m$  at the first place by swapping it with  $a_k$ .

```
void sort_impl(int *a, int k, int n) {
 if (k == n - 1) return;

 int m = k;
 for (int i = k + 1; i < n; ++i)
 if (a[i] < a[m]) m = i;

 swap(&a[m], &a[k]); // the "swap" function we defined in previous lectures
}
```

37/39

## Selection-sort

Do it **recursively**: Suppose we are going to sort  $\langle a_k, a_{k+1}, \dots, a_{n-1} \rangle$ , for some  $k$ .

- If  $k < n - 1$ :
  - i. Find the minimal number  $a_m = \min \{a_k, a_{k+1}, \dots, a_{n-1}\}$ .
  - ii. Put  $a_m$  at the first place by swapping it with  $a_k$ .
  - iii. Sort the rest part  $\langle a_{k+1}, \dots, a_{n-1} \rangle$  **recursively**.

```
void sort_impl(int *a, int k, int n) {
 if (k == n - 1) return;

 int m = k;
 for (int i = k + 1; i < n; ++i)
 if (a[i] < a[m]) m = i;

 swap(&a[m], &a[k]); // the "swap" function we defined in previous lectures

 sort_impl(a, k + 1, n); // sort the rest part recursively
}
```

38/39

## Selection-sort

```
void sort_impl(int *a, int k, int n) {
 if (k == n - 1)
 return;

 int m = k;
 for (int i = k + 1; i < n; ++i)
 if (a[i] < a[m])
 m = i;

 swap(&a[m], &a[k]); // the "swap" function we defined in previous lectures

 sort_impl(a, k + 1, n); // sort the rest part recursively
}

void sort(int *a, int n) {
 sort_impl(a, 0, n);
}
```

39/39

# CS100 Lecture 10

C Summary

## Contents

- C summary
  - Types
  - Variables
  - Expressions
  - Control flow
  - Functions
  - Standard library
  - Example: `Vector`

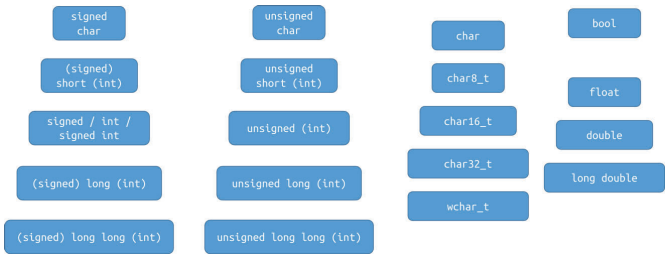
# C Summary

## Types

Types are fundamental to any program: They tell us what our data mean and what operations we can perform on those data.

C is a **statically-typed** language: The type of every expression (except those involving VLAs) is known at **compile-time**.

## Arithmetic types



## Arithmetic types

- `1 == sizeof(char) <= sizeof(short) <= sizeof(int) <= sizeof(long) <= sizeof(long long)`
- `sizeof(signed T) == sizeof(unsigned T)` for every `T ∈ { char, short, int, long, long long }`
- `short` and `int` are at least 16 bits. `long` is at least 32 bits. `long long` is at least 64 bits.
- Range of signed types:  $[-2^{N-1}, 2^{N-1} - 1]$ . Range of unsigned types:  $[0, 2^N - 1]$ .
- Whether `char` is signed or not is **implementation-defined**.
- Signed integer overflow is **undefined behavior**.
- Unsigned arithmetic **never overflows**: It is performed modulo  $2^N$ , where  $N$  is the number of bits of that type.

## Pointer types

- `PointeeType *`
- For `T ≠ U`, `T *` and `U *` are **different types**.
  - The value of a pointer of type `T *` is the **address of** an object of type `T`.
  - **Null pointer**: The pointer holding the **null pointer value**, which is a special value indicating that the pointer is "pointing nowhere".
    - A null pointer can be obtained from `NULL`.
  - `&var` returns the address of `var`. The return type is pointer to the type of `var`.
  - Only when a pointer is actually pointing to an object is it **dereferenceable**.
  - `*ptr`, where `ptr` is not dereferenceable, is **undefined behavior**.

## Array types

- `ElemType [N]`
- `T [N]`, `U [N]` and `T [M]` are **different types** for `T ≠ U` and `N ≠ M`.
  - `N` should be compile-time constant. Otherwise it is a VLA.
  - Valid index range:  $[0, N)$ . Subscript out of range is **undefined behavior**.
  - Decay: `a → &a[0]`, `T [N] → T *`.
- Pointer to array: `T (*)[N]`. Array of pointers: `T *[N]`.

## struct types

A special data type consisting of a sequence of **members**.

- The type name is `struct StructName`.
- `sizeof(struct X) ≥ ∑_{member ∈ X} sizeof(member)`

## Variables

- Declare a variable: `Type varName`
- `ElemType varName[N]` for array type `ElemType[N]`.
  - `T (*varName)[N]` for pointer to array type `T (*)[N]`.
- Initialize a variable: `= initializer`
- Brace-enclosed list initializer for arrays and `struct s: = { ... }`.
  - Designators for arrays: `= {[3] = 5, [7] = 4}`
  - Designators for `struct s: = {.mem1 = x, .mem2 = y}`.

## Initialization

If a variable is declared without explicit initializer:

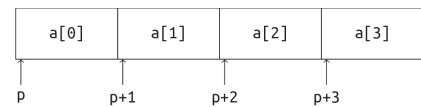
- For global or local `static` variables, they are **empty-initialized**:
  - `0` for integer types,
  - `+0.0` for floating-point types,
  - null pointer value for pointer types.
- For local non-`static` variables, they are **uninitialized**, holding indeterminate values.

These rules apply recursively to the elements of arrays and the members of `struct` s.

Any use of the value of an uninitialized variable is **undefined behavior**.

11/42

## Pointer arithmetic



- Pointer arithmetic: `p++`, `++p`, `p--`, `--p`, `p + i`, `i + p`, `p - i`, `p += i`, `p -= i`, `p1 - p2`.
- Pointer arithmetic uses the units of the pointed-to type.
  - `p + i == (char *)p + i * sizeof(*p)`
- Pointer arithmetic must be performed within an array (including its past-the-end position), otherwise **the behavior is undefined**.

16/42

## Scopes and name lookup

```
int add(int x, int y) {
 return x + y;
}
int square(int x) {
 return x * x;
}
int main(void) {
 int x; scanf("%d", &x);
 printf("%d\n", square(x));
 if (x == 42) {
 int x = 35;
 printf("%d\n", square(square(x)));
 }
 for (int x = 1; x <= 10; ++x)
 printf("%d\n", square(x + 1));
 return 0;
}
```

12/42

## Operators

`++`, `--`

- `++a` and `--a` returns the value of `a` after incrementation/decrementation.
- `a++` and `a--` returns the original value of `a`.

`<`, `<=`, `>`, `>=`, `==`, `!=`

- The operands undergo a series of type conversions to a common type before comparison.

17/42

## Expressions

Expressions = operators + operands.

- Operator precedence, associativity, and evaluation order of operands
  - `f() + g() * h()`, `f() - g() + h()`
- The only four operators whose operands have deterministic evaluation order:
  - `&&` and `||`: short-circuit evaluation
  - `?:`
  - `,` (not in a function call or in an initializer list)

13/42

## Operators

Member access: `obj.member`.

Member access through pointer: `ptr->member`, which is equivalent to `(*ptr).member`.

- `.` has higher precedence than `*`, so the parentheses around `*ptr` are necessary.

18/42

## Expressions

- If the evaluation order of `A` and `B` is unspecified, and if
  - both `A` and `B` contain a write to an object, or
  - one of them contains a write to an object, and the other one contains a read to that object

then **the behavior is undefined**.

14/42

## Control flow

- `if (cond) stmt`
- `if (cond) stmt1 else stmt2`
- `for (init_expr; cond; inc_expr) stmt`
- `while (cond) stmt`
- `do stmt while (cond);`
- `switch (integral_expr) { ... }`
- `break` and `continue`

19/42

## Arithmetic operators

`+`, `-`, `*`, `/`, `%`

- Division: truncated towards zero.
- Remainder: `(a / b) * b + (a % b) == a` always holds.
- For `+`, `-`, `*` and `/`, the operands undergo a series of type conversions to a common type.

Bitwise operators: `~`, `&`, `|`, `^`, `<<`, `>>`

Compound assignment operators: `a op= b` is equivalent to `a = a op b`.

Be careful with signed overflows.

15/42

## Functions

Function declaration: `RetType funcName(Parameters);`

- Parameter names are not necessary, but types are required.
- A function can be declared multiple times.

Function definition: `RetType funcName(Parameters) { functionBody }`

- A function can be defined only once.

20/42

## Functions

- Argument passing:
  - Use the argument to initialize the parameter.
  - The semantic is **copy**.
  - **Decay** always happens: One can never declare an array parameter.

21/42

## The `main` function

Entry point of the program (after initialization of all global and local `static` variables).

One of the following signatures:

- `int main(void) { ... }`
- `int main(int argc, char **argv) { ... }`, for passing command-line arguments.
- `/* another implementation-defined signature */`

Return value: `0` to indicate that the program exits successfully.

22/42

## Standard library

- IO library `<stdio.h>`: `scanf`, `printf`, `fgets`, `puts`, `putchar`, `getchar`, ...
- String library `<string.h>`: `strlen`, `strcpy`, `strcmp`, `strchr`, ...
- Character classification `<ctype.h>`: `isdigit`, `isalpha`, `tolower`, ...
- `<stdlib.h>`: Several general-purpose functions: `malloc` / `free`, `rand`, ...
- `<limits.h>`: Defines macros like `INT_MAX` that describe the limits of built-in types.
- `<math.h>`: Mathematical functions like `sqrt`, `sin`, `acos`, `exp`, ...

23/42

## Example: `Vector`

A "vector" in linear algebra:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}.$$

It consists of two things: A sequence of  $n$  numbers, and its dimension  $n$ .

24/42

## Example: `Vector`

```
struct Vector {
 double *entries;
 size_t dimension;
};
```

Do not name them with `x` and `n`!

[Best practice] Use meaningful names in programs.

25/42

## Creation and destruction

```
struct Vector create_vector(size_t n) {
 return (struct Vector){.entries = calloc(n, sizeof(double)),
 .dimension = n};
}
void destroy_vector(struct Vector *vec) {
 free(vec->entries);
 // Do we need to free(vec)?
}
```

Usage:

```
struct Vector v = create_vector(10);
// some operations ...
destroy_vector(&v);
```

26/42

## "Deep copy" of `Vector`

The default copy semantics of `Vector` is not satisfactory:

```
struct Vector v = something();
struct Vector u = v;
```

Now `u.entries` and `v.entries` point to the same memory block!

```
destroy_vector(&u);
destroy_vector(&v); // undefined behavior: double free!
```

27/42

## "Deep copy" of `Vector`

```
void vector_assign(struct Vector *to, const struct Vector *from) {
 to->entries = malloc(from->dimension * sizeof(double));
 memcpy(to->entries, from->entries, from->dimension * sizeof(double));
 to->dimension = from->dimension;
}
```

Is this correct?

28/42

## "Deep copy" of `Vector`

`free` the memory block that is not used anymore!

```
void vector_assign(struct Vector *to, const struct Vector *from) {
 free(to->entries); // Don't forget this!!
 to->entries = malloc(from->dimension * sizeof(double));
 memcpy(to->entries, from->entries, from->dimension * sizeof(double));
 to->dimension = from->dimension;
}
```

Is this correct?

29/42

## "Deep copy" of `Vector`

```
void vector_assign(struct Vector *to, const struct Vector *from) {
 free(to->entries); // Don't forget this!!
 to->entries = malloc(from->dimension * sizeof(double));
 memcpy(to->entries, from->entries, from->dimension * sizeof(double));
 to->dimension = from->dimension;
}
```

What happens if `to == from` ?

- This is not impossible. Consider `vector_assign(&vecs[i], &vecs[j])` where `i` and `j` have a chance to be equal.

30/42

## "Deep copy" of Vector

```
void vector_assign(struct Vector *to, const struct Vector *from) {
 free(to->entries); // Don't forget this!!
 to->entries = malloc(from->dimension * sizeof(double));
 memcpy(to->entries, from->entries, from->dimension * sizeof(double));
 to->dimension = from->dimension;
}
```

What happens if `to == from` ?

- This is not impossible. Consider `vector_assign(&x[i], &x[j])` where `i` and `j` have a chance to be equal.
- The memory block is freed, and the data are gone.

31/42

## "Deep copy" of Vector

```
void vector_assign(struct Vector *to, const struct Vector *from) {
 if (to == from)
 return;
 free(to->entries); // Don't forget this!!
 to->entries = malloc(from->dimension * sizeof(double));
 memcpy(to->entries, from->entries, from->dimension * sizeof(double));
 to->dimension = from->dimension;
}
```

Why do we declare the parameters as pointers?

32/42

## "Deep copy" of Vector

```
void vector_assign(struct Vector *to, const struct Vector *from) {
 if (to == from)
 return;
 free(to->entries); // Don't forget this!!
 to->entries = malloc(from->dimension * sizeof(double));
 memcpy(to->entries, from->entries, from->dimension * sizeof(double));
 to->dimension = from->dimension;
}
```

Why do we declare the parameters as pointers?

- For `to`, we need to modify it.
- For `from`, this is a read-only operation. Pass the address to avoid copies.

33/42

## Equality comparison

```
bool vector_equal(const struct Vector *lhs, const struct Vector *rhs) {
 if (lhs->dimension != rhs->dimension)
 return false;
 for (size_t i = 0; i != lhs->dimension; ++i)
 if (lhs->entries[i] != rhs->entries[i])
 return false;
 return true;
}
```

Here we use `!=` to compare two `double` s directly. It's better to use  $|a - b| > \epsilon$ , considering the floating-point errors.

`lhs` and `rhs` are pointers, to avoid unnecessary copies.

34/42

## Basic operations on Vector

```
struct Vector vector_add(const struct Vector *lhs, const struct Vector *rhs) {
 assert(lhs->dimension == rhs->dimension);
 struct Vector result = create_vector(lhs->dimension);
 for (size_t i = 0; i != lhs->dimension; ++i)
 result.entries[i] = lhs->entries[i] + rhs->entries[i];
 return result;
}

struct Vector vector_scale(const struct Vector *lhs, double scale) {
 struct Vector result = create_vector(lhs->dimension);
 for (size_t i = 0; i != lhs->dimension; ++i)
 result.entries[i] = lhs->entries[i] * scale;
 return result;
}
```

For `vector_add`, our design is to claim that "the behavior is undefined if the vectors have different dimensions".

35/42

## Dot product, norm and distance ( $\ell_2$ )

```
double vector_dot_product(const struct Vector *lhs, const struct Vector *rhs) {
 assert(lhs->dimension == rhs->dimension);
 double result = 0;
 for (size_t i = 0; i != lhs->dimension; ++i)
 result += lhs->entries[i] * rhs->entries[i];
 return result;
}

double vector_norm(const struct Vector *vec) {
 return sqrt(vector_dot_product(vec, vec));
}

double vector_distance(const struct Vector *lhs, const struct Vector *rhs) {
 struct Vector diff = vector_minus(lhs, rhs); // Define this on your own.
 return vector_norm(&diff);
}
```

For `vector_dot_product`, our design is to claim that "the behavior is undefined if the vectors have different dimensions".

36/42

## Print a Vector

```
void print_vector(const struct Vector *vec) {
 putchar('(');
 if (vec->dimension > 0) {
 printf("%lf", vec->entries[0]);
 for (size_t i = 1; i != vec->dimension; ++i)
 printf(", %lf", vec->entries[i]);
 }
 putchar(')');
}
```

37/42

## Exercise

What if we want to increase the dimension of a `Vector` ? Implement the related functionality that reallocates a larger block of memory when needed.

```
void vector_push_back(struct Vector *vec, double x) {
 if (/* reallocation is needed */)
 vector_grow(vec); // Implement this function
 vec->entries[vec->dimension++] = x;
}
```

You may need to add members to `struct Vector`.

38/42

## What we have done

```
struct Vector {
 double *entries;
 size_t dimension;
};

struct Vector create_vector(size_t n);
void destroy_vector(struct Vector *vec);
void vector_assign(struct Vector *to, const struct Vector *from);
bool vector_equal(const struct Vector *lhs, const struct Vector *rhs);
struct Vector vector_add(const struct Vector *lhs, const struct Vector *rhs);
struct Vector vector_minus(const struct Vector *lhs, const struct Vector *rhs);
struct Vector vector_scale(const struct Vector *lhs, double scale);
double vector_dot_product(const struct Vector *lhs, const struct Vector *rhs);
double vector_norm(const struct Vector *vec);
double vector_distance(const struct Vector *lhs, const struct Vector *rhs);
void print_vector(const struct Vector *vec);
```

39/42

## Problems of the current implementation

1. The call to `create_vector` is not mandatory. One can easily create a `Vector` with some garbage values.
2. `destroy_vector` is not called automatically. If we forget to call it manually, memory leak happens.
3. We always need to pass the address of `Vector` s to these functions. The extra `&` and `*` are annoying.
4. The "deep copy" is implemented by a function, but the default copy semantics are still there. If we forget to call `vector_assign` when copying a `Vector`, disaster will happen.
5. No prevention from modifying a `Vector`: Disaster is caused easily by a simple `free(vec->entries);`.

40/42



## Problems of the current implementation

6. The named functions are inconvenient: To compute  $\mathbf{u}^T(\mathbf{v} + 2\mathbf{w})$ , we need to write

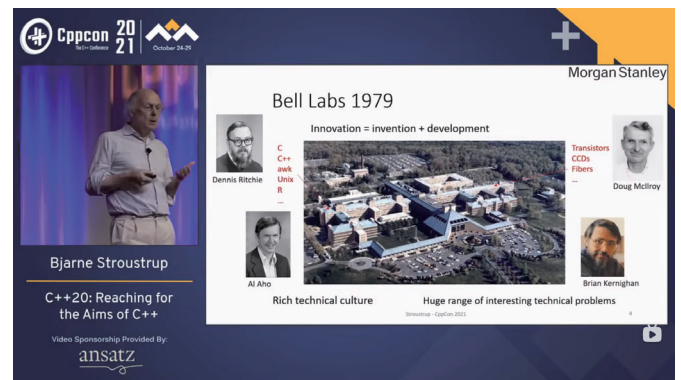
```
struct Vector scaled = vector_scale(&w, 2);
struct Vector added = vector_add(&v, &scaled);
return vector_dot_product(&u, &added);
```

Can we express it directly by `return u * (v + 2 * w);`?

7. ....

We will see the solutions to these problems in C++, by data abstraction, and by OOP (object-oriented programming).

41/42



4/37

## Enter the world of C++ ...

From *The Design and Evolution of C++*, by Bjarne Stroustrup who invented C++:

C++ is a general-purpose programming language that

- is a better C, and
- supports data abstraction, and
- supports object-oriented programming.

```
#include <iostream>

int main() {
 std::cout << "Hello world\n";
 return 0;
}
```

42/42

## C with Classes

Back to 1979, the Bell Labs: *C with Classes* was invented by Bjarne Stroustrup.

- An object-oriented C, with the ideas of "class" from *Simula* (and several other programming languages).
- Based on C, with many improvements: better type checking, better linkage, ...

The development of C++ also influences that of C.

- Today's C has also deprecated the unsafe function declarations, and has adopted better type checking.

5/37

## CS100 Lecture 11

C++ Introduction, Strings

1/37

## The birth of C++

After C with Classes was seen as a "medium success" by Stroustrup, he moved on to make a better new language - C++ (1983).

~~C++ is an object-oriented programming language.~~

C++ is a **multi-paradigm** programming language that

- is a better C,
- supports data abstraction, and that
- supports object-oriented programming.

6/37

## Contents

- Brief history of C++
- Basic IO
- Standard library
- `std::string`

2/37

## Standardization of C++

Standardization: C++98, C++11, C++14, C++17, C++20, C++23, C++26, ...

- C++98: The first ISO standard in 1998.
- C++11: Marks the beginning of **modern C++**.
- C++14/17: Some slight fixes and improvements of C++11.
- C++20: The first standard that delivers on virtually all the features that Bjarne Stroustrup dreamed of in *The Design and Evolution of C++* in 1994.
  - "D&E Complete"

CS100 is based on C++17.

7/37

## Brief history of C++

## Overview of C++

What do **embedded systems**, **game development**, **high frequency trading**, and **particle accelerators** have in common? - C++, of course!

*Effective C++* Item 1 (by Scott Meyers): **View C++ as a federation of languages.**

The easiest way is to view C++ not as a single language but as a federation of related languages ... Fortunately, there are only four:

- C.
- Object-Oriented C++.
- Template C++.
- The STL.

3/37

8/37

## "Better C"

Safer & more reasonable designs.

- `bool`, `true` and `false` are built-in. No need to `#include <stdbool.h>`. `true` and `false` are of type `bool`, not `int`.
- The return type of logical operators `&&`, `||`, `!` and comparison operators `<`, `<=`, `>`, `>=`, `==`, `!=` is `bool`, not `int`.
- The type of string literals `"hello"` is `const char [N+1]`, not `char [N+1]`.
- The type of character literals `'a'` is `char`, not `int`.

9/37

## "Better C"

Safer & more reasonable designs.

- Potentially dangerous type conversions are not allowed to happen implicitly. **They are errors, not just warnings.**
- `const` variables initialized with literals are compile-time constants. They can be used as the length of arrays.
- `int fun()` declares a function accepting no arguments. It is not accepting unknown arguments.

10/37

## Basic IO

### Hello world

C: Use `printf`

```
#include <stdio.h>

int main(void) {
 printf("Hello world\n");
 return 0;
}
```

C++: Use `std::cout`

```
#include <iostream>

int main() { // just an empty `()`
 std::cout << "Hello world\n";
 return 0;
}
```

Note on the `main` function: In C++, a function declared with an empty parameter list **accepts no arguments**, so there is no need to put a `void` there.

12/37

### A+B problem

C:

```
#include <stdio.h>

int main(void) {
 int a, b;
 scanf("%d%d", &a, &b);
 printf("a + b = %d\n", a + b);
 return 0;
}
```

C++:

```
#include <iostream>

int main() {
 int a, b;
 std::cin >> a >> b;
 std::cout << "a + b = " << a + b
 << '\n';
 return 0;
}
```

- For input: **There is no need to take the address of `a` and `b`!** C++ has a way to obtain the *reference* of the argument.
- **There is no need to write `%d`!** C++ has a way of identifying the type of the argument, and will select the correct way to handle that type.

13/37

## IOStream: Input and Output Stream

```
#include <iostream>

int main() {
 int a, b;
 std::cin >> a >> b;
 std::cout << "a + b = " << a + b << '\n';
 return 0;
}
```

`std::cin` and `std::cout`: two objects defined in the standard library file `<iostream>`.

- They are not **functions**.
- The input and output "functions" are actually the operators `<<` and `>>`, which are **overloaded** to do something different from bit shifting.
  - We will learn about operator overloading in later lectures.

14/37

## IOStream: Input and Output Stream

```
#include <iostream>

int main() {
 int a, b;
 std::cin >> a >> b;
 std::cout << "a + b = " << a + b << '\n';
 return 0;
}
```

`std::cin` and `std::cout`: two objects defined in the standard library file `<iostream>`.

- `std::cin` stands for standard input stream. `std::cout` stands for the standard output stream.

15/37

## IOStream: Input and Output Stream

`std::cin >> x`: Reads something and stores it in the variable `x`.

- `x` can be of any supported type: integers, floating-points, characters, strings, ...
- C++ has a way of identifying the type of `x` and selecting the correct way to read it. We don't need the annoying `"%d"`, `"%f"`, ... anymore.
- C++ functions have a way of obtaining the *reference* of the argument. We don't need to take the address of `x`.

16/37

## IOStream: Input and Output Stream

- `std::cin >> x` returns `std::cin`, so we can write several reads in a chained way:
  - `std::cin >> x >> y >> z` is equivalent to `((std::cin >> x) >> y) >> z`, which is equivalent to

```
std::cin >> x;
std::cin >> y;
std::cin >> z;
```

- Similarly, outputs can also be chained: `std::cout << x << y << z` is equivalent to

```
std::cout << x;
std::cout << y;
std::cout << z;
```

17/37

## Standard library

18/37

## Standard library file names

The names of C++ standard library files have **no extensions**: `<iostream>` instead of `<iostream.h>`, `<string>` instead of `<string.h>`.

19/37

## Namespace `std`

`std::cin` and `std::cout`: names from the standard library.

C++ has a large standard library with a lot of names declared.

To avoid **name collisions**, all the names from the standard library are placed in a namespace named `std`.

- Example of name collisions in C: Suppose we want to write our own quick-sort:

```
#include <stdlib.h>
void qsort(int *a, int n) { // Oops! stdlib already has one named `qsort`.
 // ...
}
```

20/37

## Namespace `std`

`std::cin` and `std::cout`: names from the standard library.

C++ has a large standard library with a lot of names declared.

To avoid **name collisions**, all the names from the standard library are placed in a namespace named `std`.

- You can write `using std::cin;` to introduce `std::cin` into the current scope, so that `cin` can be used without `std::`.
- You may write `using namespace std;` to introduce **all the names in `std`** into the current scope, but **you will be at the risk of name collisions again**.

21/37

## Compatibility with C standard library

The C++ standard library has everything from the C standard library, **but not exactly the same as in C**.

- For some historical issues, C has many awkward designs and weird defaults, some of which are **not compatible** in C++.
- The C++ version of a C standard library file `<xxx.h>` is `<cxxx>`, with all the names also introduced into namespace `std`.

```
#include <cstdio>
int main() {
 int a, b; std::scanf("%d%d", &a, &b);
 std::printf("%d\n", a + b);
}
```

**[Best practice]** Use `<cxxx>` instead of `<xxx.h>` when you need the C standard library in C++.

22/37

## `std::string`

Defined in the standard library file `<string>` (not `<string.h>`, not `<cstring>` !!)

The string.

## Define and initialize a string

```
std::string str = "Hello world";
// equivalent: std::string str("Hello world");
// equivalent: std::string str("Hello world"); (modern)
std::cout << str << std::endl;

std::string s1(7, 'a');
std::cout << s1 << std::endl; // aaaaaa

std::string s2 = s1; // s2 is a copy of s1
std::cout << s2 << std::endl; // aaaaaa

std::string s; // "" (empty string)
```

Default-initialization of a `std::string` will produce an **empty string**, not indeterminate value and has no undefined behaviors!

24/37

## Strings

- The memory of `std::string` is **allocated and deallocated automatically**.
- We can insert or erase characters in a `std::string`. **The memory of storage will be adjusted automatically**.
- `std::string` **does not need an explicit `'\0'` at the end**. It has its way of recognizing the end.
- When you use `std::string`, **pay attention to its contents** instead of the implementation details.

25/37

## Length of a string

### Member function `s.size()`

```
std::string str{"Hello world"};
std::cout << str.size() << std::endl;
```

Not `strlen`, not `sizeof` !!

### Member function `s.empty()`

```
if (str.empty()) {
 // ...
}
```

26/37

## Concatenation of strings

Use `+` and `+=` directly!

- No need to care about the memory allocation.
- No awkward functions like `strcat`.

```
std::string s1 = "Hello";
std::string s2 = "world";
std::string s3 = s1 + ' ' + s2; // "Hello world"
s1 += s2; // s1 becomes "Helloworld"
s2 += "C++string"; // s2 becomes "worldC++string"
```

27/37

## Concatenation of strings

At least one operand of `+` should be `std::string`.

```
const char *old_bad_ugly_C_style_string = "hello";
std::string good_beautiful_Cpp_string = "hello";

std::string s1 = good_beautiful_Cpp_string + "aaaaa"; // OK.
std::string s2 = "aaaaa" + good_beautiful_Cpp_string; // OK.
std::string s3 = old_bad_ugly_C_style_string + "aaaaa"; // Error
```

Is this ok?

```
std::string hello{"hello"};
std::string s = hello + "world" + "C++";
```

23/37

28/37

## Concatenation of strings

At least one operand of `+` should be `std::string`.

```
const char *old_bad_ugly_C_style_string = "hello";
std::string good_beautiful_Cpp_string = "hello";

std::string s1 = good_beautiful_Cpp_string + "aaaaa"; // OK.
std::string s2 = "aaaaa" + good_beautiful_Cpp_string; // OK.
std::string s3 = old_bad_ugly_C_style_string + "aaaaa"; // Error
```

Is this ok?

```
std::string hello{"hello"};
std::string s = hello + "world" + "C++";
```

Yes! `+` is left-associated. `(hello + "world")` is of type `std::string`.

29/37

## Use `+=`

In C, `a = a + b` is equivalent to `a += b`. This is not always true in C++.

For two `std::string s s1` and `s2`, `s1 = s1 + s2` is different from `s1 += s2`.

- `s1 = s1 + s2` constructs a temporary object `s1 + s2` (so that the contents of `s1` are copied), and then assigns it to `s1`.
- `s1 += s2` appends `s2` directly to the end of `s1`, without copying `s1`.

Try these with `n = 1000000`:

```
std::string result;
for (int i = 0; i != n; ++i)
 result += 'a'; // Fast

std::string result;
for (int i = 0; i != n; ++i)
 result = result + 'a'; // Very slow
```

30/37

## Lexicographical comparison of strings

Just use `<`, `<=`, `>`, `>=`, `==`, `!=`!

- No loops. No weird functions like `strcmp`.

## Copying a string

Just use `=`!

```
std::string s1{"Hello"};
std::string s2{"world"};
s2 = s1; // s2 is a copy of s1
s1 += 'a'; // s2 is still "Hello"
```

The copy assignment operator of `std::string` will copy the contents of it.

⇒ We will learn about copy assignment operators in later lectures.

31/37

## String IO

Use `std::cin >> s` and `std::cout << s`, as simple as handling an integer.

- Does `std::cin >> s` ignore leading whitespaces? Does it read an entire line or just a sequence of non-whitespace characters? Do some experiments on it.

`std::getline(std::cin, s)`: Reads a string starting from the current character, and stops at the first `'\n'`.

- Is the ending `'\n'` consumed? Is it stored? Do some experiments.

32/37

## Traversing a string: Use range-based `for` loops.

Example: Print all the uppercase letters in a string.

```
for (char c : s) // The range-based for loops
 if (std::isupper(c)) // in <cctype>
 std::cout << c;
std::cout << std::endl;
```

Equivalent way: Use subscripts, which is verbose and inconvenient.

```
for (std::size_t i = 0; i != s.size(); ++i)
 if (std::isupper(s[i]))
 std::cout << s[i];
std::cout << std::endl;
```

[Best practice] Use range-based `for` loops. They are modern, clear, simple, generic, and hence more recommended.

⇒ More about range-based `for` loops in later lectures.

33/37

## Conversion between strings and arithmetic numbers

For a number `x` of any arithmetic type, `std::to_string(x)` returns a string representing it.

```
int ival = 42;
double dval = 3.14;
std::string s = std::to_string(ival) + std::to_string(dval);
std::cout << s << '\n'; // Possible output: 423.140000
```

`std::stoi(s)`, `std::stol(s)`, ...: Extracts the arithmetic value represented by `s`.

See [this list](#).

34/37

## Summary

- IO: Use `<iostream>`, `std::cin >> x >> y`, `std::cout << x << y`.
- Namespace `std`.
- C++ standard library files: names with no extensions.
- Compatible with C, but use `<xxxx>` instead of `<xxx.h>`.

35/37

## Summary

`std::string`

- No need for a terminating `'\0'`.
- Automatic memory management.
- `s.size()` returns the length. `s.empty()` returns whether `s` is empty.
- Use `+` and `+=` for concatenation. Use `<`, `<=`, `>`, `>=`, `==`, `!=` for comparison. Use `=` for copying.

36/37

## Summary

`std::string`

- Use `>>` and `<<` for IO, as well as `std::getline`.
- Use `s[i]` to access the elements.
- Use range-based `for` loops to traverse a string.
- Use `std::to_string` and `std::stoi`, `std::stol`, ... for numeric conversions.
- Full list of functions related to `std::string`:  
[https://en.cppreference.com/w/cpp/string/basic\\_string](https://en.cppreference.com/w/cpp/string/basic_string)

37/37

## CS100 Lecture 12

References, `std::vector`

1/49

## Contents

- References
- `std::vector`

2/49

## References

7/49

### Declare a reference

A **reference** defines an **alternative name** for an object ("refers to" that object).

Similar to pointers, the type of a reference is `ReferredType &`, which consists of two things:

- `ReferredType` is the type of the object that it refers to, and
- `&` is the symbol indicating that it is a reference.

Example:

```
int ival = 42;
int &ri = ival; // `ri` refers to `ival`.
++x; // This has nothing to do with `ival`.
std::cout << ri << '\n'; // prints the value of `ival`, which is `42`.
++ri; // Same effect as `++ival`.
```

4/49

### Declare a reference

```
int ival = 42;
int x = ival; // `x` is another variable.
++x; // This has nothing to do with `ival`.
std::cout << ival << '\n'; // 42
int &ri = ival; // `ri` is a reference that refers to `ival`.
++ri; // This modification is performed on `ival`.
std::cout << ival << '\n'; // 43
```

Ordinarily, when we initialize a variable, the value of the initializer is **copied** into the object we are creating.

When we define a reference, instead of copying the initializer's value, we **bind** the reference to its initializer.

5/49

### A reference is an alias

When we define a reference, instead of copying the initializer's value, we **bind** the reference to its initializer.

```
int ival = 42;
int &ri = ival;
++ri; // Same as `++ival`.
ri = 50; // Same as `ival = 50`.
int a = ri + 1; // Same as `int a = ival + 1`.
```

After a reference has been defined, **all** operations on that reference are actually operations on the object to which the reference is bound.

```
ri = a;
```

What is the meaning of this?

6/49

### A reference is an alias

```
int ival = 42;
int &ri = ival;
++ri; // Same as `++ival`.
ri = 50; // Same as `ival = 50`.
int a = ri + 1; // Same as `int a = ival + 1`.
```

When we define a reference, instead of copying the initializer's value, we **bind** the reference to its initializer.

After a reference has been defined, **all** operations on that reference are actually operations on the object to which the reference is bound.

```
ri = a;
```

- This is the same as `ival = a;`. It is not rebinding `ri` to refer to `a`.

7/49

### A reference must be initialized

```
ri = a;
```

- This is the same as `ival = a;`. It is not rebinding `ri` to refer to `a`.

Once initialized, a reference remains bound to its initial object. **There is no way to rebind a reference to refer to a different object.**

Therefore, **references must be initialized.**

3/49

8/49

### References must be bound to *existing objects* ("lvalues")

It is not allowed to bind a reference to temporary objects or literals <sup>1</sup>:

```
int &r1 = 42; // Error: binding a reference to a literal
int &r2 = 2 + 3; // Error: binding a reference to a temporary object
int a = 10, b = 15;
int &r3 = a + b; // Error: binding a reference to a temporary object
```

In fact, the references we learn today are "lvalue references", which must be bound to *lvalues*. We will talk about *value categories* in later lectures.

9/49

### References are not objects

A reference is an alias. It is only an alternative name of another object, but the reference itself is **not an object**.

Therefore, there are no "references to references".

```
int ival = 42;
int &ri = ival; // binding `ri` to `ival`.
int &&rr = ri; // Error! No such thing!
```

What is the meaning of this code? Does it compile?

```
int &ri2 = ri;
```

10/49

### References are not objects

A reference is an alias. It is only an alternative name of another object, but the reference itself is **not an object**.

Therefore, there are no "references to references".

```
int ival = 42;
int &ri = ival; // binding `ri` to `ival`.
int &&rr = ri; // Error! No such thing!
```

What is the meaning of this code? Does it compile?

```
int &ri2 = ri; // Same as `int &ri2 = ival`.
```

- `ri2` is a reference that is bound to `ival`.
- Any use of a reference is actually using the object that it is bound to!

11/49

# References are not objects

A reference is an alias. It is only an alternative name of another object, but the reference itself is **not an object**.

Pointers must also point to objects. Therefore, there are no "pointers to references".

```
int ival = 42;
int &ri = ival; // binding `ri` to `ival`.
int &*pr = &ri; // Error! No such thing!
```

What is the meaning of this code? Does it compile?

```
int *pi = &ri;
```

12/49

# References are not objects

A reference is an alias. It is only an alternative name of another object, but the reference itself is **not an object**.

Pointers must also point to objects. Therefore, there are no "pointers to references".

```
int ival = 42;
int &ri = ival; // binding `ri` to `ival`.
int &*pr = ri; // Error! No such thing!
```

What is the meaning of this code? Does it compile?

```
int *pi = &ri; // Same as `int *pi = &ival`.
```

13/49

# Reference declaration

Similar to pointers, the ampersand `&` only applies to one identifier.

```
int ival = 42, &ri = ival, *pi = &ival;
// `ri` is a reference of type `int &`, which is bound to `ival`.
// `pi` is a pointer of type `int *`, which points to `ival`.
```

Placing the ampersand near the referred type does not make a difference:

```
int& x = ival, y = ival, z = ival;
// Only `x` is a reference. `y` and `z` are of type `int`.
```

14/49

# \* and &

Both symbols have many identities!

- In a declaration like `Type *x = expr`, `*` is a part of the pointer type `Type *`.
- In a declaration like `Type &r = expr`, `&` is a part of the reference type `Type &`.
- In an expression like `*opnd` where there is only one operand, `*` is the dereference operator.
- In an expression like `&opnd` where there is only one operand, `&` is the address-of operator.
- In an expression like `a * b` where there are two operands, `*` is the multiplication operator.
- In an expression like `a & b` where there are two operands, `&` is the bitwise-and operator.

15/49

# Example: Use references in range- for

Recall the range-based `for` loops (range- `for`):

```
std::string str;
std::cin >> str;
int lower_cnt = 0;
for (char c : str)
 if (std::islower(c))
 ++lower_cnt;
std::cout << "There are " << lower_cnt << " lowercase letters in total.\n";
```

The range- `for` loop in the code above traverses the string, and declares and initializes the variable `c` in each iteration as if <sup>2</sup>

```
for (std::size_t i = 0; i != str.size(); ++i) {
 char c = str[i]; // Look at this!
 if (std::islower(c))
 ++lower_cnt;
}
```

16/49

# Example: Use references in range- for

```
for (char c : str)
 // ...
```

The range- `for` loop in the code above traverses the string, and declares and initializes the variable `c` in each iteration as if <sup>2</sup>

```
for (std::size_t i = 0; i != str.size(); ++i) {
 char c = str[i];
 // ...
}
```

Here `c` is a copy of `str[i]`. Therefore, modification on `c` does not affect the contents in `str`.

17/49

# Example: Use references in range- for

What if we want to change all lowercase letters to their uppercase forms?

```
for (char c : str)
 c = std::toupper(c); // This has no effect.
```

We need to declare `c` as a reference.

```
for (char &c : str)
 c = std::toupper(c);
```

This is the same as

```
for (std::size_t i = 0; i != str.size(); ++i) {
 char &c = str[i];
 c = std::toupper(c); // Same as `str[i] = std::toupper(str[i]);`.
}
```

18/49

# Example: Pass by reference-to- const

Write a function that accepts a string and returns the number of lowercase letters in it:

```
int count_lowercase(std::string str) {
 int cnt = 0;
 for (char c : str)
 if (std::islower(c))
 ++cnt;
 return cnt;
}
```

To call this function:

```
int result = count_lowercase(my_string);
```

19/49

# Example: Pass by reference-to- const

```
int count_lowercase(std::string str) {
 int cnt = 0;
 for (char c : str)
 if (std::islower(c))
 ++cnt;
 return cnt;
}
```

```
int result = count_lowercase(my_string);
```

When passing `my_string` to `count_lowercase`, the parameter `str` is initialized as if

```
std::string str = my_string;
```

The contents of the entire string `my_string` are copied!

20/49

# Example: Pass by reference-to- const

```
int result = count_lowercase(my_string);
```

When passing `my_string` to `count_lowercase`, the parameter `str` is initialized as if

```
std::string str = my_string;
```

The contents of the entire string `my_string` are copied! Is this copy necessary?

21/49

### Example: Pass by reference-to- `const`

```
int result = count_lowercase(my_string);
```

When passing `my_string` to `count_lowercase`, the parameter `str` is initialized as if

```
std::string str = my_string;
```

The contents of the entire string `my_string` are copied! This copy is unnecessary, because `count_lowercase` is a read-only operation on `str`.

How can we avoid this copy?

22/49

### Example: Pass by reference-to- `const`

```
int count_lowercase(std::string &str) { // `str` is a reference.
 int cnt = 0;
 for (char c : str)
 if (std::islower(c))
 ++cnt;
 return cnt;
}
```

```
int result = count_lowercase(my_string);
```

When passing `my_string` to `count_lowercase`, the parameter `str` is initialized as if

```
std::string &str = my_string;
```

Which is just a reference initialization. No copy is performed.

23/49

### Example: Pass by reference-to- `const`

```
int count_lowercase(std::string &str) { // `str` is a reference.
 int cnt = 0;
 for (char c : str)
 if (std::islower(c))
 ++cnt;
 return cnt;
}
```

However, this has a problem:

```
std::string s1 = something(), s2 = some_other_thing();
int result = count_lowercase(s1 + s2); // Error: binding reference to
// a temporary object.
```

`a + b` is a temporary object, which `str` cannot be bound to.

24/49

### Example: Pass by reference-to- `const`

References must be bound to existing objects, not literals or temporaries.

There is an exception to this rule: References-to- `const` can be bound to anything.

```
const int &rci = 42; // OK.
const std::string &rscs = a + b; // OK.
```

`rscs` is bound to the temporary object returned by `a + b` as if

```
std::string tmp = a + b;
const std::string &rscs = tmp;
```

⇒ We will talk more about references-to- `const` in recitations.

25/49

### Example: Pass by reference-to- `const`

The answer:

```
int count_lowercase(const std::string &str) { // `str` is a reference-to-`const`.
 int cnt = 0;
 for (char c : str)
 if (std::islower(c))
 ++cnt;
 return cnt;
}
```

```
std::string a = something(), b = some_other_thing();
int res1 = count_lowercase(a); // OK.
int res2 = count_lowercase(a + b); // OK.
int res3 = count_lowercase("hello"); // OK.
```

26/49

### Benefits of passing by reference-to- `const`

Apart from the fact that it avoids copy, declaring the parameter as a reference-to- `const` also prevents some potential mistakes:

```
int some_kind_of_counting(const std::string &str, char value) {
 int cnt = 0;
 for (std::size_t i = 0; i != str.size(); ++i) {
 if (str[i] == value) // Oops! It should be `==`.
 ++cnt;
 else {
 // do something ...
 // ...
 }
 }
 return cnt;
}
```

`str[i] = value` will trigger a compile-error, because `str` is a reference-to- `const`.

27/49

### Benefits of passing by reference-to- `const`

1. Avoids copy.
2. Accepts temporaries and literals (*rvalues*).
3. The `const` qualification prevents accidental modifications to it.

[Best practice] Pass by reference-to- `const` if copy is not necessary and the parameter should not be modified.

28/49

### References vs pointers

A reference

- is not itself an object. It is an alias of the object that it is bound to.
- cannot be rebound to another object after initialization.
- has no "default" or "zero" value. It must be bound to an object.

A pointer

- is an object that stores the address of the object it points to.
- can switch to point to another object at any time.
- can be set to a null pointer value `nullptr`.

Both a reference and a pointer can be used to refer to an object, but references are more convenient - no need to write the annoying `*` and `&`.

Note: `nullptr` is *the* null pointer value in C++. Do not use `NULL`.

29/49

### `std::vector`

Defined in the standard library file `<vector>`.

A "dynamic array".

30/49

### Class template

`std::vector` is a class template.

Class templates are not themselves classes. Instead, they can be thought of as instructions to the compiler for *generating* classes.

- The process that the compiler uses to create classes from the templates is called **instantiation**.

For `std::vector`, what kind of class is generated depends on the type of elements we want to store, often called **value type**. We supply this information inside a pair of angle brackets following the template's name:

```
std::vector<int> v; // `v` is of type `std::vector<int>`
```

31/49

## Create a `std::vector`

`std::vector` is not a type itself. It must be combined with some `<T>` to form a type.

```
std::vector v; // Error: missing template argument.
std::vector<int> vi; // An empty vector of `int`s.
std::vector<std::string> vs; // An empty vector of strings.
std::vector<double> vd; // An empty vector of `double`s.
std::vector<std::vector<int>> vvi; // An empty vector of vector of `int`s.
 // "2-d" vector.
```

What are the types of `vi`, `vs` and `vvi`?

32/49

## Create a `std::vector`

`std::vector` is not a type itself. It must be combined with some `<T>` to form a type.

```
std::vector v; // Error: missing template argument.
std::vector<int> vi; // An empty vector of `int`s.
std::vector<std::string> vs; // An empty vector of strings.
std::vector<double> vd; // An empty vector of `double`s.
std::vector<std::vector<int>> vvi; // An empty vector of vector of `int`s.
 // "2-d" vector.
```

What are the types of `vi`, `vs` and `vvi`?

- `std::vector<int>`, `std::vector<std::string>`, `std::vector<std::vector<int>>`.

33/49

## Create a `std::vector`

There are several common ways of creating a `std::vector`:

```
std::vector<int> v{2, 3, 5, 7}; // A vector of `int`s,
 // whose elements are {2, 3, 5, 7}.
std::vector<int> v2 = {2, 3, 5, 7}; // Equivalent to ↑
std::vector<std::string> vs{"hello", "world"}; // A vector of strings,
 // whose elements are {"hello", "world"}.
std::vector<std::string> vs2 = {"hello", "world"}; // Equivalent to ↑
std::vector<int> v3(10); // A vector of ten `int`s, all initialized to 0.
std::vector<int> v4(10, 42); // A vector of ten `int`s, all initialized to 42.
```

Note that all the elements in `v3` are initialized to `0`.

- We hate uninitialized values, so does the standard library.

34/49

## Create a `std::vector`

Create a `std::vector` as a copy of another one:

```
std::vector<int> v{2, 3, 5, 7};
std::vector<int> v2 = v; // v2 is a copy of v
std::vector<int> v3(v); // Equivalent
std::vector<int> v4{v}; // Equivalent
```

No need to write a loop!

Copy assignment is also enabled:

```
std::vector<int> v1 = something(), v2 = something_else();
v1 = v2;
```

- Element-wise copy is performed automatically.
- Memory is allocated automatically. The memory used to store the old data of `v1` is deallocated automatically.

35/49

## C++17 CTAD

"Class Template Argument Deduction": As long as enough information is supplied in the initializer, the value type can be deduced automatically by the compiler.

```
std::vector v1{2, 3, 5, 7}; // vector<int>
std::vector v2{3.14, 6.28}; // vector<double>
std::vector v3(10, 42); // vector<int>, deduced from 42 (int)
std::vector v4(10); // Error: cannot deduce template argument type
```

36/49

## Size of a `std::vector`

`v.size()` and `v.empty()`: same as those on `std::string`.

```
std::vector v{2, 3, 5, 7};
std::cout << v.size() << '\n';
if (v.empty()) {
 // ...
}
```

`v.clear()`: Remove all the elements.

37/49

## Append an element to the end of a `std::vector`

`v.push_back(x)`

```
int n;
std::cin >> n;
std::vector<int> v;
for (int i = 0; i != n; ++i) {
 int x;
 std::cin >> x;
 v.push_back(x);
}
std::cout << v.size() << '\n'; // n
```

38/49

## Remove the last element of a `std::vector`

`v.pop_back()`

Exercise: Given `v` of type `std::vector<int>`, remove all the consecutive even numbers in the end.

## Remove the last element of a `std::vector`

`v.pop_back()`

Exercise: Given `v` of type `std::vector<int>`, remove all the consecutive even numbers in the end.

`v.pop_back()`

Exercise: Given `v` of type `std::vector<int>`, remove all the consecutive even numbers in the end.

```
while (!v.empty() && v.back() % 2 == 0)
 v.pop_back();
```

`v.back()`: returns the *reference* to the last element.

- How is it different from "returning the *value* of the last element"?

40/49

## `v.back()` and `v.front()`

Return the references to the last and the first elements, respectively.

It is a *reference*, through which we can modify the corresponding element.

```
v.front() = 42;
++v.back();
```

For `v.back()`, `v.front()` and `v.pop_back()`, the behavior is undefined if `v` is empty. They do not perform any bounds checking.

41/49



## Range-based for loops

A `std::vector` can also be traversed using a range-based `for` loop.

```
std::vector<int> vi = some_values();
for (int x : vi)
 std::cout << x << std::endl;
std::vector<std::string> vs = some_strings();
for (const std::string &s : vs) // use reference-to-const to avoid copy
 std::cout << s << std::endl;
```

Exercise: Use range-based `for` loops to count the number of uppercase letters in a `std::vector<std::string>`.

42/49

## Range-based for loops

Exercise: Use range-based `for` loops to count the number of uppercase letters in a `std::vector<std::string>`.

```
int cnt = 0;
for (const std::string &s : vs) { // Use reference-to-const to avoid copy
 for (char c : s) {
 if (std::isupper(c))
 ++cnt;
 }
}
```

43/49

## Access through subscripts

`v[i]` returns the **reference** to the element indexed `i`.

- $i \in [0, N)$ , where  $N = v.size()$ .
- Subscript out of range is **undefined behavior**. `v[i]` performs no bounds checking.
  - In pursuit of efficiency, most operations on standard library containers do not perform bounds checking.
- A kind of "subscript" that has bounds checking: `v.at(i)`.
  - If `i` is out of range, a `std::out_of_range` exception is thrown.

44/49

## Feel the style of STL

Basic and low-level operations are performed automatically:

- Default initialization of `std::string` and `std::vector` results in an empty string / container, not indeterminate values.
- Copy of `std::string` and `std::vector` is done automatically, which performs member-wise copy.
- Memory management is done automatically.

Interfaces are consistent:

- `std::string` also has member functions like `.push_back(x)`, `.pop_back()`, `.at(i)`, `.size()`, `.clear()`, etc. which do the same things as on `std::vector`.
- Both can be traversed by range-`for`.

45/49

## Summary

References

- A reference is an alias.
- A reference is bound to an object during initialization. After that, any use of that reference is actually using the the object it is bound to.
- A reference can only be bound to existing objects (*lvalues*). A pointer can only point to existing objects.
  - But a reference-to-`const` can be bound to anything.
- Pass arguments by reference-to-`const` : avoids copy, accepts both lvalues and rvalues, and prevents accidental modification on what should not be modified.

46/49

## Summary

`std::vector`

- `std::vector` is not a type. It must be combined with some `<T>` to form a type.
- Many ways of creation.
- Copy of a `std::vector` performs member-wise copy.
- `v.size`, `v.empty`, `v.push_back`, `v.pop_back`, `v.clear`, `v[i]`, `v.at(i)`.
- Use range-`for` to traverse a `std::vector`.

47/49

## Exercises

Write the exercises on page 26, 38, 40 and 43 on your own.

48/49

## Notes

<sup>1</sup> String literals ( `"hello"` ) are an exception to this. Integer literals, floating-point literals, character literals, boolean literals and `enum` items are rvalues, but string literals are lvalues. They do live somewhere in the memory.

<sup>2</sup> In fact, the range-`for` uses **iterators**, not subscripts.

49/49

## CS100 Lecture 13

"C" in C++

1/46

## Contents

"C" in C++

- Type System
  - Stronger Type Checking
  - Explicit Casts
  - Type Deduction
- Functions
  - Default Arguments
  - Function Overloading
- Range-Based `for` Loops Revisited

2/46

## "Better C"

C++ was developed based on C.

From *The Design and Evolution of C++*:

C++ is a general-purpose programming language that

- is a better C,
- supports data abstraction,
- supports object-oriented programming.

C++ brought up new ideas and improvements of C, some of which also in turn influenced the development of C.

3/46

## "Better C"

- `bool`, `true` and `false` are built-in. No need to `#include <stdbool.h>`. `true` and `false` are of type `bool`, not `int`.
  - This is also true since C23.
- The return type of logical operators `&&`, `||`, `!` and comparison operators `<`, `<=`, `>`, `>=`, `==`, `!=` is `bool`, not `int`.
- The type of string literals `"hello"` is `const char [N+1]`, not `char [N+1]`.
  - Recall that string literals are stored in **read-only memory**. Any attempt to modify them results in undefined behavior.
- The type of character literals `'a'` is `char`, not `int`.

4/46

## "Better C"

- `const` variables initialized with literals are compile-time constants. They can be used as the length of arrays.

```
const int maxn = 1000;
int a[maxn]; // a normal array in C++, but VLA in C
```

- `int fun()` declares a function accepting no arguments. It is not accepting unknown arguments.
  - This is also true since C23.

5/46

## Type System

### Stronger type checking

Some arithmetic conversions are problematic: They are not value-preserving.

```
int x = some_int_value();
long long y = x; // OK. Value-preserving
long long z = some_long_long_value();
int w = z; // Is this OK?
```

- Conversion from `int` to `long long` is value-preserving, without doubt.
- Conversion from `long long` to `int` may lose precision. ("narrowing")

However, no warning or error is generated for such conversions in C.

7/46

### Stronger type checking

Some arithmetic conversions are problematic: They are not value-preserving.

```
long long z = some_long_long_value();
int w = z; // "narrowing" conversion
```

Stroustrup had decided to ban all implicit narrowing conversions in C++. However,

The experiment failed miserably. Every C program I looked at contained large numbers of assignments of `int`s to `char` variables. Naturally, since these were **working programs**, most of these assignments were perfectly safe. That is, either the value was small enough not to become truncated, or the truncation was expected or at least harmless in that particular context.

In the end, narrowing conversions are not banned completely in C++. They are not allowed only in a special context in modern C++. We will see it soon.

8/46

### Stronger type checking

Some type conversions (casts) can be very dangerous:

```
const int x = 42, *pci = &x;
int *pi = pci; // Warning in C, Error in C++
++*pi; // undefined behavior
char *pc = pi; // Warning in C, Error in C++
void *pv = pi; char *pc2 = pv; // Even no warning in C! Error in C++
int y = pc; // Warning in C, Error in C++
```

- For `T ≠ U`, `T *` and `U *` are different types. Treating a `T *` as `U *` leads to undefined behavior in most cases, but the C compiler gives only a warning!
- `void *` is a hole in the type system. You can cast anything to and from it **without even a warning**.

C++ does not allow the dangerous type conversions to happen *implicitly*.

9/46

### Explicit Casts

C++ provides four **named cast operators**:

- `static_cast<Type>(expr)`
- `const_cast<Type>(expr)`
- `reinterpret_cast<Type>(expr)`
- `dynamic_cast<Type>(expr)` ⇒ will be covered in later lectures.

In contrast, the C style explicit cast `(Type)expr` looks way too innocent.

"An ugly behavior should have an ugly looking."

10/46

### `const_cast`

Cast away low-level constness (**DANGEROUS**):

```
int ival = 42;
const int &cref = ival;
int &ref = cref; // Error: casting away low-level constness
int &ref2 = const_cast<int &>(cref); // OK
int *ptr = const_cast<int *>(&cref); // OK
```

However, modifying a `const` object through a non-`const` access path (possibly formed by `const_cast`) results in **undefined behavior**!

```
const int cival = 42;
int &ref = const_cast<int &>(cival); // compiles, but dangerous
++ref; // undefined behavior (may crash)
```

6/46

11/46

### `reinterpret_cast`

Often used to perform conversion between different pointer types (**DANGEROUS**):

```
int ival = 42;
char *pc = reinterpret_cast<char *>(&ival);
```

We must never forget that the actual object addressed by `pc` is an `int`, not a character! Any use of `pc` that assumes it's an ordinary character pointer is **likely to fail** at run time, e.g.:

```
std::string str(pc); // undefined behavior
```

Wherever possible, do not use it!

12/46

## static\_cast

Other types of conversions (which often look "harmless"):

```
double average = static_cast<double>(sum) / n;
int pos = static_cast<int>(std::sqrt(n));
```

Some typical usage: ⇒ We will talk about them in later lectures.

```
static_cast<std::string &&>(str) // converts to a xvalue
static_cast<Derived *>(base_ptr) // downcast without runtime checking
```

13/46

## Minimize casting

[Best practice] Minimize casting. (*Effective C++ Item 27*)

Type systems work as a **guard** against possible errors: Type mismatch often indicates a logical error.

[Best practice] When casting is necessary, prefer C++-style casts to old C-style casts.

- With old C-style casts, you can't even tell whether it is dangerous or not!

14/46

## Type deduction

C++ is very good at **type computations**:

```
std::vector v(10, 42);
```

- It should be `std::vector<int> v(10, 42);`, but the compiler can deduce that `int` from `42`.

```
int x = 42; double d = 3.14; std::string s = "hello";
std::cout << x << d << s;
```

- The compiler can detect the types of `x`, `d` and `s` and select the correct printing functions.

15/46

## auto

When declaring a variable with an initializer, we can use the keyword `auto` to let the compiler deduce the type.

```
auto x = 42; // `int`, because 42 is an `int`.
auto y = 3.14; // `double`, because 3.14 is a `double`.
auto z = x + y; // `double`, because the type of `x + y` is `double`.
auto m;
```

`auto` can also be used to produce compound types:

```
auto &r = x; // `int &`, because `x` is an `int`.
const auto &rc = r; // `const int &`.
auto *p = &rc; // `const int *`, because `&rc` is `const int *`.
```

16/46

## auto

What about this?

```
auto str = "hello";
```

17/46

## auto

What about this?

```
auto str = "hello"; // `const char *`
```

- Recall that the type of `"hello"` is `const char [6]`, not `std::string`. This is for compatibility with C.
- When using `auto`, the array-to-pointer conversion ("decay") is performed automatically.

18/46

## auto

Deduction of return type is also allowed (since C++14):

```
auto sum(int x, int y) {
 return x + y;
}
```

- The return type is deduced to `int`.

Since C++20, `auto` can also be used for function parameters! Such a function is actually a function template.

- This is beyond the scope of CS100.

```
auto sum(auto x, auto y) {
 return x + y;
}
```

19/46

## auto

`auto` lets us enjoy the benefits of the static type system.

Some types in C++ are very long:

```
std::vector<std::string>::const_iterator it = vs.begin();
```

Use `auto` to simplify it:

```
auto it = vs.begin();
```

20/46

## auto

`auto` lets us enjoy the benefits of the static type system.

Some types in C++ are not known to anyone but the compiler:

```
auto lam = [](int x, int y) { return x + y; } // A lambda expression.
```

Every lambda expression has its own type, whose name is only known by the compiler.

21/46

## decltype

`decltype(expr)` will deduce the type of the expression `expr` **without evaluating it**.

```
auto fun(int a, int b) { // The return type is deduced to be `int`.
 std::cout << "fun() is called.\n";
 return a + b;
}
int x = 10, y = 15;
decltype(fun(x, y)) z; // Same as `int z`;
// Unlike `auto`, no initializer is required here.
// The type is deduced from the return type of `fun`.
```

- `decltype(fun(x, y))` only deduces the return type of `fun` without actually calling it. Therefore, **no output is produced**.

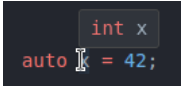
22/46

Note on auto and decltype

The detailed rules of auto and decltype (as well as their differences) are complicated, and require some deeper understanding of C++ types and templates. You don't have to remember them.

Learn about them mainly through experiments.

- A good IDE should be of great help: Place your mouse on it, and your IDE should tell you the deduction result.



C23 also has auto type deduction.

Functions

Default arguments

Some functions have parameters that are given a particular value in most, but not all, calls. In such cases, we can declare that common value as a default argument.

```
std::string get_screen(std::size_t height = 24, std::size_t width = 80,
 char background = ' ');
```

- By default, the screen is 24 × 80 filled with ' '.

```
auto default_screen = get_screen();
```

- To override the default arguments:

```
auto large_screen = get_screen(66); // 66x80, filled with ' '
auto larger_screen = get_screen(66, 256); // 66x256, filled with ' '
auto special_screen = get_screen(66, 256, '#'); // 66x256, filled with '#'
```

Default arguments

Arguments in the call are resolved by position.

```
auto scr = get_screen('#'); // Passing the ASCII value of '#' to `height`.
 // `width` and `background` are set to
 // default values (`80` and ` `).
```

- Some other languages have named parameters:

```
print(a, b, sep=" ", end="") # Python
```

There is no such syntax in C++.

Default arguments are only allowed for the last (right-most) several parameters:

```
std::string get_screen(std::size_t height = 24, std::size_t width,
 char background); // Error.
```

Function overloading

In C++, a group of functions can have the same name, as long as they can be differentiated when called.

```
int max(int a, int b) {
 return a < b ? b : a;
}
double max(double a, double b) {
 return a < b ? b : a;
}
const char *max(const char *a, const char *b) {
 return std::strcmp(a, b) < 0 ? b : a;
}
```

```
auto x = max(10, 20); // Calls max(int, int)
auto y = max(3.14, 2.5); // Calls max(double, double)
auto z = max("hello", "world"); // Calls max(const char *, const char *)
```

Overloaded functions

Overloaded functions should be distinguished in the way they are called.

```
int fun(int);
double fun(int); // Error: functions that differ only in
 // their return type cannot be overloaded.
```

```
void move_cursor(Coord to);
void move_cursor(int r, int c); // OK, differ in the number of arguments
```

Overloaded functions

Overloaded functions should be distinguished in the way they are called.

- The following are declaring the same function. They are not overloading.

```
void fun(int *);
void fun(int [10]);
```

- The following are the same for an array argument:

```
void fun(int *a);
void fun(int (&a)[10]);
int ival = 42; fun(&ival); // OK, calls fun(int *)
int arr[10]; fun(arr); // Error: ambiguous call
```

Why?

Overloaded functions

Overloaded functions should be distinguished in the way they are called.

- The following are the same for an array argument:

```
void fun(int *a);
void fun(int (&a)[10]);
int arr[10]; fun(arr); // Error: ambiguous call
```

- For fun(int (&)[10]), this is an exact match.
- For fun(int \*), this involves an array-to-pointer implicit conversion. We will see that this is also considered an exact match.

Basic overload resolution

Suppose we have the following overloaded functions.

```
void fun(int);
void fun(double);
void fun(int *);
void fun(const int *);
```

Which will be the best match for a call fun(a) ?

Basic overload resolution

Suppose we have the following overloaded functions.

```
void fun(int);
void fun(double);
void fun(int *);
void fun(const int *);
```

Obvious: The arguments and the parameters match perfectly.

Not so obvious:

```
fun(42); // fun(int)
fun(3.14); // fun(double)
const int arr[10];
fun(arr); // fun(const int *)

int ival = 42;
// fun(int *) or fun(const int *)?
fun(&ival); // fun(int) or fun(double)?
fun('a'); // fun(int) or fun(double)?
fun(3.14f); // fun(int) or fun(double)?
fun(NULL); // fun(int) or fun(int *)?
```

## Basic overload resolution

```
void fun(int);
void fun(double);
void fun(int *);
void fun(const int *);
```

- `fun(&ival)` matches `fun(int *)`
- `fun('a')` matches `fun(int)`
- `fun(3.14f)` matches `fun(double)`
- `fun(NULL)` ? We will see this later.

There are detailed rules that define these behaviors. But our program should avoid such confusing overload sets.

33/46

## Basic overload resolution

1. An exact match, including the following cases:
  - identical types
  - match through decay of array (or function) type
  - match through top-level `const` conversion
2. Match through adding low-level `const`
3. Match through integral or floating-point promotion
4. Match through numeric conversion
5. Match through a class-type conversion (in later lectures).

No need to remember all the details. But pay attention to some cases that are very common.

34/46

## The null pointer

`NULL` is a macro defined in standard library header files.

- In C, it may be defined as `(void *)0`, `0`, `(long)0` or other forms.

In C++, `NULL` cannot be `(void *)0` since the implicit conversion from `void *` to other pointer types is **not allowed**.

- It is most likely to be an integer literal with value zero.
- With the following overload declarations, `fun(NULL)` may call `fun(int)` on some platforms, and may be **ambiguous** on other platforms!

```
void fun(int);
void fun(int *);
```

```
fun(NULL); // May call fun(int),
 // or may be ambiguous.
```

35/46

## Better null pointer: `nullptr`

In short, `NULL` is a "fake" pointer.

Since C++11, a better null pointer is introduced: `nullptr` (also available in C23)

- `nullptr` has a unique type `std::nullptr_t` (defined in `<cstddef>`), which is neither `void *` nor an integer.
- `fun(nullptr)` will definitely match `fun(int *)`.

```
void fun(int);
void fun(int *);
```

```
fun(NULL); // May call fun(int),
 // or may be ambiguous.
fun(nullptr); // Calls fun(int *).
```

[Best practice] Use `nullptr` as the null pointer constant in C++.

36/46

## Avoid abuse of function overloading

Only overload operations that actually do similar things. A bad example:

```
Screen &moveHome(Screen &);
Screen &moveAbs(Screen &, int, int);
Screen &moveRel(Screen &, int, int, std::string direction);
```

If we overload this set of functions under the name `move`, some information is lost.

```
Screen &move(Screen &);
Screen &move(Screen &, int, int);
Screen &move(Screen &, int, int, std::string direction);
```

Which one is easier to understand?

```
moveHome(scrn); // OK, moves to home.
```

```
move(scrn); // Unclear: How to move?
```

37/46

## Range-based `for` loops revisited

### Range-based `for` loops

Traverse a `std::string`

```
int str_to_int(const std::string &str) {
 int value = 0;
 for (auto c : str) // char
 value = value * 10 + c - '0';
 return value;
}
```

Note: This function can be replaced by `std::stoi`.

38/46

### Range-based `for` loops

Traverse a `std::vector`

```
bool is_all_digits(const std::string &str) {
 for (auto c : str)
 if (!std::isdigit(c))
 return false;
 return true;
}

int count_numbers(const std::vector<std::string> &strs) {
 int cnt = 0;
 for (const auto &s : strs) // const std::string &s
 if (is_all_digits(s))
 ++cnt;
 return cnt;
}
```

39/46

40/46

### Traverse an array

An array can also be traversed by range-`for`:

```
int arr[100] = {}; // OK in C++ and C23.
// The following loop will read 100 integers.
for (auto &x : arr) // int &
 std::cin >> x;
```

- Note: The range-based `for` loop will traverse **the entire array**.

What else can be traversed using a range-`for`?  $\Rightarrow$  We will learn about this when introducing **iterators**.

41/46

### Pass an array by reference

```
void print(int *arr) {
 for (auto x : arr) // Error: 'arr' is a pointer, not an array.
 std::cout << x << ' ';
 std::cout << '\n';
}
```

We can declare `arr` to be a reference to array:

```
void print(const int (&arr)[100]) {
 for (auto x : arr) // OK. 'arr' is an array.
 std::cout << x << ' ';
 std::cout << '\n';
}
```

- `arr` is of type `const int (&)[100]`: a reference to an array of `100` elements, where each element is of type `const int`.

42/46

## Pass an array by reference

We can declare `arr` to be a reference to array:

```
void print(const int (&arr)[100]) {
 for (auto x : arr) // OK. 'arr' is an array.
 std::cout << x << ' ';
 std::cout << '\n';
}
```

- `arr` is of type `const int (&)[100]`: a reference to an array of `100` elements, where each element is of type `const int`.

Note that only arrays of `100 int` s can fit here.

```
int a[100] = {}; print(a); // OK.
int b[101] = {}; print(b); // Error.
double c[100] = {}; print(c); // Error.
```

43/46

2/41

## Contents

### Class basics

- Members of a class
  - Access
  - The `this` pointer
- Constructors
  - Constructor initializer list
  - Default constructors

## Pass an array by reference

To allow arrays of any type, any length: Use a template function.

```
template <typename Type, std::size_t N>
void print(const Type (&arr)[N]) {
 for (const auto &x : arr)
 std::cout << x << ' ';
 std::cout << '\n';
}
```

We will learn about this in the end of this semester.

44/46

3/41

## Summary

### Type system

- Dangerous casts must happen explicitly: pointers of different types, pointers to integers, casting away low-level `const` ness, ...
- `const_cast`: used for casting away low-level `const` ness.
- `reinterpret_cast`: used for conversion between different pointer types.
- `static_cast`: used for some normal "innocent-looking" conversions: `int` to `double`, `unsigned` to `int`, ...
- Prefer the C++-style named casts to old C-style casts.
- `auto` and `decltype`: type deduction

45/46

4/41

## A simple class

The initial idea: A `class` is a new kind of `struct` that can have member functions:

```
class Student {
 std::string name;
 std::string id;
 int entranceYear;
 void setName(const std::string &newName) {
 name = newName;
 }
 void printInfo() const {
 std::cout << "I am " << name << ", id " << id
 << ", entrance year: " << entranceYear << std::endl;
 }
 bool graduated(int year) const {
 return year - entranceYear >= 4;
 }
};
```

4/41

## Summary

### Functions

- Default arguments: used for setting defaults for some parameters.
- Function overloading: a group of functions with the same name but can be distinguished in the way they are called.

### Range-based `for` loops

- can also be used to traverse arrays.

46/46

5/41

## Member access

Member access: `a.mem`, where `a` is an object of the class type.

- Every member <sup>1</sup> belongs to an object: each student has a name, id, entrance year, etc.
  - You need to specify *whose* name / id / ... you want to obtain.

To call a member function on an object: `a.memfun(args)`.

```
Student s = someValue();
s.printInfo(); // call its printInfo() to print related info
if (s.graduated(2023)) {
 // ...
}
```

5/41

## Access control

```
class Student {
private:
 std::string name;
 std::string id;
 int entranceYear;
public:
 void setName(const std::string &newName) { name = newName; }
 void printInfo() const {
 std::cout << "I am " << name << ", id " << id
 << ", entrance year: " << entranceYear << std::endl;
 }
 bool graduated(int year) const { return year - entranceYear >= 4; }
};
```

- `private` members: Only accessible to code inside the class and `friend` s.
  - ⇒ We will introduce `friend` s in later lectures.
- `public` members: Accessible to all parts of the program.

## CS100 Lecture 14

### Class Basics I

## Access control

```
class Student {
private:
 std::string name;
 std::string id;
 int entranceYear;

public:
 void setName(const std::string &newName);
 void printInfo() const;
 bool graduated(int year) const;
};
```

Unlike some other languages (e.g. Java), an access specifier controls the access of all members after it, until the next access specifier or the end of the class definition.

7/41

## Access control

```
class Student {
// private:
 std::string name;
 std::string id;
 int entranceYear;
public:
 void setName(const std::string &newName);
 void printInfo() const;
 bool graduated(int year) const;
};
```

What if there is a group of members with no access specifier at the beginning?

- If it's `class`, they are `private`.
- If it's `struct`, they are `public`.

This is one of the only two differences between `struct` and `class` in C++.

8/41

## The `this` pointer

```
class Student {
// ...
public:
 bool graduated(int year) const;
};

Student s = someValue();
if (s.graduated(2023))
// ...
```

How many parameters does `graduated` have?

9/41

## The `this` pointer

```
class Student {
// ...
public:
 bool graduated(int year) const;
};

Student s = someValue();
if (s.graduated(2023)) // ...
```

How many parameters does `graduated` have?

- Seemingly one, but actually two: `s` is also information that must be known when calling this function!

10/41

## The `this` pointer

```
class Student {
public:
 void setName(const std::string &n) {
 name = n;
 }

 bool graduated(int year) const {
 return year - entranceYear >= 4;
 }
};

Student s = someValue();
if (s.graduated(2023))
// ...
s.setName("Alice");
```

• The code on the left can be viewed as:

```
void setName
(Student *this, const std::string &n) {
 this->name = n;
}

bool graduated
(const Student *this, int year) {
 return year - this->entranceYear >= 4;
}

Student s = someValue();
if (graduated(&s, 2023))
// ...
setName(&s, "Alice");
```

11/41

## The `this` pointer

There is a pointer called `this` in each member function of class `X` which has type `X *` or `const X *`, pointing to the object on which the member function is called.

Inside a member function, access of any member `mem` is actually `this->mem`.

We can also write `this->mem` explicitly.

```
class Student {
public:
 bool graduated(int year) const {
 return year - this->entranceYear >= 4;
 }
};
```

Many languages have similar constructs, e.g. `self` in Python. (C++23 has `self` too!)

12/41

## `const` member functions

The `const` keyword after the parameter list and before the function body `{` is used to declare a `const` member function.

- A `const` member function cannot modify its data members <sup>2</sup>.
- A `const` member function **guarantees** that no data member will be modified.
  - A non-`const` member function does not provide such guarantee.
  - In a `const` member function, calling a non-`const` member function on `*this` is not allowed.
- For a `const` object, only `const` member functions can be called on it.

[Best practice] If, logically, a member function should not modify the object's state, it should be made a `const` member function. Otherwise, it cannot be called on `const` objects.

13/41

## `const` member functions and the `this` pointer

This `const` is essentially applied to the `this` pointer:

- In `const` member functions of class `X`, `this` has type `const X *`.
- In non-`const` member functions of class `X`, `this` has type `X *`.

If `ptr` is of type `const T *`, the expression `ptr->mem` is also `const`-qualified.

- Recall that in a member function, access of a member `mem` is actually `this->mem`.
- Therefore, `mem` is also `const`-qualified in a `const` member function.

```
class Student {
public:
 void foo() const {
 name += 'a'; // Error: 'name' is 'const std::string' in a const member
 // function. It cannot be modified.
 }
};
```

14/41

## `const` member functions

*Effective C++* Item 3: Use `const` whenever possible.

Decide whether the following member functions need a `const` qualification:

```
class Student {
 std::string name, id;
 int entranceYear;
public:
 const std::string &getName(); // returns the name of the student.
 const std::string &getID(); // returns the id of the student.
 bool valid(); // verifies whether the leading four digits in 'id'
 // is equal to 'entranceYear'.
 void adjustID(); // adjust 'id' according to 'entranceYear'.
};
```

15/41

## `const` member functions

*Effective C++* Item 3: Use `const` whenever possible.

Decide whether the following member functions need a `const` qualification:

```
class Student {
 std::string name, id;
 int entranceYear;
public:
 const std::string &getName() const; // returns the name of the student.
 const std::string &getID() const; // returns the id of the student.
 bool valid() const; // verifies whether the leading four digits in 'id'
 // is equal to 'entranceYear'.
 void adjustID(); // adjust 'id' according to 'entranceYear'.
};
```

The `const` ness of member functions should be determined logically.

16/41

## const member functions

```
class Student {
 std::string name, id;
 int entranceYear;
public:
 const std::string &getName() const { return name; }
 const std::string &getID() const { return id; }
 bool valid() const { return id.substr(0, 4) == std::to_string(entranceYear); }
 void adjustID() { id = std::to_string(entranceYear) + id.substr(4); }
};
```

`str.substr(pos, len)` returns the substring of `str` starting from the position indexed `pos` with length `len`.

- If `len` is not provided, it returns the suffix starting from the position indexed `pos`.

17/41

## Order of initialization

Data members are initialized in order in which they are declared, not the order in the initializer list.

- If the initializers appear in an order different from the declaration order, the compiler will generate a warning.

Typical mistake: `entranceYear` is initialized in terms of `id`, but `id` is not initialized yet!

```
class Student {
 std::string name;
 int entranceYear; // !!!
 std::string id;
public:
 Student(const std::string &name_, const std::string &id_)
 : name(name_), id(id_), entranceYear(std::stoi(id_.substr(0, 4))) {}
};
```

22/41

## Constructors

Often abbreviated as "ctors".

18/41

## Constructor initializer list

Data members are initialized in order in which they are declared, not the order in the initializer list.

- If the initializers appear in an order different from the declaration order, the compiler will generate a warning.
- For a data member that do not appear in the initializer list:
  - If there is an **in-class initializer** (see next page), it is initialized using the in-class initializer.
  - Otherwise, it is **default-initialized**.

What does **default-initialization** mean for class types? ⇒ To be discussed later.

23/41

## Constructors

**Constructors** define how an object can be initialized.

- Constructors are often **overloaded**, because an object may have multiple reasonable ways of initialization.

```
class Student {
 std::string name;
 std::string id;
 int entranceYear;
public:
 Student(const std::string &name_, const std::string &id_, int ey)
 : name(name_), id(id_), entranceYear(ey) {}
 Student(const std::string &name_, const std::string &id_)
 : name(name_), id(id_), entranceYear(std::stoi(id_.substr(0, 4))) {}
};

Student a("Alice", "2020123123", 2020);
Student b("Bob", "2020123124"); // entranceYear = 2020
Student c; // Error: No default constructor. (to be discussed later)
```

19/41

## In-class initializers

A member can have an in-class initializer. It must be in the form `{...}` or `= ...`.<sup>4</sup>

```
class Student {
 std::string name = "Alice";
 std::string id;
 int entranceYear{2024}; // equivalent to `int entranceYear = 2024;`.
public:
 Student() {} // `name` is initialized to `\"Alice\"`,
 // `id` is initialized to an empty string,
 // and `entranceYear` is initialized to 2024.
 Student(int ey) : entranceYear(ey) {} // `name` is initialized to `\"Alice\"`,
 // `id` is initialized to an empty string,
 // and `entranceYear` is initialized to `ey`.
```

The in-class initializer provides the "default" way of initializing a member in this class, as a substitute for default-initialization.

24/41

## Constructors

```
class Student {
 std::string name;
 std::string id;
 int entranceYear;
public:
 Student(const std::string &name_, const std::string &id_)
 : name(name_), id(id_), entranceYear(std::stoi(id_.substr(0, 4))) {}
};
```

- The constructor name is the class name: `Student`.
- Constructors do not have a return type (not even `void`<sup>3</sup>). The constructor body can contain a `return;` statement, which should not return a value.
- The function body of this constructor is empty: `{}`.

20/41

## Constructor initializer list

Below is a typical way of writing this constructor without an initializer list:

```
class Student {
 // ...
public:
 Student(const std::string &name_, const std::string &id_) {
 name = name_;
 id = id_;
 entranceYear = std::stoi(id_.substr(0, 4));
 }
};
```

How are these members actually initialized in this constructor?

25/41

## Constructor initializer list

Constructors initialize **all data members** of the object.

The initialization of **all data members** is done **before entering the function body**.

How they are initialized is (partly) determined by the **constructor initializer list**:

```
class Student {
 // ...
public:
 Student(const std::string &name_, const std::string &id_)
 : name(name_), id(id_), entranceYear(std::stoi(id_.substr(0, 4))) {}
};
```

The initializer list starts with `:`, and contains initializers for each data member, separated by `,`. The initializers must be of the form `(...)` or `{...}`, not `= ...`.

21/41

## Constructor initializer list

Below is a typical way of writing this constructor without an initializer list:

```
class Student {
 // ...
public:
 Student(const std::string &name_, const std::string &id_) {
 name = name_;
 id = id_;
 entranceYear = std::stoi(id_.substr(0, 4));
 }
};
```

How are these members actually initialized in this constructor?

- First, before entering the function body, `name`, `id` and `entranceYear` are default-initialized. `name` and `id` are initialized to empty strings.
- Then, the assignments in the function body take place.

26/41



## Constructor initializer list

[Best practice] Always use an initializer list in a constructor.

- Not all types can be default-initialized. Not all types can be assigned to. (Any counterexamples?)

27/41

## Constructor initializer list

[Best practice] Always use an initializer list in a constructor.

Not all types can be default-initialized. Not all types can be assigned to.

- References `T &` cannot be default-initialized, and cannot be assigned to.
- `const` objects of built-in types cannot be default-initialized.
- `const` objects cannot be assigned to.
- A class can choose to allow or disallow default initialization or assignment. It depends on the design.  $\Rightarrow$  See next page.

Moreover, if a data member is default-initialized and then assigned when could have been initialized directly, it may lead to low efficiency.

28/41

## Default constructors

A special constructor that takes no parameters.

- Guess what it's for?

29/41

## Default Constructors

A special constructor that takes no parameters.

- It defines the behavior of **default-initialization** of objects of that class type, since no arguments need to be passed when calling it.

```
class Point2d {
 double x, y;
public:
 Point2d() : x(0), y(0) {} // default constructor
 Point2d(double x_, double y_) : x(x_), y(y_) {}
};

Point2d p1; // calls default ctor, (0, 0)
Point2d p2(3, 4); // calls Point2d(double, double), (3, 4)
Point2d p3(); // Is this calling the default ctor?
```

30/41

## Default constructors

A special constructor that takes no parameters.

- It defines the behavior of **default-initialization** of objects of that class type, since no arguments need to be passed when calling it.

```
class Point2d {
 double x, y;
public:
 Point2d() : x(0), y(0) {} // default constructor
 Point2d(double x_, double y_) : x(x_), y(y_) {}
};

Point2d p1; // calls default ctor, (0, 0)
Point2d p2(3, 4); // calls Point2d(double, double), (3, 4)
Point2d p3(); // Is this calling the default ctor?
```

Be careful! `p3` is a function that takes no parameters and returns `Point2d`.

31/41

## Is a default constructor needed?

First, if you need to use arrays, you almost certainly need a default constructor:

```
Student s[1000]; // All elements are default-initialized
 // by the default constructor.
Student s2[1000] = {a, b}; // The first two elements are initialized to
 // 'a' and 'b'. The rest are initialized by the
 // default constructor.
```

A `std::vector` does not require that:

```
// In this code, the default constructor of 'Student' is not called.
std::vector<Student> students;
for (auto i = 0; i != n; ++i)
 students.push_back(some_student());
```

32/41

## Is a default constructor needed?

If a class has no user-declared constructors, the compiler will try to synthesize a default constructor.

```
class X {}; // No user-declared constructors.
X x; // OK: calls the compiler-synthesized default constructor
```

The synthesized default constructor initializes the data members as follows:

- If a data member has an in-class initializer, it is initialized according to the in-class initializer.
- Otherwise, default-initialize that member. If it cannot be default-initialized, the compiler will give up -- no default constructor is generated.

33/41

## Is a default constructor needed?

If a class has any user-declared constructors but no default constructor, the compiler **will not** synthesize a default constructor.

You may ask for a default constructor with `= default;`:

```
class Student {
public:
 Student(const std::string &name_, const std::string &id_, int ey)
 : name(name_), id(id_), entranceYear(ey) {}

 Student(const std::string &name_, const std::string &id_)
 : name(name_), id(id_), entranceYear(std::stoi(id_.substr(0, 4))) {}

 Student() = default;
};
```

34/41

## Is a default constructor needed?

It depends on the **design**:

- If the class has a default constructor, what should be the behavior of it? Is there a reasonable "default state" for your class type?

For `Student`: What is a "default student"?

35/41

## Is a default constructor needed?

It depends on the **design**:

- If the class has a default constructor, what should be the behavior of it? Is there a reasonable "default state" for your class type?

For `Student`: What is a "default student"?

- There seems to be no such thing as a "default student" (in a normal design). Therefore, `Student` should not have a default constructor.

36/41

## Is a default constructor needed?

[Best practice] When in doubt, leave it out. If the class does not have a "default state", it should not have a default constructor!

- Do not define one arbitrarily or letting it `= default`. This leads to pitfalls.
- Calling the default constructor of something that has no "default state" should result in a **compile error**, instead of being allowed arbitrarily.

37/41

## Summary

Members of a class

- A class can have data members and member functions.
- Access control: `private`, `public`.
  - One difference between `class` and `struct`: Default access.
- The `this` pointer: has type `X *` (`const X *` in `const` member functions). It points to the object on which the member function is called.
- `const` member function: guarantees that no modification will happen.

38/41

## Summary

The followings hold for **all constructors**, no matter how they are defined:

- A constructor initializes **all** data members in order in which they are declared.
- The initialization of **all** data members is done before the function body of a constructor is executed.

In a constructor, a member is initialized as follows:

- If there is an initializer for it in the initializer list, use it.
- Otherwise, if it has an in-class initializer, use it.
- Otherwise, it is default-initialized. If it cannot be default-initialized, it leads to a compile-error.

39/41

## Summary

Default constructors

- The default constructor defines the behavior of default-initialization.
- The default constructor is the constructor with an empty parameter list.
- If we have not defined **any constructor**, the compiler will try to synthesize a **default constructor** as if it were defined as `ClassName() {}`.
  - The compiler may fail to do that if some member has no in-class initializer and is not default-initializable. In that case, the compiler gives up (without giving an error).
- We can use `= default` to ask for a synthesized default constructor explicitly.

40/41

## Notes

<sup>1</sup> Every `non-static` member belongs to an object. All data members mentioned in the slides of this lecture are `non-static`.

<sup>2</sup> A `const` member function cannot modify its data members, unless that member is marked `mutable`.

<sup>3</sup> A constructor does not have a return type according to the standard. But it behaves as if its return type is `void`. Some compilers (such as Clang) may also treat it as if it returns `void`.

<sup>4</sup> In-class initializers cannot be provided in the form `(...)`. The parentheses here will be treated as part of a function declaration.

41/41

## CS100 Lecture 15

Constructors, Destructors, Copy Control

1/49

## Contents

- Constructors and destructors
- Copy control

2/49

## Constructors and destructors

### Lifetime of an object

Lifetime of a local `non-static` object:

- Starts on initialization
- Ends when control flow goes out of its **scope**.

```
for (int i = 0; i != n; ++i) {
 do_something(i);
 // Lifetime of `s` begins.
 std::string s = some_string();
 do_something_else(s, i);
 /* end of lifetime of `s` */ }
```

Every time the loop body is executed, `s` undergoes initialization and destruction.

- `std::string` **owns** some resources (memory where the characters are stored).
- `std::string` must *somehow* release that resources (deallocate that memory) at the end of its lifetime.

4/49

### Lifetime of an object

Lifetime of a global object:

- Starts on initialization (before the first statement of `main`)
- Ends when the program terminates.

Lifetime of a heap-based object:

- Starts on initialization: A `new` expression will do this, but `malloc` does not!
- Ends when it is destroyed: A `delete` expression will do this, but `free` does not!

⇒ `new` / `delete` expressions are in this week's recitation.

5/49

## Constructors and Destructors

Take `std::string` as an example:

- Its initialization (done by its constructors) must allocate some memory for its content.
- When it is destroyed, it must *somehow* deallocate that memory.

6/49

## Constructors and Destructors

Take `std::string` as an example:

- Its initialization (done by its constructors) must allocate some memory for its content.
- When it is destroyed, it must *somehow* deallocate that memory.

A destructor of a class is the function that is automatically called when an object of that class type is destroyed.

7/49

## Constructors and Destructors

Syntax: `~ClassName() { /* ... */ }`

|                                                                                                                           |                                                                              |
|---------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <pre>struct A {     A() {         std::cout &lt;&lt; 'c';     }     ~A() {         std::cout &lt;&lt; 'd';     } };</pre> | <pre>for (int i = 0; i != 3; ++i) {     A a;     // do something ... }</pre> |
| Output:                                                                                                                   |                                                                              |
| cdcdcd                                                                                                                    |                                                                              |

8/49

## Destructor

Called **automatically** when the object is destroyed!

- How can we make use of this property?

9/49

## Destructor

Called **automatically** when the object is destroyed!

- How can we make use of this property?

We often do some **cleanup** in a destructor:

- If the object **owns some resources** (e.g. dynamic memory), destructors can be made use of to avoid leaking!

```
class A {
 SomeResourceHandle resource;
public:
 A(/* ... */) : resource(Obtain_resource(/* ... */)) {}
 ~A() {
 release_resource(resource);
 }
};
```

10/49

## Example: A dynamic array

Suppose we want to implement a "dynamic array":

- It looks like a VLA (variable-length array), but it is heap-based, which is safer.
- It should take good care of the memory it uses.

Expected usage:

```
int n; std::cin >> n;
Dynarray arr(n); // `n` is runtime determined
 // `arr` should have allocated memory for `n` `int`s now.
for (int i = 0; i != n; ++i) {
 int x; std::cin >> x;
 arr.at(i) = x * x; // subscript, looks as if `arr[i] = x * x`
}
// ...
// `arr` should deallocate its memory itself.
```

11/49

## Dynarray: members

- It should have a pointer that points to the memory, where elements are stored.
- It should remember its length.

```
class Dynarray {
 int *m_storage;
 std::size_t m_length;
};
```

- `m` stands for **member**.

[Best practice] Make data members `private` to achieve good encapsulation.

12/49

## Dynarray: constructors

- We want `Dynarray a(n);` to construct a `Dynarray` that contains `n` elements.
  - To avoid troubles, we want the elements to be **value-initialized**!
    - **Value-initialization** is like "empty-initialization" in C. (In this week's recitation.)
  - `new int[n]{};:` Allocate a block of heap memory that stores `n` `int`s, and value-initialize them.
- Do we need a default constructor?
  - Review: What is a default constructor?
    - The constructor with no parameters.
  - What should be the correct behavior of it?

13/49

## Dynarray: constructors

- We want `Dynarray a(n);` to construct a `Dynarray` that contains `n` elements.
  - To avoid troubles, we want the elements to be **value-initialized**!
- Suppose we don't want a default constructor.

```
class Dynarray {
 int *m_storage;
 std::size_t m_length;
public:
 Dynarray(std::size_t n) : m_storage(new int[n]{}), m_length(n) {}
};
```

If the class has a user-declared constructor, the compiler will not generate a default constructor.

14/49

## Dynarray: constructors

```
class Dynarray {
 int *m_storage;
 std::size_t m_length;
public:
 Dynarray(std::size_t n) : m_storage(new int[n]{}), m_length(n) {}
};
```

Since `Dynarray` has a user-declared constructor, it does not have a default constructor:

```
Dynarray a; // Error.
```

15/49

## Dynarray: destructor

- Remember: The destructor is (automatically) called when the object is "dead".
- The memory is obtained in the constructor, and released in the destructor.

```
class Dynarray {
 int *m_storage;
 std::size_t m_length;
public:
 Dynarray(std::size_t n)
 : m_storage(new int[n]{}), m_length(n) {}
 ~Dynarray() {
 delete[] m_storage; // Pay attention to `[]`!
 }
};
```

16/49

## Dynarray: some member functions

Design some useful member functions.

- A function to obtain its length (size).
- A function telling whether it is empty.

```
class Dynarray {
 // ...
public:
 std::size_t size() const {
 return m_length;
 }
 bool empty() const {
 return m_length == 0;
 }
};
```

21/49

## Dynarray: destructor

Is this correct?

```
class Dynarray {
 // ...
 ~Dynarray() {
 if (m_length != 0)
 delete[] m_storage;
 }
};
```

17/49

## Dynarray: some member functions

Design some useful member functions.

- A function returning **reference** to an element.

```
class Dynarray {
 // ...
public:
 int &at(std::size_t i) {
 return m_storage[i];
 }
 const int &at(std::size_t i) const {
 return m_storage[i];
 }
};
```

Why do we need this "const" vs non-const" overloading? ⇒ Learn it in recitations.

22/49

## Dynarray: destructor

Is this correct?

```
class Dynarray {
 // ...
 ~Dynarray() {
 if (m_length != 0)
 delete[] m_storage;
 }
};
```

**NO!** `new [0]` may also allocate some memory (implementation-defined, like `malloc`), which should also be deallocated.

18/49

## Dynarray: Usage

```
void print(const Dynarray &a) {
 for (std::size_t i = 0;
 i != a.size(); ++i)
 std::cout << a.at(i) << ' ';
 std::cout << std::endl;
}

void reverse(Dynarray &a) {
 for (std::size_t i = 0,
 j = a.size() - 1; i < j; ++i, --j)
 std::swap(a.at(i), a.at(j));
}

int main() {
 int n; std::cin >> n;
 Dynarray array(n);
 for (int i = 0; i != n; ++i)
 std::cin >> array.at(i);
 reverse(array);
 print(array);
 return 0;
 // Dtor of `array` is called here,
 // which deallocates the memory
}
```

23/49

## Dynarray: destructor

Is this correct?

```
class Dynarray {
 // ...
 ~Dynarray() {
 delete[] m_storage;
 m_length = 0;
 }
};
```

19/49

## Copy control

### Copy-initialization

We can easily construct a `std::string` to be a copy of another:

```
std::string s1 = some_value();
std::string s2 = s1; // s2 is initialized to be a copy of s1
std::string s3(s1); // equivalent
std::string s4{s1}; // equivalent, but modern
```

Can we do this for our `Dynarray`?

## Dynarray: destructor

Is this correct?

```
class Dynarray {
 // ...
 ~Dynarray() {
 delete[] m_storage;
 m_length = 0;
 }
};
```

It is correct, but `m_length = 0;` is not needed. The destructor is executed **right before** the `Dynarray` object "dies", so the value of `m_length` does not matter!

20/49

25/49

## Copy-initialization

Before we add anything, let's try what will happen:

```
Dynarray a(3);
a.at(0) = 2; a.at(1) = 3; a.at(2) = 5;
Dynarray b = a; // It compiles.
print(b); // 2 3 5
a.at(0) = 70;
print(b); // 70 3 5
```

Ooops! Although it compiles, the pointers `a.m_storage` and `b.m_storage` are pointing to the same address!

26/49

## Copy-initialization

Before we add anything, let's try what will happen:

```
Dynarray a(3);
Dynarray b = a;
```

Although it compiles, the pointers `a.m_storage` and `b.m_storage` are pointing to the same address!

This will cause disaster: consider the case if `b` "dies" before `a`:

```
Dynarray a(3);
if (some_condition) {
 Dynarray b = a; // `a.m_storage` and `b.m_storage` point to the same memory!
 // ...
} // At this point, dtor of `b` is invoked, which deallocates the memory.
std::cout << a.at(0); // Invalid memory access!
```

27/49

## Copy constructor

Let `a` be an object of type `Type`. The behaviors of **copy-initialization** (in one of the following forms)

```
Type b = a;
Type b(a);
Type b{a};
```

are determined by a constructor: **the copy constructor**.

- Note! The `=` in `Type b = a;` is not an assignment operator!

28/49

## Copy constructor

The copy constructor of a class `X` has a parameter of type `const X &`:

```
class Dynarray {
public:
 Dynarray(const Dynarray &other);
};
```

Why `const`?

- Logically, it should not modify the object being copied.

Why `&`?

- Avoid copying.** Pass-by-value is actually **copy-initialization** of the parameter, which will cause infinite recursion here!

29/49

## Dynarray: copy constructor

What should be the correct behavior of it?

```
class Dynarray {
public:
 Dynarray(const Dynarray &other);
};
```

30/49

## Dynarray: copy constructor

- We want a copy of the content of `other`.

```
class Dynarray {
public:
 Dynarray(const Dynarray &other)
 : m_storage(new int[other.size()]), m_length(other.size()) {
 for (std::size_t i = 0; i != other.size(); ++i)
 m_storage[i] = other.at(i);
 }
};
```

Now the copy-initialization of `Dynarray` does the correct thing:

- The new object allocates a new block of memory.
- The **contents** are copied, not just the address.

31/49

## Synthesized copy constructor

If the class does not have a user-declared copy constructor, the compiler will try to synthesize one:

- The synthesized copy constructor will **copy-initialize** all the members, as if

```
class Dynarray {
public:
 Dynarray(const Dynarray &other)
 : m_storage(other.m_storage), m_length(other.m_length) {}
};
```

- If the synthesized copy constructor does not behave as you expect, **define it on your own!**

32/49

## Defaulted copy constructor

If the synthesized copy constructor behaves as we expect, we can explicitly require it:

```
class Dynarray {
public:
 Dynarray(const Dynarray &) = default;
 // Explicitly defaulted: Explicitly requires the compiler to synthesize
 // a copy constructor, with default behavior.
};
```

33/49

## Deleted copy constructor

What if we don't want a copy constructor?

```
class ComplicatedDevice {
 // some members
 // Suppose this class represents some complicated device,
 // for which there is no correct and suitable behavior for "copying".
};
```

Simply not defining the copy constructor does not work:

- The compiler will synthesize one for you.

34/49

## Deleted copy constructor

What if we don't want a copy constructor?

```
class ComplicatedDevice {
 // some members
 // Suppose this class represents some complicated device,
 // for which there is no correct and suitable behavior for "copying".
public:
 ComplicatedDevice(const ComplicatedDevice &) = delete;
};
```

By saying `= delete`, we define a **deleted** copy constructor:

```
ComplicatedDevice a = something();
ComplicatedDevice b = a; // Error: calling deleted function
```

35/49

## Copy-assignment operator

Apart from copy-initialization, there is another form of copying:

```
std::string s1 = "hello", s2 = "world";
s1 = s2; // s1 becomes a copy of s2, representing "world"
```

In `s1 = s2`, `=` is the assignment operator.

`=` is the assignment operator only when it is in an expression.

- `s1 = s2` is an expression.
- `std::string s1 = s2` is in a declaration statement, not an expression. `=` here is a part of the initialization syntax.

36/49

## Dynarray: copy-assignment operator

The copy-assignment operator is defined in the form of operator overloading:

- `a = b` is equivalent to `a.operator=(b)`.
- We will talk about more on operator overloading in a few weeks.

```
class Dynarray {
public:
 Dynarray &operator=(const Dynarray &other);
};
```

- The function name is `operator=`.
- In consistent with built-in assignment operators, `operator=` returns reference to the left-hand side object (the object being assigned).
  - It is `*this`.

37/49

## Dynarray: copy-assignment operator

We also want the copy-assignment operator to copy the contents, not only an address.

```
class Dynarray {
public:
 Dynarray &operator=(const Dynarray &other) {
 m_storage = new int[other.size()];
 for (std::size_t i = 0; i != other.size(); ++i)
 m_storage[i] = other.at(i);
 m_length = other.size();
 return *this;
 }
};
```

Is this correct?

38/49

## Dynarray: copy-assignment operator

Avoid memory leaks! Deallocate the memory you don't use!

```
class Dynarray {
public:
 Dynarray &operator=(const Dynarray &other) {
 delete[] m_storage; // !!!
 m_storage = new int[other.size()];
 for (std::size_t i = 0; i != other.size(); ++i)
 m_storage[i] = other.at(i);
 m_length = other.size();
 return *this;
 }
};
```

Is this correct?

39/49

## Dynarray: copy-assignment operator

What if self-assignment happens?

```
class Dynarray {
public:
 Dynarray &operator=(const Dynarray &other) {
 // If 'other' and '*this' are actually the same object,
 // the memory is deallocated and the data are lost! (DISASTER)
 delete[] m_storage;
 m_storage = new int[other.size()];
 for (std::size_t i = 0; i != other.size(); ++i)
 m_storage[i] = other.at(i);
 m_length = other.size();
 return *this;
 }
};
```

40/49

## Dynarray: copy-assignment operator

Assignment operators should be self-assignment-safe.

```
class Dynarray {
public:
 Dynarray &operator=(const Dynarray &other) {
 int *new_data = new int[other.size()];
 for (std::size_t i = 0; i != other.size(); ++i)
 new_data[i] = other.at(i);
 delete[] m_storage;
 m_storage = new_data;
 m_length = other.size();
 return *this;
 }
};
```

This is self-assignment-safe. (Think about it.)

41/49

## Synthesized, defaulted and deleted copy-assignment operator

Like the copy constructor:

- The copy-assignment operator can also be deleted, by declaring it as `= delete;`.
- If you don't define it, the compiler will generate one that copy-assigns all the members, as if it is defined as:

```
class Dynarray {
public:
 Dynarray &operator=(const Dynarray &other) {
 m_storage = other.m_storage;
 m_length = other.m_length;
 return *this;
 }
};
```

- You can also require a synthesized one explicitly by saying `= default;`.

42/49

## [IMPORTANT] The rule of three: Reasoning

Among the copy constructor, the copy-assignment operator and the destructor:

- If a class needs a user-provided version of one of them, usually, it needs a user-provided version of each of them.
- Why?

43/49

## [IMPORTANT] The rule of three: Reasoning

Among the copy constructor, the copy-assignment operator and the destructor:

- If a class needs a user-provided version of one of them,
- usually, it is a class that manages some resources,
- for which the default behavior of the copy-control members does not suffice.
- Therefore, all of the three special functions need a user-provided version.
  - Define them in a correct, well-defined manner.
  - If a class should not be copy-constructible or copy-assignable, delete that function.

44/49

## [IMPORTANT] The rule of three: Rules

Let  $S = \{ \text{copy constructor}, \text{copy assignment operator}, \text{destructor} \}$ .

If for a class,  $\exists x, y \in S$  such that

- $x$  is user-declared, and  $y$  is not user-declared,

then the compiler *should not* generate  $y$ , according to the idea of "the rule of three".

45/49

[IMPORTANT] The rule of three: Rules

Let  $S = \{ \text{copy constructor}, \text{copy assignment operator}, \text{destructor} \}$ .

If for a class,  $\exists x, y \in S$  such that

- $x$  is user-declared, and  $y$  is not user-declared,

then the compiler **still generates  $y$** , but **this behavior has been deprecated since C++11**.

- This is a problem left over from history: At the time C++98 was adopted, the significance of the rule of three was not fully appreciated.

46/49

2/45

[IMPORTANT] The rule of three

Into modern C++: **The Rule of Five**.

- $\Rightarrow$  We will talk about it in later lectures.

Read *Effective Modern C++* Item 17 for a thorough understanding of this.

47/49

3/45

Summary

Lifetime of an object:

- depends on its **storage**: local non-`static`, global, allocated, ...
- Initialization** marks the beginning of the lifetime of an object.
  - Classes can control the way of initialization using **constructors**.
- When the lifetime of an object ends, it is **destroyed**.
  - If it is an object of class type, its **destructor** is called right before it is destroyed.

48/49

Summary

Copy control

- Usually, the **copy control members** refer to the copy constructor, the copy assignment operator and the destructor.
- Copy constructor: `ClassName(const ClassName &)`
- Copy assignment operator: `ClassName &operator=(const ClassName &)`
  - It needs to be **self-assignment safe**.
- Destructor: `~ClassName()`
- `=default`, `=delete`
- The rule of three.

49/49

CS100 Lecture 16

Class Basics II

Contents

- Type alias members
- `static` members
- `friend`
- Definition and declaration
- Destructors revisited

Type alias members

Type aliases in C++: `using`.

A better way of declaring type aliases:

```
// C-style
typedef long long LL;
// C++-style
using LL = long long;
```

It is more readable when dealing with compound types:

|                                                                                                |                                                                                                                   |
|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <pre>// C-style typedef int intarray_t[1000]; // C++-style using intarray_t = int[1000];</pre> | <pre>// C-style typedef int (&amp;ref_to_array)[1000]; // C++-style using ref_to_array = int (&amp;)[1000];</pre> |
|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|

`using` can also declare *alias templates* (in later lectures), while `typedef` cannot.

[Best practice] In C++, Use `using` to declare type aliases.

4/45

Type alias members

A class can have **type alias members**.

```
class Dynarray {
public:
 using size_type = std::size_t;
 size_type size() const { return m_length; }
};
```

Usage: `ClassName::TypeAliasName`

```
for (Dynarray::size_type i = 0; i != a.size(); ++i)
 // ...
```

Note: Here we use `ClassName::` instead of `object.`, because such members belong to the class, not one single object.

5/45

Type alias members

The class also has control over the accessibility of type alias members.

```
class A {
 using type = int;
};
A::type x = 42; // Error: Accessing private member of `A`.
```

The class has control over the accessibility of **anything that is called a *member* of it**.

1/45

6/45

## Type alias members in the standard library

All standard library containers (and `std::string`) define the type alias member `size_type` as the return type of `.size()`:

```
std::string::size_type i = s.size();
std::vector<int>::size_type j = v.size(); // Not `std::vector::size_type`!
// The template argument `<int>`
// is necessary here.
std::list<int>::size_type k = l.size();
```

Why?

7/45

## Type alias members in the standard library

All standard library containers (and `std::string`) define the type alias member `size_type` as the return type of `.size()`:

```
std::string::size_type i = s.size();
std::vector<int>::size_type j = v.size();
std::list<int>::size_type k = l.size();
```

- This type is **container-dependent**: Different containers may choose different types suitable for representing sizes.
  - The Qt containers often use `int` as `size_type`.
- Define `Container::size_type` to achieve good **consistency** and **generality**.

8/45

## static members

### static data members

A `static` data member:

```
class A {
 static int something;
 // other members ...
};
```

Just consider it as a **global variable**, except that

- its name is in the **class scope**: `A::something`, and that
- the accessibility may be restricted. Here `something` is `private`.

10/45

### static data members

A `static` data member:

```
class A {
 static int something;
 // other members ...
};
```

There is **only one** `A::something`: it does not belong to any object of `A`. It belongs to the class `A`.

- Like type alias members, we use `ClassName::` instead of `object.` to access them.

11/45

### static data members

A `static` data member:

```
class A {
 static int something;
 // other members ...
};
```

It can also be accessed by `a.something` (where `a` is an object of type `A`), but `a.something` and `b.something` refer to the same variable.

- If `f` is a function that returns an object of type `A`, `f().something` always accesses the same variable no matter what `f()` returns.
- In the very first externally available C++ compiler (Cfront 1.0, 1985), `f` in the expression `f().something` is not even called! This bug has been fixed soon.

12/45

### static data members: Example

Suppose we want to assign a unique id to each object of our class.

```
int cnt = 0;

class Dynarray {
 int *m_storage;
 std::size_t m_length;
 int m_id;
public:
 Dynarray(std::size_t n)
 : m_storage(new int[n]{}), m_length(n), m_id(cnt++) {}
 Dynarray() : m_storage(nullptr), m_length(0), m_id(cnt++) {}
 // ...
};
```

We use a global variable `cnt` as the "counter". Is this a good design?

13/45

### static data members: Example

The name `cnt` is confusing: A "counter" of what?

```
int X_cnt = 0, Y_cnt = 0, Z_cnt = 0;
struct X {
 int m_id;
 X() : m_id(X_cnt++) {}
};
struct Y {
 int m_id;
 Y() : m_id(Y_cnt++) {}
};
struct Z {
 int m_id;
 Z() : m_id(Z_cnt++) {}
};
```

- The program is in a mess with global variables all around.
- No prevention from potential mistakes:

```
struct Y {
 Y() : m_id(X_cnt++) {}
};
```

The mistake happens silently.

14/45

### static data members: Example

Restrict the name of this counter in the scope of the corresponding class, by declaring it as a `static` data member.

- This is exactly the idea behind `static` data members: A "global variable" restricted in class scope.

```
class Dynarray {
 static int s_cnt; // !!!
 int *m_storage;
 std::size_t m_length;
 int m_id;
public:
 Dynarray(/* ... */) : /* ... */, m_id(s_cnt++) {}
};
```

- `s` stands for `static`.

15/45

### static data members

```
class Dynarray {
 static int s_cnt; // !!!
 int *m_storage;
 std::size_t m_length;
 int m_id;
public:
 Dynarray(/* ... */) : /* ... */, m_id(s_cnt++) {}
};
```

You also need to give it a definition outside the class, according to some rules.

```
int Dynarray::s_cnt; // Zero-initialize, because it is `static`.
```

Or initialize it with some value explicitly:

```
int Dynarray::s_cnt = 42;
```

16/45



## static data members

Exercise: `std::string` has a `find` member function:

```
std::string s = something();
auto pos = s.find('a');
if (pos == std::string::npos) { // This means that 'a' is not found.
 // ...
} else {
 std::cout << s[pos] << '\n'; // If executed, it should print 'a'.
}
```

`std::string::npos` is returned when the required character is not found.

Define `npos` and `find` for your `Dynarray` class, whose behavior should be similar to those of `std::string`.

17/45

## static member functions

A `static` member function:

```
class A {
public:
 static void fun(int x, int y);
};
```

Just consider it as a normal non-member function, except that

- its name is in the **class scope**: `A::fun(x, y)`, and that
- the accessibility may be restricted. Here `fun` is `public`.

18/45

## static member functions

A `static` member function:

```
class A {
public:
 static void fun(int x, int y);
};
```

`A::fun` does not belong to any object of `A`. It belongs to the class `A`.

- There is no `this` pointer inside `fun`.

It can also be called by `a.fun(x, y)` (where `a` is an object of type `A`), but here `a` will not be bound to a `this` pointer, and `fun` has no way of accessing any non-`static` member of `a`.

19/45

## friend

## friend functions

Recall the `Student` class:

```
class Student {
 std::string m_name;
 std::string m_id;
 int m_entranceYear;
public:
 Student(const std::string &name, const std::string &id)
 : m_name(name), m_id(id), m_entranceYear(std::stoi(id.substr(0, 4))) {}
 auto graduated(int year) const { return year - m_entranceYear >= 4; }
 // ...
};
```

Suppose we want to write a function to display the information of a `Student`.

21/45

## friend functions

```
void print(const Student &stu) {
 std::cout << "Name: " << stu.m_name << ", id: " << stu.m_id
 << "entrance year: " << stu.m_entranceYear << '\n';
}
```

This won't compile, because `m_name`, `m_id` and `m_entranceYear` are `private` members of `Student`.

- One workaround is to define `print` as a member of `Student`.
- However, there do exist some functions that cannot be defined as a member.

22/45

## friend functions

Add a `friend` declaration, so that `print` can access the private members of `Student`.

```
class Student {
 friend void print(const Student &); // The parameter name is not used in this
 // declaration, so it is omitted.

 std::string m_name;
 std::string m_id;
 int m_entranceYear;
public:
 Student(const std::string &name, const std::string &id)
 : m_name(name), m_id(id), m_entranceYear(std::stoi(id.substr(0, 4))) {}
 auto graduated(int year) const { return year - m_entranceYear >= 4; }
 // ...
};
```

23/45

## friend functions

Add a `friend` declaration.

```
class Student {
 friend void print(const Student &);

 // ...
};
```

A `friend` is **not** a member! You can put this `friend` declaration **anywhere in the class body**. The access modifiers have **no effect** on it.

- We often declare all the `friend`s of a class in the beginning or at the end of class definition.

24/45

## friend classes

A class can also declare another class as its `friend`.

```
class X {
 friend class Y;
 // ...
};
```

In this way, any code from the class `Y` can access the private members of `X`.

25/45

## Definition and declaration

26/45

## Definition and declaration

For a function:

```
// Only a declaration: The function body is not present.
void foo(int, const std::string &);
// A definition: The function body is present.
void foo(int x, const std::string &s) {
 // ...
}
```

27/45

## Class definition

For a class, a **definition** consists of the declarations of all its members.

```
class Widget {
public:
 Widget();
 Widget(int, int);
 void set_handle(int);

 // `const` is also a part of the function type, which should be present
 // in its declaration.
 const std::vector<int> &get_gadgets() const;

 // ...
private:
 int m_handle;
 int m_length;
 std::vector<int> m_gadgets;
};
```

28/45

## Define a member function outside the class body

A member function can be declared in the class body, and then defined outside.

```
class Widget {
public:
 const std::vector<int> &get_gadgets() const; // A declaration only.
 // ...
}; // Now the definition of `Widget` is complete.

// Define the function here. The function name is `Widget::get_gadgets`.
const std::vector<int> &Widget::get_gadgets() const {
 return m_gadgets; // Just like how you do it inside the class body.
 // The implicit `this` pointer is still there.
}
```

29/45

## The :: operator

```
class Widget {
public:
 using gadgets_list = std::vector<int>;
 static int special_member;
 const gadgets_list &get_gadgets() const;
 // ...
};
const gadgets_list &Widget::get_gadgets() const {
 return m_gadgets;
}
```

- The members `Widget::gadgets_list` and `Widget::special_member` are accessed through `ClassName::`.
- The name of the member function `get_gadgets` is `Widget::get_gadgets`.

30/45

## Class declaration and incomplete type

To declare a class without providing a definition:

```
class A;
struct B;
```

If we only see the **declaration** of a class, we have no knowledge about its members, how many bytes it takes, how it can be initialized, ...

- Such class type is an **incomplete type**.
- We cannot create an object of such type, nor can we access any of its members.
- The only thing we can do is to declare a pointer or a reference to it.

31/45

## Class declaration and incomplete type

If we only see the **declaration** of a class, we have no knowledge about its members, how many bytes it takes, how it can be initialized, ...

- Such class type is an **incomplete type**.
- We cannot create an object of such type, nor can we access any of its members.
- The only thing we can do is to declare a pointer or a reference to it.

```
class Student; // We only have this declaration.

void print(const Student &stu) { // OK. Declaring a reference to it is OK.
 std::cout << stu.getName(); // Error. We don't know anything about its members.
}

class Student {
public:
 const std::string &getName() const { /* ... */ }
 // ...
};
```

32/45

## Destructors revisited

### Destructors revisited

A **destructor** (dtor) is a member function that is called automatically when an object of that class type is "dead".

- For global and `static` objects, on termination of the program.
- For local objects, when control reaches the end of its scope.
- For objects created by `new` / `new[]`, when their address is passed to `delete` / `delete[]`.

The destructor is often responsible for doing some **cleanup**: Release the resources it owns, do some logging, cut off its connection with some external objects, ...

34/45

## Destructors

```
class Student {
 std::string m_name;
 std::string m_id;
 int m_entranceYear;
public:
 Student(const std::string &, const std::string &);
 const std::string &getName() const;
 bool graduated(int) const;
 void setName(const std::string &);
 void print() const;
};
```

Does our `Student` class have a destructor?

35/45

## Destructors

Does our `Student` class have a destructor?

- It **must** have. Whenever you create an object of type `Student`, its destructor needs to be invoked somewhere in this program. <sup>1</sup>

What does `Student::~~Student` need to do? Does `Student` own any resources?

36/45

## Destructors

Does our `Student` class have a destructor?

- It **must** have. Whenever you create an object of type `Student`, its destructor needs to be invoked somewhere in this program. <sup>1</sup>

What does `Student::~~Student` need to do? Does `Student` own any resources?

- It seems that a `Student` has no resources, so nothing special needs to be done.
- However, it has two `std::string` members! Their destructors must be called, otherwise the memory is leaked!

37/45

## Destructors

To define the destructor of `Student`: Just write an empty function body, and everything is done.

```
class Student {
 std::string m_name;
 std::string m_id;
 int m_entranceYear;
public:
 ~Student() {}
};
```

38/45

## Destructors

```
class Student {
 std::string m_name;
 std::string m_id;
 int m_entranceYear;
public:
 ~Student() {}
};
```

- When the function body is executed, the object is *not yet* "dead".
  - You can still access its members.
- After the function body is executed, **all its data members** are destroyed automatically, **in reverse order** in which they are declared.
  - For members of class type, their destructors are invoked automatically.

```
~Student() { std::cout << m_name << '\n'; }
```

39/45

## Constructors vs destructors

```
Student(const std::string &name)
: m_name(name) /* ... */ {
 // ...
}
```

```
~Student() {
 // ...
}
```

- A class may have multiple ctors (overloaded).
- The data members are initialized **before** the execution of function body.
- The data members are initialized in **order** in which they are declared.
- A class has only one dtor. <sup>1</sup>
- The data members are destroyed **after** the execution of function body.
- The data members are destroyed in **reverse order** in which they are declared.

40/45

## Compiler-generated destructors

For most cases, a class needs a destructor.

Therefore, the compiler always generates one <sup>2</sup> if there is no user-declared destructor.

- The compiler-generated destructor is `public` by default.
- The compiler-generated destructor is as if it were defined with an empty function body `{}`.
- It does nothing but to destroy the data members.

We can explicitly require one by writing `= default;`, just as for other copy control members.

41/45

## Summary

Type alias members

- Type alias members belong to the class, not individual objects, so they are accessed via `ClassName::AliasName`.
- The class can controls the accessibility of type alias members.

`static` members

- `static` data members are like global variables, but in the class's scope.
- `static` member functions are like normal non-member functions, but in the class's scope. There is no `this` pointer in a `static` member function.
- A `static` member belongs to the class, instead of any individual object.

42/45

## Summary

`friend`

- A `friend` declaration allows a function or class to access private (and protected) members of another class.
- A `friend` is not a member.

Definitions and declarations

- A class definition includes declarations of all its members.
- A member function can be declared in the class body and then defined outside.
- A class type is an incomplete type if only its declaration (without a definition) is present.

43/45

## Summary

Destructors

- Destructors are called automatically when an object's lifetime ends. They often do some clean up.
- The members are destroyed **after** the function body is executed. They are destroyed in reverse order in which they are declared.
- The compiler generates a destructor (in most cases) if none is provided. It just destroys all its members.

44/45

## Notes

<sup>1</sup> Objects created by `new / new[]` are not required to be destroyed. A `delete / delete[]` expression will destroy it, but it is not mandatory. So you can still create an object with a deleted destructor (see [3](#)) by a `new` expression, but you can't `delete` it, which possibly leads to memory leak.

<sup>2</sup> A class can have many [prospective destructors](#) since C++20.

<sup>3</sup> If no user-declared destructor is provided for a class type, the compiler will always **declare** a destructor as an `inline public` member of its class.

If an implicitly-declared destructor is not deleted, it is **implicitly-defined** by the compiler when it is **odr-used**. In some very special cases the compiler may fail to define the destructor (e.g. due to a member whose destructor is inaccessible). In that case, the destructor is implicitly deleted.

45/45

## CS100 Lecture 17

Rvalue References and Move

1/43

## Contents

- Motivation: Copy is slow.
  - Rvalue references
- Move operations
  - Move constructor
  - Move assignment operator
  - The rule of five
- `std::move`
- NRVO, move and copy elision

2/43

### Motivation: Copy is slow.

```
std::string a = some_value(), b = some_other_value();
std::string s;
s = a;
s = a + b;
```

Consider the two assignments: `s = a` and `s = a + b`.

How is `s = a + b` evaluated?

3/43

### Motivation: Copy is slow.

```
s = a + b;
```

1. Evaluate `a + b` and store the result in a temporary object, say `tmp`.
2. Perform the assignment `s = tmp`.
3. The temporary object `tmp` is no longer needed, hence destroyed by its destructor.

Can we make this faster?

4/43

### Motivation: Copy is slow.

```
s = a + b;
```

1. Evaluate `a + b` and store the result in a temporary object, say `tmp`.
2. Perform the assignment `s = tmp`.
3. The temporary object `tmp` is no longer needed, hence destroyed by its destructor.

Can we make this faster?

- The assignment `s = tmp` is done by **copying** the contents of `tmp`?
- But `tmp` is about to "die"! Why can't we just *steal* the contents from it?

5/43

### Motivation: Copy is slow.

Let's look at the other assignment:

```
s = a;
```

- **Copy** is necessary here, because `a` lives long. It is not destroyed immediately after this statement is executed.
- You cannot just "steal" the contents from `a`. The contents of `a` must be preserved.

6/43

## Distinguish between the different kinds of assignments

```
s = a;
```

```
s = a + b;
```

What is the key difference between them?

- `s = a` is an assignment from an **lvalue**,
- while `s = a + b` is an assignment from an **rvalue**.

If we only have the copy assignment operator, there is no way of distinguishing them.

\* Define two different assignment operators, one accepting an lvalue and the other accepting an rvalue?

7/43

## Rvalue References

A kind of reference that is bound to **rvalues**:

```
int &r = 42; // Error: Lvalue reference cannot be bound to rvalue.
int &&rr = 42; // Correct: `rr` is an rvalue reference.
const int &cr = 42; // Also correct:
 // Lvalue reference-to-const can be bound to rvalue.
const int &&cr2 = 42; // Correct, but useless:
 // Rvalue reference-to-const is seldom used.

int i = 42;
int &&rr2 = i; // Error: Rvalue reference cannot be bound to lvalue.
int &r2 = i * 42; // Error: Lvalue reference cannot be bound to rvalue.
const int &cr2 = i * 42; // Correct
int &&rr3 = i * 42; // Correct
```

- Lvalue references (to non-`const`) can only be bound to lvalues.
- Rvalue references can only be bound to rvalues.

8/43

## Overload Resolution

Such overloading is allowed:

```
void fun(const std::string &);
void fun(std::string &&);
```

- `fun(s1 + s2)` matches `fun(std::string &&)`, because `s1 + s2` is an rvalue.
- `fun(s)` matches `fun(const std::string &)`, because `s` is an lvalue.
- Note that if `fun(std::string &&)` does not exist, `fun(s1 + s2)` also matches `fun(const std::string &)`.

We will see how this kind of overloading benefit us soon.

9/43

## Move Operations

### Overview

The move constructor and the move assignment operator.

```
struct Widget {
 Widget(Widget &&) noexcept;
 Widget &operator=(Widget &&) noexcept;
 // Compared to the copy constructor and the copy assignment operator:
 Widget(const Widget &);
 Widget &operator=(const Widget &);
};
```

- Parameter type is **rvalue reference**, instead of lvalue reference-to-`const`.
- `noexcept` is (almost always) necessary!  $\Rightarrow$  We will talk about it in later lectures.

11/43

## The Move Constructor

Take the `Dynarray` as an example.

```
class Dynarray {
 int *m_storage;
 std::size_t m_length;
public:
 Dynarray(const Dynarray &other) // copy constructor
 : m_storage(new int[other.m_length]), m_length(other.m_length) {
 for (std::size_t i = 0; i != m_length; ++i)
 m_storage[i] = other.m_storage[i];
 }
 Dynarray(Dynarray &&other) noexcept // move constructor
 : m_storage(other.m_storage), m_length(other.m_length) {
 other.m_storage = nullptr;
 other.m_length = 0;
 }
};
```

12/43

## The Move Constructor

```
class Dynarray {
 int *m_storage;
 std::size_t m_length;
public:
 Dynarray(Dynarray &&other) noexcept // move constructor
 : m_storage(other.m_storage), m_length(other.m_length) {
 }
};
```

1. *Steal* the resources of `other`, instead of making a copy.

13/43

## The Move Constructor

```
class Dynarray {
 int *m_storage;
 std::size_t m_length;
public:
 Dynarray(Dynarray &&other) noexcept // move constructor
 : m_storage(other.m_storage), m_length(other.m_length) {
 other.m_storage = nullptr;
 other.m_length = 0;
 }
};
```

1. *Steal* the resources of `other`, instead of making a copy.
2. Make sure `other` is in a valid state, so that it can be safely destroyed.

\* Take ownership of `other`'s resources!

14/43

## The Move Assignment Operator

Take ownership of `other`'s resources!

```
class Dynarray {
public:
 Dynarray &operator=(Dynarray &&other) noexcept {

 m_storage = other.m_storage; m_length = other.m_length;

 return *this;
 }
};
```

1. *Steal* the resources from `other`.

15/43

## The Move Assignment Operator

```
class Dynarray {
public:
 Dynarray &operator=(Dynarray &&other) noexcept {

 m_storage = other.m_storage; m_length = other.m_length;
 other.m_storage = nullptr; other.m_length = 0;

 return *this;
 }
};
```

1. *Steal* the resources from `other`.
2. Make sure `other` is in a valid state, so that it can be safely destroyed.

Are we done?

16/43

## The Move Assignment Operator

```
class Dynarray {
public:
 Dynarray &operator=(Dynarray &&other) noexcept {

 delete[] m_storage;
 m_storage = other.m_storage; m_length = other.m_length;
 other.m_storage = nullptr; other.m_length = 0;

 return *this;
 }
};
```

0. Avoid memory leaks!

1. *Steal* the resources from `other`.
2. Make sure `other` is in a valid state, so that it can be safely destroyed.

Are we done?

17/43

## The Move Assignment Operator

```
class Dynarray {
public:
 Dynarray &operator=(Dynarray &&other) noexcept {
 if (this != &other) {
 delete[] m_storage;
 m_storage = other.m_storage; m_length = other.m_length;
 other.m_storage = nullptr; other.m_length = 0;
 }
 return *this;
 }
};
```

0. Avoid memory leaks!

1. *Steal* the resources from `other`.
2. Make sure `other` is in a valid state, so that it can be safely destroyed.

\* Self-assignment safe!

18/43

## Lvalues are Copied; Rvalues are Moved

Before we move on, let's define a function for demonstration.

Suppose we have a function that concatenates two `Dynarray` s:

```
Dynarray concat(const Dynarray &a, const Dynarray &b) {
 Dynarray result(a.size() + b.size());
 for (std::size_t i = 0; i != a.size(); ++i)
 result.at(i) = a.at(i);
 for (std::size_t i = 0; i != b.size(); ++i)
 result.at(a.size() + i) = b.at(i);
 return result;
}
```

Which assignment operator should be called?

```
a = concat(b, c);
```

19/43

## Lvalues are Copied; Rvalues are Moved

Lvalues are copied; rvalues are moved ...

```
a = concat(b, c); // calls move assignment operator,
 // because `concat(b, c)` is an rvalue.
a = b; // calls copy assignment operator
```

20/43

## Lvalues are Copied; Rvalues are Moved

Lvalues are copied; rvalues are moved ...

```
a = concat(b, c); // calls move assignment operator,
 // because `concat(b, c)` generates an rvalue.
a = b; // copy assignment operator
```

... but rvalues are copied if there is no move operation.

```
// If Dynarray has no move assignment operator, this is a copy assignment.
a = concat(b, c)
```

21/43

Synthesized Move Operations

Like copy operations, we can use `=default` to require a synthesized move operation that has the default behaviors.

```
struct X {
 X(X &&) = default;
 X &operator=(X &&) = default;
};
```

- The synthesized move operations perform member-wise initialization/assignment of the data members, in their declaration order, from an *xvalue*<sup>1</sup> argument.
  - i.e. `X(X&& o) : mem1(std::move(o.mem1)), mem2(std::move(o.mem2)), ... {}`. (Later we will talk about `std::move`.)
- The synthesized move operations are `noexcept` if all the operations involved are `noexcept`.

Move operations can also be deleted by `=delete`, but be careful ...<sup>2</sup> 22/43

The Rule of Five: Idea

The updated *copy control members*:

- **copy constructor**
- **copy assignment operator**
- **move constructor**
- **move assignment operator**
- **destructor**

If one of them has a user-provided version, the copy control of the class is thought of to have special behaviors. (Recall "the rule of three".)

23/43

The Rule of Five: Rules

- The **move constructor** or the **move assignment operator** will not be generated if any of the rest four members have a user-declared version.
- The **copy constructor** or **copy assignment operator**, if not provided by the user, will be implicitly `deleted` if the class has a user-provided **move operation**.
- The generation of the **copy constructor** or **copy assignment operator** is **deprecated** (since C++11) when the class has a user-declared **copy operation** or a **destructor**.
  - This is why some of you see this error:

```
Implicitly-declared copy assignment operator is deprecated, because the
class has a user-provided copy constructor.
```

24/43

The Rule of Five

The *copy control members* in modern C++:

- **copy constructor**
- **copy assignment operator**
- **move constructor**
- **move assignment operator**
- **destructor**

The Rule of Five: Define zero or five of them.

25/43

How to Invoke a Move Operation?

Suppose we give our `Dynarray` a label:

```
class Dynarray {
 int *m_storage;
 std::size_t m_length;
 std::string m_label;
};
```

The move assignment operator should invoke the **move assignment operator** on `m_label`. But how?

```
m_label = other.m_label; // calls copy assignment operator,
 // because 'other.m_label' is an lvalue.
```

26/43

`std::move`

`std::move`

Defined in `<utility>`

`std::move(x)` performs an **lvalue to rvalue** cast:

```
int ival = 42;
int &&rref = ival; // Error
int &&rref2 = std::move(ival); // Correct
```

Calling `std::move(x)` tells the compiler that:

- `x` is an **lvalue**, but
- we want to treat `x` as an **rvalue**.

27/43

`std::move`

`std::move(x)` indicates that we want to treat `x` as an **rvalue**, which means that `x` will be *moved from*.

The call to `std::move` **promises** that we do not intend to use `x` again,

- except to assign to it or to destroy it.

A call to `std::move` is usually followed by a call to some function that moves the object, after which **we cannot make any assumptions about the value of the moved-from object**.

```
void foo(X &&x); // moves `x`
void foo(const X &x); // copies `x`
foo(std::move(x)); // matches `foo(X&&)` , so that `x` is moved.
```

"`std::move` does not *move* anything. It just makes a *promise*."

28/43

Use `std::move`

Suppose we give every `Dynarray` a special "label", which is a string.

```
class Dynarray {
 int *m_storage;
 std::size_t m_length;
 std::string m_label;
public:
 Dynarray(Dynarray &&other) noexcept
 : m_storage(other.m_storage), m_length(other.m_length),
 m_label(std::move(other.m_label)) { // !!
 other.m_storage = nullptr;
 other.m_length = 0;
 }
};
```

The standard library facilities ought to define efficient and correct move operations.

30/43

Use `std::move`

Suppose we give every `Dynarray` a special "label", which is a string.

```
class Dynarray {
 int *m_storage;
 std::size_t m_length;
 std::string m_label;
public:
 Dynarray &operator=(Dynarray &&other) noexcept {
 if (this != &other) {
 delete[] m_storage;
 m_storage = other.m_storage; m_length = other.m_length;
 m_label = std::move(other.m_label);
 other.m_storage = nullptr; other.m_length = 0;
 }
 return *this;
 }
};
```

The standard library facilities ought to define efficient and correct move operations.

31/43

## Use `std::move`

Why do we need `std::move`?

```
class Dynarray {
public:
 Dynarray(Dynarray &&other) noexcept
 : m_storage(other.m_storage), m_length(other.m_length),
 m_label(other.m_label) { // Isn't this correct?
 other.m_storage = nullptr;
 other.m_length = 0;
 }
};
```

`other` is an rvalue reference, so ... ?

32/43

## An rvalue reference is an lvalue.

`other` is an rvalue reference, which is an lvalue.

- To move the object that the rvalue reference is bound to, we must call `std::move`.

```
class Dynarray {
public:
 Dynarray(Dynarray &&other) noexcept
 : m_storage(other.m_storage), m_length(other.m_length),
 m_label(other.m_label) { // `other.m_label` is copied, not moved.
 other.m_storage = nullptr;
 other.m_length = 0;
 }
};
```

An rvalue reference is an lvalue! Does that make sense?

33/43

## Lvalues persist; Rvalues are ephemeral.

The lifetime of rvalues is often very short, compared to that of lvalues.

- Lvalues have persistent state, whereas rvalues are either **literals** or **temporary objects** created in the course of evaluating expressions.

An rvalue reference **extends** the lifetime of the rvalue that it is bound to.

```
std::string s1 = something(), s2 = some_other_thing();
std::string &&rr = s1 + s2; // The state of the temporary object is "captured"
 // by the rvalue reference, without which the
 // temporary object will be destroyed.
std::cout << rr << '\n'; // Now we can use `rr` just like a normal string.
```

Golden rule: **Anything that has a name is an lvalue.**

- The rvalue reference has a name, so it is an lvalue.

34/43

## NRVO, Move and Copy Elision

35/43

## Returning a Temporary (pure rvalue)

```
std::string foo(const std::string &a, const std::string &b) {
 return a + b; // a temporary
}
std::string s = foo(a, b);
```

- First, a temporary is generated to store the result of `a + b`.
- How is this temporary returned?

36/43

## Returning a Temporary (pure rvalue)

```
std::string foo(const std::string &a, const std::string &b) {
 return a + b; // a temporary
}
std::string s = foo(a, b);
```

Since C++17, **no copy or move** is made here. The initialization of `s` is the same as

```
std::string s(a + b);
```

This is called **copy elision**.

37/43

## Returning a Named Object

```
Dynarray concat(const Dynarray &a, const Dynarray &b) {
 Dynarray result(a.size() + b.size());
 for (std::size_t i = 0; i != a.size(); ++i)
 result.at(i) = a.at(i);
 for (std::size_t i = 0; i != b.size(); ++i)
 result.at(a.size() + i) = b.at(i);
 return result;
}
a = concat(b, c);
```

- `result` is a local object of `concat`.
- Since C++11, `return result` performs a **move initialization** of a temporary object, say `tmp`.
- Then a **move assignment** to `a` is performed.

38/43

## Named Return Value Optimization, NRVO

```
Dynarray concat(const Dynarray &a, const Dynarray &b) {
 Dynarray result(a.size() + b.size());
 // ...
 return result;
}
Dynarray a = concat(b, c); // Initialization
```

NRVO transforms this code to

```
// Pseudo C++ code.
void concat(Dynarray &result, const Dynarray &a, const Dynarray &b) {
 // Pseudo C++ code. For demonstration only.
 result.Dynarray::Dynarray(a.size() + b.size()); // construct in-place
 // ...
}
Dynarray a@; // Uninitialized.
concat(a@, b, c);
```

so that no copy or move is needed.

39/43

## Named Return Value Optimization, NRVO

Note:

- NRVO was invented decades ago (even before C++98).
- NRVO is an **optimization**, but not mandatory.
- Even if NRVO is performed, the move constructor should still be available.
  - Because the compiler can choose not to perform NRVO.
  - The program should be syntactically correct ("well-formed"), no matter how the compiler treats it.

40/43

## Summary

Rvalue references

- are bound to rvalues, and extends the lifetime of the rvalue.
- Functions accepting `x &&` and `const x &` can be overloaded.
- An rvalue reference is an lvalue.

Move operations

- take ownership of resources from the other object.
- After a move operation, the moved-from object should be in a valid state that can be safely assigned to or destroyed.
- `=default`
- The rule of five: Define zero or five of the special member functions.

41/43

## Summary

`std::move`

- does not move anything. It only performs an lvalue-to-rvalue cast.
- `std::move(x)` makes a promise that `x` can be safely moved from.

In modern C++, unnecessary copies are greatly avoided by:

- copy-elision, which avoids the move or copy of temporary objects, and
- move, with the `return` ed lvalue treated as an rvalue, and
- NRVO, which constructs in-place the object to be initialized.

42/43

## Notes

<sup>1</sup> An xvalue is an rvalue that has an identity. A typical xvalue is one obtained from `std::move` applied to an lvalue.

<sup>2</sup> We seldom delete move operations. In most cases, we want rvalues to be copied if move is not possible. An explicitly deleted move operation will make rvalues not copyable, because deleted functions still participate in overload resolution.

An implicitly deleted move operation will be ignored in overload resolution. This is addressed by the defect report [CWG 1402](#) and is applied retroactively to C++11. Note that this change of behavior did not come into effect when the book C++ *Primer, 5e* was published.

43/43

## CS100 Lecture 18

Smart Pointers

## Contents

- Ideas
- `std::unique_ptr`
- `std::shared_ptr`

2/48

## Ideas

## Memory management is difficult!

For raw pointers obtained from `new` / `new[]` expressions, a manual `delete` / `delete[]` is required.

```
void runGame(const std::vector<Option> &options, const Settings &settings) {
 auto pWindow = new Window(settings.width, settings.height, settings.mode);
 auto pGame = new Game(options, settings, pWindow);
 // Run the game ...
 while (true) {
 auto key = getUserKeyAction();
 // ...
 }
 delete pGame; // You must not forget this.
 delete pWindow; // You must not forget this.
}
```

Will you always remember to `delete` ?

4/48

## Will you always remember to `delete` ?

```
void runGame(const std::vector<Option> &options, const Settings &settings) {
 auto pWindow = new Window(settings.width, settings.height, settings.mode);
 auto pGame = new Game(options, settings, pWindow);
 if (/* condition1 */) {
 // ...
 return; // `pWindow` and `pGame` should also be `delete`d here!
 }
 // Run the game ...
 while (true) {
 auto key = getUserKeyAction();
 // ...
 if (/* condition2 */) {
 // ...
 return; // `pWindow` and `pGame` should also be `delete`d here!
 }
 }
 delete pGame;
 delete pWindow;
}
```

5/48

## Idea: Make use of destructors.

```
struct WindowPtr { // A "smart pointer".
 Window *ptr;
 WindowPtr(Window *p) : ptr(p) {}
 ~WindowPtr() { delete ptr; } // The destructor will `delete` the object.
};
```

When the control reaches the end of the scope in which the `WindowPtr` lives, the destructor of `WindowPtr` will be called automatically.

```
void runGame(const std::vector<Option> &options, const Settings &settings) {
 WindowPtr pWindow(new Window(settings.width, settings.height, settings.mode));
 if (/* condition1 */) {
 // ...
 return; // `pWindow` is destroyed automatically, with its destructor called.
 }
 // ...
 // `pWindow` is destroyed automatically, with its destructor called.
}
```

6/48

## What if `WindowPtr` is copied?

Now `WindowPtr` only has a compiler-generated copy constructor, which copies the value of `ptr`.

```
{
 WindowPtr pWindow(new Window(settings.width, settings.height, settings.mode));
 auto copy = pWindow; // `copy.ptr` and `pWindow.ptr` point to the same object!
} // The object is deleted twice! Disaster!
```

What should be the behavior of `auto copy = pWindow;` ? Possible designs are:

1. Copy the object, as if `WindowPtr copy(new Window(*pWindow.ptr));`.
2. Copy the pointer, as if `WindowPtr copy(pWindow.ptr);`.
  - To avoid disasters caused by multiple `delete` s, some special design is needed.
3. Disable it. If there is no unique reasonable design, disable that operation.

7/48

## What if `WindowPtr` is copied?

What should be the behavior of `auto copy = pWindow;` ? Possible designs are:

1. Copy the object, as if `WindowPtr copy(new Window(*pWindow.ptr));`.
  - "Value semantics"
  - Typical example: Standard library containers. When you copy a `std::string`, a new string is created, with the contents copied.
  - May be referred to as "deep copy" in some other languages.
2. Copy the pointer, as if `WindowPtr copy(pWindow.ptr);`.
  - To avoid disasters caused by multiple `delete` s, some special design is needed.
  - "Pointer semantics", or "Reference semantics"
  - "shallow copy" in some other languages.
3. Disable it. If there is no unique reasonable design, disable that operation.
  - In this case, `pWindow` exclusively owns the `Window` object.

3/48

8/48



## Overview of smart pointers

A "smart pointer" is a pointer that manages its resources.

Possible behaviors of copy of a smart pointer:

1. Copy the object. (Value semantics)
  - **Standard library containers**. e.g. `std::string`, `std::vector`, `std::set`, ...
2. Copy the pointer, but with some special design. (Pointer semantics)
  - `std::shared_ptr<T>`. Defined in standard library file `<memory>`.
3. Disable it. (Unique ownership)
  - `std::weak_ptr<T>`. Defined in standard library file `<memory>`.

The smart pointers `std::shared_ptr<T>`, `std::weak_ptr<T>` and `std::weak_ptr<T>` are the C++'s answer to garbage collection.

- `std::weak_ptr` is not covered in CS100.

9/48

## Overview of smart pointers

The smart pointers `std::shared_ptr<T>`, `std::weak_ptr<T>` and `std::weak_ptr<T>` are the C++'s answer to garbage collection.

Smart pointers support the similar operations as raw pointers:

- `*sp` returns reference to the pointed-to object.
- `sp->mem` is equivalent to `(*sp).mem`.
- `sp` is *contextually convertible* to `bool`: It can be treated as a "condition".
  - It can be placed at the "condition" part of `if`, `for`, `while`, `do` statements.
  - It can be used as operands of `&&`, `||`, `!` or the first operand of `?:`.
  - In all cases, the conversion result is `true` iff `sp` holds an object (not "null").

[Best practice] In modern C++, prefer smart pointers to raw pointers.

10/48

## Design: Unique ownership of the object

```
class WindowPtr {
 Window *ptr;
public:
 WindowPtr(Window *p = nullptr) : ptr(p) {}
 ~WindowPtr() { delete ptr; }
 WindowPtr(const WindowPtr &) = delete;
 WindowPtr &operator=(const WindowPtr &) = delete;
 WindowPtr(WindowPtr &&other) noexcept : ptr(other.ptr) { other.ptr = nullptr; }
 WindowPtr &operator=(WindowPtr &&other) noexcept {
 if (&other != this) {
 delete ptr; ptr = other.ptr; other.ptr = nullptr;
 }
 return *this;
 }
};
```

Move of a unique-pointer: transfer of ownership.

- Move-only type

14/48

## `std::unique_ptr`

Like `std::vector`, `std::unique_ptr` is also a class template. It is not a type itself.

- `std::unique_ptr<PointeeType>` is the complete type name, where `PointeeType` is the type of the object that it points to.
- For `T ≠ U`, `std::unique_ptr<T>` and `std::unique_ptr<U>` are **two different and independent types**.

Same for `std::shared_ptr`, which we will talk about later.

15/48

## Creating a `std::unique_ptr`: Two common ways

- Pass a pointer created by `new` to the constructor:

```
std::unique_ptr<Student> p(new Student("Bob", 2020123123));
```

- Use `std::make_unique<T>`, and pass the initializers to it.

```
std::unique_ptr<Student> p1 = std::make_unique<Student>("Bob", 2020123123);
auto p2 = std::make_unique<Student>("Alice", 2020321321);
```

- `std::make_unique<T>(args...)` *perfectly forwards* the arguments `args...` to the constructor of `T`, as if the object were created by `new T(args...)`.
- `std::make_unique<T>` returns a `std::unique_ptr<T>` to the created object.

11/48

16/48

## Design: Unique ownership of the object

```
class WindowPtr {
 Window *ptr;
public:
 WindowPtr(Window *p = nullptr) : ptr(p) {}
 ~WindowPtr() { delete ptr; }
};
```

A "unique-pointer" saves a raw pointer internally, pointing to the object it owns.

When the unique-pointer is destroyed, it disposes of the object it owns.

12/48

## Design: Unique ownership of the object

```
class WindowPtr {
 Window *ptr;
public:
 WindowPtr(Window *p = nullptr) : ptr(p) {}
 ~WindowPtr() { delete ptr; }
 WindowPtr(const WindowPtr &) = delete;
 WindowPtr &operator=(const WindowPtr &) = delete;
};
```

The unique-pointer **exclusively** owns the object. Copying a unique-pointer is not allowed.

13/48

## Default initialization of a `std::unique_ptr`

```
std::unique_ptr<T> up;
```

The default constructor of `std::unique_ptr<T>` initializes `up` to be a "null pointer".

`up` is in the state that does not own any object.

- This is a defined and deterministic behavior! It is **not** holding some indeterminate value.
  - The standard library hates indeterminate values, just as we do.

17/48

## `std::unique_ptr`: Automatic memory management

```
void foo() {
 auto pAlice = std::make_unique<Student>("Alice", 2020321321);
 // Do something...
 if (some_condition()) {
 auto pBob = std::make_unique<Student>("Bob", 2020123123);
 // ...
 } // `Student::~~Student()` is called for Bob,
 // because the lifetime of `pBob` ends.
} // `Student::~~Student()` is called for Alice,
// because the lifetime of `pAlice` ends.
```

A `std::unique_ptr` automatically calls the destructor once it gets destroyed or assigned a new value.

- No manual `delete` needed!

18/48

## std::unique\_ptr: Move-only

```
auto p = std::make_unique<std::string>(5, 'c');
std::cout << *p << std::endl; // Prints "ccccc".
auto q = p; // Error. Copy is not allowed.
auto r = std::move(p); // Correct.
// Now the ownership of this string has been transferred to `r`.
std::cout << *r << std::endl; // Prints "ccccc".
if (!p) // true
 std::cout << "p is \"null\" now." << std::endl;
```

std::unique\_ptr is not copyable, but only movable.

- Remember, only one std::unique\_ptr can point to the managed object.
- Move of a std::unique\_ptr is the transfer of ownership of the managed object.

19/48

## Express your intent precisely.

```
// Given that `pWindow` is a `std::unique_ptr<Window>`.
auto p = pWindow; // Oops, attempting to copy a `std::unique_ptr`.
```

What are you going to do with p?

- If you want p to take over the object managed by pWindow, change it to auto p = std::move(pWindow);.
  - Be careful! pWindow will no longer own that object.
- If you want to p to be another smart pointer that *shares* the ownership with pWindow, std::unique\_ptr is not suitable here. ⇒ See std::shared\_ptr later.

24/48

## std::unique\_ptr: Move-only

```
auto p = std::make_unique<std::string>(5, 'c');
std::cout << *p << std::endl; // Prints "ccccc".
auto q = p; // Error. Copy is not allowed.
auto r = std::move(p); // Correct.
// Now the ownership of this string has been transferred to `r`.
std::cout << *r << std::endl; // Prints "ccccc".
if (!p) // true
 std::cout << "p is \"null\" now." << std::endl;
```

After auto up2 = std::move(up1);, up1 becomes "null". The object that up1 used to manage now belongs to up2.

The assignment up2 = std::move(up1) destroys the object that up2 used to manage, and lets up2 take over the object managed by up1. After that, up1 becomes "null".

20/48

## Returning a std::unique\_ptr

```
struct Window {
 // A typical "factory" function.
 static std::unique_ptr<Window> create(const Settings &settings) {
 auto pw = std::make_unique<Window>(/* some arguments */);
 logWindowCreation(pw);
 // ...
 return pw;
 }
};
auto state = Window::create(my_settings);
```

A temporary is move-constructed from pw, and then is used to move-construct state.

- These two moves can be optimized out by NRVO.

25/48

## Express your intent precisely.

You may accidentally write the following code:

```
// Given that `pWindow` is a `std::unique_ptr<Window>`.
auto p = pWindow; // Oops, attempting to copy a `std::unique_ptr`.
```

The compiler gives an error, complaining about the use of deleted copy constructor.

What are you going to do?

- Change it to auto p = std::move(pWindow);.
- Give up on smart pointers, and switch back to raw pointers.
- Copy-and-paste the compiler output and ask ChatGPT.

21/48

## Other operations on std::unique\_ptr

up.reset(), up.release(), up1.swap(up2), up1 == up2, etc.

Full list of operations supported on a std::unique\_ptr.

26/48

## Express your intent precisely.

You may accidentally write the following code:

```
// Given that `pWindow` is a `std::unique_ptr<Window>`.
auto p = pWindow; // Oops, attempting to copy a `std::unique_ptr`.
```

The compiler gives an error, complaining about the use of deleted copy constructor.

- Syntactically, a std::unique\_ptr is not copyable, but you are copying it. (Direct cause of the error)
- Logically, a std::unique\_ptr must exclusively manage the pointed-to object. Why would you copy a std::unique\_ptr?
  - The root cause of the error is related to your intent: What are you going to do with p?

22/48

## std::unique\_ptr for array type

By default, the destructor of std::unique\_ptr<T> uses a delete expression to destroy the object it holds.

What happens if std::unique\_ptr<T> up(new T[n]);?

27/48

## Express your intent precisely.

```
// Given that `pWindow` is a `std::unique_ptr<Window>`.
auto p = pWindow; // Oops, attempting to copy a `std::unique_ptr`.
```

What are you going to do with p?

- If you want to copy the pointed-to object, change it to auto p = std::make\_unique<Window>(\*pWindow);.
- If you want p to be just an *observer*, write auto p = pWindow.get();.
  - pWindow.get() returns a raw pointer to the object, which is of type Window\*.
  - Be careful! As an observer, p should never interfere in the lifetime of the object. A simple delete p; will cause disaster.

23/48

## std::unique\_ptr for array type

By default, the destructor of std::unique\_ptr<T> uses a delete expression to destroy the object it holds.

What happens if std::unique\_ptr<T> up(new T[n]);?

- The memory is obtained using new[], but deallocated by delete! Undefined behavior.

28/48

## std::unique\_ptr for array type

A template specialization: `std::unique_ptr<T[]>`.

- Specially designed to represent pointers that point to a "dynamic array" of objects.
- It has some array-specific operators, e.g. `operator[]`. In contrast, it does not support `operator*` and `operator->`.
- It uses `delete[]` instead of `delete` to destroy the objects.

```
auto up = std::make_unique<int[]>(n);
std::unique_ptr<int[]> up2(new int[n]{}); // equivalent
for (auto i = 0; i != n; ++i)
 std::cout << up[i] << ' ';
```

29/48

## Motivation

Design a "shared-pointer" that allows the object it manages to be *shared*.

When should the object be destroyed?

- A `std::unique_ptr` destroys the object it manages when the pointer itself is destroyed.
- If we allow many shared-pointers to point to the same object, how can we know when to destroy that object?

34/48

## std::unique\_ptr for array type

~~A template specialization: `std::unique_ptr<T[]>`:~~

~~—Specially designed to represent pointers that point to a "dynamic array" of objects.~~  
~~—It has some array-specific operators, e.g. `operator[]`. In contrast, it does not support `operator*` and `operator->`.~~  
~~—It uses `delete[]` instead of `delete` to destroy the objects.~~

## Use standard library containers instead!

They almost always do a better job. `std::unique_ptr<T[]>` is seldom needed.

30/48

## Idea: Reference counting

Set a counter that counts how many `WindowPtr`s are pointing to it:

```
struct WindowWithCounter {
 Window theWindow;
 int refCount = 1;
};
```

When a new object is created by a `WindowPtr`, set the `refCount` to 1.

35/48

## std::unique\_ptr is zero-overhead.

`std::unique_ptr` stores nothing more than a raw pointer. <sup>1</sup>

It does nothing more than better copy / move control and automatic object destruction.

Zero-overhead: Using a `std::unique_ptr` does not cost more time or space than using raw pointers.

[Best practice] [Use `std::unique\_ptr` for exclusive-ownership resource management](#).

31/48

## Idea: Reference counting

When a `WindowPtr` is copied, let it point to the same object, and increment the counter.

```
class WindowPtr {
 WindowWithCounter *ptr;
public:
 WindowPtr(const WindowPtr &other) : ptr(other.ptr) { ++ptr->refCount; }

};
```

36/48

## Idea: Reference counting

For copy assignment: the counter of the old object should be decremented.

- If it reaches zero, destroy that object!

```
class WindowPtr {
 WindowWithCounter *ptr;
public:
 WindowPtr(const WindowPtr &other) : ptr(other.ptr) { ++ptr->refCount; }
 WindowPtr &operator=(const WindowPtr &other) {
 if (--ptr->refCount == 0)
 delete ptr;
 ptr = other.ptr;
 ++ptr->refCount;
 return *this;
 }
};
```

\* Is this correct?

37/48

## std::shared\_ptr

## Motivation

A `std::unique_ptr` exclusively owns an object, but sometimes this is not convenient.

```
struct WindowManager {
 void addWindow(const std::unique_ptr<Window> &pW) {
 mWindows.push_back(pW); // Error: Attempts to copy a `std::unique_ptr`.
 }
private:
 std::vector<std::unique_ptr<Window>> mWindows;
};

struct Window {
 static std::unique_ptr<Window> create(const Settings &settings) {
 auto pW = std::make_unique<Window>(/* some arguments */);
 logWindowCreation(pW);
 settings.getWindowManager().addWindow(pW);
 return pW;
 }
};
```

33/48

## Idea: Reference counting

Self-assignment safe!!!

```
class WindowPtr {
 WindowWithCounter *ptr;
public:
 WindowPtr(const WindowPtr &other) : ptr(other.ptr) { ++ptr->refCount; }
 WindowPtr &operator=(const WindowPtr &other) {
 ++other.ptr->refCount;
 if (--ptr->refCount == 0)
 delete ptr;
 ptr = other.ptr;
 return *this;
 }
};
```

38/48

## Idea: Reference counting

Destructor: decrement the counter, and destroy the object if the counter reaches zero.

```
class WindowPtr {
 WindowWithCounter *ptr;
public:
 WindowPtr(const WindowPtr &other) : ptr(other.ptr) { ++ptr->refCount; }
 WindowPtr &operator=(const WindowPtr &other) {
 ++other.ptr->refCount;
 if (--ptr->refCount == 0)
 delete ptr;
 ptr = other.ptr;
 return *this;
 }
 ~WindowPtr() {
 if (--ptr->refCount == 0)
 delete ptr;
 }
};
```

39/48

## Idea: Reference counting

Move: Just "steal" the object. It is also the transfer of ownership.

```
class WindowPtr {
 WindowWithCounter *ptr;
public:
 WindowPtr(WindowPtr &&other) noexcept : ptr(other.ptr) { other.ptr = nullptr; }
 WindowPtr &operator=(WindowPtr &&other) noexcept {
 if (this != &other) {
 if (--ptr->refCount == 0)
 delete ptr;
 ptr = other.ptr; other.ptr = nullptr;
 }
 return *this;
 }
};
```

40/48

## Reference counting

By maintaining a variable that counts how many shared-pointers are pointing to the object, we can know when to destroy the object.

This strategy is adopted by Python.

It can prevent memory leak in many cases, but not all cases! ⇒ See the question in the end of this lecture's slides.

41/48

## Operations

`*` and `->` can be used as if it is a raw pointer:

```
auto sp = std::make_shared<std::string>(10, 'c');
std::cout << *sp << std::endl; // "ccccccccc"
std::cout << sp->size() << std::endl; // "10"
```

`sp.use_count()`: The value of the reference counter.

```
auto sp = std::make_shared<std::string>(10, 'c');
{
 auto sp2 = sp;
 std::cout << sp.use_count() << std::endl; // 2
} // `sp2` is destroyed, but the managed object is not destroyed.
std::cout << sp.use_count() << std::endl; // 1
```

44/48

## Operations

Full list of supported operations on `std::shared_ptr`.

`std::shared_ptr` is relatively easy to use, since you are free to create many `std::shared_ptr`s pointing to one object.

However, `std::shared_ptr` has time and space overhead. Copy of a `std::shared_ptr` requires maintenance of reference counter.

45/48

## Summary

`std::unique_ptr`

- Exclusive-ownership.
- Move-only. Move is the transfer of ownership.
- Zero-overhead.

`std::shared_ptr`

- Shared-ownership.
- Uses reference counting.
  - Copy increments the reference counter.
  - When the counter is decremented to zero, the object is destroyed.

46/48

## Question

Does `std::shared_ptr` prevent memory leak in all cases? Think about what happens in the following code.

```
struct Node {
 int value;
 std::shared_ptr<Node> next;
 Node(int x, std::shared_ptr<Node> p) : value(x), next{std::move(p)} {}
};
void foo() {
 auto p = std::make_shared<Node>(1, nullptr);
 p->next = std::make_shared<Node>(2, p);
 p.reset();
}
```

47/48

## `std::shared_ptr`

A smart pointer that uses **reference counting** to manage shared objects.

Create a `shared_ptr`:

```
std::shared_ptr<Type> sp2(new Type(args));
auto sp = std::make_shared<Type>(args); // equivalent, but better
```

For example:

```
// sp points to a string "ccccccccc".
auto sp = std::make_shared<std::string>(10, 'c');

auto pWindow = std::make_shared<Window>(80, 24, my_settings.mode);
```

42/48

## Create a `shared_ptr`

Note: For `std::unique_ptr`, both of the following ways are ok (since C++17):

```
auto up = std::make_unique<Type>(args);
std::unique_ptr<Type> up2(new Type(args));
```

For `std::shared_ptr`, `std::make_shared` is preferable to directly using `new`.

```
auto sp = std::make_shared<Type>(args); // preferred
std::shared_ptr<Type> sp2(new Type(args)); // ok, but less preferred
```

Read *Effective Modern C++* Item 21. (Note that this book is based on C++14.)

[Best practice] Prefer `std::make_shared` to directly using `new` when creating a `std::shared_ptr`.

43/48

## Notes

<sup>1</sup> The `deleter` that a `std::unique_ptr` uses is customizable, which also has to be stored in a `std::unique_ptr`. By default, `std::default_delete` is used that performs `delete ptr;` to destroy the object and release the memory. `std::unique_ptr` often uses some space-saving tricks to store the deleter. If the deleter is "stateless" (e.g. an object with no non- static data members), it may be stored with no unique address so that no extra space is required. `std::default_delete` belongs to this kind. Therefore, `sizeof(std::unique_ptr<T>)` is often reasonably equal to `sizeof(T *)`.

48/48

## CS100 Lecture 19

### Operator Overloading

1/46

### Contents

- Basics
- Example: `Rational`
  - Arithmetic and relational operators
  - Increment and decrement operators (`++`, `--`)
  - IO operators (`<<`, `>>`)
- Example: `Dynarray`
  - Subscript operator (`[]`)
- Example: `WindowPtr`
  - Dereference (indirection) operator (`*`)
  - Member access through pointer (`->`)
- User-defined type conversions

2/46

### Basics

Operator overloading: Provide the behaviors of **operators** for class types.

We have already seen some:

- The **copy assignment operator** and the **move assignment operator** are two special overloads for `operator=`.
- The `IOstream` library provides overloaded `operator<<` and `operator>>` to perform input and output.
- The `string` library provides `operator+` for concatenation of strings, and `<`, `<=`, `>`, `>=`, `==`, `!=` for comparison in lexicographical order.
- Standard library containers and `std::string` have `operator[]`.
- Smart pointers have `operator*` and `operator->`.

3/46

### Basics

Overloaded operators can be defined in two forms:

- as a member function, in which the leftmost operand is bound to `this`:
  - `a[i] ⇔ a.operator[](i)`
  - `a = b ⇔ a.operator=(b)`
  - `*a ⇔ a.operator*()`
  - `f(arg1, arg2, arg3, ...)` ⇔ `f.operator()(arg1, arg2, arg3, ...)`
- as a non-member function:
  - `a == b ⇔ operator==(a, b)`
  - `a + b ⇔ operator+(a, b)`

4/46

### Basics

Some operators cannot be overloaded:

`obj.mem`, `::`, `?:`, `obj.*memptr` (not covered in CS100)

Some operators can be overloaded, but are strongly not recommended:

`cond1 && cond2`, `cond1 || cond2`

- Reason: Since `x && y` would become `operator&&(x, y)`, there is no way to overload `&&` (or `||`) that preserves the **short-circuit evaluation** property.

5/46

### Basics

- At least one operand should be a class type. Modifying the behavior of operators on built-in types is not allowed.

```
int operator+(int, int); // Error.
MyInt operator-(int, int); // Still error.
```

- Inventing new operators is not allowed.

```
double operator**(double x, double exp); // Error.
```

- Overloading does not modify the **associativity**, **precedence** and the **operands' evaluation order**.

```
std::cout << a + b; // Equivalent to `std::cout << (a + b)`.
```

6/46

### Example: `Rational`

7/46

### A class for rational numbers

```
class Rational {
 int m_num; // numerator
 unsigned m_denom; // denominator
 void simplify() { // Private, because this is our implementation detail.
 int gcd = std::gcd(m_num, m_denom); // std::gcd in <numeric> (since C++17)
 m_num /= gcd; m_denom /= gcd;
 }
public:
 Rational(int x = 0) : m_num{x}, m_denom{1} {} // Also a default constructor.
 Rational(int num, unsigned denom) : m_num{num}, m_denom{denom} { simplify(); }
 double to_double() const {
 return static_cast<double>(m_num) / m_denom;
 }
};
```

We want to have arithmetic operators supported for `Rational`.

8/46

### `Rational`: arithmetic operators

A good way: define `operator+=` and the unary `operator-`, and then define other operators in terms of them.

```
class Rational {
 friend Rational operator-(const Rational &); // Unary `operator-` as in `-x`.
public:
 Rational &operator+=(const Rational &rhs) {
 m_num = m_num * static_cast<int>(rhs.m_denom) // Be careful with `unsigned`!
 + static_cast<int>(m_denom) * rhs.m_num;
 m_denom *= rhs.m_denom;
 simplify();
 return *this; // `x += y` should return a reference to `x`.
 }
};
Rational operator-(const Rational &x) {
 return {-x.m_num, x.m_denom};
 // The above is equivalent to `return Rational(-x.m_num, x.m_denom);`.
}
```

9/46

### `Rational`: arithmetic operators

Define the arithmetic operators in terms of the compound assignment operators.

```
class Rational {
public:
 Rational &operator-=(const Rational &rhs) {
 // Makes use of `operator+=` and the unary `operator-`.
 return *this += -rhs;
 }
};
Rational operator+(const Rational &lhs, const Rational &rhs) {
 return Rational(lhs) += rhs; // Makes use of `operator+=`.
}
Rational operator-(const Rational &lhs, const Rational &rhs) {
 return Rational(lhs) -= rhs; // Makes use of `operator-=`.
}
```

10/46

## [Best practice] Avoid repetition.

```
class Rational {
public:
 Rational &operator+=(const Rational &rhs) {
 m_num = m_num * static_cast<int>(rhs.m_denom)
 + static_cast<int>(m_denom) * rhs.m_num;
 m_denom *= rhs.m_denom;
 simplify();
 return *this;
 }
};
```

The arithmetic operators for `Rational` are simple yet requires carefulness.

- Integers with different signed-ness need careful treatment.
- Remember to `simplify()`.

Fortunately, we only need to pay attention to these things in `operator+=`. Everything will be right if `operator+=` is right.

11/46

## [Best practice] Avoid repetition.

The code would be very error-prone if you implement every function from scratch!

```
class Rational {
public:
 Rational &operator+=(const Rational &rhs) {
 m_num = m_num * static_cast<int>(rhs.m_denom)
 + static_cast<int>(m_denom) * rhs.m_num;
 m_denom *= rhs.m_denom;
 simplify();
 return *this;
 }
 Rational &operator-=(const Rational &rhs) {
 m_num = m_num * static_cast<int>(rhs.m_denom)
 - static_cast<int>(m_denom) * rhs.m_num;
 m_denom *= rhs.m_denom;
 simplify();
 return *this;
 }
 friend Rational operator+(const Rational &,
 const Rational &);
 friend Rational operator-(const Rational &,
 const Rational &);
};
```

```
Rational operator+(const Rational &lhs,
 const Rational &rhs) {
 return {
 lhs.m_num * static_cast<int>(rhs.m_denom)
 + static_cast<int>(lhs.m_denom) * rhs.lhs,
 lhs.m_denom * rhs.m_denom
 };
}
Rational operator-(const Rational &lhs,
 const Rational &rhs) {
 return {
 lhs.m_num * static_cast<int>(rhs.m_denom)
 - static_cast<int>(lhs.m_denom) * rhs.lhs,
 lhs.m_denom * rhs.m_denom
 };
}
```

12/46

## Rational: arithmetic operators

Exercise: Define `operator*` (multiplication) and `operator/` (division) as well as `operator*=` and `operator/=` for `Rational`.

13/46

## Rational: arithmetic and relational operators

What if we define them (say, `operator+`) as member functions?

```
class Rational {
public:
 Rational(int x = 0) : m_num{x}, m_denom{1} {}
 Rational operator+(const Rational &rhs) const {
 return {
 m_num * static_cast<int>(rhs.m_denom)
 + static_cast<int>(m_denom) * rhs.m_num,
 m_denom * rhs.m_denom
 };
 }
};
```

14/46

## Rational: arithmetic and relational operators

What if we define them (say, `operator+`) as member functions?

```
class Rational {
public:
 Rational(int x = 0) : m_num{x}, m_denom{1} {}
 Rational operator+(const Rational &rhs) const {
 // ...
 }
};
```

```
Rational r = some_value();
auto s = r + 0; // OK, `r.operator+(0)`, effectively `r.operator+(Rational(0))`
auto t = 0 + r; // Error! `0.operator+(r)` ???
```

15/46

## Rational: arithmetic and relational operators

To allow implicit conversions on both sides, the operator should be defined as **non-member functions**.

```
Rational r = some_value();
auto s = r + 0; // OK, `operator+(r, 0)`, effectively `operator+(r, Rational(0))`
auto t = 0 + r; // OK, `operator+(0, r)`, effectively `operator+(Rational(0), r)`
```

[Best practice] The "symmetric" operators, whose operands are often exchangeable, often should be defined as non-member functions.

16/46

## Rational: relational operators

Define `<` and `==`, and define others in terms of them. (Before C++20)

- Since C++20: Define `==` and `<=>`, and the compiler will generate others.

A possible way: Use `to_double` and compare the floating-point values.

```
bool operator<(const Rational &lhs, const Rational &rhs) {
 return lhs.to_double() < rhs.to_double();
}
```

- This does not require `operator<` to be a `friend`.
- However, this is subject to floating-point errors.

17/46

## Rational: relational operators

Another way (possibly better):

```
class Rational {
 friend bool operator<(const Rational &, const Rational &);
 friend bool operator==(const Rational &, const Rational &);
};
bool operator<(const Rational &lhs, const Rational &rhs) {
 return static_cast<int>(rhs.m_denom) * lhs.m_num
 < static_cast<int>(lhs.m_denom) * rhs.m_num;
}
bool operator==(const Rational &lhs, const Rational &rhs) {
 return lhs.m_num == rhs.m_num && lhs.m_denom == rhs.m_denom;
}
```

If there are member functions to obtain the numerator and the denominator, these functions don't need to be `friend`.

18/46

## Rational: relational operators

[Best practice] Avoid repetition.

Define others in terms of `<` and `==`:

```
bool operator>(const Rational &lhs, const Rational &rhs) {
 return rhs < lhs;
}
bool operator<=(const Rational &lhs, const Rational &rhs) {
 return !(lhs > rhs);
}
bool operator>=(const Rational &lhs, const Rational &rhs) {
 return !(lhs < rhs);
}
bool operator!=(const Rational &lhs, const Rational &rhs) {
 return !(lhs == rhs);
}
```

19/46

## Relational operators

Define relational operators in a consistent way:

- `a != b` should mean `!(a == b)`
- `!(a < b)` and `!(a > b)` should imply `a == b`

C++20 has devoted some efforts to the design of consistent comparison: [P0515r3](#).

20/46

## Relational operators

Avoid abuse of relational operators:

```
struct Point2d { double x, y; };
bool operator<(const Point2d &lhs, const Point2d &rhs) {
 return lhs.x < rhs.x; // Is this the unique, best behavior?
}
// Much better design: Use a named function.
bool less_in_x(const Point2d &lhs, const Point2d &rhs) {
 return lhs.x < rhs.x;
}
```

[Best practice] Operators should be used for operations that are likely to be unambiguous to users.

- If an operator has plausibly more than one interpretation, use named functions instead. Function names can convey more information.

`std::string` has `operator+` for concatenation. Why doesn't `std::vector` have one? 21/46

### ++ and --

`++` and `--` are often defined as **members**, because they modify the object.

To differentiate the postfix version `x++` and the prefix version `++x`: **The postfix version has a parameter of type `int`.**

- The compiler will translate `++x` to `x.operator++()`, `x++` to `x.operator++(0)`.

```
class Rational {
public:
 Rational &operator++() { ++m_num; simplify(); return *this; }
 Rational operator++(int) { // This 'int' parameter is not used.
 // The postfix version is almost always defined like this.
 auto tmp = *this;
 ++*this; // Makes use of the prefix version.
 return tmp;
 }
};
```

22/46

### ++ and --

```
class Rational {
public:
 Rational &operator++() { ++m_num; simplify(); return *this; }
 Rational operator++(int) { // This 'int' parameter is not used.
 // The postfix version is almost always defined like this.
 auto tmp = *this;
 ++*this; // Make use of the prefix version.
 return tmp;
 }
};
```

The prefix version returns reference to `*this`, while the postfix version returns a copy of `*this` before incrementation.

- Same as the built-in behaviors.

23/46

## IO operators

Implement `std::cin >> r` and `std::cout << r`.

Input operator:

```
std::istream &operator>>(std::istream &, Rational &);
```

Output operator:

```
std::ostream &operator<<(std::ostream &, const Rational &);
```

- `std::cin` is of type `std::istream`, and `std::cout` is of type `std::ostream`.
- The left-hand side operand should be returned, so that we can write

```
std::cin >> a >> b >> c; std::cout << a << b << c;
```

24/46

## Rational: output operator

```
class Rational {
 friend std::ostream &operator<<(std::ostream &, const Rational &);
};
std::ostream &operator<<(std::ostream &os, const Rational &r) {
 return os << r.m_num << '/' << r.m_denom;
}
```

If there are member functions to obtain the numerator and the denominator, it don't have to be a `friend`.

```
std::ostream &operator<<(std::ostream &os, const Rational &r) {
 return os << r.get_numerator() << '/' << r.get_denominator();
}
```

25/46

## Rational: input operator

Suppose the input format is `a b` for the rational number  $\frac{a}{b}$ , where `a` and `b` are integers.

```
std::istream &operator>>(std::istream &is, Rational &r) {
 int x, y; is >> x >> y;
 if (!is) { // Pay attention to input failures!
 x = 0;
 y = 1;
 }
 if (y < 0) { y = -y; x = -x; }
 r = Rational(x, y);
 return is;
}
```

26/46

## Example: Dynarray

```
class Dynarray {
public:
 int &operator[](std::size_t n) {
 return m_storage[n];
 }
 const int &operator[](std::size_t n) const {
 return m_storage[n];
 }
};
```

The use of `a[i]` is interpreted as `a.operator[](i)`.

(C++23 allows `a[i, j, k] !`)

Homework: Define `operator[]` and relational operators for `Dynarray`.

28/46

## Example: WindowPtr

Recall the `WindowPtr` class we defined in the previous lecture.

```
struct WindowWithCounter {
 Window theWindow;
 int refCount = 1;
};
class WindowPtr {
 WindowWithCounter *m_ptr;
public:
 Window &operator*() const { // Why should it be const?
 return *m_ptr->theWindow;
 }
};
```

We want `*sp` to return reference to the managed object.

30/46



WindowPtr: indirection (dereference) operator

Why should operator\* be const?

```
class WindowPtr {
 WindowWithCounter *m_ptr;
public:
 Window &operator*() const {
 return m_ptr->theWindow;
 }
};
```

On a const WindowPtr ("top-level" const), obtaining a non-const reference to the managed object may still be allowed.

- The (smart) pointer is const, but the managed object is not.
- this is const WindowPtr \*, so m\_ptr is WindowWithCounter \*const.

WindowPtr: member access through pointer

To make operator-> consistent with operator\* (make ptr->mem equivalent to (\*ptr).mem), operator-> is almost always defined like this:

```
class WindowPtr {
public:
 Window *operator->() const {
 return std::addressof(operator*());
 }
};
```

std::addressof(x) is almost always equivalent to &x, but the latter may not return the address of x if operator& for x has been overloaded!

User-defined type conversions

Type conversions

A type conversion is a function f: T -> U for two different types T and U. Type conversions can happen either implicitly or explicitly. A conversion is explicit if and only if the target type U is written explicitly in the conversion expression.

Explicit conversions can happen in one of the following forms:

| expression         | explanation                   | example                        |
|--------------------|-------------------------------|--------------------------------|
| what_cast<U>(expr) | through named casts           | static_cast<int>(3.14)         |
| U(expr)            | looks like a constructor call | std::string("xx"), int(3.14)   |
| (U)expr            | old C-style conversion        | Not recommended. Don't use it. |

Type conversions

A type conversion is a function f: T -> U for two different types T and U. Type conversions can happen either implicitly or explicitly. A conversion is explicit if and only if the target type U is written explicitly in the conversion expression.

- Arithmetic conversions are often allowed to happen implicitly:

```
int sum = /* ... */; n = /* ... */;
auto average = 1.0 * sum / n; // 'sum' and 'n' are converted to 'double',
 // so 'average' has type 'double'.
```

- The dangerous conversions for built-in types must be explicit:

```
const int *cip = something();
auto ip = const_cast<int*>(cip); // int *
auto cp = reinterpret_cast<char*>(ip); // char *
```

Type conversions

A type conversion is a function f: T -> U for two different types T and U. Type conversions can happen either implicitly or explicitly. A conversion is explicit if and only if the target type U is written explicitly in the conversion expression.

- This is also a type conversion, isn't it?

```
std::string s = "hello"; // from 'const char [6]' to 'std::string'
```

- This is also a type conversion, isn't it?

```
std::size_t n = 1000;
std::vector<int> v(n); // from 'std::size_t' to 'std::vector<int>'
```

How do these type conversions happen? Are they implicit or explicit?

Type conversions

We can define a type conversion for our class X in one of the following ways:

- A constructor with exactly one parameter of type T is a conversion from T to X.
  - Example: std::string has a constructor accepting a const char \*. std::vector has a constructor accepting a std::size\_t.
- A type conversion operator: a conversion from X to some other type.

```
class Rational {
public:
 // conversion from 'Rational' to 'double'.
 operator double() const { return 1.0 * m_num / m_denom; }
};
Rational r(3, 4);
double dval = r; // 0.75
```

Type conversion operator

A type conversion operator is a member function of class X, which defines the type conversion from X to some other type T.

```
class Rational {
public:
 // conversion from 'Rational' to 'double'.
 operator double() const { return 1.0 * m_num / m_denom; }
};
Rational r(3, 4);
double dval = r; // 0.75
```

- The name of the function is operator T.
- The return type is T, which is not written before the name.
- A type conversion is usually a read-only operation, so it is usually const.

Explicit type conversion

Some conversions should be allowed to happen implicitly:

```
void foo(const std::string &str) { /* ... */ }
foo("hello"); // implicit conversion from 'const char [6]' to 'const char *',
 // and then to 'std::string'.
```

Some should never happen implicitly!

```
void bar(const std::vector<int> &vec) { /* ... */ }
bar(1000); // ??? Too weird!
bar(std::vector<int>(1000)) // OK.
std::vector<int> v1(1000); // OK.
std::vector<int> v2 = 1000; // No! This should never happen. Too weird!
```

Explicit type conversion

To disallow the implicit use of a constructor as a type conversion, write explicit before the return type:

```
class string { // Suppose this is the 'std::string' class.
public:
 string(const char *cstr); // Not marked 'explicit'. Implicit use is allowed.
};

template <typename T> class vector { // Suppose this is the 'std::vector' class.
public:
 explicit vector(std::size_t n); // Implicit use is not allowed.
};

class Dynarray {
public:
 explicit Dynarray(std::size_t n) : m_length{n}, m_storage{new int[n]{} } {}
};
```



## Explicit type conversion

To disallow the implicit use of a type conversion operator, also write `explicit`:

```
class Rational {
public:
 explicit operator double() const { return 1.0 * m_num / m_denom; }
};
Rational r(3, 4);
double d = r; // Error.
void foo(double x) { /* ... */ }
foo(r); // Error.
foo(double(r)); // OK.
foo(static_cast<double>(r)); // OK.
```

41/46

### [Best practice] Avoid the abuse of type conversion operators.

Type conversion operators can lead to unexpected results!

```
class Rational {
public:
 operator double() const { return 1.0 * m_num / m_denom; }
 operator std::string() const {
 return std::to_string(m_num) + " / " + std::to_string(m_denom);
 }
};
int main() {
 Rational r(3, 4);
 std::cout << r << '\n'; // Oops! Is it `0.75` or `3 / 4`?
}
```

In the code above, either mark the type conversions as `explicit`, or remove them and define named functions like `to_double()` and `to_string()` instead.

42/46

## Contextual conversion to `bool`

A special rule for conversion to `bool`.

Suppose `expr` is an expression of a class type `X`, and suppose `X` has an `explicit` type conversion operator to `bool`. In the following contexts, that conversion is applicable even if it is not written as `bool(expr)` or `static_cast<bool>(expr)`:

- if `(expr)`, `while (expr)`, `for (...; expr; ...)`, `do ... while (expr)`
- as the operand of `!`, `&&`, `||`
- as the first operand of `?:`: `expr ? something : something_else`

43/46

## Contextual conversion to `bool`

Exercise: We often test whether a pointer is non-null like this:

```
if (ptr) {
 // ...
}
auto val = ptr ? ptr->some_value : 0;
```

Define a conversion from `WindowPtr` to `bool`, so that we can test whether a `WindowPtr` is non-null in the same way.

- Should this conversion be allowed to happen implicitly? If not, mark it `explicit`.

44/46

## Summary

Operator overloading

- As a non-member function: `@a ⇔ operator@(a)`, `a @ b ⇔ operator@(a, b)`
- As a member function: `@a ⇔ a.operator@()`, `a @ b ⇔ a.operator@(b)`
  - The postfix `++` and `--` are special: They have a special `int` parameter to make them different from the prefix ones.
  - The arrow operator `->` is special: Although it looks like a binary operator in `ptr->mem`, it is unary and involves special rules.
    - You don't need to understand the exact rules for `->`.
- Avoid repetition.
- Avoid abuse of operator overloading.

45/46

## Summary

Type conversions

- Implicit vs explicit
- User-defined type conversions: either through a constructor or through a type conversion operator.
- To disable the implicit use of the user-defined type conversion: `explicit`
- Avoid abuse of type conversion operators.
- Conversion to `bool` has some special rules (*contextual conversion*).

46/46

## CS100 Lecture 20

Iterators and Algorithms

1/46

## Contents

- Iterators
- Algorithms

2/46

## Iterators

A generalized "pointer" used for accessing elements in different containers.

Every container has its iterator, whose type is `Container::iterator`.

e.g. `std::vector<int>::iterator`, `std::forward_list<std::string>::iterator`

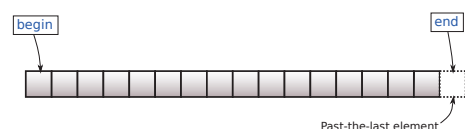
- `auto` comes to our rescue!

3/46

## Iterators

For any container object `c`,

- `c.begin()` returns the iterator to the first element of `c`.
- `c.end()` returns the iterator to the position following the last element of `c` ("off-the-end", "past-the-end").



4/46

Iterators

A pair of iterators ( `b` , `e` ) is often used to indicate a range [ `b` , `e` ) .

Such ranges are **left-inclusive**. Benefits:

- `e - b` is the **length (size)** of the range, i.e. the number of elements. There is no extra `+1` or `-1` in this expression.
- If `b == e`, the range is empty. In other words, to check whether the range is empty, we only need to do an equality test, which is easily supported by all kinds of iterators.

5/46

Iterators: dereferenceable

Like pointers, an iterator can be dereferenced ( `*it` ) only when it refers to an existing element. ("dereferenceable")

- `*v.end()` is undefined behavior.
- `++it` is undefined behavior if `it` is not dereferenceable. In other words, moving an iterator out of the range [ `begin` , `off_the_end` ] is undefined behavior.

10/46

Iterators

Basic operations, supported by almost all kinds of iterators:

- `*it` : returns a reference to the element that `it` refers to.
- `it->mem` : equivalent to `(*it).mem` .
- `++it` , `it++` : moves `it` one step forward, so that `it` refers to the "next" element.
  - `++it` returns a reference to `it` , while `it++` returns a copy of `it` before incrementation.
- `it1 == it2` : checks whether `it1` and `it2` refer to the same position in the container.
- `it1 != it2` : equivalent to `!(it1 == it2)` .

These are supported by the iterators of all sequence containers, as well as `std::string` .

6/46

Iterators: invalidation

```
Type *storage = new Type[n];
Type *iter = storage;
delete[] storage;
// Now `iter` does not refer to any existing element.
```

Some operations on some containers will **invalidate** some iterators:

- make these iterators not refer to any existing element.

For example:

- `push_back(x)` on a `std::vector` may cause the reallocation of storage. All iterators obtained previously are invalidated.
- `pop_back()` on a `std::vector` will invalidate the iterators that points to the deleted element.

11/46

Iterators

Use the basic operations to traverse a sequence container:

```
void swapcase(std::string &str) {
 for (auto it = str.begin(); it != str.end(); ++it) {
 if (std::islower(*it))
 *it = std::toupper(*it);
 else if (std::isupper(*it))
 *it = std::tolower(*it);
 }
}

void print(const std::vector<int> &vec) {
 for (auto it = vec.begin(); it != vec.end(); ++it)
 std::cout << *it << ' ';
}
```

7/46

Never use invalidated iterators or references!

```
void foo(std::vector<int> &vec) {
 auto it = vec.begin();
 while (some_condition(vec))
 vec.push_back(*it++); // Undefined behavior.
}
```

After several calls to `push_back` , `vec` may reallocate a larger chunk of memory to store its elements. This will invalidate all pointers, references and iterators that point to somewhere in the previous memory block.

12/46

Iterators

Built-in pointers are also iterators: They are the iterator for built-in arrays.

For an array `Type a[N]` :

- The "begin" iterator is `a` .
- The "end" (off-the-end) iterator is `a + N` .

The standard library functions `std::begin(c)` and `std::end(c)` (defined in `<iterator>` and many other header files):

- return `c.begin()` and `c.end()` if `c` is a container;
- return `c` and `c + N` if `c` is an array of length `N` .

8/46

More operations on iterators

The iterators of containers that support `*it` , `it->mem` , `++it` , `it++` , `it1 == it2` and `it1 != it2` are **ForwardIterators**.

**BidirectionalIterator**: a ForwardIterator that can be moved in both directions

- supports `--it` and `it--` .

**RandomAccessIterator**: a BidirectionalIterator that can be moved to any position in constant time.

- supports `it + n` , `n + it` , `it - n` , `it += n` , `it -= n` for an integer `n` .
- supports `it[n]` , equivalent to `*(it + n)` .
- supports `it1 - it2` , returns the **distance** of two iterators.
- supports `<` , `<=` , `>` , `>=` .

13/46

Range-for demystified

The range-based for loop

```
for (@declaration : container)
 @loop_body
```

is equivalent to

```
{
 auto b = std::begin(container);
 auto e = std::end(container);
 for (; b != e; ++b) {
 @declaration = *b;
 @loop_body
 }
}
```

9/46

More operations on iterators

The iterators of containers that support `*it` , `it->mem` , `++it` , `it++` , `it1 == it2` and `it1 != it2` are **ForwardIterators**.

**BidirectionalIterator**: a ForwardIterator that can be moved in both directions

- supports `--it` and `it--` .

**RandomAccessIterator**: a BidirectionalIterator that can be moved to any position in constant time.

- supports `it + n` , `n + it` , `it - n` , `it += n` , `it -= n` , `it[n]` , `it1 - it2` , `<` , `<=` , `>` , `>=` .
- `std::string::iterator` and `std::vector<T>::iterator` are in this category.

Which category is the built-in pointer in?

14/46

More operations on iterators

The iterators of containers that support `*it`, `it->mem`, `++it`, `it++`, `it1 == it2` and `it1 != it2` are **ForwardIterators**.

**BidirectionalIterator**: a ForwardIterator that can be moved in both directions

- supports `--it` and `it--`.

**RandomAccessIterator**: a BidirectionalIterator that can be moved to any position in constant time.

- supports `it + n`, `n + it`, `it - n`, `it += n`, `it -= n`, `it[n]`, `it1 - it2`, `<`, `<=`, `>`, `>=`.
- `std::string::iterator` and `std::vector<T>::iterator` are in this category.

Which category is the built-in pointer in? - RandomAccessIterator.

15/46

Algorithms: interfaces

**Parameters**: The algorithms suffixed `_n` use a beginning iterator `begin` and an integer `n` to represent a range `[begin, begin + n)`.

Example: Use STL algorithms to rewrite the constructors of `Dynarray` :

```
Dynarray::Dynarray(const int *begin, const int *end)
: m_storage(new int[end - begin]), m_length(end - begin) {
 std::copy(begin, end, m_storage);
}
Dynarray::Dynarray(const Dynarray &other)
: m_storage(new int[other.size()]), m_length(other.size()) {
 std::copy_n(other.m_storage, other.size(), m_storage);
}
Dynarray::Dynarray(std::size_t n, int x = 0)
: m_storage(new int[n]), m_length(n) {
 std::fill_n(m_storage, m_length, x);
}
```

20/46

Initialization from iterator range

`std::string`, `std::vector`, as well as other standard library containers, support the initialization from an iterator range:

```
std::vector<char> v = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i'};
std::vector v2(v.begin() + 2, v.end() - 3); // {'c', 'd', 'e', 'f'}
std::string s(v.begin(), v.end()); // "abcdefghi"
```

16/46

Algorithms: interfaces

**Return values**: "Position" is typically represented by an iterator. For example:

```
std::vector<int> v = someValues();
auto pos = std::find(v.begin(), v.end(), 42);
assert(*pos == 42);
auto maxPos = std::max_element(v.begin(), v.end());
```

- `pos` is an **iterator** pointing to the first occurrence of `42` in `v`.
- `maxPos` is an **iterator** pointing to the max element in `v`.

"Not found" / "No such element" is often indicated by returning `end`.

```
if (std::find(v.begin(), v.end(), something) != v.end()) {
 // ...
}
```

21/46

if: new syntax in C++17

"Not found" / "No such element" is often indicated by returning `end`.

```
if (std::find(v.begin(), v.end(), something) != v.end()) { /* (*) */ }
```

If we want to use the returned iterator in (\*):

```
if (auto pos = std::find(v.begin(), v.end(), something); pos != v.end())
 std::cout << *pos << '\n';
```

The new syntax of `if` in C++17: `if (init_expr; condition)`.

- `init_expr` is just like the first part of the `for` statement.
- The scope of the variable declared in `init_expr` is within this `if` statement (containing the `else` clause, if present).

22/46

Algorithms

Algorithms

Full list of standard library algorithms can be found [here](#).

No one can remember all of them, but some are quite commonly used.

17/46

Algorithms: requirements

An algorithm may have **requirements** on

- the iterator categories of the passed-in iterators, and
- the type of elements that the iterators point to.

Typically, `std::sort` requires *RandomAccessIterators*, while `std::copy` allows any *InputIterators*.

Typically, all algorithms that need to compare elements rely only upon `operator<` and `operator==` of the elements.

- You don't have to define all the six comparison operators of `X` in order to `sort` a `vector<X>`. `sort` only requires `operator<`.

23/46

Algorithms: interfaces

**Parameters**: The STL algorithms accept pairs of iterators to represent "ranges":

```
int a[N], b[N]; std::vector<int> v;
std::sort(a, a + N);
std::sort(v.begin(), v.end());
std::copy(a, a + N, b); // copies elements in [a, a+N) to [b, b+N)
std::sort(v.begin(), v.begin() + 10); // Only the first 10 elements are sorted.
```

Since C++20, `std::ranges::xxx` can be used, which has more modern interfaces

```
std::ranges::sort(a);
std::ranges::copy(a, b);
```

19/46

Algorithms

Since we pass **iterators** instead of **containers** to algorithms, the **standard library algorithms never modify the length of the containers**.

- STL algorithms never insert or delete elements in the containers (unless the iterator passed to them is some special *iterator adapter*).

For example: `std::copy` only **copies** elements, instead of inserting elements.

```
std::vector<int> a = someValues();
std::vector<int> b(a.size());
std::vector<int> c{};
std::copy(a.begin(), a.end(), b.begin()); // OK
std::copy(a.begin(), a.end(), c.begin()); // Undefined behavior!
```

24/46

Some common algorithms (<algorithm>)

Non-modifying sequence operations:

- count(begin, end, x), find(begin, end, x), find\_end(begin, end, x), find\_first\_of(begin, end, x), search(begin, end, pattern\_begin, pattern\_end)

Modifying sequence operations:

- copy(begin, end, dest), fill(begin, end, x), reverse(begin, end), ...
- unique(begin, end) : drop duplicate elements.
  - requires the elements in the range [begin, end) to be sorted (in ascending order by default).
  - It does not remove any elements! Instead, it moves all the duplicated elements to the end of the sequence, and returns an iterator pos, so that [begin, pos) has no duplicate elements.

25/46

Some common algorithms (<algorithm>)

Example: unique

```
std::vector v{1, 1, 2, 2, 2, 3, 5};
auto pos = std::unique(v.begin(), v.end());
// Now [v.begin(), pos) contains {1, 2, 3, 5}.
// [pos, v.end()) has the values {1, 2, 2}, but the exact order is not known.
v.erase(pos, v.end()); // Typical use with the container's 'erase' operation
// Now v becomes {1, 2, 3, 5}.
```

unique does not remove the duplicate elements! To remove them, use the container's erase operation.

26/46

Some common algorithms (<algorithm>)

Partitioning, sorting and merging algorithms:

- partition, is\_partitioned, stable\_partition
- sort, is\_sorted, stable\_sort
- nth\_element
- merge, inplace\_merge

Binary search on sorted ranges:

- lower\_bound, upper\_bound, binary\_search, equal\_range

Heap algorithms:

- is\_heap, make\_heap, push\_heap, pop\_heap, sort\_heap

Learn the underlying algorithms and data structures of these functions in CS101!

27/46

Some common algorithms

Min/Max and comparison algorithms: (<algorithm>)

- min\_element(begin, end), max\_element(begin, end), minmax\_element(begin, end)
- equal(begin1, end1, begin2), equal(begin1, end1, begin2, end2)
- lexicographical\_compare(begin1, end1, begin2, end2)

Numeric operations: (<numeric>)

- accumulate(begin, end, initialValue) : Sum of elements in [begin, end), with initial value initialValue.
  - accumulate(v.begin(), v.end(), 0) returns the sum of elements in v.
- inner\_product(begin1, end1, begin2, initialValue) : Inner product of two vectors a<sup>T</sup>b, added with the initial value initialValue.

28/46

Predicates

Consider the Point2d class:

```
struct Point2d {
 double x, y;
};
std::vector<Point2d> points = someValues();
```

Suppose we want to sort points in ascending order of the x coordinate.

- std::sort requires operator< in order to compare the elements,
- but it is not recommended to overload operator< here! (What if we want to sort some Point2d s in another way?)

(C++20 modern way: std::ranges::sort(points, {}, &Point2d::x);)

29/46

Predicates

std::sort has another version that accepts another argument cmp :

```
bool cmp_by_x(const Point2d &lhs, const Point2d &rhs) {
 return lhs.x < rhs.x;
}
std::sort(points.begin(), points.end(), cmp_by_x);
```

sort(begin, end, cmp)

- cmp is a Callable object. When called, it accepts two arguments whose type is the same as the element type, and returns bool.
- std::sort will use cmp(x, y) instead of x < y to compare elements.
- After sorting, cmp(v[i], v[i + 1]) is true for every i ∈ [0, v.size()-1).

30/46

Predicates

To sort numbers in reverse (descending) order:

```
bool greater_than(int a, int b) { return a > b; }
std::sort(v.begin(), v.end(), greater_than);
```

To sort them in ascending order of absolute values:

```
bool abs_less(int a, int b) { return std::abs(a) < std::abs(b); } // <cmath>
std::sort(v.begin(), v.end(), abs_less);
```

31/46

Predicates

Many algorithms accept a Callable object. For example, find\_if(begin, end, pred) finds the first element in [begin, end) such that pred(element) is true.

```
bool less_than_10(int x) {
 return x < 10;
}
std::vector<int> v = someValues();
auto pos = std::find_if(v.begin(), v.end(), less_than_10);
```

for\_each(begin, end, operation) performs operation(element) for each element in the range [begin, end).

```
void print_int(int x) { std::cout << x << ' '; }
std::for_each(v.begin(), v.end(), print_int);
```

32/46

Predicates

Many algorithms accept a Callable object. For example, find\_if(begin, end, pred) finds the first element in [begin, end) such that pred(element) is true.

What if we want to find the first element less than k, where k is determined at run-time?

```
struct LessThan {
 int k_;
 LessThan(int k) : k_{k} {}
 bool operator()(int x) const {
 return x < k_;
 }
};
auto pos = std::find_if(v.begin(), v.end(), LessThan(k));
```

- LessThan(k) constructs an object of type LessThan, with the member k\_ initialized to k.
- This object has an operator() overloaded: the function-call operator.
  - LessThan(k)(x) is equivalent to LessThan(k).operator()(x), which is x < k.

34/46

Function objects

Modern way:

```
struct LessThan {
 int k_; // No constructor is needed, and k_ is public.
 bool operator()(int x) const { return x < k_; }
};
auto pos = std::find_if(v.begin(), v.end(), LessThan{k}); // {} instead of ()
```

A **function object** (aka "functor") is an object `fo` with `operator()` overloaded.

- `fo(arg1, arg2, ...)` is equivalent to `fo.operator()(arg1, arg2, ...)`. Any number of arguments is allowed.

Function objects

Exercise: use a function object to compare integers by their absolute values.

```
struct AbsCmp {
 bool operator()(int a, int b) const {
 return std::abs(a) < std::abs(b);
 }
};
std::sort(v.begin(), v.end(), AbsCmp{});
```

Lambda expressions

Defining a function or a function object is not good enough:

- These functions or function objects are almost used only once, but
- too many lines of code is needed, and
- you have to add names to the global scope.

Is there a way to define an **unnamed**, immediate callable object?

Lambda expressions

To sort by comparing absolute values:

```
std::sort(v.begin(), v.end(),
 [](int a, int b) -> bool { return std::abs(a) < std::abs(b); });
```

To sort in reverse order:

```
std::sort(v.begin(), v.end(),
 [](int a, int b) -> bool { return a > b; });
```

To find the first element less than `k`:

```
auto pos = std::find_if(v.begin(), v.end(),
 [k](int x) -> bool { return x < k; });
```

Lambda expressions

The return type can be omitted and deduced by the compiler.

```
std::sort(v.begin(), v.end(),
 [](int a, int b) { return std::abs(a) < std::abs(b); });
```

```
std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; });
```

```
auto pos = std::find_if(v.begin(), v.end(), [k](int x) { return x < k; });
```

Lambda expressions

A lambda expression has the following syntax:

```
[capture_list](params) -> return_type { function_body }
```

The compiler will generate a function object according to it.

```
int k = 42;
auto f = [k](int x) -> bool { return x < k; };
bool b1 = f(10); // true
bool b2 = f(100); // false
```

Lambda expressions

```
[capture_list](params) -> return_type { function_body }
```

It is allowed to write complex statements in `function_body`, just as in a function.

```
struct Point2d { double x, y; };
std::vector<Point2d> points = somePoints();
// prints the l2-norm of every point
std::for_each(points.begin(), points.end(),
 [](const Point2d &p) {
 auto norm = std::sqrt(p.x * p.x + p.y * p.y);
 std::cout << norm << std::endl;
 });
```

Lambda expressions: capture

To capture more variables:

```
auto pos = std::find_if(v.begin(), v.end(),
 [lower, upper](int x) { return lower <= x && x <= upper; });
```

To capture by reference (so that copy is avoided)

```
std::string str = someString();
std::vector<std::string> wordList;
// finds the first string that is lexicographically greater than `str`,
// but shorter than `str`.
auto pos = std::find_if(wordList.begin(), wordList.end(),
 [&str](const std::string &s) { return s > str && s.size() < str.size(); });
```

Here `&str` indicates that `str` is captured by referece. `&` here is not the address-of operator!

More on lambda expressions

- C++ *Primer* Section 10.3
- Effective Modern C++* Chapter 6 (Item 31-34)

Note that C++ *Primer* (5th edition) is based on C++11 and *Effective Modern C++* is based on C++14. Lambda expressions are evolving at a very fast pace in modern C++, with many new things added and many limitations removed.

More fancy ways of writing lambda expressions are not covered in CS100.

Back to algorithms

So many things in the algorithm library! How can we remember them?

- Remember the **conventions**:
  - No insertion/deletion of elements
  - Iterator range `[begin, end)`
  - Functions named with the suffix `_n` uses `[begin, begin + n)`
  - Pass functions, function objects, and lambdas for customized operations
  - Functions named with the suffix `_if` requires a boolean predicate
- Remember the common ones: `copy`, `find`, `for_each`, `sort`, ...
- Look them up in [cppreference](#) before use.

## Summary

### Iterators

- A generalized "pointer" used for accessing elements in different containers.
- Iterator range: a left-inclusive interval `[b, e)`.
- `c.begin()`, `c.end()`
- Basic operations: `*it`, `it->mem`, `++it`, `it++`, `it1 == it2`, `it1 != it2`.
- Range-based `for` loops are in fact traversal using iterators.
- More operations: BidirectionalIterator supports `it--` and `--it`.  
RandomAccessIterator supports all pointer arithmetics.
- Initialization of standard library containers from an iterator range.

45/46

## Summary

### Algorithms

- Normal functions accept iterator range `[b, e)`. Functions with `_n` accept an iterator and an integer, representing the range `[begin, begin + n)`.
- Position is represented by an iterator.
- STL algorithms never insert or delete elements in the containers.
- Some algorithms accept a predicate argument, which is a callable object. It can be a function, a pointer to function, an object of some type that has an overloaded `operator()`, or a lambda.
- Lambda: `[capture_list][params] -> return_type { function_body }`

46/46

## CS100 Lecture 21

### Inheritance and Polymorphism I

## Contents

- Inheritance
- Dynamic binding and polymorphism

2/45

## Inheritance

3/45

## Example: An item for sale

```
class Item {
 std::string m_name;
 double m_price = 0.0;
public:
 Item() = default;
 Item(const std::string &name, double price)
 : m_name(name), m_price(price) {}
 const auto &getName() const { return m_name; }
 auto netPrice(int cnt) const {
 return cnt * m_price;
 }
};
```

4/45

## Defining a subclass

A discounted item is an item, and has more information:

- `std::size_t m_minQuantity;`
- `double m_discount;`

The net price for such an item is

$$\text{netPrice}(n) = \begin{cases} n \cdot \text{price}, & \text{if } n < \text{minQuantity}, \\ n \cdot \text{discount} \cdot \text{price}, & \text{otherwise.} \end{cases}$$

5/45

## Defining a subclass

Use inheritance to model the "is-a" relationship:

- A discounted item is an item.

```
class DiscountedItem : public Item {
 int m_minQuantity = 0;
 double m_discount = 1.0;
public:
 // constructors
 // netPrice
};
```

6/45

## protected members

A `protected` member is private, except that it is accessible in subclasses.

- `m_price` needs to be `protected`, of course.
- Should `m_name` be `protected` or `private`?
  - `private` is ok if the subclass does not modify it. It is accessible through the public `getName` interface.
  - `protected` is also reasonable.

7/45

## protected members

```
class Item {
 std::string m_name;
protected:
 double m_price = 0.0;
public:
 Item() = default;
 Item(const std::string &name, double price)
 : m_name(name), m_price(price) {}
 const auto &getName() const { return m_name; }
 auto netPrice(int cnt) const {
 return cnt * m_price;
 }
};
```

8/45

## Inheritance

By defining `DiscountedItem` to be a subclass of `Item`, every `DiscountedItem` object contains a subobject of type `Item`.

- Every data member and member function, except the ctors and dtors, is inherited, **no matter what access level they have**.

What can be inferred from this?

9/45

## Inheritance

By defining `DiscountedItem` to be a subclass of `Item`, every `DiscountedItem` object contains a subobject of type `Item`.

- Every data member and member function, except the ctors and dtors, is inherited, **no matter what access level they have**.

What can be inferred from this?

- A constructor of `DiscountedItem` must first initialize the base class subobject by calling a constructor of `Item`'s.
- The destructor of `DiscountedItem` must call the destructor of `Item` after having destroyed its own members (`m_minQuantity` and `m_discount`).
- `sizeof(Derived) >= sizeof(Base)`

10/45

## Inheritance

Key points of inheritance:

- Every object of the derived class (subclass) contains a base class subobject.
- Inheritance should not break the encapsulation of the base class.
  - e.g. To initialize the base class subobject, **we must call a constructor of the base class**. It is not allowed to initialize data members of the base class subobject directly.

11/45

## Constructor of `DiscountedItem`

```
class DiscountedItem : public Item {
 int m_minQuantity = 0;
 double m_discount = 1.0;
public:
 DiscountedItem(const std::string &name, double price,
 int minQ, double disc)
 : Item(name, price), m_minQuantity(minQ), m_discount(disc) {}
};
```

It is not allowed to write this:

```
DiscountedItem(const std::string &name, double price,
 int minQ, double disc)
 : m_name(name), m_price(price), m_minQuantity(minQ), m_discount(disc) {}
```

12/45

## Constructor of derived classes

Before the initialization of the derived class's own data members, the base class subobject **must** be initialized by having one of its ctors called.

- What if we don't call the base class's ctor explicitly?

```
DiscountedItem(...)
 : /* ctor of Item is not called */ m_minQuantity(minQ), m_discount(d) {}
```

13/45

## Constructor of derived classes

Before the initialization of the derived class's own data members, the base class subobject **must** be initialized by having one of its ctors called.

- What if we don't call the base class's ctor explicitly?
  - The default constructor of the base class is called.
  - If the base class is not default-constructible, an error.
- What does this constructor do?

```
DiscountedItem() = default;
```

14/45

## Constructor of derived classes

Before the initialization of the derived class's own data members, the base class subobject **must** be initialized by having one of its ctors called.

- What if we don't call the base class's ctor explicitly?
  - The default constructor of the base class is called.
  - If the base class is not default-constructible, an error.
- What does this constructor do?

```
DiscountedItem() = default;
```

- Calls `Item::Item()` to default-initialize the base class subobject before initializing `m_minQuantity` and `m_discount`.

15/45

## Constructor of derived classes

In the following code, does the constructor of `DiscountedItem` compile?

```
class Item {
protected:
 std::string m_name;
 double m_price;
public:
 Item(const std::string &name, double p) : m_name(name), m_price(p) {}
};
class DiscountedItem : public Item {
 int m_minQuantity;
 double m_discount;
public:
 DiscountedItem(const std::string &name, double p, int mq, double disc) {
 m_name = name; m_price = p; m_minQuantity = mq; m_discount = disc;
 }
};
```

16/45

## Constructor of derived classes

In the following code, does the constructor of `DiscountedItem` compile?

```
class Item {
 // ...
public:
 // Since `Item` has a user-declared constructor, it does not have
 // a default constructor.
 Item(const std::string &name, double p) : m_name(name), m_price(p) {}
};
class DiscountedItem : public Item {
 // ...
public:
 DiscountedItem(const std::string &name, double p, int mq, double disc)
 // Before entering the function body, `Item::Item()` is called --> Error!
 { /* ... */ }
};
```

[Best practice] Use constructor initializer lists whenever possible.

17/45

## Dynamic binding

18/45

## Upcasting

If `D` is a subclass of `B`:

- A `B*` can point to a `D`, and
- A `B&` can be bound to a `D`.

```
DiscountedItem di = someValue();
Item &ir = di; // correct
Item *ip = &di; // correct
```

Reason: The is-a relationship! A `D` is a `B`.

But on such references or pointers, only the members of `B` can be accessed.

19/45

## Upcasting: Example

```
void printItemName(const Item &item) {
 std::cout << "Name: " << item.getName() << std::endl;
}
DiscountedItem di("A", 10, 2, 0.8);
Item i("B", 15);
printItemName(i); // "Name: B"
printItemName(di); // "Name: A"
```

`const Item &item` can be bound to either an `Item` or a `DiscountedItem`.

20/45

## Static type and dynamic type

- **static type** of an expression: The type known at compile-time.
- **dynamic type** of an expression: The real type of the object that the expression is representing. This is known at run-time.

```
void printItemName(const Item &item) {
 std::cout << "Name: " << item.getName() << std::endl;
}
```

The static type of the expression `item` is `const Item`, but its dynamic type is not known until run-time. (It may be `const Item` or `const DiscountedItem`.)

21/45

## virtual functions

`Item` and `DiscountedItem` have different ways of computing the net price.

```
void printItemInfo(const Item &item) {
 std::cout << "Name: " << item.getName()
 << ", price: " << item.netPrice(1) << std::endl;
}
```

- Which `netPrice` should be called?
- How do we define two different `netPrice`s?

22/45

## virtual functions

```
class Item {
public:
 virtual double netPrice(int cnt) const {
 return m_price * cnt;
 }
 // other members
};
class DiscountedItem : public Item {
public:
 double netPrice(int cnt) const override {
 return cnt < m_minQuantity ? cnt * m_price : cnt * m_price * m_discount;
 }
 // other members
};
```

Note: `auto` cannot be used to deduce the return type of `virtual` functions.

23/45

## Dynamic binding

```
void printItemInfo(const Item &item) {
 std::cout << "Name: " << item.getName()
 << ", price: " << item.netPrice(1) << std::endl;
}
```

The dynamic type of `item` is determined at run-time.

Since `netPrice` is a `virtual` function, which version is called is also determined at run-time:

- If the dynamic type of `item` is `Item`, it calls `Item::netPrice`.
- If the dynamic type of `item` is `DiscountedItem`, it calls `DiscountedItem::netPrice`.

late binding, or dynamic binding

24/45

## virtual - override

To **override** (覆盖/覆写) a `virtual` function,

- The function parameter list must be the same as that of the base class's version.
- The return type should be **identical to** (or *covariant with*) that of the corresponding function in the base class.
  - We will talk about "covariant with" in later lectures or recitations.
- The **const** ness should be the same!

To make sure you are truly overriding the `virtual` function (instead of making a overloaded version), use the `override` keyword.

\* Not to be confused with "overloading" (重载).

25/45

## virtual - override

An overriding function is also `virtual`, even if not explicitly declared.

```
class DiscountedItem : public Item {
 virtual double netPrice(int cnt) const override; // correct, explicitly virtual
};
class DiscountedItem : public Item {
 double netPrice(int cnt) const; // also correct, but not recommended
};
```

The `override` keyword lets the compiler check and report if the function is not truly overriding.

[Best practice] To override a virtual function, write the `override` keyword explicitly. The `virtual` keyword can be omitted.

26/45

## virtual destructors

```
Item *ip = nullptr;
if (some_condition)
 ip = new Item(/* ... */);
else
 ip = new DiscountedItem(/* ... */);
// ...
delete ip;
```

Whose destructor should be called?

- Only looking at the static type of `*ip` is not enough.

27/45

## virtual destructors

```
Item *ip = nullptr;
if (some_condition)
 ip = new Item(/* ... */);
else
 ip = new DiscountedItem(/* ... */);
// ...
delete ip;
```

Whose destructor should be called? - It needs to be determined at run-time!

- To use dynamic binding correctly, you almost always need a `virtual` destructor.

28/45



## virtual destructors

```
Item *ip = nullptr;
if (some_condition)
 ip = new Item(/* ... */);
else
 ip = new DiscountedItem(/* ... */);
// ...
delete ip;
```

- The implicitly-defined (compiler-generated) destructor is **non-virtual**, but we can explicitly require a **virtual** one:

```
virtual ~Item() = default;
```

- If the dtor of the base class is **virtual**, the compiler-generated dtor for the derived class is also **virtual**.

29/45

## Synthesized copy-control members

Guess!

- What are the behaviors of the compiler-generated copy-control members?
- In what cases will they be **delete** d?

34/45

## (Almost) completed Item and DiscountedItem

```
class Item {
 std::string m_name;

protected:
 double m_price = 0.0;

public:
 Item() = default;
 Item(const std::string &name, double price) : m_name(name), m_price(price) {}
 const auto &getName() const { return name; }
 virtual double net_price(int n) const {
 return n * price;
 }
 virtual ~Item() = default;
};
```

30/45

## Synthesized copy-control members

Remember that the base class's subobject is always handled first.

These rules are quite natural:

- What are the behaviors of the compiler-generated copy-control members?
  - First, it calls the base class's corresponding copy-control member.
  - Then, it performs the corresponding operation on the derived class's own data members.
- In what cases will they be **delete** d?
  - If the base class's corresponding copy-control member is not accessible (e.g. non-existent, or **private**).
  - or if any of the data members' corresponding copy-control member is not accessible.

35/45

## (Almost) completed Item and DiscountedItem

```
class DiscountedItem : public Item {
 int m_minQuantity = 0;
 double m_discount = 1.0;

public:
 DiscountedItem(const std::string &name, double price,
 int minQ, double disc)
 : Item(name, price), m_minQuantity(minQ), m_discount(disc) {}
 double netPrice(int cnt) const override {
 return cnt < m_minQuantity ? cnt * m_price : cnt * m_price * m_discount;
 }
};
```

31/45

## Slicing

Dynamic binding only happens on references or pointers to base class.

```
DiscountedItem di("A", 10, 2, 0.8);
Item i = di; // What happens?
auto x = i.netPrice(3); // Which netPrice?
```

36/45

## Usage with smart pointers

Smart pointers are implemented by wrapping the raw pointers, so they can also be used for dynamic binding.

```
std::vector<std::shared_ptr<Item>> myItems;
for (auto i = 0; i != n; ++i) {
 if (someCondition) {
 myItems.push_back(std::make_shared<Item>(someParams));
 } else {
 myItems.push_back(std::make_shared<DiscountedItem>(someParams));
 }
}
```

A `std::unique_ptr<Derived>` can be implicitly converted to a `std::unique_ptr<Base>`.

A `std::shared_ptr<Derived>` can be implicitly converted to a `std::shared_ptr<Base>`.

32/45

## Slicing

Dynamic binding only happens on references or pointers to base class.

```
DiscountedItem di("A", 10, 2, 0.8);
Item i = di; // What happens?
auto x = i.netPrice(3); // Which netPrice?
```

`Item i = di;` calls the **copy constructor** of `Item`

- but `Item`'s copy constructor handles only the base part.
- So `DiscountedItem`'s own members are **ignored**, or "sliced down".
- `i.netPrice(3)` calls `Item::netPrice`.

37/45

## Copy-control

Remember to copy/move the base subobject! One possible way:

```
class Derived : public Base {
public:
 Derived(const Derived &other)
 : Base(other), /* Derived's own members */ { /* ... */ }
 Derived &operator=(const Derived &other) {
 Base::operator=(other); // call Base's operator= explicitly
 // copy Derived's own members
 return *this;
 }
 // ...
};
```

Why `Base(other)` and `Base::operator=(other)` work?

- The parameter type is `const Base &`, which can be bound to a `Derived` object.

33/45

## Downcasting

```
Base *bp = new Derived();
```

If we only have a `Base` pointer, but we are quite sure that it points to a `Derived` object

- Accessing the members of `Derived` through `bp` is not allowed.
- How can we perform a "downcasting"?

38/45

# Polymorphic class

A class is said to be **polymorphic** if it has (declares or inherits) at least one virtual function.

- Either a `virtual` normal member function or a `virtual` dtor is ok.

If a class is polymorphic, all classes derived from it are polymorphic.

- There is no way to "refuse" to inherit any member functions, so `virtual` member functions must be inherited.
- The dtor must be `virtual` if the dtor of the base class is `virtual`.

39/45

# Summary

## Inheritance

- Every object of type `Derived` contains a subobject of type `Base`.
  - Every member of `Base` is inherited, no matter whether it is accessible or not.
- Inheritance should not break the base class's encapsulation.
  - The access control of inherited members is not changed.
  - Every constructor of `Derived` calls a constructor of `Base` to initialize the base class subobject **before** initializing its own data members.
  - The destructor of `Derived` calls the destructor of `Base` to destroy the base class subobject **after** destroying its own data members.

44/45

# Downcasting: For polymorphic class only

`dynamic_cast<Target>(expr)`.

```
Base *bp = new Derived{};
Derived *dp = dynamic_cast<Derived*>(bp);
Derived &dr = dynamic_cast<Derived&>(*bp);
```

- `Target` must be a **reference** or a **pointer** type.
- `dynamic_cast` will perform **runtime type identification (RTTI)** to check the dynamic type of the expression.
  - If the dynamic type is `Derived`, or a derived class (direct or indirect) of `Derived`, the downcasting succeeds.
  - Otherwise, the downcasting fails. If `Target` is a pointer, returns a null pointer. If `Target` is a reference, throws an exception `std::bad_cast`.

40/45

# Summary

## Dynamic binding

- Upcasting: A pointer, reference or smart pointer to `Base` can be bound to an object of type `Derived`.
  - static type and dynamic type
- `virtual` functions: A function that can be overridden by derived classes.
  - The base class and the derived class can provide different versions of this function.
- Dynamic (late) binding
  - A call to a virtual function on a pointer or reference to `Base` will actually call the corresponding version of that function according to the dynamic type.
- Avoid downcasting if possible.

45/45

# dynamic\_cast can be very slow

`dynamic_cast` performs a runtime **check** to see whether the downcasting should succeed, which uses runtime type information.

This is **much slower** than other types of casting, e.g. `const_cast`, or arithmetic conversions.

[Best practice] Avoid `dynamic_cast` whenever possible.

# Guaranteed successful downcasting: Use `static_cast`.

If the downcasting is guaranteed to be successful, you may use `static_cast`

```
auto dp = static_cast<Derived*>(bp); // quicker than dynamic_cast,
// but performs no checks. If the dynamic type is not Derived, UB.
```

41/45

# CS100 Lecture 22

## Inheritance and Polymorphism II

1/41

# Avoiding `dynamic_cast`

Typical abuse of `dynamic_cast`:

```
struct A {
 virtual ~A() {}
};
struct B : A {};
struct C : A {};

std::string getType(const A *ap) {
 if (dynamic_cast<const B*>(ap))
 return "B";
 else if (dynamic_cast<const C*>(ap))
 return "C";
 else
 return "A";
}
```

Click here to see how large and slow the generated code is:

<https://godbolt.org/z/3367efGd7>

42/45

# Contents

- Abstract base class
- More on the "is-a" relationship (*Effective C++* Item 32)
- Inheritance of interface vs inheritance of implementation (*Effective C++* Item 34)

2/41

# Avoiding `dynamic_cast`

Use a group of `virtual` functions!

```
struct A {
 virtual ~A() {}
 virtual std::string name() const {
 return "A";
 }
};
struct B : A {
 std::string name()const override{
 return "B";
 }
};
struct C : A {
 std::string name()const override{
 return "C";
 }
};

auto getType(const A *ap) {
 return ap->name();
}
```

• This time:  
<https://godbolt.org/z/KosbcaGnT>

The generated code is much simpler!

43/45

# Abstract base class

3/41

Shapes

Define different shapes: Rectangle, Triangle, Circle, ...

Suppose we want to draw things like this:

```
void drawThings(ScreenHandle &screen,
 const std::vector<std::shared_ptr<Shape>> &shapes) {
 for (const auto &shape : shapes)
 shape->draw(screen);
}
```

and print information:

```
void printShapeInfo(const Shape &shape) {
 std::cout << "Area: " << shape.area()
 << "Perimeter: " << shape.perimeter() << std::endl;
}
```

Shapes

Define a base class `Shape` and let other shapes inherit it.

```
class Shape {
public:
 Shape() = default;
 virtual void draw(ScreenHandle &screen) const;
 virtual double area() const;
 virtual double perimeter() const;
 virtual ~Shape() = default;
};
```

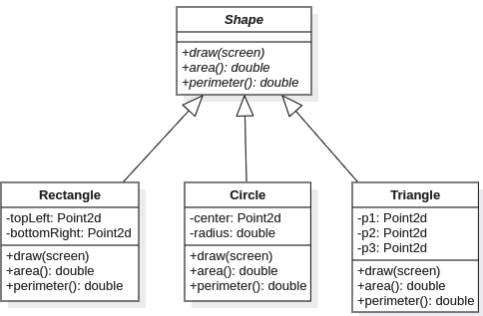
Different shapes should define their own `draw`, `area` and `perimeter`, so these functions should be `virtual`.

Shapes

```
class Rectangle : public Shape {
 Point2d mTopLeft, mBottomRight;

public:
 Rectangle(const Point2d &t1, const Point2d &br)
 : mTopLeft(t1), mBottomRight(br) {} // Base is default-initialized
 void draw(ScreenHandle &screen) const override { /* ... */ }
 double area() const override {
 return (mBottomRight.x - mTopLeft.x) * (mBottomRight.y - mTopLeft.y);
 }
 double perimeter() const override {
 return 2 * (mBottomRight.x - mTopLeft.x + mBottomRight.y - mTopLeft.y);
 }
};
```

Shapes



Pure virtual functions

How should we define `Shape::draw`, `Shape::area` and `Shape::perimeter`?

- For the general concept "Shape", there is no way to determine the behaviors of these functions.

Pure virtual functions

How should we define `Shape::draw`, `Shape::area` and `Shape::perimeter`?

- For the general concept "Shape", there is no way to determine the behaviors of these functions.
- Direct call to `Shape::draw`, `Shape::area` and `Shape::perimeter` should be forbidden.
- We shouldn't even allow an object of type `Shape` to be instantiated! The class `Shape` is only used to define the concept "Shape" and required interfaces.

Pure virtual functions

If a `virtual` function does not have a reasonable definition in the base class, it should be declared as `pure virtual` by writing `=0`.

```
class Shape {
public:
 virtual void draw(ScreenHandle &) const = 0;
 virtual double area() const = 0;
 virtual double perimeter() const = 0;
 virtual ~Shape() = default;
};
```

Any class that has a `pure virtual` function is an **abstract class**. `Pure virtual` functions (usually) cannot be called<sup>1</sup>, and abstract classes cannot be instantiated.

Pure virtual functions and abstract classes

Any class that has a `pure virtual` function is an **abstract class**. `Pure virtual` functions (usually) cannot be called<sup>1</sup>, and abstract classes cannot be instantiated.

```
Shape shape; // Error.
Shape *p = new Shape; // Error.
auto sp = std::make_shared<Shape>(); // Error.
std::shared_ptr<Shape> sp2 = std::make_shared<Rectangle>(p1, p2); // OK.
```

We can define pointer or reference to an abstract class, but never an object of that type!

Pure virtual functions and abstract classes

An impure `virtual` function **must be defined**. Otherwise, the compiler will fail to generate necessary runtime information (the virtual table), which leads to an error.

```
class X {
public:
 virtual void foo(); // Declaration, without a definition
 // Even if 'foo' is not used, this will lead to an error.
};
```

Linkage error:

```
/usr/bin/ld: /tmp/ccV9TNfM.o: in function `main':
a.cpp:(.text+0x1e): undefined reference to `vtable for X'
```

Make the interface robust, not error-prone.

Is this good?

```
class Shape {
public:
 virtual double area() const {
 return 0;
 }
};
```

What about this?

```
class Shape {
public:
 virtual double area() const {
 throw std::logic_error{"area() called on Shape!"};
 }
};
```

## Make the interface robust, not error-prone.

```
class Shape {
public:
 virtual double area() const {
 return 0;
 }
};
```

If `Shape::area` is called accidentally, the error will happen *silently*!

14/41

## Make the interface robust, not error-prone.

```
class Shape {
public:
 virtual double area() const {
 throw std::logic_error{"area() called on Shape!"};
 }
};
```

If `Shape::area` is called accidentally, an exception will be raised.

However, a good design should make errors fail to compile.

[Best practice] If an error can be caught in compile-time, don't leave it until run-time.

15/41

## Polymorphism (多态)

Polymorphism: The provision of a single interface to entities of different types, or the use of a single symbol to represent multiple different types.

- Run-time polymorphism: Achieved via **dynamic binding**.
- Compile-time polymorphism: Achieved via **function overloading**, **templates**, **concepts** (since C++20), etc.

Run-time polymorphism:

```
struct Shape {
 virtual void draw() const = 0;
};
void drawStuff(const Shape &s) {
 s.draw();
}
```

Compile-time polymorphism:

```
template <typename T>
concept Shape = requires(const T x) {
 x.draw();
};
void drawStuff(Shape const auto &s) {
 s.draw();
}
```

16/41

## More on the "is-a" relationship

Effective C++ Item 32

17/41

## Public inheritance: The "is-a" relationship

By writing that class `D` publicly inherits from class `B`, you are telling the compiler (as well as human readers of your code) that

- Every object of type `D` *is also an* object of type `B`, but not vice versa.
- `B` represents a **more general** concept than `D`, and that `D` represents a **more specialized** concept than `B`.

More specifically, you are asserting that **anywhere an object of type `B` can be used, an object of type `D` can be used just as well.**

- On the other hand, if you need an object of type `D`, an object of type `B` won't do.

18/41

## Example: Every student *is a* person.

```
class Person { /* ... */ };
class Student : public Person { /* ... */ };
```

- Every student *is a* person, but not every person is a student.
- Anything that is true of a person is also true of a student:
  - A person has a date of birth, so does a student.
- Something is true of a student, but not true of people in general.
  - A student is enrolled in a particular school, but a person may not.

The notion of a person is **more general** than is that of a student; a student is a **specialized type** of person.

19/41

## Example: Every student *is a* person.

The is-a relationship: Anywhere an object of type `Person` can be used, an object of type `Student` can be used just as well, **but not vice versa**.

```
void eat(const Person &p); // Anyone can eat.
void study(const Student &s); // Only students study.
Person p;
Student s;
eat(p); // Fine. `p` is a person.
eat(s); // Fine. `s` is a student, and a student is a person.
study(s); // Fine.
study(p); // Error! `p` isn't a student.
```

20/41

## Your intuition can mislead you.

- A penguin is a bird.
- A bird can fly.

If we naively try to express this in C++, our effort yields:

```
class Bird {
public:
 virtual void fly(); // Birds can fly.
 // ...
};
class Penguin : public Bird { // A penguin is a bird.
 // ...
};
```

```
Penguin p;
p.fly(); // Oh no!! Penguins cannot fly, but this code compiles!
```

21/41

## No. Not every bird can fly.

*In general*, birds have the ability to fly.

- Strictly speaking, there are several types of non-flying birds.

Maybe the following hierarchy models the reality much better?

```
class Bird { /* ... */ };
class FlyingBird : public Bird {
 virtual void fly();
};
class Penguin : public Bird { // Not FlyingBird
 // ...
};
```

22/41

## No. Not every bird can fly.

Maybe the following hierarchy models the reality much better?

```
class Bird { /* ... */ };
class FlyingBird : public Bird {
 virtual void fly();
};
class Penguin : public Bird { // Not FlyingBird
 // ...
};
```

- Not necessarily.** If your application has much to do with beaks and wings, and nothing to do with flying, the original two-class hierarchy might be satisfactory.
- There is no one ideal design for every software.** The best design depends on what the system is expected to do.

23/41

## What about report a runtime error?

```
void report_error(const std::string &msg); // defined elsewhere
class Penguin : public Bird {
public:
 virtual void fly() {
 report_error("Attempt to make a penguin fly!");
 }
};
```

24/41

## What about report a runtime error?

```
void report_error(const std::string &msg); // defined elsewhere
class Penguin : public Bird {
public:
 virtual void fly() { report_error("Attempt to make a penguin fly!"); }
};
```

No. This does not say "Penguins can't fly." This says "Penguins can fly, but it is an error for them to actually try to do it."

To actually express the constraint "Penguins can't fly", you should prevent the attempt from **compiling**.

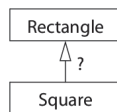
```
Penguin p;
p.fly(); // This should not compile.
```

[Best practice] Good interfaces prevent invalid code from compiling.

25/41

## Example: A square *is a* rectangle.

Should class `Square` publicly inherit from class `Rectangle`?



26/41

## Example: A square *is a* rectangle.

Consider this code.

```
class Rectangle {
public:
 virtual void setHeight(int newHeight);
 virtual void setWidth(int newWidth);
 virtual int getHeight() const;
 virtual int getWidth() const;
 // ...
};
void makeBigger(Rectangle &r) {
 r.setWidth(r.getWidth() + 10);
}

class Square : public Rectangle {
 // A square is a rectangle,
 // where height == width.
 // ...
};
Square s(10); // A 10x10 square.
makeBigger(s); // Oh no!
```

27/41

## Is this really an "is-a" relationship?

We said before that the "is-a" relationship means that **anywhere an object of type `B` can be used, an object of type `D` can be used just as well.**

However, something applicable to a rectangle is not applicable to a square!

**Conclusion:** Public inheritance means "is-a". Everything that applies to base classes must also apply to derived classes, because every derived class object is a base class object.

28/41

## Inheritance of interface vs inheritance of implementation

*Effective C++* Item 34

29/41

### Example: Airplanes for XYZ Airlines.

Suppose XYZ has only two kinds of planes: the Model A and the Model B, and both are flown in exactly the same way.

```
class Airplane {
public:
 virtual void fly(const Airport &destination) {
 // Default code for flying an airplane to the given destination.
 }
};
class ModelA : public Airplane { /* ... */ };
class ModelB : public Airplane { /* ... */ };
```

- `Airplane::fly` is declared `virtual` because *in principle*, different airplanes should be flown in different ways.
- `Airplane::fly` is defined, to avoid copy-and-pasting code in the `ModelA` and `ModelB` classes.

30/41

### Example: Airplanes for XYZ Airlines.

Now suppose that XYZ decides to acquire a new type of airplane, the Model C, which is flown differently from the Model A and the Model B.

XYZ's programmers add the class `ModelC` to the hierarchy, but forget to redefine the `fly` function!

```
class ModelC : public Airplane {
 // `fly` is not overridden.
 // ...
};
```

This surely leads to a disaster:

```
auto pc = std::make_unique<ModelC>();
pc->fly(PVG); // No! Attempts to fly Model C in the Model A/B way!
```

31/41

## Impure virtual function: Interface + default implementation

The problem here is not that `Airplane::fly` has default behavior, but that `ModelC` was allowed to inherit that behavior **without explicitly saying that it wanted to**.

**\* By defining an impure virtual function, we have the derived class inherit a function *interface as well as a default implementation*.**

- Interface: Every class inheriting from `Airplane` can `fly`.
- Default implementation: If `ModelC` does not override `Airplane::fly`, it will have the inherited implementation automatically.

32/41

## Separate default implementation from interface

To sever the connection between the *interface* of the virtual function and its *default implementation*:

```
class Airplane {
public:
 virtual void fly(const Airport &destination) = 0; // pure virtual
 // ...
protected:
 void defaultFly(const Airport &destination) {
 // Default code for flying an airplane to the given destination.
 }
};
```

- The pure virtual function `fly` provides the **interface**: Every derived class can `fly`.
- The **default implementation** is written in `defaultFly`.

33/41

## Separate default implementation from interface

If `ModelA` and `ModelB` want to adopt the default way of flying, they simply make a call to `defaultFly`.

```
class ModelA : public Airplane {
public:
 virtual void fly(const Airport &destination) {
 defaultFly(destination);
 }
 // ...
};
class ModelB : public Airplane {
public:
 virtual void fly(const Airport &destination) {
 defaultFly(destination);
 }
 // ...
};
```

34/41

39/41

## Separate default implementation from interface

For `ModelC`:

- Since `Airplane::fly` is pure virtual, `ModelC` must define its own version of `fly`.
- If it **does** want to use the default implementation, it **must say it explicitly** by making a call to `defaultFly`.

```
class ModelC : public Airplane {
public:
 virtual void fly(const Airport &destination) {
 // The "Model C way" of flying.
 // Without the definition of this function, `ModelC` remains abstract,
 // which does not compile if we create an object of such type.
 }
};
```

35/41

40/41

## Still not satisfactory?

Some people object to the idea of having separate functions for providing the interface and the default implementation, such as `fly` and `defaultFly` above.

- For one thing, it pollutes the class namespace with closely related function names.
  - This really matters, especially in complicated projects. Extra mental effort might be required to distinguish the meaning of overly similar names.

Read the rest part of *Effective C++* Item 34 for another solution to this problem.

36/41

41/41

## Inheritance of interface vs inheritance of implementation

We have come to the conclusion that

- Pure virtual functions specify **inheritance of interface** only.
- Simple (impure) virtual functions specify **inheritance of interface + a default implementation**.
  - The default implementation can be overridden.

Moreover, non-virtual functions specify **inheritance of interface + a mandatory implementation**.

Note: In public inheritance, *interfaces are always inherited*.

37/41

1/51

## Summary

Pure virtual function and abstract class

- A pure virtual function is a virtual function declared `= 0`.
  - Call to a pure virtual function is not allowed.<sup>1</sup>
  - Pure virtual functions define the interfaces and force the derived classes to override it.
- A class that has a pure virtual function is an abstract class.
  - We cannot create an object of an abstract class type.
  - Abstract classes are often used to represent abstract, general concepts.

38/41

2/51

## Summary

Public inheritance models the "is-a" relationship.

- Everything that applies to base classes must also apply to derived classes.
- The "Birds can fly, and a penguin is a bird" example.
- The "A square is a rectangle" example.

## Summary

Inheritance of interface vs inheritance of implementation

- In public inheritance, interfaces are always inherited.
- Pure virtual functions: inheritance of **interface** only.
- Simple (impure) virtual functions: inheritance of **interface + a default implementation**.
  - The default implementation can be overridden.
- non-virtual functions: inheritance of **interface + a mandatory implementation**.

## Notes

<sup>1</sup> A pure virtual function can have a definition. In that case, it can be called via the syntax `ClassName::functionName(args)`, not via a virtual function call (dynamic binding).

In some cases, we want a class to be made abstract, but it does not have any pure virtual function. A possible workaround is to declare the destructor to be pure virtual, and then provide a definition for it:

```
struct Foo {
 virtual ~Foo() = 0;
};
Foo::~~Foo() = default; // Provide a definition outside the class.
```

The "another solution" mentioned in page 36 is also related to this.

## CS100 Lecture 23

More on STL: Sequence containers and associative containers

## Contents

More on STL: Sequence containers and associative containers

- Overview of STL
- Sequence containers
- Associative containers

## Overview of STL

### Standard Template Library

Added into C++ in 1994.

- Containers
- Iterators (In Lecture 20)
- Algorithms (In Lecture 20)
- Function objects
- Some other adaptors, like container adaptors and iterator adaptors
- Allocators

The next generation: [C++20 Ranges](#)

3/51

### Containers

- Sequence containers
  - `vector`, `list`, `deque`, `array` (since C++11), `forward_list` (since C++11)
- Associative containers
  - `set`, `map`, `multiset`, `multimap` (often implemented with *binary search trees*)
- Unordered associative containers (since C++11)
  - `unordered_set`, `unordered_map`, `unordered_multiset`, `unordered_multimap` (implemented with *hash tables*)
- Container adaptors: provide a different interface for sequential containers, but they are not containers themselves.
  - `stack`, `queue`, `priority_queue`
  - (since C++23) `flat_set`, `flat_map`, `flat_multiset`, `flat_multimap`

4/51

### Iterators

Without iterators:

- Traverse an array

```
for (int i = 0; i != sizeof(a) / sizeof(a[0]); ++i)
 do_something(a[i]);
```

- Traverse a `vector`

```
for (std::size_t i = 0; i != v.size(); ++i)
 do_something(v[i]);
```

- Traverse a linked-list?

```
for (ListNode *p = l.head(); p; p = p->next)
 do_something(p->data);
```

6/51

### Iterators

A generalization of pointers, used to access elements in different containers in a uniform manner.

With iterators:

The following works no matter whether `c` is an array, a `std::string`, or any container.

```
for (auto it = std::begin(c); it != std::end(c); ++it)
 do_something(*it);
```

Equivalent way: range-based for loops

```
for (auto &x : c) do_something(x);
```

7/51

### Algorithms

The algorithms library defines functions for a variety of purposes:

- searching, sorting, counting, manipulating, ...

Examples:

```
// assign every element in `a` with the value `x`.
std::fill(a.begin(), a.end(), x);
// sort the elements in `b` in ascending order.
std::sort(b.begin(), b.end());
// find the first element in `b` that is equal to `x`.
auto pos = std::find(b.begin(), b.end(), x);
// reverse the elements in `c`.
std::reverse(c.begin(), c.end());
```

8/51

### Algorithms

Example: Obtain the rank of each number in a sequence.

```
auto getRank(const std::vector<int> &data) {
 auto tmp = data;
 std::sort(tmp.begin(), tmp.end()); // sort
 auto pos = std::unique(tmp.begin(), tmp.end()); // drop duplicates
 auto ret = data;
 for (auto &x : ret)
 x = std::lower_bound(tmp.begin(), pos, x) - tmp.begin(); // binary search
 return ret;
}
```

9/51

### Function objects

Things that look like "functions": *Callable*

- functions, and also function pointers
- objects of a class type that has an overloaded `operator()` (the function-call operator)
- lambda expressions

The standard library has defined some common function objects: `std::less`, `std::greater`, ...

```
std::sort(a.begin(), a.end(), std::greater<>()); // Sort in descending order.
```

10/51

### Adaptors

Container adaptors: `std::stack`, `std::queue`, `std::priority_queue`

- Represent the stack, queue and the priority-queue data structures respectively.
- They are **not** containers themselves. They are based on some underlying container, and provide the interfaces of the corresponding data structures.

```
std::stack<int> stk; // By default, uses `std::deque<int>` as
 // the underlying container.
std::stack<int, std::vector<int>> stk2; // Uses `std::vector<int>`.
```

Iterator adaptors: To be discussed in recitations.

11/51

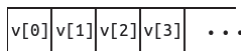
### Sequence containers

Note: `std::string` is not treated as a container but behaves much like one.

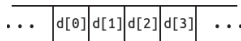
12/51

## Sequence containers

- `std::vector<T>` : dynamic contiguous array (we are quite familiar with)



- `std::deque<T>` : double-ended queue (often pronounced as "deck")
  - `std::deque<T>` supports fast insertion and deletion **at both its beginning and its end**. (`push_front`, `pop_front`, `push_back`, `pop_back`)

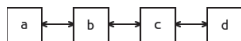


- `std::array<T, N>` : same as `T[N]`, but it is a **container**
  - It will never decay to `T*`.
  - Container interfaces are provided: `.at(i)`, `.front()`, `.back()`, `.size()`, ..., as well as iterators.

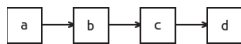
13/51

## Sequence containers

- `std::list<T>` : doubly-linked list
  - `std::list<T>` supports fast insertion and deletion **anywhere in the container**,
  - but fast random access is not supported (i.e. no `operator[]`).
  - Bidirectional traversal is supported.



- `std::forward_list<T>` : singly-linked list
  - Intended to save time and space (compared to `std::list`).
  - Only forward traversal is supported.



14/51

## Interfaces

STL containers have consistent interfaces. See [here](#) for a full list.

Element access:

- `c.at(i)`, `c[i]` : access the element indexed `i`. `at` performs bounds checking, and throws `std::out_of_range` if `i` exceeds the valid range.
- `c.front()`, `c.back()` : access the front/back element.

15/51

## Interfaces

Size and capacity: `c.size()` and `c.empty()` are what we already know.

- `c.resize(n)`, `c.resize(n, x)` : adjust the container to be with exactly `n` elements. If `n > c.size()`, `n - c.size()` elements will be appended.
  - `c.resize(n)` : Appended elements are **value-initialized**.
  - `c.resize(n, x)` : Appended elements are copies of `x`.
- `c.capacity()`, `c.reserve(n)`, `c.shrink_to_fit()` : only for `string` and `vector`.
  - `c.capacity()` : returns the capacity (number of elements that *can* be stored in the current storage)
  - `c.reserve(n)` : reserves space for at least `n` elements.
  - `c.shrink_to_fit()` : requests to remove the unused capacity, so that `c.capacity() == c.size()`.

16/51

## Interfaces

Modifiers:

- `c.push_back(x)`, `c.emplace_back(args...)`, `c.pop_back()` : insert/delete elements at the end of the container.
- `c.push_front(x)`, `c.emplace_front(args...)`, `c.pop_front()` : insert/delete elements at the beginning of the container.
- `c.clear()` removes all the elements in `c`.

17/51

## Interfaces

Modifiers:

- `c.insert(...)`, `c.emplace(...)`, `c.erase(...)` : insert/delete elements at a specified location.
  - Warning:** For containers that need to maintain contiguous storage (`string`, `vector`, `deque`), insertion and deletion somewhere in the middle can be **very slow** ( $O(n)$ ).
- These functions have a lot of overloads. Remember a few common ones, and STFW (Search The Friendly Web) when you need to use them.

18/51

## Interfaces

Some of these member functions are not supported on some containers, **depending on the underlying data structure**. For example:

- Any operation that modifies the length of the container is not allowed for `array`.
- `push_front`, `emplace_front` and `pop_front` are not supported on `string`, `vector` and `array`.
- `size` is not supported on `forward_list` in order to save time and space.
- `operator[]` and `at` are not supported on linked-lists.

[This table](#) tells you everything.

19/51

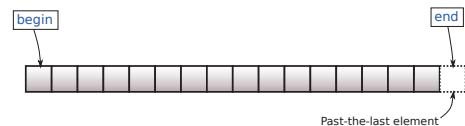
## Iterators

Every container has its iterator: `Container::iterator`. e.g. `std::vector<int>::iterator`, `std::forward_list<std::string>::iterator`

- `auto` comes to our rescue!

`c.begin()` returns the iterator to the first element of `c`.

`c.end()` returns the iterator to **the element following the last element** of `c`.



20/51

## Iterator categories

**ForwardIterators:** supports `*it`, `it->mem`, `++it`, `it++`, `it1 == it2`, `it1 != it2`

**BidirectionalIterator:** a ForwardIterator that can be moved in both directions

- supports `--it` and `it--`.

**RandomAccessIterator:** a BidirectionalIterator that can be moved to point to any element in constant time.

- supports `it + n`, `n + it`, `it - n`, `it += n`, `it -= n` for an integer `n`.
- supports `it[n]`, equivalent to `*(it + n)`.
- supports `it1 - it2`, returns the **distance** of two iterators.
- supports `<`, `<=`, `>`, `>=`.

21/51

## Iterator categories

**ForwardIterators:** an iterator that can be moved forward.

- `forward_list<T>::iterator`

**BidirectionalIterator:** a ForwardIterator that can be moved in both directions

- `list<T>::iterator`

**RandomAccessIterator:** a BidirectionalIterator that can be moved to point to any element in constant time.

- `string::iterator`, `vector<T>::iterator`, `deque<T>::iterator`, `array<T,N>::iterator`

22/51



Iterator categories

To know the category of an iterator of a container, consult its type alias member `iterator_category`.

```
using vec_iter = std::vector<int>::iterator;
using category = vec_iter::iterator_category;
```

Put your mouse on `category`, and the IDE will tell you what it is.

It is one of the following tags: `std::forward_iterator_tag`, `std::bidirectional_iterator_tag`, `std::random_access_iterator_tag`.

Note: Two other categories `InputIterator` and `OutputIterator` will be discussed in recitations.

23/51

std::set

Defined in `<set>`.

- `std::set<T>` is a set whose elements are of type `T`. `operator<(const T, const T)` should be supported, because it is usually implemented as Red-black trees.
- `std::set<T, Cmp>` is also available. `x < y` will be replaced with `cmp(x, y)`, where `cmp` is a function object of type `Cmp`.

```
std::set<int> s1; // An empty set of ints
std::set<std::string> s2{"hello", "world"}; // A set of strings,
// initialized with two elements
struct Student { std::string name; int id; };
std::set<Student> s3; // No operator< for Student is available.
// This line alone does not cause error, but you cannot
// insert elements into it.
s3.insert(Student{"Alice", 42}); // Error: No operator< available.
```

28/51

std::set

Defined in `<set>`.

- `std::set<T>` is a set whose elements are of type `T`. `operator<(const T, const T)` should be supported, because `set` is usually implemented as Red-black trees.
- `std::set<T, Cmp>` is also available. `x < y` will be replaced with `cmp(x, y)`, where `cmp` is a function object of type `Cmp`.

```
struct Student { std::string name; int id; };
struct CmpStudentByName {
 bool operator()(const Student &a, const Student &b) const {
 return a.name < b.name;
 }
};
std::set<Student, CmpStudentByName> students; // OK
students.insert(Student{"Alice", 42}); // OK
```

29/51

std::set

Constructors

```
std::set<Type> s1{a, b, c, ...};
std::set<Type> s2(begin, end); // An iterator range [begin, end)
```

C++17 CTAD (Class Template Argument Deduction) also applies:

```
std::set s1{a, b, c, ...}; // Element type is deduced according to the list
std::set s2(begin, end); // Element type is deduced according to
// the type of elements pointed by `begin` and `end`.
```

Besides, `std::set` is copy-constructible, copy-assignable, move-constructible and move-assignable, just as the sequence containers we have learned.

`std::set` does not contain duplicate elements. These constructors will ignore duplicate elements.

30/51

std::set: operations

Common operations: `s.empty()`, `s.size()`, `s.clear()`.

Insertion: `insert` and `emplace`. Duplicate elements will not be inserted.

- `s.insert(x)`, `s.insert({a, b, ...})`, `s.insert(begin, end)`.

```
std::set s{3, 2, 5, 5, 1}; // {1, 2, 3, 5}. The duplicate 5 is removed.
std::cout << s.size() << std::endl; // 4
s.insert(42); // {1, 2, 3, 5, 42}
s.insert(42); // Nothing is inserted. (No errors.)
int a[]{10, 20, 30};
s.insert(a, a + 3); // An iterator range.
// s now contains {1, 2, 3, 5, 10, 20, 30, 42}.
s.insert({11, 12}); // {1, 2, 3, 5, 10, 11, 12, 20, 30, 42}.
```

26/51

31/51

std::set: insertion

Insertion: `insert` and `emplace`. Duplicate elements will not be inserted.

- `s.emplace(args...)`. Forwards the arguments `args...` to the constructor of the element type, and constructs the element in-place.

```
std::set<std::string> s;
s.emplace(10, 'c'); // inserts a string "ccccccccc"
```

`s.insert(x)` and `s.emplace(args...)` returns `std::pair<iterator, bool>`:

- On success, `.first` is an `iterator` pointing to the inserted element, and `.second` is `true`.
- On failure, `.first` is an `iterator` pointing to the element that prevented the insertion, and `.second` is `false`.

32/51

Constructors of containers

All sequence containers can be constructed in the following ways:

- Container `c(b, e)`, where `[b, e)` is an iterator range.
  - Copies elements from the iterator range `[b, e)`.
- Container `c(n, x)`, where `n` is a nonnegative integer and `x` is a value.
  - Initializes the container with `n` copies of `x`.
- Container `c(n)`, where `n` is a nonnegative integer.
  - Initializes the container with `n` elements. All elements are value-initialized.
  - This is not supported by `string`. (Why?)

24/51

Constructors of containers

All sequence containers can be constructed in the following ways:

- Container `c(b, e)`, where `[b, e)` is an iterator range.
  - Copies elements from the iterator range `[b, e)`.
- Container `c(n, x)`, where `n` is a nonnegative integer and `x` is a value.
  - Initializes the container with `n` copies of `x`.
- Container `c(n)`, where `n` is a nonnegative integer.
  - Initializes the container with `n` elements. All elements are value-initialized.
  - This is not supported by `string`, because it is meaningless to have `n` value-initializes `char` `s` (all of them will be `'\0'`)!

25/51

Associative containers

Motivation: set

Represent a "set":

- Quick insertion, lookup and deletion of elements.
- Order does not matter.

Sequence containers do not suffice:

- Lookup of elements is  $O(n)$ .
- Quick insertion/deletion only happens at certain positions for some containers.
  - e.g. `vector` only supports quick insertion/deletion at the end.
- The order of elements is preserved, which is not important.

You will learn the appropriate data structures in CS101.

27/51

## std::set: iterators

`s.begin()` , `s.end()` : Begin and off-the-end iterators.

The iterator of `std::set` is `BidirectionalIterator`:

- Supports `*it`, `it->mem`, `++it`, `it++`, `--it`, `it--`, `it1 == it2`, `it1 != it2`.

The elements are in ascending order: The following assertion always succeeds (if both `tmp` and `iter` are dereferenceable).

```
auto tmp = iter;
++iter;
assert(*tmp < *iter);
```

33/51

## std::set: iterators

Elements in a `set` cannot be modified directly: `*iter` returns a reference-to-`const`.

- The elements are stored in specific positions in the red-black tree, according to their values.
- You cannot change their values arbitrarily.

34/51

## std::set: traversal

Range-for still works!

```
std::set<int> s{5, 5, 7, 3, 20, 12, 42};
for (auto x : s)
 std::cout << x << ' ';
std::cout << std::endl;
```

Output: 3, 5, 7, 12, 20, 42 . The elements are in ascending order.

Equivalent way: Use iterators

```
for (auto it = s.begin(); it != s.end(); ++it)
 std::cout << *it << ' ';
std::cout << std::endl;
```

35/51

## std::set: deletion

Delete elements: `erase`

- `s.erase(x)`, `s.erase(pos)`, `s.erase(begin, end)`, where `pos` is an iterator pointing to some element in `s`, and `[begin, end)` is an iterator range in `s`.
- `s.erase(x)` removes the element that is equivalent to `x`, if any.
  - returns 0 or 1, indicating the number of elements removed.

```
std::set<int> s{5, 5, 7, 3, 20, 12, 42};
std::cout << s.erase(42) << std::endl; // 42 is removed. output: 1
// s is now {3, 5, 7, 12, 20}.
s.erase(++s.begin()); // 7 is removed.
```

36/51

## std::set: element lookup

`s.find(x)`, `s.count(x)`, and some other functions.

`s.find(x)` returns an iterator pointing to the element equivalent to `x` (if found), or `s.end()` (if not found).

```
std::set<int> s = someValues();
if (s.find(x) != s.end()) // x is found
 // ...
```

37/51

## std::set: pros and cons

The time complexity of insertion, deletion, and lookup of elements in a `std::set` is logarithmic in the size of the container. ( $O(\log n)$ )

- Compared to sequence containers, this is (almost) a huge improvement.

Elements are sorted automatically.

Fast random access like `v[i]` is not supported.

38/51

## Other kinds of sets:

Sets based on red-black trees:

- `std::set`
- `std::multiset`: allows duplicate elements

Sets based on hash-tables: (since C++11)

- `std::unordered_set`: hash-table version of `std::set`
- `std::unordered_multiset`: allows duplicate elements

Sets based on hash-tables provides (average-case)  $O(1)$  time operations, but requires the data to be hashable.

39/51

## Motivation: map

Represent a map:  $f : S \rightarrow T$ .

- For sequence containers `Container<Type>`:  $S = \{0, 1, 2, \dots, N - 1\}$  (index),  $T$  is the set of values of type `Type`.
- For `std::set<Type>`:  $T = \{\text{exist, not-exist}\}$ ,  $S$  is the set of values of type `Type`.

`std::map<Key, Value>`: defined in `<map>`

- `Key` is the type of elements in  $S$ , and `Value` is the type of elements in  $T$ .
- Stores "key-value" pairs.

40/51

## Motivation: map

Example: Count the occurrences of strings.

```
std::map<std::string, int> counter; // maps every string to an integer
std::string word;
while (std::cin >> word)
 ++counter[word]; // !!
```

Now for any string `str`, `counter[str]` is an integer indicating how many times `str` has occurred.

41/51

## std::map: comparison with std::set

`std::map<Key, Value>` has two template parameters: `Key` and `Value`.

- If we ignore `Value`, it is a `std::set<Key>`.
  - Duplicate keys are not allowed.
  - `operator<(const Key, const Key)` is required.
  - Elements are stored in ascending order of keys.
  - Keys cannot be modified directly.
- The element type of `std::map<Key, Value>` is `std::pair<const Key, Value>`.
  - `*iter` returns `std::pair<const Key, Value> &`.

42/51

## std::map: comparison with std::set

Constructors:

- std::map<Key, Value> m{{key1, value1}, {key2, value2}, ...};
- std::map<Key, Value> m(begin, end), but the elements should be pairs:

```
std::vector<std::pair<int, int>> v{{1, 2}, {3, 4}};
std::map<int, int> m(v.begin(), v.end());
```

Insertion:

- m.insert({key, value})
- m.insert({{key1, value1}, {key2, value2}, ...})
- m.insert(begin, end)

43/51

## std::map: comparison with std::set

Deletion:

- m.erase(pos), m.erase(begin, end) : same as std::set<T>::erase.
- m.erase(key) : Removes the element whose key is key.

Iterators: BidirectionalIterator, pointing to std::pair<const Key, Value>.

```
std::map<std::string, int> counter = someValues();
for (auto it = counter.begin(); it != counter.end(); ++it)
 std::cout << it->first << " occurred " << it->second << " times.\n";
```

44/51

## std::map: traversal

Use range-for:

```
for (const auto &kvpair : counter)
 std::cout << kvpair.first << " occurred " << kvpair.second << " times.\n";
```

It's so annoying to deal with the pair stuff...

45/51

## std::map: traversal

Use range-for:

```
for (const auto &kvpair : counter)
 std::cout << kvpair.first << " occurred " << kvpair.second << " times.\n";
```

It's so annoying to deal with the pair stuff...

C++17 structured binding kills the game!

```
for (const auto &[word, occ] : counter)
 std::cout << word << " occurred " << occ << " times.\n";
```

(Looks very much like Python unpacking.)

46/51

## std::map-specific: operator[]

m[key] finds the key-value pair whose key is equivalent to key.

- If such key does not exist, inserts {key, Value{}} - the value is value-initialized.
- Then, returns reference to the value.

```
std::map<std::string, int> counter;
std::string word;
while (std::cin >> word)
 ++counter[word]; // If 'word' does not exist in 'counter',
 // a pair {word, 0} is inserted first.
```

47/51

## std::map: element lookup

m.find(key), m.count(key), and some other member functions.

Note: m.find(key) does not insert elements. m[key] will insert an element if that key does not exist.

48/51

## Other kinds of maps:

Maps based on red-black trees:

- std::map
- std::multimap: allows duplicate keys

Maps based on hash-tables: (since C++11)

- std::unordered\_map: hash-table version of std::map
- std::unordered\_multimap: allows duplicate keys

Maps based on hash-tables provides (average-case)  $O(1)$  time operations, but requires the key to be hashable.

49/51

## Summary

Sequence containers

- std::vector<T> : dynamic contiguous array (we are quite familiar with)

v[0] v[1] v[2] v[3] ...

- std::deque<T> : double-ended queue (often pronounced as "deck")

... d[0] d[1] d[2] d[3] ...

- std::array<T, N> : same as T[N], but it is a container

- std::list<T> : doubly-linked list

a ↔ b ↔ c ↔ d

- std::forward\_list<T> : singly-linked list

a → b → c → d

50/51

## Summary

Associative containers

- std::set<T> : A finite set  $\{e_1, e_2, \dots, e_n\}$  where elements are of type T.
- std::map<Key, Value> : A map  $f : S \mapsto T$ , where S and T are the sets of values of type Key and Value respectively.
- std::set and std::map are ordered: T and Key need to have an ordering, either in the form of operator< or some user-supplied comparator.
- std::unordered\_set and std::unordered\_map are unordered and hash-based.

51/51

## CS100 Lecture 25

Templates I

1/46

## Contents

- Function templates
- Class templates
- Alias templates, variable templates and non-type template parameters

2/46

## Function templates

3/46

### Motivation: function template

```
int compare(int a, int b) {
 if (a < b) return -1;
 if (b < a) return 1;
 return 0;
}
int compare(double a, double b) {
 if (a < b) return -1;
 if (b < a) return 1;
 return 0;
}
int compare(const std::string &a, const std::string &b) {
 if (a < b) return -1;
 if (b < a) return 1;
 return 0;
}
```

4/46

### Motivation: function template

```
template <typename T>
int compare(const T &a, const T &b) {
 if (a < b) return -1;
 if (b < a) return 1;
 return 0;
}
```

- Type parameter: `T`
- A template is a **guideline** for the compiler:
  - When `compare(42, 40)` is called, `T` is deduced to be `int`, and the compiler generates a version of the function for `int`.
  - When `compare(s1, s2)` is called, `T` is deduced to be `std::string`, ...
  - The generation of a function based on a function template is called the **instantiation** (实例化) of that function template.

5/46

### Type parameter

```
template <typename T>
int compare(const T &a, const T &b) {
 if (a < b) return -1;
 if (b < a) return 1;
 return 0;
}
```

- The keyword `typename` indicates that `T` is a **type**.
- Here `typename` can be replaced with `class`. **They are totally equivalent here.** Using `class` doesn't mean that `T` should be a class type.

6/46

### Type parameter

```
template <typename T>
int compare(const T &a, const T &b) {
 if (a < b) return -1;
 if (b < a) return 1;
 return 0;
}
```

- Why do we use `const T &`? Why do we use `b < a` instead of `a > b`?

7/46

### Type parameter

```
template <typename T>
int compare(const T &a, const T &b) {
 if (a < b) return -1;
 if (b < a) return 1;
 return 0;
}
```

- Why do we use `const T &`? Why do we use `b < a` instead of `a > b`?
  - Because we are dealing with an unknown type!
  - `T` may not be copyable, or copying `T` may be costly.
  - `T` may only support `operator<` but does not support `operator>`.

\* Template programs should try to minimize the number of requirements placed on the argument types.

8/46

### Template argument deduction

To instantiate a function template, every template argument must be known, but not every template argument has to be specified.

When possible, the compiler will deduce the missing template arguments from the function arguments.

```
template <typename To, typename From>
To my_special_cast(From f);

double d = 3.14;
int i = my_special_cast<int>(d); // calls my_special_cast<int, double>(double)
char c = my_special_cast<char>(d); // calls my_special_cast<char, double>(double)
```

9/46

### Template argument deduction

Suppose we have the following function

```
template <typename P>
void fun(P p);
```

and a call `fun(a)`, where the type of `a` (ignoring references) is `A`.

- `A` is never considered to be a reference. For example,

```
const int &cr = 42;
fun(cr); // A is considered to be const int, not const int &
```

10/46

### Template argument deduction

Suppose we have the following function

```
template <typename P>
void fun(P p);
```

and a call `fun(a)`, where the type of `a` (ignoring references) is `A`.

- Passing-by-value ignores references and top-level `const` qualifications. For example,

```
const int ci = 42; const int &cr = ci;
fun(ci); // A = const int, P = int
fun(cr); // A = const int, P = int
```

11/46

## Template argument deduction

Suppose we have the following function

```
template <typename P>
void fun(P p);
```

and a call `fun(a)`, where the type of `a` (ignoring references) is `A`.

- If `A` is an array or a function type, the *decay* to the corresponding pointer types takes place:

```
int a[10]; int foo(double, double);
fun(a); // A = int[10], P = int *
fun(foo); // A = int(double, double), P = int(*) (double, double)
```

12/46

## Template argument deduction

Suppose we have the following function

```
template <typename P>
void fun(P p);
```

and a call `fun(a)`, where the type of `a` (ignoring references) is `A`.

- If `A` is `const`-qualified, the top-level `const` is ignored;
- otherwise `A` is an array or a function type, the *decay* to the corresponding pointer types takes place.

This is exactly the same thing as in `auto p = a;` !

13/46

## Template argument deduction

Let `T` be the type parameter of the template.

- The deduction for the parameter `T x` with argument `a` is the same as that in `auto x = a;`
- The deduction for the parameter `T &x` with argument `a` is the same as that in `auto &x = a;`. `a` must be an lvalue expression.
- The deduction for the parameter `const T x` with the argument `a` is the same as that in `const auto x = a;`

The deduction rule that `auto` uses is **totally the same** as the rule used here!

14/46

## Template argument deduction

Deduction forms can be nested:

```
template <typename T>
void fun(const std::vector<T> &vec);

std::vector<int> vi;
std::vector<std::string> vs;
std::vector<std::vector<int>> vvi;

fun(vi); // T = int
fun(vs); // T = std::string
fun(vvi); // T = std::vector<int>
```

Exercise: Write a function `map(func, vec)`, where `func` is any unary function and `vec` is any vector. Returns a vector obtained from `vec` by replacing every element `x` with `func(x)`.

15/46

## Template argument deduction

Exercise: Write a function `map(func, vec)`, where `func` is any unary function and `vec` is any vector. Returns a vector obtained from `vec` by replacing every element `x` with `func(x)`.

```
template <typename Func, typename T>
auto map(Func func, const std::vector<T> &vec) {
 std::vector<T> result;
 for (const auto &x : vec)
 result.push_back(func(x));
 return result;
}
```

Usage:

```
std::vector v{1, 2, 3, 4};
auto v2 = map([](int x) { return x * 2; }, v); // v2 == {2, 4, 6, 8}
```

16/46

## Forwarding reference

Also known as "universal reference" (万能引用).

Suppose we have the following function

```
template <typename T>
void fun(T &&x);
```

A call `fun(a)` happens for an expression `a`.

- If `a` is an rvalue expression of type `E`, `T` is deduced to be `E` so that the type of `x` is `E &&`, an rvalue reference.
- If `a` is an lvalue expression of type `E`, `T` will be deduced to `E &`.
  - The type of `x` is `T & && ???` (We know that there are no "references to references" in C++.)

17/46

## Reference collapsing

If a "reference to reference" is formed through type aliases or templates, the **reference collapsing** (引用折叠) rule applies:

- `&&&`, `&&&` and `&&&` collapse to `&`;
- `&&&` collapses to `&&`.

```
using lref = int &;
using rref = int &&;
int n;

lref& r1 = n; // type of r1 is int&
lref&& r2 = n; // type of r2 is int&
rref& r3 = n; // type of r3 is int&
rref&& r4 = 1; // type of r4 is int&&
```

18/46

## Forwarding reference

Suppose we have the following function

```
template <typename T>
void fun(T &&x);
```

A call `fun(a)` happens for an expression `a`.

- If `a` is an rvalue expression of type `E`, `T` is deduced to be `E` so that the type of `x` is `E &&`, an rvalue reference.
- If `a` is an lvalue expression of type `E`, `T` will be deduced to `E &` and the reference collapsing rule applies, so that the type of `x` is `E &`.

19/46

## Forwarding reference

Suppose we have the following function

```
template <typename T>
void fun(T &&x);
```

A call `fun(a)` happens for an expression `a`.

- If `a` is an rvalue expression of type `E`, `T` is `E` and `x` is of type `E &&`.
- If `a` is an lvalue expression of type `E`, `T` is `E &` and `x` is of type `E &`.

As a result, `x` is always a reference depending on the value category of `a` ! Such reference is called a **universal reference** or **forwarding reference**.

The same thing happens for `auto &&x = a;` !

20/46

## Perfect forwarding

A forwarding reference can be used for **perfect forwarding**:

- The value category is not changed.
- If there is a `const` qualification, it is not lost.

Example: Suppose we design a `std::make_unique` for one argument:

- `make_unique<Type>(arg)` forwards the argument `arg` to the constructor of `Type`.
- The value category must not be changed: `make_unique<std::string>(s1 + s2)` must move `s1 + s2` instead of copying it.
- The `const`-qualification must not be changed: No `const` is added if `arg` is non-`const`, and `const` should be preserved if `arg` is `const`.

21/46

## Perfect forwarding

- The value category must not be changed: `make_unique<std::string>(s1 + s2)` must move `s1 + s2` instead of copying it.
- The `const`-qualification must not be changed: No `const` is added if `arg` is non-`const`, and `const` should be preserved if `arg` is `const`.

The following does not meet our requirements: An rvalue will become an lvalue, and `const` is added.

```
template <typename T, typename U>
auto make_unique(const U &arg) {
 return std::unique_ptr<T>(new T(arg));
}
```

22/46

## Perfect forwarding

- The value category must not be changed: `make_unique<std::string>(s1 + s2)` must move `s1 + s2` instead of copying it.
- The `const`-qualification must not be changed: No `const` is added if `arg` is non-`const`, and `const` should be preserved if `arg` is `const`.

We need to do this:

```
template <typename T, typename U>
auto make_unique(U &&arg) {
 // For an lvalue argument, U=E& and arg is E&.
 // For an rvalue argument, U=E and arg is E&&.
 if (/* U is an lvalue reference type */)
 return std::unique_ptr<T>(new T(arg));
 else
 return std::unique_ptr<T>(new T(std::move(arg)));
}
```

23/46

## Perfect forwarding

`std::forward<T>(arg)`: defined in `<utility>`.

- If `T` is an lvalue reference type, returns an lvalue reference to `arg`.
- Otherwise, returns an rvalue reference to `std::move(arg)`.

```
template <typename T, typename U>
auto make_unique(U &&arg) {
 return std::unique_ptr<T>(new T(std::forward<U>(arg))).
}
```

24/46

## Variadic templates

To support an unknown number of arguments of unknown types:

```
template <typename... Types>
void foo(Types... params);
```

It can be called with any number of arguments, of any types:

```
foo(); // OK: `params` contains no arguments
foo(42); // OK: `params` contains one argument: int
foo(42, 3.14); // OK: `params` contains two arguments: int and double
```

`Types` is a **template parameter pack**, and `params` is a **function parameter pack**.

25/46

## Parameter pack

The types of the function parameters can also contain `const` or references:

```
// All arguments are passed by reference-to-const
template <typename... Types>
void foo(const Types &...params);
// All arguments are passed by forwarding reference
template <typename... Types>
void foo(Types &&...params);
```

26/46

## Pack expansion

A **pattern** followed by `...`, in which the name of at least one parameter pack appears at least once, is **expanded** into zero or more comma-separated instantiations of the pattern.

- The name of the parameter pack is replaced by each of the elements from the pack, in order.

```
template <typename... Types>
void foo(Types &&...params) {
 // Suppose Types is T1, T2, T3 and params is p1, p2, p3.
 // ¶ms... is expanded to &p1, &p2, &p3.
 // func(params)... is expanded to func(p1), func(p2), func(p3).
 // func(params...) is expanded to func(p1, p2, p3)
 // std::forward<Types>(params)... is expanded to
 // std::forward<T1>(p1), std::forward<T2>(p2), std::forward<T3>(p3)
}
```

27/46

## Perfect forwarding: final version

Perfect forwarding for any number of arguments of any types:

```
template <typename T, typename... Ts> auto make_unique(Ts &&...params) {
 return std::unique_ptr<T>(new T(std::forward<Ts>(params)...));
}
```

This is also how `std::vector<T>::emplace_back` forward arguments.

```
template <typename T> class vector {
public:
 template <typename... Types> reference emplace_back(Types &&...args) {
 if (size() == capacity())
 reallocate(capacity() == 0 ? 1 : capacity() * 2);
 construct_at(m_end_of_elems, std::forward<Types>(args)...);
 ++m_end_of_elems;
 return *(m_end_of_elems - 1);
 }
};
```

28/46

## `auto` and template argument deduction

We have seen that the deduction rule that `auto` uses is exactly the template argument deduction rule.

If we declare the parameter types in an **lambda expression** with `auto`, it becomes a **generic lambda**:

```
auto less = [](const auto &lhs, const auto &rhs) { return lhs < rhs; };
std::string s1, s2;
bool b1 = less(10, 15); // 10 < 15
bool b2 = less(s1, s2); // s1 < s2
```

29/46

## `auto` and template argument deduction

We have seen that the deduction rule that `auto` uses is exactly the template argument deduction rule.

Since C++20: Function parameter types can be declared with `auto`.

```
auto add(const auto &a, const auto &b) {
 return a + b;
}
// Equivalent way: template
template <typename T, typename U>
auto add(const T &a, const U &b) {
 return a + b;
}
```

30/46

## Class templates

31/46

Define a class template

Let's make our `Dynarray` a template `Dynarray<T>` to support any element type `T`.

```
template <typename T>
class Dynarray {
 std::size_t m_length;
 T *m_storage;

public:
 Dynarray();
 Dynarray(const Dynarray<T> &);
 Dynarray(Dynarray<T> &&) noexcept;
 // other members ...
};
```

32/46

Member functions of a class template

A member function itself can also be a template:

```
template <typename T>
class Dynarray {
public:
 template <typename Iterator>
 Dynarray(Iterator begin, Iterator end)
 : m_length(std::distance(begin, end)), m_storage(new T[m_length]) {
 std::copy(begin, end, m_storage);
 }
};

std::vector<std::string> vs = someValues();
Dynarray<std::string> ds(vs.begin(), vs.end()); // Values are copied from vs.
```

37/46

Define a class template

A class template is a **guideline** for the compiler: When a type `Dynarray<T>` is used for a certain type `T`, the compiler will **instantiate** that class type according to the class template.

For different types `T` and `U`, `Dynarray<T>` and `Dynarray<U>` are different types.

```
template <typename T>
class X {
 int x = 42; // private

public:
 void foo(X<double> xd) {
 x = xd.x; // access a private member of X<double>
 // This is valid only when T is double.
 }
};
```

33/46

Member functions of a class template

A member function itself can also be a template. To define it outside the class, **two** template declarations are needed:

```
// This cannot be written as `template <typename T, typename Iterator>`
template <typename T>
template <typename Iterator>
Dynarray<T>::Dynarray(Iterator begin, Iterator end)
 : m_length(std::distance(begin, end)), m_storage(new T[m_length]) {
 std::copy(begin, end, m_storage);
}
```

38/46

Define a class template

Inside the class template, the template parameters can be omitted when we are referring to the self type:

```
template <typename T>
class Dynarray {
 std::size_t m_length;
 T *m_storage;

public:
 Dynarray();
 Dynarray(const Dynarray &); // Parameter type is const Dynarray<T> &
 Dynarray(Dynarray &&) noexcept; // Parameter type is Dynarray<T> &&
 // other members ...
};
```

34/46

Alias templates, variable templates and non-type template parameters

```
// ...
```

39/46

Member functions of a class template

If we want to define a member function outside the template class, a template declaration is also needed.

```
class Dynarray {
public:
 int &at(std::size_t n);
};

int &Dynarray::at(std::size_t n) {
 return m_storage[n];
}
```

```
template <typename T>
class Dynarray {
public:
 int &at(std::size_t n);
};

template <typename T>
int &Dynarray<T>::at(std::size_t n) {
 return m_storage[n];
}
```

35/46

Alias templates

The `using` declaration can also declare **alias templates**:

```
template <typename T>
using pii = std::pair<T, T>;
pii<int> p1(2, 3); // std::pair<int, int>
```

40/46

Member functions of a class template

A member function will not be instantiated if it is not used!

```
template <typename T>
struct X {
 T x;
 void foo() { x = 42; }
 void bar() { x = "hello"; }
};
```

```
X<int> xi; // OK
xi.foo(); // OK
X<std::string> xs; // OK
xs.bar(); // OK
```

No compile-error occurs: `X<int>::bar()` and `X<std::string>::foo()` are not instantiated because they are not called.

36/46

Variable templates (since C++14)

Variables can also be templates. Typical example: the C++20 `<numbers>` library:

```
namespace std::numbers {
 template <typename T>
 inline constexpr T pi_v = /* unspecified */;
}

auto pi_d = std::numbers::pi_v<double>; // The `double` version of π
auto pi_f = std::numbers::pi_v<float>; // The `float` version of π
```

41/46

## Non-type template parameters

Apart from types, a template parameter can also be an integer, an lvalue reference, a pointer, ...

Since C++20, floating-point types and *literal class types* with certain properties are also allowed.

```
template <typename T, std::size_t N>
class array {
 T data[N];
 // ...
};
array<int, 10> a;
```

42/46

1/37

## Non-type template parameters

Define a function that accepts an array of **any length, any type**:

```
template <typename T, std::size_t N>
void print_array(T (&a)[N]) {
 for (const auto &x : a)
 std::cout << x << ' ';
 std::cout << std::endl;
}
int a[10]; double b[20];
print_array(a); // T=int, N=10
print_array(b); // T=double, N=20
```

43/46

2/37

## Summary

Idea of templates: **Generic programming**.

- Write code that can deal with different types. e.g. Containers parameterized on element types.
- The compiler generates (*instantiates*) the actual code in terms of the template code.

Function template:

- Template argument deduction:
  - Pass-by-value: Reference and top-level `const` -qualification are ignored. Decay happens.
  - Pass-by-reference: Reference collapsing happens.
  - Same as the deduction rule of `auto`.

44/46

3/37

## Summary

Perfect forwarding:

- Forwarding reference: `auto &&`, or `T &&` where `T` is a template type parameter.
  - Deduced to lvalue-reference if the argument is an lvalue, and rvalue-reference if the argument is an rvalue.
  - `const` qualifications are preserved.
- `std::forward<T>(x)` : Casts `x` to `T &&`.

Variadic template and pack expansion: Used to accept an unknown number of arguments of unknown types, and perform some operations on them.

- "unknown" is for the author of the templates. The compiler generates the required versions of them.

45/46

4/37

## Summary

Class templates:

- The class is parameterized on some template parameter. Different template arguments yield different classes.
- The member functions are instantiated only when they are called.

Other things:

- Alias templates
- Variable templates
- Non-type template parameters

46/46

## CS100 Lecture 26

Templates II

## Contents

- Template specialization
- Variadic templates: an example
- Curiously Recurring Template Pattern (CRTP)
- Introduction to template metaprogramming

## Template specialization

Templates are for **generic programming**, but some things need special treatments.

## Specialization for a function template

```
template <typename T>
int compare(T const &lhs, T const &rhs) {
 if (lhs < rhs) return -1;
 else if (rhs < lhs) return 1;
 else return 0;
}
```

What happens for C-style strings?

```
const char *a = "hello", *b = "world";
auto x = compare(a, b);
```

This is comparing two pointers, instead of comparing the strings!

## Specialization for a function template

```
template <typename T>
int compare(T const &lhs, T const &rhs) {
 if (lhs < rhs) return -1;
 else if (rhs < lhs) return 1;
 else return 0;
}
template <> // specialized version for T = const char *
int compare(const char *(const char *const &lhs, const char *const &rhs) {
 return std::strcmp(lhs, rhs);
}
```

Write a specialized version of that function with the template parameters taking a certain group of values.

The type `const T &` with `T = const char *` is `const char *const &`: A reference bound to a `const` pointer which points to `const char`.

5/37



## Specialization for a function template

It is also allowed to omit `<const char *>` following the name:

```
template <typename T>
int compare(T const &lhs, T const &rhs) {
 if (lhs < rhs) return -1;
 else if (rhs < lhs) return 1;
 else return 0;
}
template <>
int compare(const char *const &lhs, const char *const &rhs) {
 return std::strcmp(lhs, rhs);
}
```

6/37

## Specialization for a function template

Is this a specialization?

```
template <typename T>
int compare(T const &lhs, T const &rhs);
template <typename T>
int compare(const std::vector<T> &lhs, const std::vector<T> &rhs);
```

No! These functions constitute **overloading** (allowed).

7/37

## Specialization for a function template

Is this a specialization?

```
template <typename T>
int compare(T const &lhs, T const &rhs);
template <typename T>
int compare<std::vector<T>>(const std::vector<T> &lhs,
 const std::vector<T> &rhs);
```

- Since we write `int compare<std::vector<T>>(...)`, this is a specialization.
- However, such specialization is a **partial specialization**: The specialized function is still a function template.
  - **Partial specialization for function templates is not allowed.**

8/37

## Specialization for a class template

It is allowed to write a specialization for class templates.

```
template <typename T>
struct Dynarray { /* ... */ };
template <> // specialization for T = bool
struct Dynarray<bool> { /* ... */ };
```

Partial specialization is also allowed:

```
template <typename T, typename Alloc>
class vector { /* ... */ };
// specialization for T = bool, while Alloc remains a template parameter.
template <typename Alloc>
class vector<bool, Alloc> { /* ... */ };
```

9/37

## Variadic templates: an example

## A print function

```
template <typename First, typename... Rest>
void print(std::ostream &os, const First &first, const Rest &...rest) {
 os << first;
 if (/* `rest` is not empty */) // How to test this?
 print(os, rest...);
}
int i = 42; double d = 3.14; std::string s = "hello";
print(std::cout, s, d, i);
```

First, understand the different meanings of `...`.

- `typename... Rest` indicates that `Rest` is a template parameter pack.
- `const Rest &...rest` indicates that `rest` is a function parameter pack.
- `rest...` in `print(os, rest...)` is **pack expansion**.

11/37

## Compile-time recurrence

```
template <typename First, typename... Rest>
void print(std::ostream &os, const First &first, const Rest &...rest) {
 os << first;
 if (/* `rest` is not empty */) // How to test this?
 print(os, rest...);
}
std::string s = "hello"; double d = 3.14; int i = 42;
print(std::cout, s, d, i);
```

`print(std::cout, s, d, i)` leads to the instantiation of the following functions:

```
void print(std::ostream &os, const std::string &first, const double &rest0,
 const int &rest1);
void print(std::ostream &os, const double &first, const int &rest0);
void print(std::ostream &os, const int &first);
```

Note: `first` is not a parameter pack, so `print` must have at least **two** arguments.

12/37

## sizeof...(pack)

How many arguments are there in a pack? Use the `sizeof...` operator, which is evaluated at compile-time.

```
template <typename First, typename... Rest>
void print(std::ostream &os, const First &first, const Rest &...rest) {
 os << first;
 if (sizeof...(Rest) > 0)
 print(os, rest...);
}
std::string s = "hello"; double d = 3.14; int i = 42;
print(std::cout, s, d, i);
```

Looks good ... But a compile-error?

13/37

## sizeof...(pack)

Looks good ... But a compile-error?

```
b.cpp: In instantiation of 'void print(std::ostream&, const First&, const Rest& ...)
[with First = int; Rest = {}; std::ostream = std::basic_ostream<char>]':
b.cpp:11:8: required from here
b.cpp:7:10: error: no matching function for call to 'print(std::ostream&)'
 7 | print(os, rest...);
```

It says that when `First = int`, `Rest = {}`, we are trying to call `print(os)` (with nothing to print).

14/37

## Compile-time if

Let's see what the function looks like when `Rest = {}`:

```
template <typename First>
void print(std::ostream &os, const First &first) {
 os << first;
 if (false) // sizeof...(Rest) == 0
 print(os); // Oops! `print` needs at least two arguments!
}
```

The problem is that `if` is a **run-time** control flow statement! The statements must **compile** even if the condition is 100% `false`!

We need a **compile-time** `if`.

10/37

15/37

## Compile-time if: if constexpr

```
if constexpr (condition)
 statement1
```

```
if constexpr (condition)
 statement1
else
 statement2
```

`condition` must be a compile-time constant.

Only when `condition` is `true` will `statement1` be compiled.

Only when `condition` is `false` will `statement2` be compiled.

16/37

## Example 1: Uncopyable

```
template <typename Derived>
class Uncopyable {
 Uncopyable(const Uncopyable &) = delete;
 Uncopyable &operator=(const Uncopyable &) = delete;

public:
 Uncopyable() = default;
};

class Airplane : public Uncopyable<Airplane> {};
class MonaLisa : public Uncopyable<MonaLisa> {};
```

Now `Airplane` and `MonaLisa` inherit from different bases: `Uncopyable<Airplane>` and `Uncopyable<MonaLisa>` are different types.

21/37

## Use if constexpr

```
template <typename First, typename... Rest>
void print(std::ostream &os, const First &first, const Rest &...rest) {
 os << first;
 if constexpr (sizeof...(Rest) > 0)
 print(os, rest...);
}
```

Solution without `if constexpr`: overloading.

```
template <typename T>
void print(std::ostream &os, const T &x) { os << x; }

template <typename First, typename... Rest>
void print(std::ostream &os, const First &first, const Rest &...rest) {
 print(os, first);
 print(os, rest...);
}
```

17/37

## Example 2: Incrementable

|                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>template &lt;typename T&gt; class Iterator {     T *cur;  public:     auto &amp;operator++() {         ++cur;         return *this;     }     auto operator++(int) {         auto tmp = *this;         ++*this;         return tmp;     } };</pre> | <pre>class Rational {     int num;     unsigned denom;  public:     auto &amp;operator++() {         num += denom;         return *this;     }     auto operator++(int) {         auto tmp = *this;         ++*this;         return tmp;     } };</pre> | <pre>class AtomicCounter {     int cnt;     std::mutex m;  public:     auto &amp;operator++() {         std::lock_guard l(m);         ++cnt;         return *this;     }     auto operator++(int) {         auto tmp = *this;         ++*this;         return tmp;     } };</pre> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

22/37

## Example 2: Incrementable

With the prefix incrementation operator `operator++` defined, the postfix version is always defined as follows:

```
auto operator++(int) {
 auto tmp = *this;
 ++*this;
 return tmp;
}
```

How can we avoid repeating ourselves?

18/37

23/37

## Curiously Recurring Template Pattern (CRTP)

### Example 1: Uncopyable

We have seen this `Uncopyable` in Homework 6:

```
class Uncopyable {
 Uncopyable(const Uncopyable &) = delete;
 Uncopyable &operator=(const Uncopyable &) = delete;

public:
 Uncopyable() = default;
};

class ComplexDevice : public Uncopyable { /* ... */};
```

A class can be made uncopyable by inheriting `Uncopyable`.

19/37

### Example 2: Incrementable

```
template <typename Derived>
class Incrementable {
public:
 auto operator++(int) {
 // Since we are sure that the dynamic type of `*this` is `Derived`,
 // we can use `static_cast` here to perform the downcasting.
 auto real_this = static_cast<Derived*>(*this);
 auto tmp = *real_this;
 ++*real_this;
 return tmp;
 }
};

class A : public Incrementable<A> {
public:
 A &operator++(int) { /* ... */ }
 // The operator++(int) is inherited from Incrementable<A>.
};
```

24/37

### Example 1: Uncopyable

But if two classes inherit from `Uncopyable` publicly, odd things may happen ...

```
class Uncopyable {
 Uncopyable(const Uncopyable &) = delete;
 Uncopyable &operator=(const Uncopyable &) = delete;

public:
 Uncopyable() = default;
};

class Airplane : public Uncopyable {}; // Copying an airplane is too costly.
class MonaLisa : public Uncopyable {}; // An artwork is not copyable.

Uncopyable *foo1 = new Airplane();
Uncopyable *foo2 = new MonaLisa();
```

Ooops ... A `Uncopyable*` can point to two things that are **totally unrelated to each other!**

20/37

## Curiously Recurring Template Pattern

By writing the common parts of `X`, `Y`, `Z`, ... in a base class `Base`,

- we can avoid repeating ourselves.
- However, `X`, `Y` and `Z` have a common base (which may lead to weird things), and `Base` does not know *who* is inheriting from it.

By letting `X`, `Y`, `Z`, ... inherit from `Base<X>`, `Base<Y>`, `Base<Z>`, ... respectively,

- each class inherits from a unique base class, and
- the base class knows what the derived class is, so a safe downcast can be performed.

CRTP idiom adopted in the standard library: `std::enable_shared_from_this`.

25/37

## Introduction to template metaprogramming

26/37

### Know whether two types are the same?

```
template <typename T, typename U>
struct is_same {
 static const bool result = false;
};
template <typename T> // specialization for U = T
struct is_same<T, T> {
 static const bool result = true;
};
```

- `is_same<int, double>::result` is false.
- `is_same<int, int>::result` is true.
- Are `int` and `signed` the same type? Let `is_same` tell you!

27/37

### Know whether a type is a pointer?

```
template <typename T>
struct is_pointer {
 static const bool result = false;
};
template <typename T>
struct is_pointer<T*> { // specialization for <T*> for some T.
 static const bool result = true;
};
```

- `is_pointer<int*>::result` is true.
- `is_pointer<int>::result` is false.
- Is `std::vector<int>::iterator` actually a pointer? Is `int[10]` the same thing as `int*`? Consult these "functions"!

28/37

### <type\_traits>

`std::is_same`, `std::is_pointer`, as well as a whole bunch of other "functions": [Go to this standard library](#).

This is part of the **metaprogramming** library.

29/37

### Compute $n!$ in compile-time?

```
template <unsigned N>
struct Factorial {
 static const unsigned long long value = N * Factorial<N - 1>::value;
};
template <>
struct Factorial<0> {
 static const unsigned long long value = 1;
};

int main() {
 int a[Factorial<5>::value]; // 120, which is a compile-time constant.
}
```

30/37

### Check whether an integer is a prime in compile-time?

```
template <unsigned N, unsigned Div> struct PrimeTest {
 static const bool result = (N % Div != 0) && PrimeTest<N, Div + 1>::result;
};
template <unsigned N> struct PrimeTest<N, N> { // end
 static const bool result = true;
};
template <unsigned N> struct IsPrime {
 static const bool result = PrimeTest<N, 2>::result;
};
template <> struct IsPrime<1> {
 static const bool result = false;
};

static_assert(IsPrime<197>::result); // 197 is a prime
static_assert(!IsPrime<42>::result); // 42 is not
```

31/37

### Seven basic quantities in physics

When performing computations in physics, the correctness in **dimensions** is important.

```
double mass = getMass();
double acceleration = getAcc();
double force = mass + acceleration; // Oops! A mistake here!
```

Can we avoid such mistakes in **compile-time**? That is, to make mistakes in dimensions a **compile error**.

32/37

### Seven basic quantities in physics

Each of the seven basic quantities corresponds to a template parameter:

```
template <int mass, int length, int time, int charge,
 int temperature, int intensity, int amount_of_substance>
struct quantity { /* ... */ };

using mass = quantity<1, 0, 0, 0, 0, 0, 0>;
using force = quantity<1, 1, -2, 0, 0, 0, 0>;
using pressure = quantity<1, -1, -2, 0, 0, 0, 0>;
using acceleration = quantity<0, 1, -2, 0, 0, 0, 0>;

mass m = getMass();
acceleration a = getAcc();
force f = m * a; // Error! No match operator+ for 'mass' and 'acceleration'!
force f = m * a; // Correct.
```

If the arithmetic operations of different `quantity`s are defined correctly, we can avoid dimension mistakes in **compile-time**!

33/37

### Template metaprogramming

Template metaprogramming is a very special and powerful technique that makes use of the compile-time computation of C++ compilers. (It is **Turing-complete** and **pure functional programming**.)

Learn a little bit more in recitations.

In modern C++, there are many more things that facilitate compile-time computations:

`constexpr`, `constexpr`, `constexpr`, `constexpr`, `constexpr`, `constexpr`, ...

34/37

### Summary

Template specialization

- Specializes the template function / class when the template argument satisfies certain properties.
- Partial specialization: The specialization still has template parameters.
- Full specialization: The specialization no longer has no template parameters.
- Function templates cannot have partial specializations.

35/37

## Summary

Variadic template example: A `print` function.

- `pack...`: pack expansion.
- `sizeof...(pack)` returns the number of arguments in a parameter pack. It is compile-time evaluated.
- `if constexpr`: compile-time `if`: *compile* statements conditioned on a compile-time boolean expression.

36/37

## Summary

Curiously Recurring Template Pattern (CRTP)

- Let `X` inherit from `Base<X>`.
- Each class inherits from a unique base class.
- The base class knows what the derived class is, so a safe downcast can be performed.

Template metaprogramming (TMP)

- TMP can shift work from runtime to compile-time, thus enabling earlier error detection and higher runtime performance.

37/37

## CS100 Lecture 27

Other Facilities in the Standard Library

## Contents

- C++17 library facilities
  - `function`
  - `optional`
  - `string_view`
  - `pair` and `tuple`
- Going into C++20:
  - Ranges library
  - Formatting library
  - Future

2/41

## C++17 library facilities

## function

Defined in `<functional>`

`std::function<Ret(Args...)>` is a general-purpose function wrapper that stores any callable object that can be called with arguments of types `Args...` and returns `Ret`.

```
Polynomial poly({3, 2, 1}); // `Polynomial` in homework 5
std::function<double(double)> f1(poly);
std::cout << f1(0) << '\n';

std::function<void()> f2 = []() { std::cout << 42 << '\n'; };
f2(); // prints 42
```

4/41

## Recap: callable

A callable object in C++ might be a function, a pointer-to-function, or an object of class type that has an overloaded `operator()`<sup>1</sup>.

- Lambdas belong to the last category, whose type is compiler-generated.

A function has an address! When the program is executed, the program instructions (machine code) are loaded into the memory.

```
int add(int a, int b) { return a + b; }
int main() {
 auto *padd = &add;
 std::cout << (*padd)(3, 4) << '\n';
 std::cout << padd(3, 4) << '\n'; // Also correct.
}
```

A pointer-to-function itself is also callable. `pfunc(...)` is the same as `(*pfunc)(...)`.

5/41

## Example: Calculator

A more fancy way of implementing a calculator:

```
std::map<char, std::function<double(double, double)>> funcMap{
 {'+', std::plus<>()},
 {'-', std::minus<>()},
 {'*', std::multiplies<>()},
 {'/', std::divides<>()}
};
double lhs, rhs; char op;
std::cin >> lhs >> op >> rhs;
std::cout << funcMap[op](lhs, rhs) << '\n';
```

`std::plus`, `std::minus`, etc. are defined in the standard library header `<functional>`.

1/41

6/41

## Example: Calculator

Combining different ways of using `std::function`:

```
double add(double a, double b) { return a + b; }
struct Divides {
 double operator()(double a, double b) const { return a / b; }
};
int main() {
 std::map<char, std::function<double(double, double)>> funcMap{
 {'+', add}, // A function (in fact, a pointer-to-function)
 {'-', std::minus<>()}, // An object of type `std::minus<>`
 {'*', [](double a, double b) { return a * b; }}, // A lambda
 {'/', Divides{}} // An object of type `Divides`
 };
 double lhs, rhs; char op;
 std::cin >> lhs >> op >> rhs;
 std::cout << funcMap[op](lhs, rhs) << '\n';
}
```

7/41

## optional

Defined in the header `<optional>`.

`std::optional<T>` manages either an object of type `T`, or nothing.

- Algebraically: Let  $\mathcal{T}$  be the value set of `T`, and let  $\mathcal{O}$  be the value set of `std::optional<T>`. We have

$$\mathcal{O} = \mathcal{T} \cup \{\text{std::nullopt}\},$$

where `std::nullopt` is a special object that represents the state of *nothing*.

3/41

8/41

## Example: Solving quadratic equation in $\mathbb{R}$ .

A typical example: Use `std::optional<Solution>` when there may be no solutions.

```
std::optional<std::pair<double, double>> solve(double a, double b, double c) {
 auto delta = b * b - 4 * a * c;
 if (delta < 0)
 return std::nullopt; // No solution.
 auto sqrtDelta = std::sqrt(delta);
 // An 'std::optional<T>' can be initialized directly from 'T'.
 return std::pair{(-b - sqrtDelta) / (2 * a), (-b + sqrtDelta) / (2 * a)};
}
```

9/41

## Example: Solving quadratic equation in $\mathbb{R}$ .

```
void printSolution(const std::optional<std::pair<double, double>> &sln) {
 if (sln) { // conversion to bool tests whether it contains an object
 auto [x1, x2] = sln.value(); // .value() returns the contained object.
 std::cout << "The solutions are " << x1 << " and " << x2 << ' '.
 << std::endl;
 } else
 std::cout << "No solutions." << std::endl;
}

int main() {
 auto sln1 = solve(1, -2, -3);
 printSolution(sln1);
 auto sln2 = solve(1, 0, 1);
 printSolution(sln2);
 return 0;
}
```

10/41

## How will you implement `std::optional<T>`?

Is this good?

```
template <typename T>
struct Optional {
 T object;
 bool hasObject;
 // ...
};
```

11/41

## How will you implement `std::optional<T>`?

Is this good?

```
template <typename T>
struct Optional {
 T object;
 bool hasObject;
 // ...
};
```

NO! It models  $\mathcal{O} = \mathcal{T} \times \{\text{true}, \text{false}\}$ . The `object` is alive even when `hasObject` is `false`!

- This also requires the "nothing" state to be represented by default-initializing `object`, but the default-initialization of `T` may be expensive or disabled!

12/41

## How will you implement `std::optional<T>`?

Is this good?

```
template <typename T>
struct Optional {
 std::unique_ptr<T> pObject; // "Nothing" is represented by nullptr.
 // ...
};
```

13/41

## How will you implement `std::optional<T>`?

Is this good?

```
template <typename T>
struct Optional {
 std::unique_ptr<T> pObject; // "Nothing" is represented by nullptr.
 // ...
};
```

It does model  $\mathcal{O} = \mathcal{T} \cup \{\text{std::nullopt}\}$ , but it requires dynamic memory allocation.

If I just need something to represent "no solution", why would I have to store the solution on dynamic memory?

- Such overhead is not acceptable!

An `std::optional` models an **object**, not a pointer!

14/41

## How will you implement `std::optional<T>`?

The implementation is not trivial. See [this page](#) if you are interested.

- It requires careful treatment of memory, possibly using a `union`.

15/41

## Other member functions of `std::optional`

Some common ones:

- `*o` : returns the stored object. The behavior is undefined if it does not contain one.
- `o->mem` : equivalent to `(*o).mem`.
- `std::optional<T>` does not model a pointer, although it provides `*` and `->`.
- `o.value_or(x)` : returns the stored object, or `x` if it does not contain one.
- `o1.swap(o2)`
- `o.reset()` : destroys any contained object
- `o.emplace(args...)` : constructs the contained object in-place.

Refer to [cppreference](#) for a full list.

16/41

## `string_view`

The old question: How do you pass a string?

```
void some_operation(const std::string &str) {
 // ...
}
```

Pass-by-reference-to-`const` seems to be quite good: It accepts both lvalues and rvalues, whether `const`-qualified or not, and avoids copy.

- Wait ... Does it really avoid copy?

17/41

## `string_view`

The old question: How do you pass a string?

```
void some_operation(const std::string &str) {
 // ...
}
```

```
std::string s = something();
some_operation(s); // Copy is avoided, of course.
some_operation("The quick red fox jumps over the slow red turtle."); // Oops!
```

- When we pass a string literal, a temporary `std::string` is created first, during which the content of the string is still copied!

18/41

## string\_view

What do `char[N]`, `"hello"`, `std::string`, `str = new char[N]{...}` have in common?

19/41

## string\_view

What do `char[N]`, `"hello"`, `std::string`, `str = new char[N]{...}` have in common?

- A pointer to the first position, and a length!

```
struct StringView {
 const char *start;
 std::size_t length;

 StringView(const char *cstr) : start(cstr), length(std::strlen(cstr)) {}
 StringView(const std::string &str) : start(str.data()), length(str.size()) {}

 std::size_t size() const { return length; }
 const char &operator[](std::size_t n) const { return start[n]; }

 // ...
};
```

20/41

## string\_view

Defined in header `<string_view>`.

`std::string_view`: An **non-owning reference** to a string. It is often used to refer to a string that we don't modify.

```
// `std::string_view` is usually passed by value directly,
// since it is light-weighted and models a "pointer".
void some_operation(std::string_view str);
int main() {
 std::string s1 = something(), s2 = something_else();
 some_operation(s1);
 some_operation(s1 + s2);
 some_operation("hello");
}
```

- No copy is performed, even for `"hello"`.

21/41

## Avoid dangling string\_view!

Let's use `std::string_view` everywhere, shall we?

```
struct Student {
 std::string_view name;
 // ...

 Student(std::string_view name_) : name(name_) {}
};

int main() {
 std::string s1 = something(), s2 = something_else();
 Student stu(s1 + s2);
 std::cout << stu.name << '\n'; // Undefined behavior!
}
```

22/41

## Avoid dangling string\_view!

Let's use `std::string_view` everywhere, shall we?

```
struct Student {
 std::string_view name;
 // ...

 Student(std::string_view name_) : name(name_) {}
};

int main() {
 std::string s1 = something(), s2 = something_else();
 Student stu(s1 + s2); // `s1 + s2` is a temporary!
 std::cout << stu.name << '\n'; // Undefined behavior! `stu.name` is dangling!
}
```

`stu.name` refers to a **temporary** created by `s1 + s2`! It is destroyed immediately when the initialization of `stu` ends.

23/41

## Avoid dangling string\_view!

The same thing happens if you try to use reference-to-`const` as a member:

```
struct Student {
 const std::string &name;
 // ...

 Student(const std::string &name_) : name(name_) {}
};

int main() {
 std::string s1 = something(), s2 = something_else();
 Student stu(s1 + s2); // `s1 + s2` is a temporary!
 std::cout << stu.name << '\n'; // Undefined behavior! `stu.name` is dangling!
}
```

24/41

## string\_view

Using a `string_view` parameter can accept strings of any form, and avoid copy.

- The use of `string_view` as a parameter is often safe, because the lifetime of the argument should be longer than the execution of the function.
- In other cases, be extremely careful to avoid dangling `string_view`!

25/41

## pair and tuple

`pair` and `tuple` can be thought of as a "quick and dirty" data structure.

- `std::pair<T, U>`: defined in `<utility>`. It models
$$\mathcal{T} \times \mathcal{U} = \{(t, u) \mid t \in \mathcal{T}, u \in \mathcal{U}\}.$$
- `std::tuple<T1, T2, ...>`: defined in `<tuple>`. It models

$$\mathcal{T}_1 \times \mathcal{T}_2 \times \dots \times \mathcal{T}_n,$$

where  $n$  is compile-time known non-negative constant integer.

26/41

## pair and tuple

`std::pair<T, U>` is defined almost just like this:

```
template <typename T, typename U>
struct pair {
 T first;
 U second;
};
```

It comes from C++98. At that time, there was no **variadic templates** which is necessary for building a `tuple`.

`std::tuple<Types...>` is an extension of `std::pair<T1, T2>`, which can contain an arbitrary number of things.

27/41

## pair and tuple in modern C++

With the increasing support for **aggregates** and **structured binding** in modern C++, `pair` and `tuple` are seldom needed now.

A user-defined type can also be used conveniently:

```
template <typename T> struct Set {
 struct InsertResult {
 bool success;
 Iterator position;
 };
 InsertResult insert(const T &);
};

// structured binding
auto [ok, pos] = mySet.insert(something);
if (ok)
 do_something(pos);
```

28/41

## pair and tuple in modern C++

Which one do you prefer?

```
template <typename T> struct Set {
 struct InsertResult {
 bool success;
 Iterator position;
 };
 InsertResult insert(const T &);
};

auto result = mySet.insert(x);
if (result.success)
 do_something(result.position);
```

```
template <typename T> struct Set {
 std::pair<bool, Iterator>
 insert(const T &);
};

auto result = mySet.insert(x);
if (result.first)
 do_something(result.second);
```

[Best practice] Prefer a self-defined type with meaningfully named members to `pair` and `tuple`.

29/41

## Others

Other things in the C++17 standard library we have not touched:

- `std::variant<T1, T2, ...>`: A type-safe `union` that models  $\mathcal{V} = \mathcal{T}_1 \cup \mathcal{T}_2 \cup \dots \cup \mathcal{T}_n$ .
- `std::any`: A type-safe container that contains a single object of any copy-constructible type.
- `<regex>`: Standard library support for **regular expressions**.
- `<filesystem>`: Standard library support for **filesystem operations**.
- Concurrency support: `<thread>`, `<atomic>`, `<mutex>`, ...

30/41

## Going into C++20

### C++20 is historic!

CppCon2021 Talk by Bjarne Stroustrup: C++20: Reaching the aims of C++

C++20 is the first C++ standard that delivers on virtually all the features that Bjarne Stroustrup dreamed of in *The Design and Evolution of C++* in 1994.

- Coroutines (Talk)
- Concepts and requirements (`concept`, `requires`) (Talk)
- Modules (Talk) (Talk on the implementation by MSVC)
- Ranges library
- Formatting library
- Three-way comparison (`operator<=>`, `std::partial_ordering`, ...)

32/41

### Ranges library: The next generation of STL.

An extension and generalization of the algorithms and iterator libraries that makes them more powerful by making them **composable** and less error-prone.

A range is represented by one object, instead of `(begin, end)` or `(begin, n)`.

## C++20 constrained algorithms

Given `Student` defined as

```
struct Student { std::string name; int id; };
```

C++17:

```
void sortStudentsByID(std::vector<Student> &students) {
 std::sort(students.begin(), students.end(),
 [](const Student &a, const Student &b) { return a.id < b.id; });
}
```

C++20:

```
void sortStudentsByID(std::vector<Student> &students) {
 std::ranges::sort(students, {}, &Student::id);
}
```

34/41

### Ranges are composable

Enumerate the last 10 even numbers in a vector in reverse order:

```
using namespace std::views;
for (auto x : vec | filter([](auto x) { return x % 2 == 0; }) | take(10)
 | reverse)
 do_something(x);
```

It looks very much like the Unix pipes: The following Linux shell command will list all the installed packages related with LaTeX, sort them, and display the first five lines.

```
apt list --installed | grep latex | sort | head --lines 5
```

35/41

### Formatting library

Some may think that

```
printf("%d + %d == %d\n", a, b, a + b)
```

is better than

```
cout << a << " + " << b << " == " << a + b << '\n';
```

However, the existing `printf` family of functions have at least three drawbacks:

- The format string is parsed at runtime, which could have been done in compile-time. An error in the format string should be a compile-error.
- Not type-safe. Arguments are passed via `__VA_ARGS__`, which should have been done via variadic templates in modern C++.
- Not extensible. We cannot use `printf` to print objects of our self-defined types.

36/41

### Formatting library

The text formatting library offers a safe and extensible alternative to the `printf` family of functions. It is intended to complement the existing C++ I/O streams library.

```
std::cout << std::format("{} + {} == {}\n", a, b, a + b);
```

37/41

### The C++23 `print` library

With C++23 `print`, we can print a formatted string directly:

```
std::print("{} + {} == {}\n", a, b, a + b);
std::println("{} + {} == {}.", a, b, a + b); // equivalent way
```

Furthermore, the C++23 `print` functions are able to handle Unicode! The following should never produce garbled characters.

```
std::print("你好，世界!");
```

33/41

38/41

Future

- The graph library (Talk) (P1709R3), which may be in C++26?
- Standard library support for linear algebra algorithms: `<linalg>` in C++26.
- Debugging support: `<debugging>` in C++26.
- .....

Summary

- `function`: A function wrapper that stores a callable object.
- `optional`: Either a value or nothing.
- `string_view`: A reference to a string of any form, which is often used as a parameter.
- `pair` and `tuple`: A "quick and dirty" data structure, which should be seldomly used in modern C++.

Notes

<sup>1</sup> A pointer-to-member is also a callable.

CS100 Lecture 28

Compile-time Computations and Metaprogramming

Contents

- Example: Binary literals
- `constexpr` and `constexpr`
- `concept` and constraints
- Summary

Example: Binary literals

Binary literals

Built-in binary literals support: C++14 and C23.

```
switch (rv32inst.opcode) { // 32-bit RISC-V instruction opcode
case 0b0110011: /* R-format */
case 0b0010011: /* I-format (not load) */
case 0b0000011: /* I-format: load */
case 0b100011: /* S-format */
// ...
}
```

How do people write binary literals when there is no built-in support?

Runtime solution? No!

This is not satisfactory: The value is computed at run-time!

```
int dec2bin(int x) {
 int result = 0, pow_two = 1;
 while (x > 0) {
 result += (x % 10) * pow_two;
 x /= 10;
 pow_two *= 2;
 }
 return result;
}

void foo(const RV32Inst &inst) {
 const int forty_two = dec2bin(101010); // correct, but slow.
 switch (inst.opcode) {
 case dec2bin(110011): // Error! 'case' label must be compile-time constant!
 // ...
 }
}
```

Preprocessor metaprogramming solution

`#` and `##` operators: Both are used in function-like macros.

`#x`: stringify `x`.

```
#define SHOW_VALUE(x) std::cout << #x << " == " << x

int ival = 42;
SHOW_VALUE(ival); // std::cout << "ival" << " == " << ival;
```

`a##b`: Concatenate `a` and `b`.

```
#define DECLARE_HANDLER(name) void handler_##name(int err_code)
DECLARE_HANDLER(overflow); // void handler_overflow(int err_code);
```

Preprocessor metaprogramming solution

<https://stackoverflow.com/a/68931730/8395081>

```
#define BX_0000 0
#define BX_0001 1
#define BX_0010 2
//
#define BX_1110 E
#define BX_1111 F

#define BIN_A(x) BX_##x

#define BIN_B(x, y) 0x##x##y
#define BIN_C(x, y) BIN_B(x, y)

#define BIN(x, y) BIN_C(BIN_A(x), BIN_A(y))

const int x = BIN(0010, 1010); // 0x##BX_0010##BX_1010 ==> 0x2a ==> 42
```



## Template metaprogramming solution

```
template <unsigned N> struct Binary {
 static const unsigned value = Binary<N / 10>::value * 2 + (N % 10);
};
template <> struct Binary<0> {
 static const unsigned value = 0;
};

void foo(const RV32Inst &inst) {
 const auto x = Binary<101010>::value; // 42
 switch (inst.opcode) {
 case Binary<110011>::value: // OK.
 // ...
 }
}
```

Compared to preprocessor metaprogramming, template metaprogramming is more powerful, and less error-prone.

8/43

## Modern C++: constexpr function

Just mark the function `constexpr`, and the compiler will be able to execute it!

```
constexpr int dec2bin(int x) {
 int result = 0, pow_two = 1;
 while (x > 0) {
 result += (x % 10) * pow_two; x /= 10; pow_two *= 2;
 }
 return result;
}

void foo(const RV32Inst &inst) {
 switch (inst.opcode) {
 case dec2bin(101010): // OK. Since 101010 is a compile-time constant,
 // the function is executed at compile-time and
 // produces a compile-time constant.
 // ...
 }
}
```

9/43

## Metaprogramming

Metaprogramming is a programming technique in which computer programs have the ability to **treat other programs as their data**.

- Read, generate, analyze or transform other programs, and even itself.

10/43

## Typical problems that needs metaprogramming

Write a function that selects different behaviors **at compile-time** according to the argument?

- e.g. The `std::distance` function (in this week's recitation).

Generate some code according to the members of my class, without too much manual modification?

- e.g. Serialization: Generate `operator>>` automatically for my class that prints the members one-by-one?
- e.g. "Metaclasses": Generate the getters and setters automatically for each of my data members?

.....

11/43

## Compile-time computations

How much work can be done in compile-time?

- Call to numeric functions with compile-time known arguments?
  - e.g. Can `std::acos(-1)` be computed in compile-time?
- Manipulation of compile-time known strings?
  - e.g. Preparation of compile-time known regular expressions: [CTRE](#)
- Even crazier: [Compile-time raytracer?](#)
  - The computations are done entirely in compile-time. At run-time, the only work is to output the image.

Anything can be computed in compile-time, provided that the arguments are compile-time known!

12/43

## constexpr and consteval

### constexpr

**Constant expressions:** expressions that are evaluated at compile-time.

`constexpr` variables:

```
constexpr double dval = 5.2;
const int ival = 42;
```

By declaring a variable `constexpr`, we mean that its value is compile-time known, and will not change.

- A `constexpr` variable is implicitly `const`.
- It must be initialized from a constant expression. Otherwise, an compile-error.

A `const` variable initialized from a constant expression is also a constant expression.

14/43

### constexpr functions

`constexpr` functions are **potentially** executed at compile-time:

- When the arguments are constant expressions, it is run at compile-time and produces a constant expression.
- When the arguments are not constant expressions, it is run at run-time just like a normal function.

```
constexpr int gcd(int a, int b) {
 while (b != 0) { a = std::exchange(b, a % b); }
 return a;
}

int main() {
 const int x = 10, y = 16;
 constexpr auto result = gcd(x, y); // OK. The result is a constant expression.
 int n, m; std::cin >> n >> m;
 std::cout << gcd(n, m) << '\n'; // OK. It is computed at run-time.
}
```

15/43

### constexpr member functions

Member functions may also be `constexpr`. This is particularly useful for some very simple classes:

```
class StringView { // The 'StringView' class in lecture 27.
 const char *mStart(nullptr);
 std::size_t mLength{0};
public:
 // constructors
 constexpr StringView(const char *cstr);
 constexpr StringView(const std::string &str);
 // length
 constexpr std::size_t size() const { return mLength; }
 constexpr bool empty() const { return mStart; }
 // searching
 constexpr std::size_t find(char c, std::size_t pos = 0) const;
 // ...
};
```

16/43

## Evolution of constexpr functions

`constexpr` was first introduced in C++11, with many restrictions:

- A single `return` statement only. No loops or branches.
- `constexpr` member functions are implicitly `const`: They cannot modify the data members.
- `virtual` functions cannot be `constexpr`.
- Very little standard library support.
- .....

At that time, it was *almost* Turing-complete.

17/43

Evolution of `constexpr` functions

In C++14:

- Multiple statements, loops and branches are allowed.
- `constexpr` member functions are no longer implicitly `const`.
- `constexpr` lambdas are still not yet allowed.
- Definitely Turing-complete.

In C++17:

- Much more standard library support: A lot more functions are made `constexpr` since C++17.
- Lambdas are automatically `constexpr` when it can be.

`constexpr`: Immediate functions

`constexpr` generates an *immediate* function.

- Every call of an immediate function generates a constant expression that is executed at compile-time.

`constexpr`

- cannot be applied to destructors.
- has the same requirements as a `constexpr` function.

Evolution of `constexpr` functions

C++20: A huge step!

- `constexpr` functions can perform **dynamic memory allocations!**
  - Memory allocated at compile-time must also be released at compile-time.
- **Destructors** can be `constexpr`!
- Standard library **containers** like `std::vector`, `std::string` can be `constexpr`!
- Standard library **algorithms** are `constexpr`!
- **virtual** functions can be `constexpr`!

`constexpr`: Immediate functions

- `constexpr`: **potentially** executed at compile-time.
- `constexpr`: **must be** executed at compile-time.

```
constexpr int sqr(int n) { return n * n; }
constexpr int r = sqr(100); // OK.
int x = 100; // 'x' is not a constant expression!
int r2 = sqr(x); // Error: 'sqr' must be called with constant expressions.
```

Note: A non-`const` variable is not treated as a constant expression, even if initialized from a constant expression.

C++20: `constexpr` support in the standard library

```
#include <vector>
#include <algorithm>

constexpr int find_or_42(const std::vector<int> &vec, int target) {
 auto found = std::ranges::find(vec, target); // compile-time search
 return found == vec.end() ? 42 : *found;
}

int main() {
 // 'vec' is initialized in compile-time!
 constexpr auto result_1 = find_or_42({1, 4, 2, 8, 5, 7}, 10); // 42
 constexpr auto result_2 = find_or_42({2, 3, 5, 7}, 3); // 3
 static_assert(result_1 == 42);
 static_assert(result_2 == 3);
}
```

`concept` and constraints

`constexpr` numeric functions

Since C++23, some simple numeric functions in `<cmath>`, like `abs`, `ceil`, `floor`, `trunc`, `round`, ... are `constexpr`.

Since C++26, the **power**, **square/cubic root**, **trigonometric**, **hyperbolic**, **exponential** and **logarithmic** functions are all `constexpr`!

`constexpr` functions are pure

Pure functions (mathematical functions):

- Produce the same result when given the same arguments.
- Have no side effects. They cannot modify the value of variables outside them.
- Don't change the state of the program.

Motivating example: sorting a `std::list`.

How should we sort a `std::list`?

```
std::list<int> l = some_list();
std::sort(l.begin(), l.end()); // Is this correct?
```



## Motivating example: sorting a `std::list`.

How should we sort a `std::list` ?

```
std::list<int> l = some_list();
std::sort(l.begin(), l.end()); // No! We should use 'l.sort()'.
```

Ooops! `std::sort` requires a pair of *RandomAccessIterators*, but

`std::list<T>::iterator` is not.

- But how can I understand the output of the compiler? It is complaining about "Invalid operands to operator~" .....

28/43

## Example: Constrained algorithms

This time, the compiler will produce some human-readable output on violation of the requirements:

```
std::list<int> l = some_list();
std::ranges::sort(l);
```

```
b.cpp:8:3: error: no matching function for call to object of type 'const __sort_fn'
std::ranges::sort(l);
 ^
/usr/bin/../lib/gcc/x86_64-linux-gnu/13/../../../../include/c++/13/bits/ranges_algo.h:1794:7: note: candidate
template ignored: constraints not satisfied [with _Range = std::list<int> &, _Comp = ranges::less, _Proj = 1
identity]
operator()(_Range&& _r, _Comp _comp = {}, _Proj _proj = {}) const
~
/usr/bin/../lib/gcc/x86_64-linux-gnu/13/../../../../include/c++/13/bits/ranges_algo.h:1790:14: note: because
'std::list<int> &' does not satisfy 'random_access_range'
template<random_access_range _Range,
```

33/43

## Motivating example: sorting a `std::list`.

Let's look at how `std::sort` is declared:

```
template <typename Iterator, typename Pred>
void sort(Iterator begin, Iterator end, Pred compare);
```

`Iterator` is just a type deduced from the argument without any restrictions!

- In fact, this declaration accepts *anything* we pass to it.
- Errors can only be reported inside the function body, which might be in the form of "no match for .....".

29/43

## Example: Compile-time polymorphism

Run-time polymorphism:

- "Shape" is a general concept, so we define an abstract base class.

Compile-time polymorphism:

- `Shape` is a `concept` !

```
struct Shape {
 virtual void draw() const = 0;
 virtual ~Shape() = default;
};
struct Rectangle : Shape {
 void draw() const override;
};
struct Circle : Shape {
 void draw() const override;
};
void drawStuff(const Shape &s) {
 s.draw();
}
```

```
template <typename T>
concept Shape = requires(const T x) {
 x.draw();
};
struct Rectangle {
 void draw() const; // non-virtual
};
struct Circle {
 void draw() const; // non-virtual
};
template <Shape T>
void drawStuff(const T &s) {
 s.draw();
}
```

34/43

## Constraining the template arguments

```
template <typename Iterator, typename Pred>
void sort(Iterator begin, Iterator end, Pred compare);
```

Can we declare the function with some constraints on the type `Iterator` ?

- We need something like

```
template <RandomAccessIterator Iterator, typename Pred>
void sort(Iterator begin, Iterator end, Pred compare);
```

30/43

## How `concept` and `requires` benefit template code

Without the C++20 `concept` and `requires`, we need to write the requirements in a "hacky" way through `SFINAE` (Substitution Failure Is Not An Error):

```
template <typename T, typename = std::enable_if_t<std::is_integral_v<T>>>
void increment(T &x) { ++x; }
```

With C++20:

```
template <std::integral T>
void increment(T &x) { ++x; }
```

Or even simpler:

```
void increment(std::integral auto &x) { ++x; }
```

35/43

## `concept`

A `concept` is a named set of requirements often used to constrain the template arguments.

Examples:

```
template <typename T>
concept movable = std::is_object_v<T> &&
 std::move_constructible<T> &&
 std::assignable_from<T &, T> &&
 std::swappable<T>;

template <typename T>
concept Incrementable = requires (T x) {
 { x++ } -> std::same_as<T>;
}
```

31/43

## Summary

### The past

Back to 1979, the Bell Labs: C with Classes made by Bjarne Stroustrup.

- An object-oriented C with the ideas of "class" from Simula (and several other languages).
- Member functions, derived classes, constructors and destructors, protection mechanisms (`public`, `private`, `friend`), copy control through `operator=`, ...
- Better syntax, better type checking, ...

37/43

## Example: Constrained algorithms

The C++20 algorithms in `std::ranges` have well-defined constraints on the arguments:

```
template <std::random_access_iterator I, std::sentinel_for<I> S,
 typename Comp = ranges::less, class Proj = std::identity>
requires std::sortable<I, Comp, Proj>
constexpr I sort(I first, S last, Comp comp = {}, Proj proj = {});

template<ranges::random_access_range R, typename Comp = ranges::less,
 class Proj = std::identity>
requires std::sortable<ranges::iterator_t<R>, Comp, Proj>
constexpr ranges::borrowed_iterator_t<R>
sort(R&& r, Comp comp = {}, Proj proj = {});
```

32/43

## The past

After C with Classes was seen as a "medium success" by Stroustrup, he moved on to make a better new language - C++ was born (1983).

- Virtual functions, overloading, references, `const`, type checking, ...
- Templates, exceptions, RTTI, namespaces, STL were added in the 1990s.

By the year 1998, C++ had become matured and standardized with the four major parts (*Effective C++* Item 1):

- C
- Object-Oriented C++
- Template C++
- The STL

```
CS100::~~CS100();
```

38/43

43/43

## Entering Modern C++

A huge step since 2011:

- Rvalue references, move semantics, variadic templates, perfect forwarding
- Better template metaprogramming support
- Smart pointers
- `auto` and `decltype`: More benefit from the static type system
- Lambdas, `std::function` and `std::bind`: Functional support
- The concurrency library ( `std::thread`, `std::mutex`, `std::atomic`, ...)
- `constexpr`: Support for more straightforward compile-time computations.
- .....

39/43

## Evolution since C++11

More specialized library facilities:

- `optional`, `any`, `variant`, `tuple`
- `filesystem`: Standardized filesystem library
- `regex`: The regular expression library
- `string_view`: Heading towards the C++20 `views` and `ranges`

More compile-time computation support:

- More restrictions on `constexpr` functions and `auto` deduction are removed.
- Class Template Argument Deduction (CTAD)

40/43

## C++20 is historic!

[CppCon2021 Talk by Bjarne Stroustrup: C++20: Reaching the aims of C++](#)

C++20 is the first C++ standard that delivers on virtually all the features that Bjarne Stroustrup dreamed of in *The Design and Evolution of C++* in 1994.

- Coroutines ([Talk](#))
- Concepts and requirements ( `concept`, `requires` ) ([Talk](#))
- Modules ([Talk](#)) ([Talk on the implementation by MSVC](#))
- Ranges library
- Formatting library
- Three-way comparison ( `operator<=>`, `std::partial_ordering`, ...)

41/43

## Future

- Static reflection and metaprogramming ([Talk](#)) (P2237R0) (P1240R1) (P2320R0)
- Metaclasses: Generative C++ (P0707R3)
- Pattern matching ([Talk](#)) (Herb Sutter's [cppfront](#) project)
- Structured concurrency support (executors) ([Talk](#)) (P2300R6)
- Internal representation of C++ code suitable for analysis ([Talk](#)) ([GitHub Page](#))
- .....

42/43